

基于跨日随机控制与价值反馈的微网购电和储能协同调度

摘 要

微网调度同时受到光伏和负荷预测误差、合同提前确定、紧急购电价格较高以及储能跨日库存价值的影响。午夜库存连接次日供需，合同承诺与实时动作又遵守不同的信息截止时刻。本文把四问统一为一个带合同、库存和实时反馈的随机动态系统，构造“成熟历史残差场景—因果合同规划—凸 Markov 动态规划反馈—滚动重规划—真实闭环结算”的控制框架。

针对问题一，证明合同费用与库存变化的紧急供能费用均为凸分段线性函数，通过同时充放电消元建立确定性线性规划。典型日购电量为 **59,482.70 kWh**，费用为 **35,126.95 元**，首末库存均为 6,000 kWh。

针对问题二，在中间午夜连续、真实年末统一为 6,000 kWh 的条件下，从成熟历史残差块中按时间覆盖选取 7 条等权轨迹，在因果仿射策略类内规划购电，并以有限状态 Markov 近似递推连续库存价值。2025 年 2 月至 12 月 334 日实际费用为 **1,384.42 万元**，较同样允许跨日的贪心基线节省 **20.24 万元**。

针对问题三，在合法发布节点融入光伏预报并重订剩余合同，费用为 **1,346.18 万元**，较对应跨日贪心基线节省 **23.13 万元**。保持策略框架和调约权限，删除 12 时预报使当年回放费用增加 **17.93 万元**。

针对问题四，将净需求和价格的历史误差成对用于规划，实际电价在执行后结算。日前、日内机制费用分别为 **1,465.46 万元**、**1,425.21 万元**，较各自跨日贪心基线节省 **15.29 万元**、**22.89 万元**。

四种机制均保持真实初末库存 6,000 kWh。相同仿射框架下，解除中间午夜复位带来 **22.66 至 33.36 万元**的闭环节省；库存价值反馈相对跨日仿射节省 **58.58 至 76.27 万元**。相对跨日贪心，三种块长的配对重采样区间均位于正值侧，11 个评价月份均累计节省。理论保证对应声明的凸模型和信息条件，实验支持当前单年度回放。

关键词：微网调度；跨日储能；因果合同；凸动态规划；滚动优化

1 问题重述与控制视角

微网的费用同时落在三个时间尺度上。日前要把交付合同定下来；日内只能在题目允许的时刻，根据已经发布的预测去改尚未交付的承诺；电池则按十分钟响应眼前的真实净需求。电池可以跨时段转移富余电量，转移规模受效率、容量和功率限制。若今晚剩下的电要支撑次日早晨，或低价补能要等到明天，午夜库存就是连接两天的状态。

四问共用同一套物理和计费规则，差别只在当时知道什么、允许改什么。问题一给定典型日完整负荷、光伏和电价，求逐段购电与电池动作，并且当天首末库存相同。问题二在固定分时电价下，必须在 0 时确定当天不可调整的合同，用历史信息应对尚未发生的净需求误差。问题三增加每日 0、6、12、18 时发布的光伏预报，并允许在这些节点修订尚未交付的合同。问题四再引入交付电价不确定，分别考察日前和日内两种权限。题目要求的指定日期和时段结果从实际轨迹直接读出，电量一律在交流母线侧计量。

据此，策略被拆成两层：上层承诺必须对签约时刻的信息可测；下层电池在观测当前净需求后，权衡即时供能与未来库存价值。除整年首末固定为 6,000 kWh 外，中间午夜不另加复位。微网滚动调度与储能多阶段随机规划已有实验和算法基础 [1, 2, 3]；本文把合同限制在仿射扰动反馈策略类中 [4, 5]，用经验场景平均近似期望费用 [6, 7]，下层则以有限状态 Markov 决策的值函数近似电池动作 [10, 11]。后文先写能量和计费的凸结构，再写信息如何约束策略，最后给出价值递推与四问账单。理论命题采用结论类型标记：A 表示数学性质，B 表示所声明近似模型内的最优性或下界；实验结果和适用范围直接用文字说明。

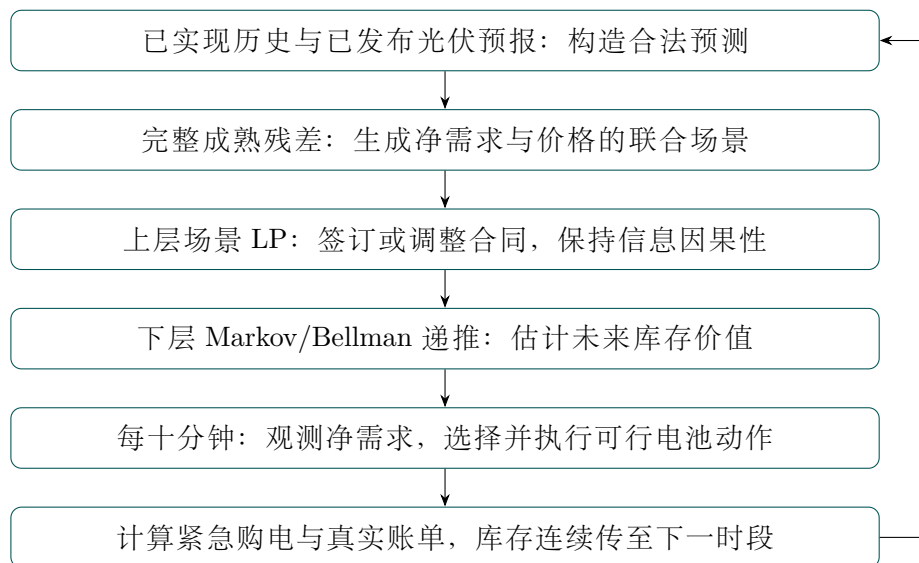


图 1 预测、合同规划、实时价值反馈与闭环结算的总体框架

2 假设、符号与信息结构

2.1 模型假设

1. 附件功率按对应十分钟区间均值积分；电量统一在交流母线侧计量，内部库存单独递推。

2. 将题给 90% 解释为充电和放电各自的转换效率；不额外引入题面未给的自放电、线路损耗和循环退化参数。
3. 不向外网售电，允许免费弃置母线富余；紧急购电无题外容量上限。
4. 采用退购退款后支付 50% 违约金的主口径；另报告不退款口径的账单敏感性。
5. 合同只使用签约前已知信息；电池可观测本时段净需求，实际浮动电价在动作后用于结算。
6. 问题一主方案选定周期初值 6000 kWh，并补充自由周期初值对照；问题二至四正式比较统一真实初末库存 6000 kWh。

2.2 物理参数、时间计量与符号

容量、功率与库存上下界取自题目。将每十分钟作为一个交付时段，时段长度为六分之一小时。输入的 00:10 至 24:00 数值解释为此前十分钟的区间均值，输出从 00:00–00:10 标记至 23:50–24:00。负荷与光伏功率除以 6 得到时段电量。单向效率 0.9 采用上文对题给 90% 的解释。于是

$$\eta = 0.9, \quad M = 5000/6, \quad E_{\min} = 1200, \quad E_{\max} = 10800. \quad (1)$$

额定充放电功率为 5,000 kW，允许库存为 1,200 至 10,800 kWh。系统允许以规定价格从外网补电，无额外外网容量限制；不向外网售电，母线富余可免费弃置且不设弃电上限。当前模型无循环奖励、启停时间和电池退化费用，价格非负。这些条件对应给定调度模型，也是后文消元及可达性命题的条件。弃电变量表示母线总富余，不能直接换算为纯光伏弃电率。合同必须先定，电池动作可以后做，真实电价只用于结算。

表 1 主要符号及计量口径

符号	含义
E_t, δ_t	时段初库存及本段库存增量, kWh
q_t, r_t	原始合同及最终有效交付合同, kWh
c_t, d_t, e_t, w_t	充电、放电、紧急购电、母线弃电, kWh
n_t, p_t	净需求电量及实际交付价格, kWh、元/kWh
ξ_t	外生扰动, 包括净需求、价格误差及预报创新
a, H, S	当前规划时段、该次时域长度、经验轨迹数
$K_t, V_{t,k}$	净需求箱标签、该标签下的库存继续费用函数
$\mathcal{F}_t^-, \mathcal{F}_t$	合同可用信息及当前净需求实现后的控制信息

2.3 严格的观测与行动次序

先签约，再观察当前净需求并操作电池，最后按实际交付价格结算。

$$\mathcal{F}_t^- \subseteq \mathcal{F}_t, \quad (q_t, r_t) \in L^0(\mathcal{F}_t^-), \quad (c_t, d_t, e_t, w_t) \in L^0(\mathcal{F}_t). \quad (2)$$

其中，原始合同还须对所属日签约时刻的信息可测，修订合同须对最后合法修订节点的信息可测；式(2)是共同必要条件，并不赋予逐时重新签约权。净需求实现后新增的观测

不能倒流进入该时段合同。变动价机制的当段真实电价不作为动作输入，控制使用此前可得的预测价格，真实价格留在结算环节。

库存是物理状态，但一般不是完整随机控制状态。一个充分的状态表述须包含库存、未交付合同和足以更新条件分布的历史或预测器状态；也可以直接以完整合法历史定义策略，不预设有限维马尔可夫性。后文的二元状态是**冻结本次名义合同后的经验近似状态**，不是对原始随机过程充分状态性的证明。

3 统一凸能量模型

在题目的计费规则和电池约束下，合同费用是凸的；电价不为负时，存在不同时充放电的最优解，因而不必给电池加 0-1 开关。紧急购电关于库存变化同样是凸分段线性。若年末要回到 6,000 kWh，每一步就只能走到还来得及抵达终点的区间里。四问的差别不在这些等式，而在合同权限和信息集。

3.1 状态方程和随机控制目标

令净需求等于负荷电量减光伏电量，则电池递推与母线平衡为

$$E_{t+1} = E_t + \eta c_t - d_t / \eta, \quad (3)$$

$$r_t + e_t + d_t - c_t - w_t = n_t. \quad (4)$$

同时满足非负电量、容量边界及单段充放电量不超过 M 。对遵守各机制合同权限、信息约束和真正初末条件的策略集合，统一目标为

$$J^* = \inf_{\pi \in \Pi} \mathbb{E} \left[\sum_{t=0}^{T-1} p_t \{ \phi(q_t, r_t) + 5e_t \} \right]. \quad (5)$$

这里的期望对应声明的外生过程分布。实际计算以有限经验场景上的样本平均近似该目标 [6, 7]，并作滚动执行。问题一为确定性且无紧急购电的特例；问题二固定价格并约束最终合同等于原合同；问题三放开合法修订并增加已发布预报；问题四使用随机价格结算。

3.2 合同结算凸性

采用题解中已经实现的退购解释：退购部分原电费由 50% 违约费替代，因此相对日初合同退回 50% 的减少部分电费。对每个交付段，计费相对原合同与最终合同净额计算，不把多次发布的变化绝对值重复累加。当最终合同增大时，费用对应原合同加 1.5 倍增购；减小时，原合同扣除 0.5 倍退购。因而

$$\begin{aligned} \phi(q, r) &= q + 1.5(r - q)^+ - 0.5(q - r)^+ \\ &= \begin{cases} 1.5r - 0.5q, & r \geq q, \\ 0.5r + 0.5q, & r < q, \end{cases} \\ &= \max\{1.5r - 0.5q, 0.5r + 0.5q\} = r + 0.5|r - q|. \end{aligned} \quad (6)$$

命题 1 (A：合同费用的凸性) 函数关于两个合同变量联合凸且分段线性。

证明. 两仿射表达式之差为最终合同减原合同, 故在各自区间恰由较大者取值; 仿射函数的逐点最大值为凸分段线性函数。□

因此可用上图变量 F 及两条线性不等式表示合同费用。非负价格保证最小化上图费用与原目标的最优值一致; 价格为零时不要求所有上图变量均紧。合同修订没有引入新的非凸项, 且与库存价值的凸结构兼容。若采用不同的退购结算解释, 则须改变目标并重新计算, 除另行标明的计费敏感性外, 主结果均对应式(6)。

另一种违约解释的对照 若原计划费照付, 再对退购加收 50% 违约金, 则应改为

$$\phi_+(q, r) = q + 1.5(r - q)^+ + 0.5(q - r)^+, \quad \phi_+(q, r) - \phi(q, r) = (q - r)^+. \quad (7)$$

以下将同一 334 日主轨迹按两种规则重计费, 以隔离口径变化; 它不是新口径下重新优化的年度最优费用。

表 2 同终点原策略在两种退购计费解释下的全年账单; 新口径未重新优化

问题	原口径 (元)	附加违约金口径 (元)	增幅 (%)
第二问	13844170.06	13844170.06	0.000
第三问	13461842.35	14073209.01	4.541
第四问日前	14654611.38	14654611.38	0.000
第四问日内	14252083.03	14907515.48	4.599

在非负价格、免费且不受限的弃电条件下, 将 $r < q$ 改为 $r' = q$ 、保持原电池动作并增加弃电 $q - r$, 物理平衡不变, 新口径费用减少 $0.5p(q - r)$ 。因此该条件下总存在不退购的最优动作, 结论不覆盖负价或弃电受限情形。

另在 1 月 25–31 日保持初末 6000 kWh、预测和权限一致, 改变上层计费上图并重新滚动执行: 问题三新口径费用 345349.60 元, 较原策略直接重计费的 350544.87 元少 5195.27 元; 问题四日内为 384732.76 元, 较 390984.78 元少 6252.02 元。新策略的退购量均为数值零。这是开发窗口的计费敏感性, 不替换主口径全年结果。

3.3 同时充放电消元与紧急电量

命题 2 (A: 存在单向最优解) 在上述免费无上限弃电、非负价格、无循环奖励条件下, 若原问题最优解存在, 则存在逐时不同时充放电的最优解。

证明. 对任一同时充放电时段, 构造

$$u = \min\{c, d/\eta^2\}, \quad c' = c - u, \quad d' = d - \eta^2 u. \quad (8)$$

二者保持非负且不超过原值, 至少一者为零。进一步有

$$\eta c' - d'/\eta = \eta c - d/\eta, \quad (c' - d') - (c - d) = -(1 - \eta^2)u. \quad (9)$$

令弃电增加同样的母线富余量, 保持合同和紧急电量不变, 即可保持平衡、库存与成本。逐时施加该变换得到单向最优解。该证明是存在性结论, 并不要求所有最优解均单向。□

以库存增量代替两个动作变量，单向恢复公式为

$$\delta = E_{t+1} - E_t, \quad c = \delta^+ / \eta, \quad d = \eta(-\delta)^+, \quad -M/\eta \leq \delta \leq \eta M. \quad (10)$$

其母线净耗电为两个仿射式的最大值，最小紧急电量及相应弃电为

$$\begin{aligned} g(\delta) &= \max\{\eta\delta, \delta/\eta\}, \\ e(r, \delta; n) &= \max\{0, n - r + \eta\delta, n - r + \delta/\eta\}, \\ w &= r + e - g(\delta) - n. \end{aligned} \quad (11)$$

命题 3 (A: 即时费用的凸性) 对固定净需求、合同及非负价格，紧急费用关于库存增量为凸分段线性函数。

证明. 式(11)是三个仿射函数的最大值，乘以非负系数保持凸性及分段线性。□

这一消元把功率方向与供需追索压缩进一个库存变量。模型采用母线聚合平衡，没有增加紧急电不得流向电池等分源约束；相关结论均基于这一实际实现。

3.4 一步可达域与真正终点

命题 4 (A: 单步可达闭区间) 给定当前库存，下一库存的物理可达域为

$$\mathcal{X}(E) = [E_{\min}, E_{\max}] \cap [E - M/\eta, E + \eta M]. \quad (12)$$

证明. 式(10)的增量上下界分别来自最大放电和充电；加入容量界即得必要条件。区间中任一点均可由式(10)恢复合规单向动作，并由紧急购电和免费弃电完成母线平衡，故也充分。□

图中斜带随当前库存平移，对角线表示不动作，上下斜边表示最大充放电，容量边界在两端截断可行带。

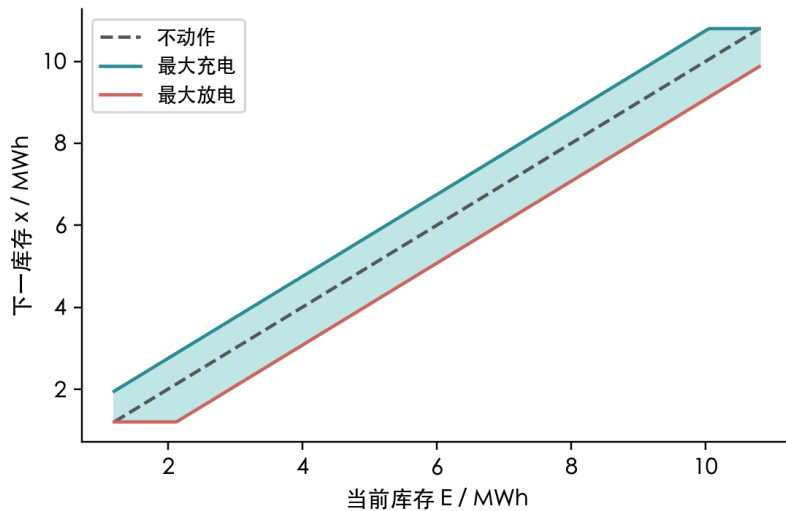


图 2 容量与功率共同决定的单步库存可达域

命题 5 (A: 真正终点的可达性) 本次动作后若还剩 N 步, 终点目标位于容量区间, 则下一库存必须且只须位于下列剩余可达区间; 实际执行再与单步可达域相交:

$$\mathcal{R}_N = [E_{\min}, E_{\max}] \cap [E_T^* - N\eta M, E_T^* + NM/\eta], \quad E_{t+1} \in \mathcal{X}(E_t) \cap \mathcal{R}_N. \quad (13)$$

证明. 必要性来自剩余 N 步累计充、放电能力。充分性方面, 当 N 为正且下一库存低于目标时, 将差值均匀分配到剩余 N 次充电; 高于目标时均匀分配到放电。区间条件保证每步不越功率界, 单调趋近目标保证中间库存不越容量界。外网追索与弃电保证供需可行。 N 为零时区间退化为目标单点。 $\square$

本稿主口径的真实评价终点为 12 月 31 日 24 时, 目标 6,000 kWh。除每日闭合对照外, 中间午夜不是终点; 滚动规划域的末点也不自动施加真实终点等式。执行器把目标库存投影到式(13)的交集后恢复动作, 从而保证真实末端可达。

4 确定性问题一: 凸模型的基准验证

题目要求日初与日末库存相同, 并未对该典型日另行固定周期库存。主方案参照附录的初始状态, 选择两端均为 6000 kWh; 该值是本文的实施口径。另令周期初值参与优化, 以检验这一选择的费用影响。

问题一没有预测, 也没有跨日: 典型日的负荷、光伏和分时电价全部已知, 当天首末库存均为 6,000 kWh, 并禁止紧急购电。它用来核对凸模型能否给出题目要求的指定时段结果, 也给后面的随机机制提供一个可对照的确定性基准。

典型日共 144 个交付段, 供需和分时电价已知, 原合同与最终合同相等, 并禁止紧急电。模型可直接写为

$$\begin{aligned} \min \quad & \sum_{t=0}^{143} p_t q_t, \\ & q_t + d_t - c_t - w_t = n_t, \\ & E_{t+1} = E_t + \eta c_t - d_t / \eta, \\ & 0 \leq c_t, d_t \leq M, \quad q_t, w_t \geq 0, \\ & E_{\min} \leq E_t \leq E_{\max}, \quad E_0 = E_{144} = 6000. \end{aligned} \quad (14)$$

命题 2 允许保留连续松弛并恢复单向动作。所有约束线性, 目标线性, 使用 HiGHS 求解 [15]。**模型内结论:** 求解器返回最优状态; 独立线性规划与对偶账单在数值容差内相符。**结果:** 全天购电 59,482.70 kWh, 充电 20,740.67 kWh, 放电 16,799.94 kWh, 费用 35,126.95 元; 原始对偶差约为 7.28×10^{-12} 元。该数值证书对应本确定性模型和浮点精度。

表 3 问题一购电结果 (电量 kWh, 费用元)

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	480.41	14:00–14:10	0.00
16:00–16:10	445.43	18:00–18:10	531.89	20:00–20:10	0.00
全天购电量	59,482.70	全天购电费	35,126.95		

表 4 问题一充放电与首末库存 (kWh)

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	4,500.00	0.00	04:00–08:00	833.33	6,365.84
08:00–12:00	4,787.96	1,703.00	12:00–16:00	5,286.04	91.10
16:00–20:00	0.00	5,780.13	20:00–24:00	5,333.33	2,859.87
0:00 库存		6,000.00	24:00 库存		6,000.00

表 5 问题一周期初始库存的边界条件对照

边界条件	起末库存 (kWh)	费用 (元)	节省 (元)
固定起末库存	6000	35126.95	0
优化周期起点	8550	35118.60	8.35

自由周期模型仅将固定边界替换为 $E_0 = E_{144}$ ，并令两端库存共同满足 $[1200, 10800]$ 的容量范围，仍是线性规划。一组最优周期起点为 8550 kWh，费用 35118.60 元，比固定 6000 少 8.35 元；最优周期起点不据此宣称唯一。前者对应可选择周期库存的设计，后者对应给定初始状态的实施，不能把改变实际起点库存视为免费操作。全天首末库存相同，故由状态递推求和，总放电量等于总充电量的效率平方倍，即 81%。图示库存随电价和供需共同变化：低价时段补能，高价值用电时段放能，最终回到规定库存。费用目标与外购电量目标不同，效率损耗并不妨碍低价向高价转移形成费用收益。

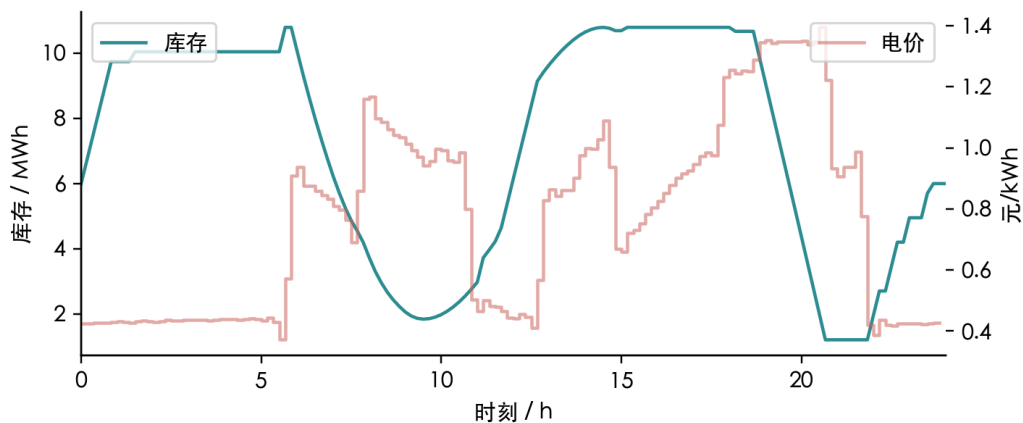


图 3 典型日库存与分时电价：解释跨时段移能机制

5 跨日随机规划：在合法信息中安排承诺

问题二至四的合同必须在真实净需求揭晓之前确定。规划只用当时已经允许看到的历史和预报，把已经走完的残差接到当前预测后面，作为若干条可能的后续轨迹；文件里尚未发生的数不得进入场景。电池如何响应当前缺口，留到下一节。

5.1 成熟残差与等权经验场景

预测族在 1 月 15 日至 31 日按 MAE 比较前一日、前一周、近七日均值和趋势四种候选；负荷选前一周、光伏选七日均值、价格选前一周。日内利用此前已观测的近 36 段误差作衰减修正。设当前绝对时段为 a ，规划长度为 H ，历史块起点为 h ，目标采用半开区间，则成熟条件为

$$[h, h + H) \subseteq [0, a), \quad h + H \leq a. \quad (15)$$

命题 6 (A：成熟场景的非预见性) 若历史块满足式(15)，且预测器参数、残差重建和轨迹选取均由当前已知信息确定，则当前场景不依赖当前之后的真实目标。

证明. 历史块中的最后一个目标发生于 a 之前，所有真实目标均属于当前信息。历史原点预测的输入前缀按该原点截断；用当前已确定的预测族进行重建及残差运算，仍是当前已知数据的可测函数。按时间下标选取轨迹也不引入未来目标。只要求历史起点早于当前时点不足以保证这一结论。 $\square$

历史残差的重建使用冻结预测族，并非声称这些预测族在每个更早历史原点已经被实际选中。1 月开发段与预测族选择区间重叠，因而该段是内部开发比较；2 月至 12 月才是冻结选择之后的时间顺序评价。

实际场景来自最近最多 28 个完整成熟残差块，按下标等距覆盖选取最多 7 条，赋予等权。净需求场景构造为当前预测加历史残差，价格场景使用同一历史块的价格残差：

$$n_{a+j}^s = \hat{n}_{a+j|a} + n_{h_s+j} - \hat{n}_{h_s+j|h_s}, \quad p_{a+j}^s = \max\{10^{-3}, \hat{p}_{a+j|a} + p_{h_s+j} - \hat{p}_{h_s+j|h_s}\}. \quad (16)$$

变动价格预测作正值截断；固定价机制直接使用给定分时价。此设计保留历史块内的联合时间结构，属于确定性时间覆盖的经验分布，不是独立同分布抽样，也没有采用概率度量下的最优场景缩减 [8, 9]。场景是否适应当前规划，最终由同终点闭环账单判断。

预测误差与可复核对照 用 MAE 和 RMSE 分别衡量平均绝对偏差及较大误差：

$$\text{MAE} = N^{-1} \sum_i |y_i - \hat{y}_i|, \quad \text{RMSE} = \sqrt{N^{-1} \sum_i (y_i - \hat{y}_i)^2}. \quad (17)$$

表 6 冻结预测器与朴素候选的误差核验；开发期为 1 月 15–31 日，正式期为 2–12 月

对象	预测方法	开发 MAE	开发 RMSE	正式 MAE	正式 RMSE
负荷 (kW)	前一日	658.48	1148.64	650.02	1128.39
负荷 (kW)	前一周 (选定)	142.55	177.25	176.80	244.29
负荷 (kW)	近七日均值	781.87	961.04	781.21	955.92
负荷 (kW)	周趋势	248.25	310.44	297.49	390.92
光伏 (kW)	前一日	119.48	261.96	185.35	375.17
光伏 (kW)	前一周	137.16	280.30	214.69	419.95
光伏 (kW)	近七日均值 (选定)	96.18	203.87	153.40	303.86
光伏 (kW)	周趋势	186.87	402.53	338.36	681.30
电价 (元/kWh)	前一日	0.08576	0.12877	0.08433	0.12384
电价 (元/kWh)	前一周 (选定)	0.03829	0.04878	0.04715	0.06535
电价 (元/kWh)	近七日均值	0.08160	0.10473	0.08267	0.10527
电价 (元/kWh)	周趋势	0.06447	0.08176	0.07920	0.10519

表中正式期是冻结预测族后的逐日评价，不用其误差重新选型；开发期分数属于回顾性选择窗口。按四个合法发布点分别评估随后六小时，历史修正与官方预报下的净需求 MAE 分别为 264.29 和 207.41 kW。预测族较简单是模型局限；完美信息费用差同时含交易与控制差异，不能单独用来诊断预测器。

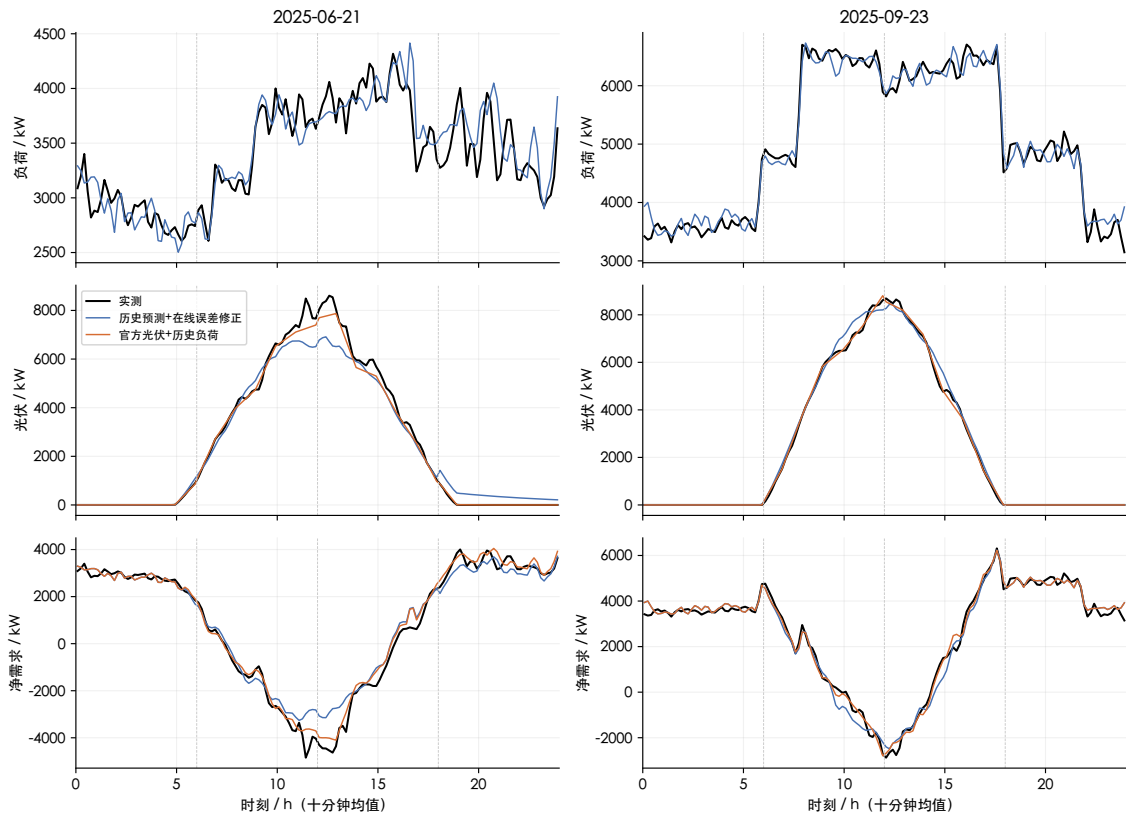


图 4 冻结预测与实际轨迹对照；图中各预报遵循其发布时刻

5.2 有限维因果扰动反馈

合同与库存请求写成对已实现扰动特征的仿射反馈。仿射扰动反馈把策略搜索变成凸规划，是可调鲁棒优化与约束鲁棒控制中的标准参数化 [4, 5]；本文把它用在经验场景的期望费用上，并按题目的签约时刻截断特征，而不是对任意扰动集合求最坏情形。场景误差以该次场景中心为基准，构造当前误差、EMA 和近 36 段均误差。EMA 递推及一般反馈形式为

$$z_j^s = 0.8z_{j-1}^s + 0.2\varepsilon_j^s, \quad z_{-1}^s = 0, \quad (18)$$

$$\begin{aligned} q_j^s &= a_j^q + G_{\nu_q(j)}^q \psi_{\rho_q(j)}^s, & r_j^s &= a_j^r + G_{\nu_r(j)}^r \psi_{\rho_r(j)}^s, \\ E_{j+1}^s &= a_j^E + G_{\nu_E(j)}^E (\varepsilon_j^s, z_j^s)^\top. \end{aligned} \quad (19)$$

截止下标由决策权限决定：未来原合同使用所属日开始前的误差特征，修订合同使用所属六小时块开始前的特征，库存请求可使用当前段误差及 EMA。合同特征接近 36 段均误差、EMA 的顺序组成二维列向量；日或块内共享两个增益，增益逐分量限定在负 10 至 10。当前已锁定的合同不是自由变量。

命题 7 (A：仿射策略的因果性) 给定规划根节点可测的系数，若特征仅由其合法截止时刻的信息构成，则该策略满足非预见性。

证明. 相同历史前缀给出相同特征，根节点拟合系数已经确定，所以对应决策一致；可测函数的仿射组合仍可测。库存特征可以比合同特征多含当前净需求，实现先承诺后响应的权限区分。 $\square$

该策略类没有纳入未来尚未发布的官方预报创新，后者在实际发布后由重新规划使用。**模型内定位 (B)：**上层最优性对应给定经验轨迹、特征和增益界内的受限仿射类；场景内可行不直接推出全部外样本可行，实际供需由执行器追索保证。

5.3 上层场景线性规划及继续价值

将式(19)代入各场景，以合同上图和紧急电量上图表示费用。目标是有限等权场景上的样本平均近似 [6, 7]：

$$\begin{aligned} \min \quad & \frac{1}{S} \sum_{s=1}^S \left[\sum_{j=0}^{H-1} p_j^s (F_j^s + 5e_j^s) + \Psi_a(E_H^s) \right], \\ & F_j^s \geq 1.5r_j^s - 0.5q_j^s, \quad F_j^s \geq 0.5r_j^s + 0.5q_j^s, \\ & e_j^s \geq 0, \quad e_j^s \geq n_j^s - r_j^s + \eta(E_{j+1}^s - E_j^s), \\ & e_j^s \geq n_j^s - r_j^s + (E_{j+1}^s - E_j^s)/\eta, \\ & E_{j+1}^s \in \mathcal{X}(E_j^s), \quad E_0^s = E_a, \quad q_j^s, r_j^s \geq 0, \\ & -10 \leq G^q, G^r, G^E \leq 10. \end{aligned} \quad (20)$$

还须加入机制规定的合同固定等式、式(19)及已抵达真实终点时的库存等式。日前机制始终保持最终合同等于原合同。选择线性末值时，继续费用取

$$\Psi_a(E) = -\lambda_a E, \quad \lambda_a = Q_{0.25}(p_{H-\min(144,H):H}^{1:S})/\eta. \quad (21)$$

这一近似按未来低价补能的边际费用给库存估值，用于抑制短时域末点的过度放电倾向。滚动域到达真正评价终点时令末值系数归零，并由上层硬等式和实际执行可达集合管理末端。该目标是规划评分，正式账单始终按真实交付费逐段求和，不把虚拟末值记入现金节省。

每日 0、6、12、18 时重规划，时域覆盖至第二个午夜并在真正终点截断。这与微网模型预测控制的滚动时域一致 [1, 2]，但合同变量始终受式(2)约束，不会在十分钟尺度上重新签约。0 时落实当日原合同，合法修订节点落实剩余日内合同，未来场景动作作为价值计算参考。问题二和问题四日前机制的六小时重算不产生额外签约权限。

6 凸 Markov 动态规划：用库存价值响应当前净需求

合同按上一节锁定之后，电池看见眼前缺口，需要判断当前库存中的单位电量是现在用来少买紧急电，还是留到后面。下层把该子问题写成有限状态 Markov 决策过程 [10]，并以凸分段线性值函数作近似动态规划 [11]。斜率合并用来计算下确界卷积，而不必对每个库存点另解一次线性规划。

6.1 状态压缩和固定名义合同

每次上层求解后，以其未来名义合同作为本次下层计算的固定序列。将该次经验场景在每段按净需求三分位分成三箱，当前观测经相同阈值映射为标签。于是下层近似状态为

$$(E_t, K_t), \quad K_t \in \{0, 1, 2\}. \quad (22)$$

相邻场景标签的计数形成时变转移矩阵。每个单元加三分之一伪计数，每行总伪计数为 1:

$$P_j(k, k') = \frac{N_j(k, k') + 1/3}{\sum_{\ell=0}^2 N_j(k, \ell) + 1}. \quad (23)$$

箱内净需求采用该箱场景经验分布，空箱退回全部场景；价格使用箱内条件均价。这是每次规划重建的经验 Markov 近似，并非从全年资料拟合的固定平稳链。箱内价格压缩不保留剩余净需求与价格相关性的全部信息。到下一合法节点，重新求上层 LP、更新名义合同和经验链。

6.2 Bellman 方程与观测顺序

用下一段转移概率组合继续费用，令容量域之外的值为正无穷。固定合同费用与当前库存选择无关，可从该次电池子问题中略去。箱内净需求先被观察，电池再决策，因此值函数采用先最小化再组内平均：

$$W_{j,k}(x) = \sum_{k'=0}^2 P_j(k, k') V_{j+1,k'}(x), \quad (24)$$

$$V_{j,k}(E) = \frac{1}{|B_{j,k}|} \sum_{s \in B_{j,k}} \min_{x \in \mathcal{X}(E)} \{5\bar{p}_{j,k} e(r_j, x - E; n_j^s) + W_{j,k}(x)\}. \quad (25)$$

空箱时使用全部场景作为组内集合。将净需求先取均值再最小化会改变信息顺序，并非式(25)。终端函数在一般滚动末点采用线性末值或凸割包络；本次 DP 遇到给定终点时，源码使用有限绝对值罚

$$V_{H,k}(E) = 1000|E - 6000|. \quad (26)$$

本文没有把 1000 证明为精确罚系数；真正末端由执行器的硬可达集合保证。该区别将价值引导与物理执行保证分开。

6.3 Bellman 凸性保持定理

令库存消耗量为当前库存减下一库存，定义扩展实值即时函数

$$\begin{aligned} h_{j,k,n}(y) &= 5\bar{p}_{j,k} \max\{0, n - r_j - \eta y, n - r_j - y/\eta\} + \mathcal{I}_{[-\eta M, M/\eta]}(y), \\ (W \square h)(E) &= \inf_{x+y=E} \{W(x) + h(y)\}. \end{aligned} \quad (27)$$

指标函数在集合内为 0、集合外为正无穷；在 W 中同样加入容量域指标。于是每个组内最小化都是下确界卷积。

定理 1 (A: 凸分段线性保持) 若继续函数和即时函数在各自非空有限闭区间上为有限凸分段线性函数，并在域外取正无穷，则其下确界卷积在可行域上为闭凸分段线性函数；有限非负概率平均仍保持该结构。因此式(25)从声明的凸终端函数向后递推时保持凸分段线性。

证明. 即时成本凸性由命题 3 及区间指标得到。对两个可行库存及其最优分配，取相同权重的凸组合仍满足总量等式和区间约束；函数凸性给出组合分配成本不超过原最优成本的加权和，因此最优值凸。有限闭区间保证可行最小值达到；PWL 函数的上图为多面体，将联合上图按总量等式投影仍是多面体，故值函数闭且 PWL。最后，有限个这样的函数按非负概率求和仍凸且 PWL。完整域条件及归纳见附录。□

该论证也可由凸分析中部分最小化与下确界卷积的上图投影性质理解 [12, 13]。**推论：**值函数斜率随库存单调不减；把额外库存带来的继续费用减少称为边际收益时，其值为负斜率，单调不增。这里不要求严格单调，也不宣称整个域上库存收益始终为正，特别是终点惩罚附近。

6.4 斜率合并与连续动作

命题 8 (A: 斜率合并算法) 对有限区间凸 PWL 函数，下确界卷积可由两组非降斜率作有序合并，同时保留每段横向长度构造。两函数从左端点开始分配总增量。先使用边际费用较低的线段；凸性保证各函数内部斜率有序，全局合并仍遵守各自前缀。附录以小量交换给出完整证明。该方法把逐库存重复求 LP 化为线段合并，是下层计算的结构依据。

实现把值函数存储在 321 个库存节点上，卷积后回到网格作分段线性插值。凸函数的弦插值在节点间位于被插值函数上方，但这一局部关系不构成原随机问题的成本界。执行时，用当前净需求对应箱的价格均值，比较价值折点、即时缺口折点和物理边界，得到连续库存请求：

$$x^{\text{req}} \in \arg \min_{x \in \mathcal{X}(E)} \{5\bar{p}_{j,k} e(r, x - E; n) + \widetilde{W}_{j,k}(x)\}. \quad (28)$$

实际候选包括网格节点、当前库存和两个缺口折点，控制输入不限制为 321 个离散值。随后执行器施加真正终点可达投影；投影后的动作不宣称仍是带收缩区间目标的重新精确最小解。321 是数值存储参数，数学凸性来自函数结构。

7 信息更新、价格不确定性与理论边界

问题三把已经发布的光伏预报送进剩余合同，问题四再让电价随历史残差一起波动；二者都不改物理模型和合同结算式。精确模型里，信息变多或拿掉中间午夜复位，最优值都不会变差。实际闭环还经过场景、压缩和滚动近似，省多少钱仍以同终点回放为准。

7.1 预报如何进入问题三

每次实际发布后，将 24 个未来小时预测与发布前已观测起点拼成 25 个插值节点，再插值到十分钟段，并与历史预测衔接。控制使用已发布版本，当前时刻之后的新预报尚不可得。净需求预测变化首先改变剩余合同，再经继续费用影响电池留电量。图示 3 月 20 日原合同、最终合同和紧急购电的差异，用于说明合同修订与即时追索承担不同职责。

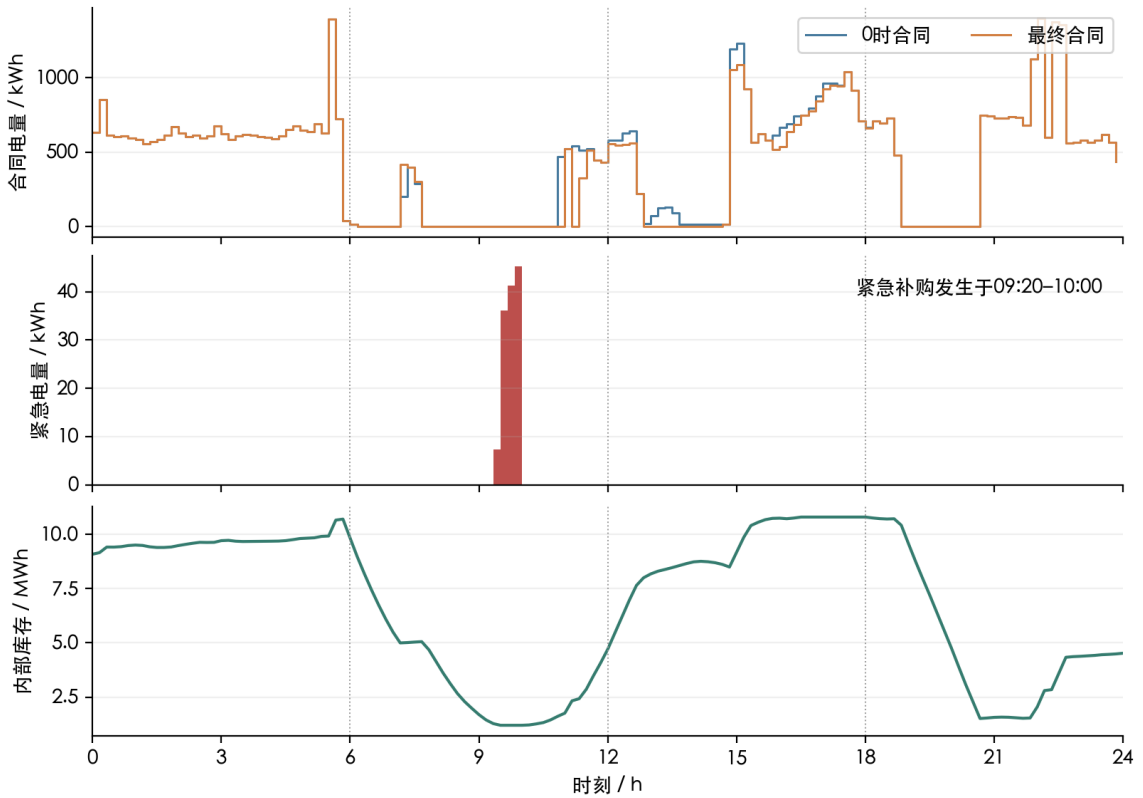


图 5 问题三 2025 年 3 月 20 日：合同更新与 09:20–10:00 紧急补购

信息价值结论 若两个精确优化问题除嵌套信息集外，控制权限、费用、物理条件和真正初末约束完全相同，而且较丰富信息下可以忽略新增信息，则

$$\mathcal{I}_1 \subseteq \mathcal{I}_2 \implies \Pi(\mathcal{I}_1) \subseteq \Pi(\mathcal{I}_2) \implies J^*(\mathcal{I}_2) \leq J^*(\mathcal{I}_1). \quad (29)$$

证明是将少信息策略原样嵌入多信息策略集合，再比较下确界。

适用范围 实际预报删减是冻结闭环框架的年度配对回放，重新构建经验场景及价值函数，并非分别求两种信息集下的精确总体最优解；其费用差表示本策略对已有信息的使用效果。

7.2 问题四的联合价格场景

问题四保持物理结构和相应合同权限，把净需求残差与价格残差按同一历史块加入当前预测。上层采用每条轨迹的费用，能够反映历史净需求与价格的成对变化。下层再压缩为净需求箱条件均价，以减小有限样本状态规模。实际账单始终使用交付电价，价格预测失准的影响自然进入评价结果。日前与日内机制分别与拥有相同权限的基线比较，不把信息、调约权利和算法三个因素混合解释成单一收益。

7.3 跨日策略集合包含关系

命题 9 (A: 解除中间复位约束的模型关系) 对同一精确模型、同一信息结构和同一真实初末条件，若每日闭合策略集合是在跨日策略集合中增加各午夜固定库存约束得到，则

$$\Pi_{\text{daily}} \subseteq \Pi_{\text{cross}}, \quad J_{\text{cross}}^* \leq J_{\text{daily}}^*. \quad (30)$$

证明. 任一满足额外午夜等式的策略也满足原跨日模型全部约束，故前者可行集合被包含；对包含关系取同一目标的下确界得到结论。□

适用范围 已实施的仿射请求、Markov 压缩和滚动重规划属于近似闭环。不同近似策略的实际账单并不由式(30)排序；下节用同初末库存的配对轨迹量化跨日机制。

8 实验与机制验证

主表同时列出每日闭合贪心、同样允许跨日的贪心，以及主策略。主结果以跨日贪心为对照；每日闭合贪心作为附加参照，午夜复位的单因素作用由同一仿射框架的配对消融识别。机制表再把「中间午夜是否复位」和「电池是否按库存价值动作」分开比较。各列对照对象不同，节省不宜叠加。

8.1 统一时间范围与主结果

正式评价为 2025 年 2 月 1 日至 12 月 31 日，共 334 日 48,096 段。1 月用于开发与公共初始化，不纳入正式评价。所有正式主策略与基线从 6,000 kWh 开始并在真实年末回到 6,000 kWh，避免借用不同末库存形成账面差。

表 7 同真实初末库存下的 334 日账单（万元；后两列不能相加）

机制	每日闭合基线	跨日贪心	主策略	相对跨日贪心	相对每日闭合
问题二	1418.20	1404.66	1384.42	20.24	33.78
问题三	1382.80	1369.31	1346.18	23.13	36.62
问题四日前	1501.06	1480.75	1465.46	15.29	35.60
问题四日内	1468.17	1448.10	1425.21	22.89	42.96

主结果：四种机制较同样允许跨日的贪心基线节省 15.29 至 23.13 万元，降幅 1.03% 至 1.69%，七日块重采样区间下界均为正。该基线使用场景共同合同与即时电量平衡，中间午夜连续，仅在真实年末回到 6,000 kWh。相对每日闭合贪心的总节省为 33.78 至 42.96 万元，该对照同时包含跨日权限与控制策略差异。跨日权限的单因素作用另由同一仿射框架的午夜复位消融衡量。不同对照的节省不能相加。两类策略均全额满足实际净需求，库存递推、跨午夜衔接和 5,000 kW 功率边界逐段核验。附录完整列出四个指定日期的购电表、全天电量与费用、六个四小时充放电块、首末库存及紧急购电区间；工作簿提供逐时明细。

8.2 跨日与价值反馈分别承担什么论证

第一项对照固定仿射策略框架，只改变中间午夜库存等式，衡量跨日状态连续的闭环效果。第二项对照保留跨日和线性末值，将仿射库存响应换为 Markov 价值反馈，衡量反馈架构的闭环效果。**结果：**跨日仿射相对每日闭合仿射节省 22.66 至 33.36 万元，

表 8 分别隔离跨日状态与库存价值反馈的闭环对照（万元）

机制	每日闭合仿射	跨日仿射	跨日节省	反馈再节省
问题二	1480.63	1457.98	22.66	73.56
问题三	1435.18	1404.76	30.42	58.58
问题四日前	1569.03	1541.73	27.30	76.27
问题四日内	1518.16	1484.80	33.36	59.59

Markov 反馈相对跨日仿射节省 58.58 至 76.27 万元。该比较表明，所实施 Markov 反馈闭环优于仿射库存响应闭环。各对照的上层合同在真实状态下重规划，因此第二项是完整反馈闭环的差异，不是固定所有合同后的电池动作净效应。两个机制对照与主结果的跨日贪心基线不同，不能将其节省叠加到主结果上。

8.3 已有预报的实际使用效果

预报删减实验保留 0、6、12、18 时的计算和调约权限，只改变可读取的预报版本，并统一真实终点。该设计将预报内容与合同调整权限的作用分离。**结果：**删除 12 时预报增加 17.93 万元，删除 6 时增加 7.20 万元，删除 18 时增加约 0.10 万元；只保留 0 时增加 31.68 万元。完整预报总体费用最低，但各发布贡献不能相加，因为信息更新通过库存、合同与后续动作相互作用。结论支持保留题目已提供的预报接口，不据此给出新增未提供时点的预测价值。

表 9 固定主策略框架下删减已提供预报的配对回放（万元）

保留预报	总费用	相对完整预报增加
全部	1346.18	0.00
只保留 0 时	1377.87	31.68
删除 6 时	1353.38	7.20
删除 12 时	1364.11	17.93
删除 18 时	1346.28	0.10

8.4 费用分解：节省来自哪些账目

将总费逐项分解为原计划、增购、退购净额和紧急电。退购净费用按负值记账，与式(6)保持一致。图中的分项变化均以各自跨日贪心基线为参照，负值表示减少支出，四项相加恰等于总费变化。

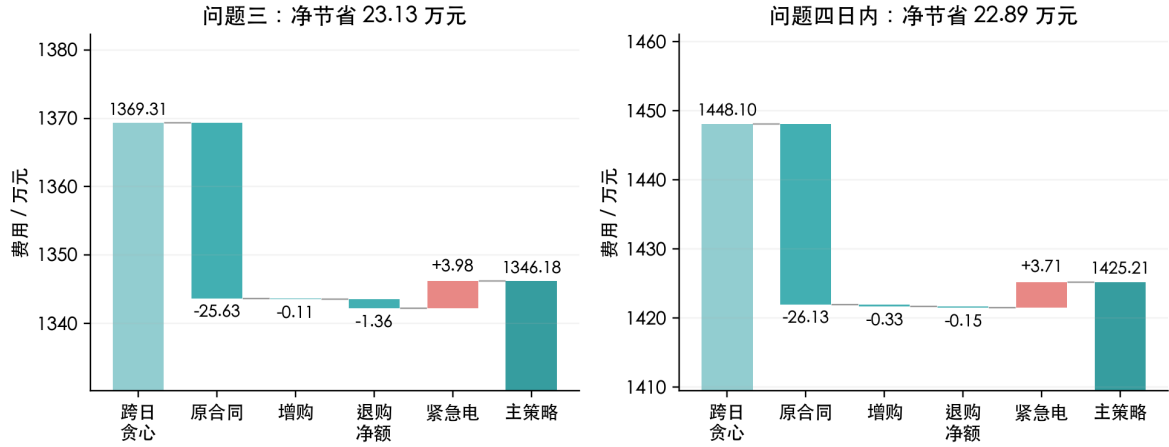


图 6 两个日内机制相对基线的费用变化：逐项核对节省来源

原计划减少不必单独追求最大，因为减少承诺可能提高紧急购电；增购与退购同样应按净账单评价。以问题三为例，原合同费用减少 25.63 万元，增购和退购净额合计再减少 1.48 万元，紧急费用增加 3.98 万元，净节省 23.13 万元。该策略通过联合安排合同与库存降低总账单；紧急费增支是已计入净收益的成本。

8.5 配对移动块重采样与月度稳定性

以跨日贪心账单减主策略账单定义每日配对差，两者均保持相同真实初末库存和中间午夜连续：

$$\Delta_d = C_d^{\text{base}} - C_d^{\text{main}}, \quad A = \sum_{d=1}^{334} \Delta_d, \quad \bar{\Delta} = A/334. \quad (31)$$

同一年的连续天气与跨日库存可能产生时间依赖，采用移动块重采样保留块内短期结构 [16]。从重叠的 3、7、14 日块中有放回抽取、拼接并截断至 334 日，每种配置重复 10,000 次，以日均差的 2.5% 和 97.5% 分位数形成区间。种子和累计区间由同一统计登记文件保存。

表 10 相对跨日贪心的日均节省与移动块 95% 区间 (元/日)

机制	日均节省	3 日块	7 日块	14 日块
问题二	606.1	[434.4, 797.9]	[447.4, 773.3]	[452.4, 755.4]
问题三	692.5	[565.1, 806.6]	[533.0, 826.5]	[514.9, 841.4]
问题四日前	457.7	[313.4, 601.0]	[314.3, 588.8]	[312.3, 585.0]
问题四日内	685.4	[562.7, 793.2]	[529.6, 811.2]	[509.8, 822.4]

表 11 相对跨日贪心的日、月稳定性

机制	正日数	负日数	正月数/11	一阶相关
问题二	272	62	11/11	0.205
问题三	296	38	11/11	0.328
问题四日前	262	72	11/11	0.061
问题四日内	298	36	11/11	0.381

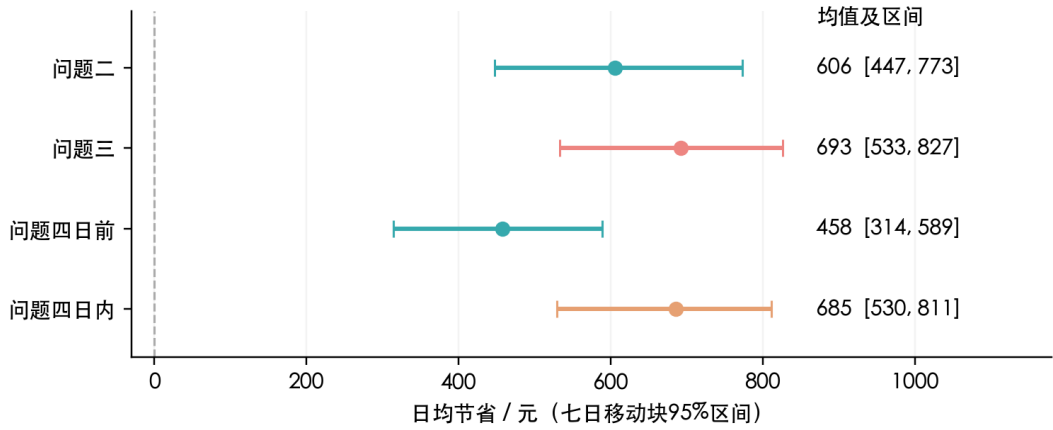


图 7 7 日块区间说明单年度配对节省的时间稳定性

结果：三种块长下四机制日均节省区间均位于正值侧，11 个月的累计节省均为正。日表同时报告节省与增支日期；连续库存允许在某日补能以服务后日，月度累计与运行期总账单衡量这种跨日安排的整体效果。一阶相关系数用于报告依赖的方向与程度，不把它称为正式显著性检验。

适用范围 这些区间用于本年度时间稳定性诊断，跨年度效果需由新增年份的顺序回放评价。

8.6 完美信息参照与可解释的性能边界

完美信息关系 令事后参照已知整条路径，以普通电价购电并保持相同电池条件、时间范围与真实初末库存。对任意可行因果策略，保留其电池和弃电，令参照普通购电为

$$q_t^{\text{PI}} = r_t + e_t, \quad p_t(r_t + e_t) \leq p_t\{\phi(q_t, r_t) + 5e_t\}. \quad (32)$$

不等式由非负电价、非负紧急电和合同恒等式直接得到，因此逐路径有

$$L_{\text{PI}}(\omega) \leq C_{\pi}(\omega). \quad (33)$$

同一分布下取期望并对全部因果策略取下确界，得到

$$\mathbb{E}[L_{\text{PI}}] \leq J_{\text{causal}}^* \leq \mathbb{E}[C_{\pi}]. \quad (34)$$

单条 2025 路径不能替代式(34)中的总体期望。数值参照采用全 48,096 段线性规划，并独立修复、计算对偶下界以检查浮点误差。

表 12 同路径、同真实终点的完美信息参照

机制	主策略/万元	乐观参照/万元	参照距离
问题二	1384.42	1223.04	13.19%
问题三	1346.18	1223.04	10.07%
问题四日前	1465.46	1278.30	14.64%
问题四日内	1425.21	1278.30	11.49%

表中相对参照距离以费用差除以参照费用计算。固定价两档大约贵 13.19% 和 10.07%，变动价两档大约贵 14.64% 和 11.49%。

适用范围 这同时放松未来信息和成交渠道，并与有限场景、状态压缩、策略类限制、滚动时域及末值近似有关，不能称为算法最优性 gap，不能将差额承诺为在线可实现节省。

9 参数、辅助 SDDP 与模型边界

末端库存估价服务于滚动执行，候选的取舍依据闭环实际账单。随机对偶动态规划通过阶段 LP 的对偶信息构造继续费用下界 [14, 3]。本节用同口径对照说明线性末值的选取依据，并通过数值敏感性确定主模型的计算精度。

9.1 按闭环实际账单选择模型

开发期比较场景共同合同贪心、仿射反馈、Markov 反馈及两类 SDDP 末值候选。原选型脚本只读取 1 月 25 至 31 日自由终点开发费用，规则为最小费用，若线性末值 Markov 费用处于最小值的 0.01% 以内则优先选择它：

$$J_{\min} = \min_m J_{\text{dev}}(m), \quad \hat{m} = \begin{cases} m_{\text{ML}}, & J_{\text{dev}}(m_{\text{ML}}) \leq 1.0001 J_{\min}, \\ \arg \min_m J_{\text{dev}}(m), & \text{otherwise.} \end{cases} \quad (35)$$

选择过程不读取 2 月至 12 月费用。正式论文统一固定真实终点后，补做同初末 6,000 kWh 的开发期复核，四机制按原规则仍选择线性末值 Markov。这是事后的口径一致性增强证据，不倒置为先于评价结果的全新预注册。完整费用见附录。

评价目标是实际闭环账单，而不是对某个辅助继续值模型逼近得更精细。简单末值通过相同开发规则被保留，SDDP 承担候选价值代理的论证职责。

9.2 末值的模型选择作用

辅助 SDDP 的状态包含库存和未交付合同，按两小时聚合、有限阶段独立噪声训练；其割的下界性质仅针对该辅助模型，证明置于附录。按已冻结的开发期账单规则，主策略采用线性末值；既有自由终点同口径全年 SDDP 对照未降低真实账单，两者分别承担选型和补充评价职责。局部窗口的末值倍率敏感性说明，当前控制账单对这一价格代理较稳定；据此采用线性末值，把计算资源用于合同重规划和库存价值反馈。零末值、倍率变化及 SDDP 的完整对照均保留在附录。

9.3 参数的作用及诊断

表 13 参数的数学角色及解释范围

参数	作用与诊断
321 个库存节点	值函数存储、插值的数值参数；以 161 和 641 节点作局部敏感性核对。
7 条场景	时间覆盖下的计算预算与有限样本表示，未声明为最优场景数。
28 日窗口	近期分布适应和历史样本量之间的经验折中。
EMA 系数 0.2	短期误差记忆的经验收缩系数。
增益界 $[-10, 10]$	限定仿射策略类的正则化约束；单独采集全年规划解检查触边。
三状态	在少量场景下平衡状态表达与转移稀疏，采用显式伪计数。
末值价格四分位	近似未来低价补能价值；用零末值和两倍末值核对敏感性。

既有 3 月 14 至 20 日七日实验在自由末端口径下，161、321、641 节点账单分别为 242,988.81、243,002.90、243,028.94 元；相对 321 节点变化约负 14.09、正 26.03 元。数值变化较小，支持该局部窗口的网格稳定性，未声称账单单调收敛。零末值与基准费用在数值精度内相同，两倍末值增加 6.20 元，说明该窗口对精确末值倍率不敏感；三日视野在该窗口减少 2,158.03 元。它们是既有局部诊断，不用于重新选择正式评价配置，也不混入固定终点主表。

表 14 全年主策略规划 LP 增益触边诊断（容差 10^{-6} ）

机制	规划次数	增益数	触边数	触边比例
问题二	1336	40008	20010	50.01%
问题三	1336	40008	8009	20.02%
问题四日前	1336	40008	20010	50.01%
问题四日内	1336	40008	8009	20.02%

表 15 1 月 25–31 日增益边界诊断；首末库存均为 6000 kWh

情境	增益列数	结构零列	有效列触边	界限 10 费用	界限 20 费用
问题二	768	388	0/380	347089.63	347089.63
问题三	768	160	0/608	341917.12	341902.65
问题四：日前	768	388	0/380	386128.80	386128.80
问题四：日内	768	160	0/608	380647.04	380647.04

补充诊断在 1 月 25–31 日逐列检查约束矩阵与目标中的精确零列，将其从有效反馈分母剔除。两种界限下有效列触边均为零；问题三费用相差 14.47 元，仅 0.00423%，另三问不变。放宽界限可能影响非唯一最优解及浮点择优，不据此宣称策略类扩张产生了稳定收益，故不修改冻结的界限 10。此表是开发期诊断，不能将其有效列触边比例外推为全年比例。全年主轨迹的规划 LP 在保存的真实状态处独立重解，检查所有具有增益界的变量，并核对生效合同重放一致性。总触边统计包括结构性未激活或退化的增益列，这些列可以在不改变策略输出时触边，因此该比例是求解器变量层面的诊断，不能直接认定有效反馈饱和。它也不能单独证明扩大增益会改善闭环账单，或证明增益限制不影响结果。完整比例、重放误差和容差保留在核验文件中。

9.4 规划时域与预报融合独立诊断

延长前瞻会增加纳入规划的未来供需时段，但也改变成熟历史池、末值取价区间及规划边界，因此应由相同结算和真实终点下的闭环对照评价。保持场景生成方法、反馈结构与预报权重不变，分别比较第二个午夜、固定 48 小时、固定 72 小时三种视域，完整费用见附录。48 小时延长至 72 小时，四种机制费用分别变化为节省 2.57 万元、增支 1.13 万元、节省 6.12 万元及节省 0.81 万元。时域作用随合同权限和电价机制而变；问题二相对两午夜视域的增量日均节省 78.52 元，七日块区间为 $[-101.14, 288.44]$ 元/日，尚不足以支持稳定正增益。该组事后诊断保留完整正负结果，不据评价期账单重新选择主配置。

对光伏官方预报与历史预测，进一步以配对误差二阶矩解释融合权重。两个一月窗口的样本 MSE 最优权重分别为 0.8341 和 0.7617；0.75 在预设离散候选中取得最低开发期 MAE。其依据是预测层的内部开发比较，年度经济评价还需同时计入合同调整与紧急购电。相关推导、误差矩和选择范围见附录；主结果、预报删减实验及结果工作簿统一采用两午夜视域；日内机制使用已发布官方预报，融合权重为 1。

10 模型评价与推广

10.1 模型优点

模型用连续库存连接相邻日期，分别约束合同与实时动作的信息权限；利用费用凸性和一维库存值函数减少计算复杂度。全部费用依据逐时执行轨迹重算，并通过相同初末库存的对照分离跨日效应。在共同允许跨日的条件下，主策略节省 15.29 至 23.13 万元，量化了合同规划与库存价值反馈的联合收益。

10.2 适用条件与推广

在已比较候选中，因果合同规划与 Markov 库存价值反馈的组合取得主表费用优势：单独执行仿射库存响应的账单高于贪心，加入 Markov 反馈后形成主表收益。模型采用历史预测、有限经验场景、状态分箱和固定名义合同下的库存价值近似，结论对应这些信息与计算条件。增益约束及退购解释分别通过独立对照检查；完美信息差额衡量同时放宽信息与成交条件的整体费用空间。如获得新的带发布时间戳预报或多年度样本，可在开发期比较条件场景和增强状态；电池退化、自放电及并网容量仅在参数有依据时作为独立扩展。新增模块须通过同终点闭环对照，而不能仅以预测误差或辅助优化目标的改善代替费用验证。

11 结论

四问以同一能量平衡和信息因果约束组织。问题一采用确定性 LP，典型日费用 35,126.95 元。问题二至四把同一套凸能量模型接到场景合同与库存价值反馈上。334 日固定初末库存下，四种机制费用分别为 1,384.42、1,346.18、1,465.46、1,425.21 万元，较跨日贪心基线分别少花 20.24、23.13、15.29、22.89 万元。11 个评价月份均实现累计节省。跨日连续使库存服务于后续供需，价值反馈使当段放电服从未来边际价值；两者共同构成费用改善的控制机制。

AI 工具使用声明

本参赛队在竞赛过程中使用了生成式人工智能工具，主要用于语言润色、代码辅助和文稿核验；全部报告数值由本地程序根据题目附件计算，并由独立脚本复核。工具名称、使用方式与生成内容范围见支撑材料。

参考文献

- [1] Parisio A, Rikos E, Glielmo L. A model predictive control approach to microgrid operation optimization[J]. IEEE Transactions on Control Systems Technology, 2014, 22(5): 1813–1827. DOI:10.1109/TCST.2013.2295737.
- [2] Parisio A, Rikos E, Glielmo L. Stochastic model predictive control for economic/environmental operation management of microgrids: An experimental case study[J]. Journal of Process Control, 2016, 43: 24–37. DOI:10.1016/j.jprocont.2016.04.008.
- [3] Bhattacharya A, Kharoufeh J P, Zeng B. Managing energy storage in microgrids: A multi-stage stochastic programming approach[J]. IEEE Transactions on Smart Grid, 2018, 9(1): 483–496. DOI:10.1109/TSG.2016.2618621.
- [4] Ben-Tal A, Goryashko A, Guslitzer E, Nemirovski A. Adjustable robust solutions of uncertain linear programs[J]. Mathematical Programming, 2004, 99(2): 351–376. DOI:10.1007/s10107-003-0454-y.
- [5] Goulart P J, Kerrigan E C, Maciejowski J M. Optimization over state feedback policies for robust control with constraints[J]. Automatica, 2006, 42(4): 523–533. DOI:10.1016/j.automatica.2005.08.023.
- [6] Kleywegt A J, Shapiro A, Homem-de-Mello T. The sample average approximation method for stochastic discrete optimization[J]. SIAM Journal on Optimization, 2002, 12(2): 479–502. DOI:10.1137/S1052623499363220.

- [7] Shapiro A, Dentcheva D, Ruszczyński A. Lectures on Stochastic Programming: Modeling and Theory[M]. 2nd ed. Philadelphia: SIAM, 2014.
- [8] Dupačová J, Gröwe-Kuska N, Römisch W. Scenario reduction in stochastic programming: An approach using probability metrics[J]. *Mathematical Programming*, 2003, 95(3): 493–511. DOI:10.1007/s10107-002-0331-0.
- [9] Kaut M, Wallace S W. Evaluation of scenario-generation methods for stochastic programming[J]. *Pacific Journal of Optimization*, 2007, 3(2): 257–271.
- [10] Puterman M L. Markov Decision Processes: Discrete Stochastic Dynamic Programming[M]. New York: Wiley, 1994.
- [11] Powell W B. Approximate Dynamic Programming: Solving the Curses of Dimensionality[M]. 2nd ed. Hoboken: Wiley, 2011.
- [12] Rockafellar R T. Convex Analysis[M]. Princeton: Princeton University Press, 1970.
- [13] Boyd S, Vandenberghe L. Convex Optimization[M]. Cambridge: Cambridge University Press, 2004. <https://web.stanford.edu/~boyd/cvxbook/>.
- [14] Pereira M V F, Pinto L M V G. Multi-stage stochastic optimization applied to energy planning[J]. *Mathematical Programming*, 1991, 52: 359–375.
- [15] HiGHS Development Team. HiGHS: High performance software for linear optimization[EB/OL]. <https://highs.dev/>, 2026-09-11.
- [16] Künsch H R. The jackknife and the bootstrap for general stationary observations[J]. *Annals of Statistics*, 1989, 17(3): 1217–1241. DOI:10.1214/aos/1176347265.
- [17] SDDP.jl Documentation. Words of warning[EB/OL]. <https://sddp.dev/stable/tutorial/warnings/>, 2026-09-11.

A 凸结构与可达性的完整证明

A.1 合同上图与单向消元的等价性

式(6)的两分支在合同相等处连续。引入 F 满足两条上图不等式后，对任意原可行解可令 F 等于两仿射式最大值，得到同费用的 LP 可行解；反向对任一 LP 可行解，将 F 降至其最大值不会破坏约束，非负价格下费用不增。因此两个问题的最优值相同。价格为零时 LP 可能返回松弛 F ，但账单重新计算仍按原合同函数。

命题 2 的母线构造也可逐项验证。保持外网合同与紧急电不变，令

$$w' = w + (1 - \eta^2)u. \quad (36)$$

则

$$r + e + d' - c' - w' = r + e + d - c - w = n. \quad (37)$$

若弃电另有限制，上式新增富余未必能被消纳；若存在循环奖励，去除循环未必维持目标值。因此本文的单向存在性以正文条件为边界。对单向动作，净母线耗电在正增量为增量除效率，在负增量为效率乘增量，得到凸折线 g 。给定 g 后，最小非负紧急电等于净需求加 g 减合同的正部，弃电由平衡恢复，分别得到式(11)。其关于库存增量的凸性无需净需求非负。

A.2 终点区间的充要构造及递归可行性

假设下一库存为 x ，目标为 z ，尚余 N 步。若 N 为正且 x 不高于 z ，取每次库存增加量为

$$\delta_i = (z - x)/N, \quad i = 1, \dots, N. \quad (38)$$

区间左界保证此值不超过最大充电库存增量；所有中间点位于 x 与 z 之间。若 x 高于 z ，取同样等步长的负增量，区间右界保证其绝对值不超过最大放电库存减量。恢复母线动作并加入必要外网补电或弃电，构造了满足所有当前约束的完整后缀。

执行器每步将下一库存置于剩余可达域，所以选定动作后仍存在到达目标的后缀。以剩余步数递减归纳，可得真实终点的递归可行性。初始状态本身必须可达，才有非空交集；本文 6,000 至 6,000 的初末设置显然具有保持库存的可行路径。对中间午夜实施同样目标是每日闭合对照的额外约束，不是实际电池的物理必需条件。

B Bellman 与斜率合并的完整证明

B.1 真性、闭性和分段线性的域条件

设两个函数分别在有限闭区间上有限、连续、凸 PWL，区间外为正无穷。它们均为真函数，即不取负无穷且不恒为正无穷，并具有多面体上图。可行总量域为两区间的 Minkowski 和，对可行总量，分配集合是非空紧集，连续目标在其上达到最小值。设上图变量为 z ，则卷积上图可表示为

$$\{(E, z) : \exists x, y, u, v, E = x + y, u \geq W(x), v \geq h(y), z \geq u + v\}. \quad (39)$$

这是有限多条线性不等式和等式描述的多面体的线性投影，故仍是多面体。它是闭凸函数的上图，因此卷积为闭凸 PWL。若只给一般真凸函数而没有这里的紧域和 PWL 条件，则不应跳过闭性及最小值达到的讨论。

在 Bellman 中， W 是下一标签值函数的有限概率平均，加入容量区间指标后保留上述性质； h 的有限域是单步放能变量区间。每个当前标签组内的函数对所有物理库存均可行，因为外网追索无上限，库存可保持不变。有限平均后的共同域仍是容量区间。线性末值、最大仿射割与有限绝对值罚均为凸 PWL，故向后归纳成立。

若对精确理论 DP 采用硬终端单点指标，须按扩展实值函数限制有效域，并处理逐步扩展的可达区间。本稿实现采用有限罚和独立执行投影，正文没有将二者混称为硬终端精确 DP。

B.2 斜率合并的资源交换证明

设 f 在区间 $[a, A]$ 、 g 在 $[b, B]$ 上为凸 PWL。 f 各线段斜率、长度依次为

$$(s_i, \ell_i)_{i=1}^m, \quad s_1 \leq \dots \leq s_m, \quad \ell_i > 0, \quad (40)$$

g 对应斜率长度为

$$(t_j, d_j)_{j=1}^n, \quad t_1 \leq \dots \leq t_n, \quad d_j > 0. \quad (41)$$

总量为 a 加 b 加 L 时，相当于分别给 f 和 g 分配非负横向增量，增量之和为 L 。函数增值等于其非降边际斜率沿前缀的积分。

将两组斜率稳定有序合并，保留每段原长度；相同斜率任意交织，但保持各自原有次序。取合并序列前 L 长度，得到两函数各自的合法前缀：若某函数的后段被选择而更早段未被选择，则后段斜率不低于前段，有序选择可以把相同或更低斜率前段先纳入，且不增成本。

为避免交换破坏原函数的前缀约束，先把各线段视为可独立取量、上限为该段长度的资源，暂时放松前缀约束。现设这一放松问题的另一分配在尚未使用较低边际成本片段时使用了较高片段。选择不超过两片段可用长度的正小量，从高边际片段移到低边际片段，保持总增量且不增费用；严格高低时费用严格降低。重复交换可得到放松问题的有序合并最优分配。前段已经证明该排序分配同时满足两函数各自的前缀约束，因此放松问题的最优解也在原问题可行，二者最优值相同。因而合并前 L 长度的成本达到最小值。

令合并后第 k 段斜率、长度为 v 和 h ，则卷积折点与函数值从左端初值开始累计：

$$z_0 = a + b, \quad F_0 = f(a) + g(b), \quad z_k = z_{k-1} + h_k, \quad F_k = F_{k-1} + v_k h_k. \quad (42)$$

两组斜率已排序时合并复杂度与线段总数成正比；源码直接排序全部段，成本还包括排序。数学正确性来自有序边际资源分配，而不是网格点个数。

B.3 网格插值与动作折点

对凸函数，任意相邻网格节点的弦不低于真实函数。代码在每次递推后插值回网格，再把网格折线视为下一次卷积的输入，所以它对当前插值函数执行斜率合并，而不是对原始完整随机过程进行精确运算。

对固定当前净需求，即时成本折点来自以下三处（仅保留处于可行区间者）：

$$x = E, \quad x = E - (n - r)/\eta, \quad x = E - \eta(n - r). \quad (43)$$

与继续值折点和区间边界合并后，相邻候选之间目标为仿射，故最小值可在端点取得。这解释了动作候选为何允许连续值。终点可达投影使用新的收缩区间，源码未重新完整枚举该区间目标，因此不据此宣称投影动作具有精确 Bellman 最优性。

C 辅助 SDDP 割的有效性证明

固定两小时聚合模型、有限时域和阶段独立噪声分布。设真实下一阶段值为 V ，其已有割包络为下界。给定输入状态及一个有限噪声实现，替换下一值后得到近似阶段 LP 最优值。由于下界替换降低或保持目标，对所有可行输入状态有

$$\underline{Q}_t(X, \xi) \leq Q_t(X, \xi). \quad (44)$$

在访问状态处，强对偶给出输入状态固定等式的对偶乘子。该乘子是近似阶段值的次梯度，因此

$$\underline{Q}_t(\bar{X}, \xi) + \beta_t(\xi)^\top (X - \bar{X}) \leq \underline{Q}_t(X, \xi) \leq Q_t(X, \xi). \quad (45)$$

按固定有限噪声概率完整求和，形成期望阶段值的全局仿射下界：

$$\sum_{\xi} p_{\xi} \left\{ \underline{Q}_t(\bar{X}, \xi) + \beta_t(\xi)^\top (X - \bar{X}) \right\} \leq \sum_{\xi} p_{\xi} Q_t(X, \xi). \quad (46)$$

辅助末端费用为零且阶段成本非负，零割有效；从终端向前归纳得到全部生成割的有效性。这里支撑的是**近似阶段值**，不能把访问点近似目标误当真实继续值。状态中尚未交付合同必须保留；根节点签约会重置合同，因此根割可以进一步仅依赖库存。

阶段独立性使固定噪声概率无需再依赖被省略的历史状态；若用于历史相关的正式预测过程，必须扩充状态或改变模型，不能沿用本证明直接获得原问题下界。当前 SDDP 割作为末值代理的效果由开发期实际闭环账单判断。

D 可复现实验设计与独立核验

D.1 完整执行伪代码

1. 固定预测族与候选选择规则，声明开发期和评价期。初始化实际库存为 6,000 kWh，保留未交付合同。
2. 在 0、6、12、18 时计算滚动末点；从成熟历史残差构造等权场景，读取此刻已发布的预报，设置未来末值或真实终点等式。
3. 解受限因果仿射场景 LP。0 时落实当日原合同；其他时刻只在机制允许时落实剩余修订合同。
4. 固定本次未来名义合同，将场景分箱，加入伪计数构造转移，向后递推凸库存价值并存入网格。
5. 每十分钟观察当前净需求，使用对应箱条件均价及未来价值求连续库存请求；投影至单步和真正终点可达交集，恢复单向充放电、紧急购电和弃电。

6. 用实际交付电价结算本段，更新实际库存与历史信息；到合法重规划点返回第 2 步，午夜直接继承真实库存。
7. 评价终点核对 6,000 kWh，逐段重算母线和状态残差，汇总费用并输出 CSV、工作簿与登记表。

D.2 一月因果公共初始化

首日无前一完整日数据，保留库存 6000 kWh 并由紧急购电满足缺口；1 月 2 日至 31 日只使用前一完整日的负荷与光伏构造合同，以每日闭合的贪心电池策略执行，不使用 1 月 31 日才选定的预测器。所有候选共用该初始化轨迹，每日末库存 6000 kWh，故 2 月 1 日的库存和可用历史与原正式回放完全相同。预测使用的负荷、光伏和电价不受该初始化动作影响，正式 334 日输出无需改变。

表 16 一月公共初始化账单（万元，不进入正式评价）

电价	原全月应急初始化	因果预测与闭合贪心初始化
固定价	945.64	294.07
浮动价	1064.20	335.38

原初始化保留为历史复现记录，本文采用因果公共初始化作为完整运行年的实施说明。该比较不是正式期策略收益；首日仍因缺少历史而采用应急方案，后续日不虚构过去样本。

D.3 固定真实终点数据链

冻结结果中的固定终点文件含完整 334 日。对于当前两午夜线性末值实现，12 月 30 日前滚动域尚未到达真实终点，且剩余可达约束未收紧物理可行域，所以原前 332 日可复用。重放最后两日时先以自由末端核对保存轨迹，再以 6,000 kWh 硬末端重算并拼接。预报删减实验采用相同方法，新脚本逐变量验证自由后缀匹配后才生成固定后缀。

这一前缀复用结论针对当前实现及相同数据、预测器和重规划时域，不推广到任意更长视域或不同继续值依赖的算法。主表、统计登记、工作簿和 CSV 全部读取拼接后的完整固定终点轨迹；自由终点与局部敏感性保留在独立位置。

D.4 移动块实现与解释

对块长 b 形成全部重叠、非环绕的长度 b 连续块，起点为 1 至 334 减 b 加 1。每次独立等概率抽取足够多的块，按抽取顺序拼接，再保留前 334 个日差。计算其均值，重复 10,000 次后取百分位区间。累计区间等于对应均值区间乘 334，不把每次重复当作新的独立年份。

原每日闭合基线的统计对每一机制和块长使用独立声明的 SeedSequence。主文跨日贪心对照对各机制、各块长分别重置 default_rng(20260911)，同一块长的四机制共享抽样索引；3、7、14 日块各重复 10,000 次。原每日闭合对照作为补充统计单独登记。源

脚本保存每日正负数、逐月差、均值与累计区间及一阶相关，避免图、正文和统计口径分离。移动块近似保留块内依赖；年度季节变化和分布漂移不因重采样消失，故本稿不把区间作为跨年泛化保证。

表 17 主策略相对跨日贪心的逐月累计节省（万元）

月份	问题二	问题三	问题四日前	问题四日内
2025-02	1.82	2.18	1.76	2.32
2025-03	1.98	1.44	1.13	1.46
2025-04	2.23	1.43	1.24	1.45
2025-05	0.72	1.79	0.21	1.34
2025-06	2.47	1.60	0.79	1.58
2025-07	2.38	2.41	2.60	2.35
2025-08	2.25	3.72	2.28	3.75
2025-09	1.52	2.61	0.98	2.40
2025-10	2.08	1.93	1.61	1.69
2025-11	1.85	2.19	1.89	2.45
2025-12	0.94	1.83	0.79	2.11

表 18 固定终点开发期复核：2025 年 1 月 25 日至 31 日（元）

候选	问题二	问题三	问题四日前	问题四日内
cross_baseline	349411.02	348277.53	387150.67	387623.51
affine_mpc	361685.60	352670.78	402363.20	392636.03
markov_mpc	347089.63	341917.12	386128.80	380647.04
sddp_mpc	361692.33	352665.36	402313.49	392636.03
sddp_markov	347089.63	341942.34	386128.76	380647.04

表 19 既有七日敏感性实验：3 月 14 日至 20 日，自由末端

配置	费用/元	末库存/kWh	相对参考/元
reference	243002.90	1200.00	0.00
grid161	242988.81	1200.00	-14.09
grid641	243028.94	1200.00	26.03
scenarios14	242792.43	1200.00	-210.47
horizon3	240844.87	1200.00	-2158.03
tail_zero	243002.90	1200.00	-0.00
tail_double	243009.10	1200.00	6.20
stress_load_up_pv_down	528779.59	1200.00	285776.69

D.5 独立 LP 与控制器核验

既有独立核验针对费用上图、未来真实数据与预报变异、可达执行、库存连续、凸递推和 SDDP 辅助模型。正式轨迹独立重算的原合同费、增购费、退购净费、紧急费，

并独立检查供需平衡、库存递推、首末条件和充放电互斥。数学审稿另将源码斜率卷积与全折点枚举对照，检查其在声明域上的数值一致性。

全时域完美信息 LP 还用对偶乘子构造数值证书：母线行乘子截断至零与实际电价之间，重新计算约化成本，并以变量上下界修正对偶目标。该过程核验浮点下界的有效数值精度，不是有理数精确证明。SDDP 小规模确定性等价模型与后向割的对照只支持辅助模型内的性质。

D.6 固定时域的闭环对照与边界状态

下表每项均来自本地完整 334 日轨迹，统一真实初末 6000 kWh，保持预报权重 1、28 日窗口、7 条等权场景、321 库存网格与三箱 Markov 反馈。第二个午夜视域随重规划时刻取 48、42、36、30 小时，固定视域则分别取 48 或 72 小时，评价终点统一截断。

表 20 同配置下的时域诊断（万元）

机制	第二个午夜	固定 48 小时	固定 72 小时	48 至 72 小时节省
问题二	1384.42	1384.36	1381.79	+2.57
问题三	1346.18	1346.13	1347.26	-1.13
问题四日前	1465.46	1465.43	1459.31	+6.12
问题四日内	1425.21	1425.21	1424.40	+0.81

改变时域还会改变完整成熟残差的可用集合和末值定价时段，因此该表识别配置变化的闭环总效应。固定 72 小时在 6、12、18 时起算时，规划末点落在日内；完整继续状态应包含未交付合同，而当前实现用库存线性末值近似。实际年度账单覆盖全部交付，这一近似影响规划继续价值。上述时域对照同时改变历史池和末值时段，不能独立量化边界近似误差；其识别需要在相同 72 小时条件下增加完整合同继续状态的匹配回放。主策略的滚动末点位于午夜。

问题二固定 72 小时相对两午夜视域，334 日节省 26,225.00 元，日均 78.52 元。3、7、14 日块 95% 区间分别为 $[-122.81, 295.64]$ 、 $[-101.14, 288.44]$ 、 $[-74.14, 285.42]$ 元/日；节省日 156、增支日 177、近零日 1，正累计月份 5 个。区间跨零表示该年度增量证据尚不足以确认稳定正收益，不表示真实效果已被证明为零。该诊断对应固定 72 小时候选，不替代正文的强跨日贪心对照。

D.7 预报融合的二阶矩推导与开发诊断

记官方和历史预测误差为 e_O, e_H ，融合误差为 $e_\beta = \beta e_O + (1 - \beta)e_H$ 。假设二阶矩有限，令

$$A = \mathbb{E}[e_O^2], \quad B = \mathbb{E}[e_H^2], \quad C = \mathbb{E}[e_O e_H], \quad D = A + B - 2C. \quad (47)$$

则

$$\mathbb{E}[e_\beta^2] = B + 2(C - B)\beta + D\beta^2, \quad D = \mathbb{E}[(e_O - e_H)^2] \geq 0. \quad (48)$$

当 $D > 0$ 时，凸二次式在 $[0, 1]$ 的最小点为

$$\beta^* = \text{proj}_{[0,1]} \frac{B - C}{D}. \quad (49)$$

证明由求导及闭区间投影直接得到。当 $D = 0$ 时，两误差几乎处处相等，所有权重具有相同 MSE。二阶矩包含偏差平方，不能在有偏预测下直接以方差替代。将期望替换为同一组配对段平均，可得到对应样本 MSE 的最优权重。

表 21 配对光伏误差矩与样本最优权重（二阶矩单位 kW^2 ）

一月窗口	段数	A	B	C	D	$\hat{\beta}^*$
15 至 24 日	1440	10849.55	33705.63	5172.59	34210.01	0.8341
25 至 31 日	1008	23689.77	47340.10	12923.33	45183.21	0.7617

每个 0、6、12、18 时原点预测后续 36 段，每个目标在所属窗口计入一次。15 至 24 日的离散候选权重为 0、0.25、0.5、0.75、1，MAE 分别为 96.68、75.89、59.42、50.50、52.04 kW ；因此 0.75 的选取具有该候选集的开发期 MAE 依据。25 至 31 日中 0.75 的 MSE 为 21,130.50 kW^2 ，样本最优权重 0.7617 对应 21,124.30 kW^2 ，两者接近。

历史预测族已用 1 月 15 至 31 日选型，因此两个窗口均属于内部开发诊断；按原点截断未来输入不能消除预测族选型的日期重叠。这里不使用 2 至 12 月样本估计融合矩，也不将样本 MSE 最优权重解释为无偏留出估计或账单最优权重。全年账单受净需求尾部、合同调整费与库存约束共同影响，预测误差降低的经济作用需由固定时域下的完整配对执行验证。

E 指定日期完整结果

各表对应固定真实初末库存 6000 kWh 的同一 334 日主轨迹。表 1 格式同时报告最终合同量和合同费用；另列实际购电量（最终合同加紧急量）与含紧急费的总账单。工作簿“调整购电量”工作表中的时段数值表示最终有效合同量 r_t ，调整增量为 $r_t - q_t$ ，各次发布另见调整发布记录 CSV。表 2 的每个四小时块是分时动作累计，不表示同时充放电。指定日期的紧急购电区间集中列于本节末表，数值零阈值为 10^{-6} kWh。

问题二：2025-03-20

表 22 问题二 2025-03-20 购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	451.12	14:00–14:10	0.00
16:00–16:10	456.20	18:00–18:10	667.10	20:00–20:10	0.00
全天合同购电量	62,914.06	全天合同购电费	39,375.08		
全天实际购电量	63,230.08	全天总购电费	40,779.74		

表 23 问题二 2025-03-20 储能结果（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	1,521.77	144.03	04:00–08:00	1,300.17	4,796.12
08:00–12:00	5,483.51	2,777.38	12:00–16:00	4,555.91	856.38
16:00–20:00	201.18	5,402.67	20:00–24:00	3,404.99	2,978.93
0:00 库存		8,459.80	24:00 库存		4,441.13

问题二：2025-06-21

表 24 问题二 2025-06-21 购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	315.89	18:00–18:10	427.56	20:00–20:10	0.00
全天合同购电量		50,429.12	全天合同购电费		28,888.20
全天实际购电量		50,429.12	全天总购电费		28,888.20

表 25 问题二 2025-06-21 储能结果（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	3,712.39	0.00	04:00–08:00	1,287.73	2,182.26
08:00–12:00	4,100.00	0.00	12:00–16:00	2,828.81	833.33
16:00–20:00	285.45	5,139.64	20:00–24:00	8,533.33	2,543.57
0:00 库存		3,414.63	24:00 库存		10,200.00

问题二：2025-09-23

表 26 问题二 2025-09-23 购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	443.17	14:00–14:10	0.00
16:00–16:10	705.16	18:00–18:10	681.37	20:00–20:10	50.57
全天合同购电量		71,072.71	全天合同购电费		44,591.63
全天实际购电量		71,072.71	全天总购电费		44,591.63

表 27 问题二 2025-09-23 储能结果（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	2,448.27	60.61	04:00–08:00	1,243.37	6,274.98
08:00–12:00	5,933.38	1,896.62	12:00–16:00	3,984.98	446.84
16:00–20:00	238.51	5,690.01	20:00–24:00	8,117.22	2,744.95
0:00 库存		8,014.38	24:00 库存		8,767.98

问题二：2025-12-21

表 28 问题二 2025-12-21 购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	968.47	14:00–14:10	0.00
16:00–16:10	903.47	18:00–18:10	722.74	20:00–20:10	0.00
全天合同购电量		100,928.13	全天合同购电费		65,624.75
全天实际购电量		100,928.13	全天总购电费		65,624.75

表 29 问题二 2025-12-21 储能结果（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	2,355.58	101.36	04:00–08:00	1,683.40	1,702.64
08:00–12:00	5,066.40	6,275.70	12:00–16:00	7,344.72	3,006.22
16:00–20:00	34.37	5,635.15	20:00–24:00	7,950.94	2,879.45
0:00 库存		8,282.60	24:00 库存		8,496.11

问题三：2025-03-20

表 30 问题三 2025-03-20 购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00 (0.00)	12:00–12:10	553.76 (577.45)	14:00–14:10	0.00 (17.26)
16:00–16:10	538.48 (667.17)	18:00–18:10	661.19 (666.75)	20:00–20:10	0.81 (0.81)
全天合同购电量		62,297.93	全天合同购电费		40,402.97
全天实际购电量		62,427.84	全天总购电费		41,031.60

购电量列为最终合同量，括号内为 0 时原合同。

表 31 问题三 2025-03-20 储能结果（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	852.91	156.43	04:00–08:00	1,205.92	5,960.90
08:00–12:00	3,901.25	2,647.46	12:00–16:00	7,002.86	237.46
16:00–20:00	96.58	5,464.91	20:00–24:00	3,404.26	2,975.01
0:00 库存		9,085.70	24:00 库存		4,522.91

问题三：2025-06-21

表 32 问题三 2025-06-21 购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00 (0.00)	12:00–12:10	0.00 (0.00)	14:00–14:10	0.00 (0.00)
16:00–16:10	268.09 (268.09)	18:00–18:10	52.12 (52.12)	20:00–20:10	0.00 (52.12)
全天合同购电量		43,976.63	全天合同购电费		25,206.74
全天实际购电量		43,976.63	全天总购电费		25,206.74

购电量列为最终合同量，括号内为 0 时原合同。

表 33 问题三 2025-06-21 储能结果（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	1,178.37	361.54	04:00–08:00	0.00	3,013.46
08:00–12:00	4,333.33	0.00	12:00–16:00	6,333.33	0.00
16:00–20:00	213.24	5,054.59	20:00–24:00	7,759.18	2,618.06
0:00 库存		3,889.47	24:00 库存		9,450.00

问题三：2025-09-23

表 34 问题三 2025-09-23 购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00 (0.00)	12:00–12:10	8.58 (8.58)	14:00–14:10	0.00 (8.58)
16:00–16:10	532.56 (625.55)	18:00–18:10	664.56 (664.56)	20:00–20:10	37.10 (37.10)
全天合同购电量		67,194.60	全天合同购电费		43,196.46
全天实际购电量		67,194.60	全天总购电费		43,196.46

购电量列为最终合同量，括号内为 0 时原合同。

表 35 问题三 2025-09-23 储能结果（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	1,743.36	61.25	04:00–08:00	1,106.98	6,719.05
08:00–12:00	5,229.50	1,896.62	12:00–16:00	5,476.55	482.53
16:00–20:00	388.01	5,869.92	20:00–24:00	8,181.26	2,992.37
0:00 库存		8,688.73	24:00 库存		8,577.68

问题三：2025-12-21

表 36 问题三 2025-12-21 购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	5.88 (5.88)	12:00–12:10	892.63 (983.88)	14:00–14:10	0.00 (20.56)
16:00–16:10	851.88 (886.78)	18:00–18:10	632.06 (719.15)	20:00–20:10	0.00 (3.75)
全天合同购电量		97,424.90	全天合同购电费		63,557.57
全天实际购电量		97,441.01	全天总购电费		63,656.05

购电量列为最终合同量，括号内为 0 时原合同。

表 37 问题三 2025-12-21 储能结果（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	1,895.92	55.71	04:00–08:00	1,806.65	1,831.00
08:00–12:00	5,435.79	6,919.14	12:00–16:00	7,357.01	2,616.41
16:00–20:00	0.00	5,785.05	20:00–24:00	7,649.87	2,862.99
0:00 库存		8,645.57	24:00 库存		8,075.95

问题四日前：2025-03-20

表 38 问题四日前 2025-03-20 购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	430.13	14:00–14:10	0.00
16:00–16:10	458.09	18:00–18:10	459.15	20:00–20:10	748.57
全天合同购电量		63,797.91	全天合同购电费		41,395.65
全天实际购电量		64,020.67	全天总购电费		42,399.63

表 39 问题四日前 2025-03-20 储能结果（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	5,401.96	1,273.09	04:00–08:00	2,724.69	4,497.02
08:00–12:00	4,848.08	2,777.38	12:00–16:00	4,318.44	1,049.45
16:00–20:00	154.19	6,907.93	20:00–24:00	452.28	1,522.47
0:00 库存		5,633.35	24:00 库存		1,712.66

问题四日前：2025-06-21

表 40 问题四日前 2025-06-21 购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	0.00	14:00–14:10	0.00
16:00–16:10	323.08	18:00–18:10	0.00	20:00–20:10	0.00
全天合同购电量		51,618.83	全天合同购电费		25,513.51
全天实际购电量		51,618.83	全天总购电费		25,513.51

表 41 问题四日前 2025-06-21 储能结果（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	4,144.24	2,568.06	04:00–08:00	3,921.77	2,447.64
08:00–12:00	4,266.67	0.00	12:00–16:00	3,700.00	1,134.00
16:00–20:00	293.99	4,492.65	20:00–24:00	6,754.83	2,512.90
0:00 库存		3,203.58	24:00 库存		9,360.00

问题四日前：2025-09-23

表 42 问题四日前 2025-09-23 购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	452.16	14:00–14:10	0.00
16:00–16:10	707.33	18:00–18:10	0.00	20:00–20:10	766.26
全天合同购电量		69,912.16	全天合同购电费		46,115.60
全天实际购电量		69,912.16	全天总购电费		46,115.60

表 43 问题四日前 2025-09-23 储能结果（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	9,795.98	1,130.50	04:00–08:00	1,112.11	6,745.85
08:00–12:00	6,084.33	1,896.62	12:00–16:00	3,854.74	470.98
16:00–20:00	335.85	7,140.26	20:00–24:00	1,438.08	1,298.03
0:00 库存		3,239.73	24:00 库存		2,840.66

问题四日前：2025-12-21

表 44 问题四日前 2025-12-21 购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00	12:00–12:10	967.91	14:00–14:10	0.00
16:00–16:10	906.23	18:00–18:10	722.74	20:00–20:10	0.00
全天合同购电量		97,957.08	全天合同购电费		75,920.52
全天实际购电量		97,957.08	全天总购电费		75,920.52

表 45 问题四日前 2025-12-21 储能结果（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	1,691.71	807.24	04:00–08:00	1,804.88	2,255.30
08:00–12:00	5,345.18	6,502.30	12:00–16:00	7,266.67	2,943.00
16:00–20:00	71.05	5,652.34	20:00–24:00	6,695.14	2,881.46
0:00 库存		10,170.00	24:00 库存		7,377.58

问题四日内：2025-03-20

表 46 问题四日内 2025-03-20 购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00 (0.00)	12:00–12:10	555.31 (578.13)	14:00–14:10	0.00 (20.73)
16:00–16:10	679.20 (684.48)	18:00–18:10	420.58 (420.58)	20:00–20:10	747.30 (747.30)
全天合同购电量		63,514.17	全天合同购电费		42,369.44
全天实际购电量		63,514.17	全天总购电费		42,369.44

购电量列为最终合同量，括号内为 0 时原合同。

表 47 问题四日内 2025-03-20 储能结果（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	5,350.91	1,253.82	04:00–08:00	2,107.07	5,958.88
08:00–12:00	3,508.90	2,777.38	12:00–16:00	7,080.30	134.63
16:00–20:00	184.46	6,872.40	20:00–24:00	451.40	1,531.41
0:00 库存		5,641.21	24:00 库存		1,868.69

问题四日内：2025-06-21

表 48 问题四日内 2025-06-21 购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00 (0.00)	12:00–12:10	0.00 (0.00)	14:00–14:10	0.00 (0.00)
16:00–16:10	239.15 (239.15)	18:00–18:10	50.73 (50.73)	20:00–20:10	0.00 (50.73)
全天合同购电量		42,942.68	全天合同购电费		21,833.05
全天实际购电量		42,942.68	全天总购电费		21,833.05

购电量列为最终合同量，括号内为 0 时原合同。

表 49 问题四日内 2025-06-21 储能结果（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	4,030.87	3,373.01	04:00–08:00	1,400.00	3,402.00
08:00–12:00	4,300.00	0.00	12:00–16:00	6,366.67	0.00
16:00–20:00	150.16	5,054.80	20:00–24:00	5,656.03	2,429.21
0:00 库存		3,840.00	24:00 库存		7,710.00

问题四日内：2025-09-23

表 50 问题四日内 2025-09-23 购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	0.00 (0.00)	12:00–12:10	7.65 (7.65)	14:00–14:10	0.00 (7.65)
16:00–16:10	532.56 (626.32)	18:00–18:10	7.54 (9.51)	20:00–20:10	788.19 (788.19)
全天合同购电量		66,850.30	全天合同购电费		45,213.48
全天实际购电量		66,850.30	全天总购电费		45,213.48

购电量列为最终合同量，括号内为 0 时原合同。

表 51 问题四日内 2025-09-23 储能结果（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	9,653.73	1,130.50	04:00–08:00	1,063.78	7,221.07
08:00–12:00	5,240.05	1,896.62	12:00–16:00	5,471.82	492.83
16:00–20:00	430.33	7,239.32	20:00–24:00	1,531.53	1,648.87
0:00 库存		3,367.76	24:00 库存		2,609.64

问题四日内：2025-12-21

表 52 问题四日内 2025-12-21 购电结果（电量 kWh，费用元）

时间段	购电量	时间段	购电量	时间段	购电量
10:00–10:10	2.57 (2.57)	12:00–12:10	892.79 (984.09)	14:00–14:10	0.00 (21.19)
16:00–16:10	851.88 (886.78)	18:00–18:10	631.79 (718.33)	20:00–20:10	0.00 (3.68)
全天合同购电量		95,134.35	全天合同购电费		73,732.59
全天实际购电量		95,150.06	全天总购电费		73,838.36

购电量列为最终合同量，括号内为 0 时原合同。

表 53 问题四日内 2025-12-21 储能结果（kWh）

时间段	充电量	放电量	时间段	充电量	放电量
00:00–04:00	1,678.06	796.18	04:00–08:00	1,911.52	2,362.58
08:00–12:00	5,616.58	7,079.23	12:00–16:00	7,317.34	2,579.05
16:00–20:00	0.00	5,789.31	20:00–24:00	6,642.95	2,858.83
0:00 库存		10,170.00	24:00 库存		7,169.61

表 54 指定日期紧急购电 (kWh; 连续非零区间, 数值零阈值为 10^{-6} kWh)

问题二							
2025-03-20		2025-06-21		2025-09-23		2025-12-21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
16:00–16:50	290.37	无	0.00	无	0.00	无	0.00
17:20–17:40	7.58						
18:10–18:20	18.08						
问题三							
2025-03-20		2025-06-21		2025-09-23		2025-12-21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
09:20–10:00	129.92	无	0.00	无	0.00	20:40–20:50	16.11
问题四日前							
2025-03-20		2025-06-21		2025-09-23		2025-12-21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
16:00–16:40	215.80	无	0.00	无	0.00	无	0.00
17:30–17:40	6.97						
问题四日内							
2025-03-20		2025-06-21		2025-09-23		2025-12-21	
时间段	购电量	时间段	购电量	时间段	购电量	时间段	购电量
无	0.00	无	0.00	无	0.00	20:40–20:50	15.71

重复合同数值核验 问题三 6 月 21 日 18 时与 20 时的原合同, 以及 9 月 23 日 12 时与 14 时原合同, 问题四日内对应两对值, 均由原数组直接提取。对相关规划 LP 重新求解, 与保存合同一致; 四对中三对全精度相等, 另一对相差 3.55×10^{-15} kWh。当前待签合同的场景反馈增益被禁用, 因此不能把这些重复值归因为当日共享反馈增益。它们是所求 LP 的数值解特征, 不是排版复制; 表内仍保留真实结果, 不人为扰动数值。

F 支撑文件与完整源程序

源程序全文列于本附录；同名可运行文件同时保存在支撑材料中。原始附件放入 inputs 目录，安装依赖后按运行说明复算。新增审稿对照采用独立脚本和结果路径，不覆盖原已验证轨迹。调度主链四个文件按可读字号排印，其余脚本保持紧凑排版。

文件/目录	作用
src/	原始数据处理、预测、LP、Markov 反馈、SDDP、回放、导出完整源码。
nextgen-source/	固定时域、融合与有限状态控制诊断所用版本的完整源码副本。
review/	原模型独立物理、计费和控制核验。
reference/scripts/	底稿统计、全年增益复演与同终点消融生产程序。
revision/	新增 Q1、计费、预测、增益、强基线和指定日期表复算脚本与证据。
artifacts/	原已执行结果；主固定终点数据读取同级原题解的 global-terminal 目录。
计算结果/	固定真实终点的五份工作簿与 CSV。
LaTeX 正文、附录、表格与图像	正文、证明、完整题定表与图。
source-code.tex	本附录全部源程序的逐文件引用。
AI 工具使用说明	使用 Codex 辅助语言润色、代码编写、数学一致性核查及排版；数值由模型程序计算并由独立脚本复核。
README.md	环境、入口、版本与复算说明。

下列四个文件构成预测、合同规划、电池动作与全年回放主链，按正文相近字号排印。

src/forecasting.py

```
1 """Absolute-time, as-of forecasts and fully matured multiday residual blocks."""
2 import numpy as np
3
4 FAMILIES=('lag1','lag7','mean7','trend7')
5
6 def base_forecast(data,key,asof,targets,family):
7     day=asof//144; td=targets//144; tt=targets%144; x=data[key]
8     if day<1: raise ValueError('No preceding complete day')
9     if family=='mean7': return x[max(0,day-7):day].mean(0)[tt]
10    lag=1 if family=='lag1' else 7
11    hd=np.minimum(td-lag,day-1);hd=np.maximum(0,hd)
12    if family=='trend7':
13        older=np.maximum(0,np.minimum(td-14,day-1))
14        return np.maximum(0,2*x[hd,tt]-x[older,tt])
15    return x[hd,tt].copy()
```

```

16
17 def select_families(data):
18     out={}
19     for key in ['load','pv','price']:
20         scores={f:float(np.mean([np.abs(base_forecast(data,key,d*144,np.ar_ ↵
           ange(d*144,(d+1)*144),f)-data[key][d]).mean() for d in ↵
           range(14,31)])) for f in FAMILIES}
21         out[key]={'selected':min(scores,key=scores.get),'january_mae':scores}
22     return out
23
24 def forecast_as_of(data,selection,asof,targets,official,variable):
25     """Only completed days, already observed intraday values, latest ↵
       release."""
26     pred={k:base_forecast(data,k,asof,targets,selection[k]['selected']) for ↵
       k in ['load','pv','price']}
27     day,t=divmod(asof,144)
28     # A conservative observed-error correction, with a six-hour fading memory.
29     # Coefficient 0.8 is a declared shrinkage, unchanged across annual runs.
30     if t:
31         past=np.arange(max(day*144,asof-36),asof)
32         fade=np.exp(-(targets-asof)/36.)
33         for k in ['load','pv','price']:
34             err=data[k].ravel()[past]-base_forecast(data,k,day*144,past,se_ ↵
               lection[k]['selected'])
35             pred[k]=np.maximum(0,pred[k]+.8*err.mean()*fade)
36     official_covered=0
37     if official:
38         issue=(asof//36)*36; d,k=divmod(issue//36,4)
39         anchor=data['pv'].ravel()[issue-1] if issue else 0.
40         endpoints=np.arange(25)*6+issue
41         mask=(targets+1<=issue+144)&(targets+1>=issue)
42         pred['pv'][mask]=np.interp(targets[mask]+1,endpoints,np.r_[anchor, ↵
               data['forecast'][d,k]])
43         official_covered=int(mask.sum())
44     if not variable: pred['price']=data['day_price'][targets%144]
45     pred['price']=np.maximum(.001,pred['price'])
46     pred['net']=(pred['load']-pred['pv'])/6
47     pred['official_covered_slots']=official_covered
48     return pred
49
50 def scenarios(data,selection,asof,end,official,variable>window=28,count=7):
51     targets=np.arange(asof,end);T=len(targets); day,t=divmod(asof,144)
52     # A block must be mature at the current decision time, including its ↵
       last target.
53     eligible=[h for h in range(max(7,day-window-3),day) if h*144+t+T<=asof]
54     eligible=eligible[-window:]
55     if not eligible: raise ValueError('No fully mature historical blocks')
56     if count and len(eligible)>count:

```

```

57     eligible=[eligible[j] for j in
               np.linspace(0,len(eligible)-1,count).round().astype(int)]
58     current=forecast_as_of(data,selection,asof,targets,official,variable)
59     ns=[];ps=[]
60     for h in eligible:
61         issue=h*144+t; ht=np.arange(issue,issue+T)
62         old=forecast_as_of(data,selection,issue,ht,official,variable)
63         actual=(data['load'].ravel()[ht]-data['pv'].ravel()[ht])/6
64         ns.append(current['net']+actual-old['net'])
65         ps.append(np.maximum(.001,current['price']+data['price'].ravel()[h
               t]-old['price']) if variable else current['price'])
66     return np.array(ns),np.array(ps),{'as_of':asof,'end':end,'history_days'
               ':eligible',
67         'latest_training_target_exclusive':max(eligible)*144+t+T,
68         'official_covered_slots':current['official_covered_slots']}

```

src/control.py

```

1  """Causal affine-policy SAA MPC and one-dimensional convex Markov DP."""
2  from pathlib import Path
3  from types import SimpleNamespace
4  import sys
5  import numpy as np
6  from scipy.sparse import coo_matrix,csr_matrix,vstack,eye
7  sys.path.insert(0,str(Path(__file__).resolve().parents[1]/'vendor'))
8  import highspy
9
10 ETA=.9; M=5000/6; EMIN=1200.; EMAX=10800.; INITIAL=6000.
11
12 def lp_solve(obj,A,b,bounds,solver='ipm'):
13     A=A.tocsr();p=highspy.HighsLp();p.num_col_=len(obj);p.num_row_=len(b)
14     p.col_cost_=np.asarray(obj);p.col_lower_=np.array([a if a is not None
               else -np.inf for a,z in bounds]);p.col_upper_=np.array([z if z is not
               None else np.inf for a,z in bounds])
15     p.row_lower_=np.full(len(b),-np.inf);p.row_upper_=np.asarray(b)
16     p.a_matrix_.format_=highspy.MatrixFormat.kRowwise;p.a_matrix_.start_=A
               .indptr;p.a_matrix_.index_=A.indices;p.a_matrix_.value_=A.data
17     h=highspy.Highs();h.setOptionValue('output_flag',False);h.setOptionVal
               ue('threads',1);h.setOptionValue('solver',solver);h.passModel(p);h.run
               ()
18     if h.getModelStatus()!=highspy.HighsModelStatus.kOptimal:raise
               RuntimeError(h.modelStatusToString(h.getModelStatus()))
19     x=np.array(h.getSolution().col_value)
20     return SimpleNamespace(x=x,fun=h.getObjectiveValue(),violation=float(m
               ax(0,np.max(A@x-b))))
21
22 def features(net):
23     err=net-net.mean(0);ema=np.zeros_like(err);avg=np.zeros_like(err);z=np
               .zeros(len(net))

```

```

24     for t in range(net.shape[1]):
25         z=.8*z+.2*err[:,t];ema[:,t]=z;avg[:,t]=err[:,max(0,t-35):t+1].mean(1)
26     return err,ema,avg
27
28 def affine_plan(net,prices,asof,E,base=None,adjust=False,affine=True,tail_ ↵
    price=0.,terminal=None,tail_cuts=None,daily_closed=False,solver='ipm'):
29     """Node-time affine recourse. Future contracts use ONLY pre-release errors.
30
31     Scenario states are feasible in-sample; unseen paths are projected ↵
    physically.
32     Future official forecast innovations are omitted from this restricted ↵
    policy
33     class, while actual current releases are used by the scenario producer.
34     """
35     S,T=net.shape;K=S*T;clock=np.arange(asof,asof+T); day0=asof//144
36     blocks=clock//36-clock[0]//36;B=int(blocks.max()+1);days=clock//144-da ↵
    y0;D=int(days.max()+1)
37     err,ema,avg=features(net)
38     # a_q, a_r, a_E, g_q, g_r, g_E, emergency, delivered contract charge.
39     aq=0; ar=T; ae=2*T;gq=3*T;gr=gq+2*D;ge=gr+2*B;em=ge+2*B;bill=em+K;thet ↵
    a=bill+K;N=theta+(S if tail_cuts else 0)
40     def expr(which):
41         rows=[];cols=[];vals=[];constant=np.zeros(K)
42         for s in range(S):
43             for j,ts in enumerate(clock):
44                 row=s*T+j;di=days[j];bi=blocks[j]
45                 if which=='Q' and di==0 and base is not None:
46                     constant[row]=base[j];continue
47                 if which=='Q': a=aq+j;g=gq+2*di;reveal=(ts//144)*144-asof
48                 elif which=='R': a=ar+j;g=gr+2*bi;reveal=(ts//36)*36-asof
49                 else:a=ae+j;g=ge+2*bi;reveal=j+1
50                 rows.append(row);cols.append(a);vals.append(1.)
51                 if affine and reveal>0:
52                     values=(err[s,j],ema[s,j]) if which=='E' else ↵
                        (avg[s,reveal-1],ema[s,reveal-1])
53                     for k,val in enumerate(values):rows.append(row);cols.a ↵
                        ppend(g+k);vals.append(val)
54         return coo_matrix((vals,(rows,cols)),shape=(K,N)).tocsr(),constant
55     Q,q0=expr('Q'); R,r0=expr('R') if adjust else (Q,q0);X,x0=expr('E')
56     if adjust:
57         firstblock=np.tile(clock%144<36,S).astype(float)
58         from scipy.sparse import diags
59         keep=diags(firstblock);R=keep@Q+(eye(K)-keep)@R;r0=firstblock*q0+( ↵
            1-firstblock)*r0
60     shift=coo_matrix((np.ones(S*(T-1)),(np.concatenate([np.arange(s*T+1,(s ↵
        +1)*T) for s in range(S)]),np.concatenate([np.arange(s*T,(s+1)*T-1) for ↵
        s in range(S)]))),shape=(K,K)).tocsr()
61     Delta=X-shift@X; delta0=np.zeros(K);delta0[np.arange(S)*T]=-E

```

```

62 U=coo_matrix((np.ones(K),(np.arange(K),em+np.arange(K))),shape=(K,N)).
    tocsr()
63 F=coo_matrix((np.ones(K),(np.arange(K),bill+np.arange(K))),shape=(K,N)
    ).tocsr()
64 mats=[-Q,-R,X,-X,Delta,-Delta,ETA*Delta-R-U,Delta/ETA-R-U,1.5*R-.5*Q-F
    ,.5*R+.5*Q-F]
65 rhs=[q0,r0,np.full(K,EMAX),np.full(K,-EMIN),np.full(K,ETA*M)-delta0,np
    .full(K,M/ETA)+delta0,
66         r0-net.ravel()-ETA*delta0,r0-net.ravel()-delta0/ETA,.5*q0-1.5*r0,
    -.5*q0-.5*r0]
67 last=np.arange(S)*T+T-1
68 if terminal is not None:
69     mats +=[X[last],-X[last]];rhs
    +=[np.full(S,terminal),np.full(S,-terminal)]
70 if daily_closed:
71     mids=np.flatnonzero(np.tile((clock+1)%144==0,S))
72     mats +=[X[mids],-X[mids]];rhs
    +=[np.full(len(mids),6000.),np.full(len(mids),-6000.)]
73 if tail_cuts:
74     Theta=coo_matrix((np.ones(S),(np.arange(S),theta+np.arange(S))),sh
    ape=(S,N)).tocsr()
75     for alpha,beta in tail_cuts:mats.append(beta*X[last]-Theta);rhs.ap
    pend(np.full(S,-alpha))
76 obj=np.zeros(N);obj[em:bill]=(5*prices/S).ravel();obj[bill:theta]=(pri
    ces/S).ravel()
77 if tail_cuts:obj[ttheta:]=1/S
78 obj-=tail_price*np.asarray(X[last].mean(0)).ravel()
79 bounds=[(None,None)]*(3*T)+[(-10,10) if affine else
    (0,0)]*(2*D+4*B)+[(0,None)]*(2*K+(S if tail_cuts else 0))
80 sol=lp_solve(obj,vstack(mats),np.concatenate(rhs),bounds,solver);x=sol.x
81 qs=(Q*x+q0).reshape(S,T);rs=(R*x+r0).reshape(S,T);xs=(X*x).reshape(S,T)
82 prev=np.c_[np.full(S,E),xs[:,-1]];delta=x-prev
83 emer=np.maximum(net-rs+np.maximum(ETA*delta,delta/ETA),0)
84 true=float(np.mean(np.sum(prices*(np.maximum(1.5*rs-.5*qs,.5*rs+.5*qs)
    +5*emer),axis=1))-tail_price*xs[:,-1].mean())
85 if tail_cuts:true+=float(np.maximum(0,np.max([a+b*xs[:,-1] for a,b in
    tail_cuts],axis=0)).mean())
86 return {'q':qs.mean(0),'r':rs.mean(0),'a':x[ae:ae+T],'g':x[ge:ge+2*B]
    .reshape(B,2),
87         'center':net.mean(0),'asof':asof,'lp_violation':sol.violation,
    'objective_gap':abs(true-sol.fun),
88         'scenario_q':qs,'scenario_r':rs,'scenario_state':xs,'forecast_
    contract_gain':x[gq:gr]}
89
90 def prune_cuts(cuts):
91     """Remove globally inactive lines on [EMIN, EMAX], preserving the
    envelope."""
92     by_slope={}
93     for a,b in [[0.,0.]]+list(cuts):by_slope[b]=max(a,by_slope.get(b,-np.inf))

```

```

94     hull=[];starts=[]
95     for b,a in sorted(by_slope.items()):
96         start=-np.inf
97         while hull:
98             pa,pb=hull[-1];start=(pa-a)/(b-pb)
99             if start>starts[-1]:break
100            hull.pop();starts.pop()
101            if not hull:start=-np.inf
102            hull.append([a,b]);starts.append(start)
103     keep=[line for i,line in enumerate(hull) if starts[i]<=EMAX and      ↵
          (i==len(hull)-1 or starts[i+1]>=EMIN)]
104     probes=np.linspace(EMIN,EMAX,301)
105     assert np.max(np.abs(np.max([a+b*probes for a,b in      ↵
          [[0.,0.]+list(cuts)],axis=0)-np.max([a+b*probes for a,b in      ↵
          keep],axis=0)))<1e-6
106     return keep
107
108 def execute(E,target,q,net,remaining,closed=False,t=0,final=6000.):
109     lo=max(EMIN,E-M/ETA);hi=min(EMAX,E+ETA*M)
110     if closed: remaining=143-t
111     if final is not None:
112         lo=max(lo,final-remaining*ETA*M);hi=min(hi,final+remaining*M/ETA)
113     if lo>hi+1e-6:raise RuntimeError('Unreachable terminal inventory')
114     nex=float(np.clip(target,lo,hi));delta=nex-E;c=max(delta,0)/ETA;d=max(      ↵
          -delta,0)*ETA
115     balance=q+d-c-net
116     return nex,c,d,max(-balance,0),max(balance,0),abs(nex-target)
117
118 def inf_convolution(grid,V,net_minus_q,price):
119     """Exact convex PL infimal convolution, then interpolate on the state grid.
120
121     min_x V(x)+5p max(0,n-q+g(x-E)). Slopes of two convex PL
122     functions are merged. Interpolation is an upper chord approximation.
123     """
124     a=net_minus_q;ys=np.array(sorted(set([-ETA*M,M/ETA,0.,float(np.clip(a/      ↵
          ETA,-ETA*M,M/ETA)),float(np.clip(a*ETA,-ETA*M,M/ETA))])))      ↵
125     cost=5*price*np.maximum(0,a+np.maximum(-ETA*ys,-ys/ETA))
126     lengths=np.r_[np.diff(grid),np.diff(ys)];slopes=np.r_[np.diff(V)/np.di      ↵
          ff(grid),np.diff(cost)/np.diff(ys)]      ↵
127     order=np.argsort(slopes,kind='stable');knots=np.r_[grid[0]+ys[0],grid[      ↵
          0]+ys[0]+np.cumsum(lengths[order])]      ↵
128     values=np.r_[V[0]+cost[0],V[0]+cost[0]+np.cumsum(lengths[order]*slopes      ↵
          [order])]      ↵
129     return np.interp(grid,knots,values)
130
131 class MarkovDP:
132     """Observable net-load bins, conditional price means, continuous actions.
133
134     Future provisional contracts use the planner's common nominal quantities.

```

```

135 Grid value interpolation approximates convex Bellman functions; it is not
136 advertised as SDDP or a global lower bound.
137 """
138 def __init__(self,net,prices,q,tail_price,grid_size=321,terminal=None,
    ↪ tail_cuts=None):
139     S,T=net.shape;self.grid=np.linspace(EMIN,EMAX,grid_size);self.T=T
140     cuts=np.quantile(net,[1/3,2/3],axis=0).T;self.cuts=cuts
141     labels=(net>cuts[:,0]).astype(int)+(net>cuts[:,1]).astype(int)
142     self.future=np.zeros((T,3,grid_size));self.price=np.zeros((T,3))
143     # A finite convex terminal penalty approximates the target in this DP;
144     # hard true-end reachability is enforced separately in execution.
145     v=np.repeat((-tail_price*self.grid)[None,:],3,axis=0)
146     if tail_cuts:v=np.repeat(np.maximum(0,np.max([a+b*self.grid for a,b
    ↪ in tail_cuts],axis=0))[None,:],3,axis=0)
147     if terminal is not None:v=np.repeat((1000*np.abs(self.grid-terminal)
    ↪ 1))[None,:],3,axis=0)
148     for j in range(T-1,-1,-1):
149         counts=np.zeros((3,3))
150         if j<T-1:np.add.at(counts,(labels[:,j],labels[:,j+1]),1.)
151         P=(counts+1/3)/(counts.sum(1,keepdims=True)+1)
152         nxt=P@v;self.future[j]=nxt;cur=np.empty_like(v)
153         for k in range(3):
154             ids=np.flatnonzero(labels[:,j]==k)
155             if not len(ids):ids=np.arange(S)
156             price=float(prices[ids,j].mean());self.price[j,k]=price
157             cur[k]=np.mean([inf_convolution(self.grid,nxt[k],net[s,j]-
    ↪ q[j],price) for s in ids],axis=0)
158         v=cur
159     self.initial_values=v
160
161     def action(self,j,E,net,q):
162         k=int(net>self.cuts[j,0])+int(net>self.cuts[j,1]);a=net-q
163         lo=max(EMIN,E-M/ETA);hi=min(EMAX,E+ETA*M)
164         # A convex PL minimum occurs at a value knot or an immediate-cost kink.
165         candidates=np.unique(np.r_[self.grid[(self.grid>=lo)&(self.grid<=h
    ↪ i)],lo,hi,np.clip([E,E-a/ETA,E-a*ETA],lo,hi)])
166         delta=candidates-E;cost=5*self.price[j,k]*np.maximum(0,a+np.maximu
    ↪ m(ETA*delta,delta/ETA))+np.interp(candidates,self.grid,self.future
    ↪ [j,k])
167         return float(candidates[np.argmin(cost)])

```

src/dispatch.py

```

1 """Convex sample-average planning and causal battery execution; all energy
    ↪ in kWh."""
2 from functools import lru_cache
3 import numpy as np
4 from scipy.optimize import linprog
5 from scipy.sparse import eye, hstack, vstack, kron, csr_matrix, diags

```

```

6
7 @lru_cache(maxsize=256)
8 def matrices(T,S,adjust,eta):
9     I=eye(T,format='csr'); Z=csr_matrix((T,T));
10    R=kron(np.ones((S,1)),I,format='csr')
11    # -q + c - d - emergency <= -net demand
12    Aub=hstack([-R,R,-R,csr_matrix((S*T,T)),-eye(S*T)],format='csr')
13    D=eye(T)-diags(np.ones(T-1),-1,shape=(T,T))
14    Aeq=hstack([Z,-eta*I,I/eta,D,csr_matrix((T,S*T))],format='csr')
15    if adjust:
16        Aub=hstack([Aub,csr_matrix((S*T,2*T))],format='csr')
17        Aeq=hstack([Aeq,csr_matrix((T,2*T))],format='csr')
18        Aeq=vstack([Aeq,hstack([I,Z,Z,Z,csr_matrix((T,S*T)),-I,I])],format='csr')
19    return Aub,Aeq
20 def solve(net,prices,initial=6000.,terminal=6000.,base=None,refund=True,eta=.9,
21         power=5000.,emin=1200.,emax=10800.,penalty=5.,hard=False):
22     S,T=net.shape; adjust=base is not None; p=prices.mean(0)
23     Aub,Aeq=matrices(T,S,adjust,eta)
24     obj=np.r_[np.zeros(T) if adjust else
25              p,np.zeros(2*T),np.zeros(T),penalty*prices.ravel()/S]
26     if adjust: obj=np.r_[obj,1.5*p,(-.5 if refund else .5)*p]
27     rhs=np.zeros(T);rhs[0]=initial
28     if adjust:rhs=np.r_[rhs,base]
29     bounds=[(0,None)]*T+[(0,power/6)]*(2*T)+[(emin,emax)]*T+[(0,0 if hard
30     else None)]*(S*T)
31     bounds[4*T-1]=(terminal,terminal)
32     if adjust: bounds += [(0,None)]*(2*T)
33     res=linprog(obj,A_ub=Aub,b_ub=-net.ravel(),A_eq=Aeq,b_eq=rhs,bounds=boun
34     ds,method='highs',
35     options={'dual_feasibility_tolerance':1e-8,'primal_feasibility_tolerance':1e-8})
36     if not res.success: raise RuntimeError(f'LP failed: {res.message}')
37     x=res.x; q=x[:T];c=x[T:2*T];d=x[2*T:3*T]
38     # Analytic removal of any numerical simultaneous charge/discharge,
39     preserving SOC.
40     remove=np.minimum(c,d/eta**2);c=c-remove;d=d-eta**2*remove
41     state=np.r_[initial,initial+np.cumsum(eta*c-d/eta)]
42     em=np.maximum(net+c-d-q,0)
43     if adjust:
44         cost=float(np.sum(1.5*p*np.maximum(q-base,0)+(-.5 if refund else
45         .5)*p*np.maximum(base-q,0))+
46                    penalty*np.mean(np.sum(prices*em,axis=1)))
47     else: cost=float(p@q+penalty*np.mean(np.sum(prices*em,axis=1)))
48     return {'q':q,'c':c,'d':d,'state':state,'expected_cost':cost,'lp_objective':float(res.fun),
49            'eq_residual':float(np.max(np.abs(Aeq@x-rhs))),
50            'ineq_violation':float(max(0,np.max(Aub@x+net.ravel()))),

```

```

46         'duality_gap':float(abs(res.fun-(np.dot(rhs,res.eqlin.marginal_ ↵
        s)+np.dot(-net.ravel(),res.ineqlin.marginals)+
47         sum(lo*m for (lo,hi),m in zip(bounds,res.lower.marginals) ↵
        if lo is not None)+
48         sum(hi*m for (lo,hi),m in zip(bounds,res.upper.marginals) ↵
        if hi is not None))))}
49
50 def execute_slot(q,net,E,c_plan,d_plan,t,controller='fixed',eta=.9,power=5 ↵
    000.,emin=1200.,emax=10800.,terminal=6000.):
51     if controller=='fixed': c,d=c_plan,d_plan
52     elif controller=='greedy':
53         balance=q-net
54         nextE=E+eta*min(max(balance,0),power/6,(emax-E)/eta)-min(max(-bala ↵
        nce,0),power/6,eta*(E-emin))/eta
55         # Backward reachable terminal set prevents last-day battery ↵
        depletion borrowing.
56         left=143-t
57         lower=max(emin,terminal-left*eta*power/6)
58         upper=min(emax,terminal+left*power/(6*eta))
59         nextE=np.clip(nextE,lower,upper)
60         c=max(0,(nextE-E)/eta);d=max(0,(E-nextE)*eta)
61     else: raise ValueError(controller)
62     nextE=E+eta*c-d/eta
63     balance=q+d-c-net
64     emergency=max(-balance,0);spill=max(balance,0)
65     return c,d,nextE,emergency,spill
66
67 def settlement(q,r,emergency,price,refund=True,penalty=5.):
68     planned=price*q
69     increase=1.5*price*np.maximum(r-q,0)
70     reduction=(-.5 if refund else .5)*price*np.maximum(q-r,0)
71     emerg=penalty*price*emergency
72     return {'planned_cost':float(planned.sum()),'increase_cost':float(incr ↵
        ease.sum()),
73           'reduction_net_cost':float(reduction.sum()),'emergency_cost':f ↵
        loat(emerg.sum()),
74           'total_cost':float((planned+increase+reduction+emerg).sum())}

```

src/run.py

```

1  """Chronological contest replay; whole trajectories, never independent days."""
2  from pathlib import Path
3  from concurrent.futures import ProcessPoolExecutor,as_completed
4  import argparse,json,sys,time,platform,hashlib
5  import numpy as np
6  from forecasting import select_families,scenarios
7  from control import affine_plan,MarkovDP,execute,ETA,prune_cuts
8  from dispatch import settlement,solve
9  from sddp import train_tail

```

```

10
11 ROOT=Path(__file__).resolve().parents[1]
12 KINDS={'q2':(False,False),'q3':(True,False),'q4_2':(False,True),'q4_3':(Tr  ↵
    ue,True)}
13 CANDIDATES={'closed_baseline':(False,'greedy','linear',True),
14             'closed_affine':(True,'affine','linear',True),
15             'cross_baseline':(False,'greedy','linear',False),
16             'affine_mpc':(True,'affine','linear',False),
17             'markov_mpc':(True,'dp','linear',False),
18             'sddp_mpc':(True,'affine','sddp',False),
19             'sddp_markov':(True,'dp','sddp',False)}
20
21 def dump(path,obj):Path(path).write_text(json.dumps(obj,ensure_ascii=False  ↵
    ,indent=2,allow_nan=False)+'\n')
22
23 def warmup(data):
24     net=(data['load'][:31]-data['pv'][:31])/6
25     # No unearned February reset: hold the battery from the specified  ↵
    January initial state.
26     a={'state':np.full((31,145),6000.),'q':np.zeros((31,144)), 'r':np.zeros  ↵
    ((31,144)),
27     'c':np.zeros((31,144)), 'd':np.zeros((31,144)), 'emergency':np.maximu  ↵
    m(net,0), 'spill':np.maximum(-net,0)}
28     np.savez_compressed(ROOT/'artifacts/january-warmup.npz',**a)
29     return {'policy':'Zero planned purchase and no battery operation;  ↵
    current shortages covered by emergency purchase.',
30            'initial_energy':6000., 'february_initial_energy':float(a['stat  ↵
    e'][-1,-1]),
31            'days':31, 'fixed_price_cost':float(np.sum(5*data['day_price']*  ↵
    a['emergency'])),
32            'variable_price_cost':float(np.sum(5*data['price'][:31]*a['eme  ↵
    rgency'])),
33            'note':'Same causal initialization for every candidate. January  ↵
    cost reported separately; no inventory transfer benefit
    hidden.'}
34
35 def tail_file(kind,cutoff,days=3):return  ↵
    ROOT/f"artifacts/tails/{kind}_{cutoff}{' ' if days==3 else
    '_D'+str(days)}.json"  ↵
36
37 def train_job(job):
38     kind,cutoff,iterations,steps,days=job;official,variable=KINDS[kind]
39     data=dict(np.load(ROOT/'artifacts/data.npz'));res=train_tail(data,cuto  ↵
    ff,variable,official,K=steps,days=days,iterations=iterations)
40     res['execution']={'command':'python3 '+'  ↵
    '.join(sys.argv),'worker_job':list(job),'exit_code':0,'source_sha256':  ↵
    hashlib.sha256((ROOT/'src/sddp.py').read_bytes()).hexdigest()}
41     dump(tail_file(kind,cutoff,days),res)

```

```

42     return {'kind':kind,'cutoff':cutoff,'runtime_seconds':res['runtime_sec'] ↵
        onds'],'evaluation':res['evaluation']}
43
44 def simulate(data,selection,days,kind,candidate,count=7,horizon_days=2,tai ↵
    l_scale=1.,grid=321,final=None,initial=6000.):
45     official,variable=KINDS[kind];affine,lower,tail,closed=CANDIDATES[cand ↵
        idate]
46     N=len(days);arrays={k:np.zeros((N,144)) for k in ↵
        ['q','r','c','d','emergency','spill','price','projection']}
47     arrays['state']=np.zeros((N,145));arrays['releases']=np.full((N,4,144) ↵
        ,np.nan)
48     E=float(initial);logs=[];decisions=[];lpv=gap=0.;start=time.perf_count ↵
        er();dp=None;cache={}
49     evaluation_end=(days[-1]+1)*144
50     for i,day in enumerate(days):
51         arrays['state'][i,0]=E;actual=(data['load'][day]-data['pv'][day])/6
52         price=data['price'][day] if variable else ↵
            data['day_price'];arrays['price'][i]=price
53         for t in range(144):
54             now=day*144+t
55             if t%36==0:
56                 end=min((day+horizon_days)*144,evaluation_end)
57                 net,prices,meta=scenarios(data,selection,now,end,official, ↵
                    variable,count=count)
58                 base=None if t==0 else ↵
                    np.r_[arrays['q'][i,t:],np.zeros(end-(day+1)*144)]
59                 terminal=6000. if closed else final if end==evaluation_end ↵
                    else None
60                 cuts=None
61                 lam=tail_scale*float(np.quantile(prices[:,-min(144,len(pri ↵
                    ces[0])):],.25))/ETA
62                 if end==evaluation_end or closed:lam=0.
63                 elif tail=='sddp':
64                     # Monthly model version is fixed before the month begins.
65                     cutoff=int(day)-(int(str(data['dates'][day])[8:10])-1)
66                     if day<31:cutoff=14
67                     tail_days=min(3,(evaluation_end-end)//144)
68                     path=tail_file(kind,cutoff,tail_days)
69                     if path not in ↵
                        cache:cache[path]=json.loads(path.read_text())
70                     cuts=prune_cuts(cache[path]['root_cuts']);lam=0.
71                 plan=affine_plan(net,prices,now,E,base,official,affine,lam ↵
                    ,terminal,cuts,daily_closed=closed)
72                 lpv=max(lpv,plan['lp_violation']);gap=max(gap,plan['object ↵
                    ive_gap'])
73                 if t==0:
74                     arrays['q'][i]=np.maximum(plan['q'][:144],0);arrays['r ↵
                        '][i]=arrays['q'][i]
75                     arrays['releases'][i,0]=arrays['q'][i]

```

```

76         elif official:
77             arrays['r'][i,t:]=np.maximum(plan['r'][:144-t],0)
78             arrays['releases'][i,t//36,t:]=arrays['r'][i,t:]
79             effective=plan['r'].copy();until=36 if official else 144-t
80             effective[:until]=arrays['r'][i,t:t+until]
81             if lower=='dp':dp=MarkovDP(net,prices,effective,lam,grid,t,
82             erminial,cuts)
83             active=t;ema=0.
84             decisions.append(**meta,'date':str(data['dates'][day]),'r'
85             elease':t//36,'inventory':E,'tail_price':lam,'tail_model_c'
86             utoff':cutoff if cuts else None,'tail_model_days':tail_days
87             if cuts else None,'tail_cut_count':len(cuts) if cuts else
88             0})
89         j=t-active
90         if lower=='affine':
91             error=actual[t]-plan['center'][j];ema=.8*ema+.2*error
92             target=plan['a'][j]+plan['g'][0]@np.array([error,ema])
93         elif lower=='dp':target=dp.action(j,E,actual[t],arrays['r'][i,t])
94         else:
95             balance=arrays['r'][i,t]-actual[t]
96             target=E+ETA*max(balance,0)-max(-balance,0)/ETA
97             remaining=evaluation_end-now-1
98             E,c,d,em,sp,projection=execute(E,target,arrays['r'][i,t],actua
99             l[t],remaining,closed,t,6000. if closed else final)
100             for k,v in [('c',c),('d',d),('emergency',em),('spill',sp),('pr
101             ojection',projection)]:arrays[k][i,t]=v
102             arrays['state'][i,t+1]=E
103             cost=settlement(arrays['q'][i],arrays['r'][i],arrays['emergency'][i],price)
104             logs.append({'date':str(data['dates'][day]),**cost,'emergency_ener
105             gy':float(arrays['emergency'][i].sum()),
106             'spill_energy':float(arrays['spill'][i].sum()),'ending_inventory'
107             :E,'projection_count':int((arrays['projection'][i]>1e-6).sum())
108             })
109             balance=arrays['r']+arrays['emergency']+arrays['d']-arrays['c']-arrays
110             ['spill']-(data['load'][days]-data['pv'][days])/6
111             val={'balance_max_abs':float(abs(balance).max()),'soc_recurrence_max_a
112             bs':float(abs(np.diff(arrays['state'],axis=1)-ETA*arrays['c']+arrays['
113             d']/ETA).max()),
114             'midnight_jump_max_abs':float(abs(arrays['state'][1:,0]-arrays['stat
115             e'][:-1,-1]).max()) if N>1 else 0.,
116             'soc_min':float(arrays['state'].min()),'soc_max':float(arrays['state'
117             ].max()),'power_max_kw':float(max(arrays['c'].max(),arrays['d'].max()
118             ())*6),
119             'simultaneous_max':float(np.minimum(arrays['c'],arrays['d']).max()),
120             'lp_violation_max':lpv,'lp_objective_recompute_gap':gap,
121             'initial_inventory':float(arrays['state'][0,0]),'final_inventory':fl
122             oat(E),

```

```

105     'negative_energy_violation':float(max(0,-min(arrays[k].min() for k in
    ['q','r','c','d','emergency','spill'])))}
106 assert val['balance_max_abs']<1e-5 and
    val['soc_recurrence_max_abs']<1e-5 and
    val['midnight_jump_max_abs']<1e-5
107 assert val['soc_min']>=1200+1e-5 and val['soc_max']<=10800+1e-5 and
    val['power_max_kw']<=5000+1e-5
108 if final is not None:assert abs(E-final)<1e-5
109 totals={k:float(sum(z[k] for z in logs)) for k in
    ['planned_cost','increase_cost','reduction_net_cost','emergency_cost',
    'total_cost','emergency_energy','spill_energy','projection_count']}
110 arrays['days']=np.array(days);arrays['dates']=data['dates'][days]
111 metrics={'kind':kind,'candidate':candidate,'configuration':{'count':count,
    'horizon_days':horizon_days,'tail_scale':tail_scale,'grid':grid,'final':final,
    'initial':initial,'closed_daily':closed,'start_day':days[0],
    'end_day':days[-1]},
    'totals':totals,'validation':val,'daily':logs,'decisions':decisions,
    'runtime_seconds':time.perf_counter()-start}
112
113 return arrays,metrics
114
115 def replay_job(job):
116     phase,kind,candidate,days,count,horizon,scale,grid,final=job
117     data=dict(np.load(ROOT/'artifacts/data.npz'));selection=json.loads((ROOT/'artifacts/forecast-selection.json').read_text())
118     a,m=simulate(data,selection,days,kind,candidate,count,horizon,scale,grid,final)
119     folder=ROOT/'artifacts'/phase;folder.mkdir(exist_ok=True)
120     np.savez_compressed(folder/f'{kind}_{candidate}.npz',**a);dump(folder/f'{kind}_{candidate}.json',m)
121     return {'kind':kind,'candidate':candidate,'total_cost':m['totals']['total_cost'],
    'runtime_seconds':m['runtime_seconds']}
122
123 def main():
124     p=argparse.ArgumentParser();p.add_argument('--phase',default='pilot');
    p.add_argument('--workers',type=int,default=4);p.add_argument('--kinds',
    nargs='+',default=list(KINDS));p.add_argument('--candidates',nargs='+',
    default=list(CANDIDATES));p.add_argument('--count',type=int,default=7);
    p.add_argument('--horizon',type=int,default=2);p.add_argument('--tail-scale',
    type=float,default=1.);p.add_argument('--grid',type=int,default=321);
    p.add_argument('--final',type=float,default=None);p.add_argument('--iterations',
    type=int,default=100);p.add_argument('--steps',type=int,default=12);
    p.add_argument('--tail-days',type=int,default=3);p.add_argument('--cutoffs',
    nargs='+',type=int);p.add_argument('--start',type=int);p.add_argument('--stop',
    type=int);args=p.parse_args()
125     data=dict(np.load(ROOT/'artifacts/data.npz'))
126     if not (ROOT/'artifacts/forecast-selection.json').exists():dump(ROOT/'artifacts/forecast-selection.json',select_families(data))
127     if args.phase=='initialize':

```

```

128     dump(ROOT/'artifacts/warmup.json',warmup(data));n=(data['day_load'] ↵
    ]-data['day_pv'])/6
129     q1=solve(n[None,:],data['day_price'][None,:],hard=True)
130     np.savez_compressed(ROOT/'artifacts/q1.npz',**{k:v for k,v in ↵
    q1.items() if isinstance(v,np.ndarray)})
131     dump(ROOT/'artifacts/q1.json',{k:v for k,v in q1.items() if not ↵
    isinstance(v,np.ndarray)});return
132 if args.phase=='train':
133     (ROOT/'artifacts/tails').mkdir(exist_ok=True)
134     cutoffs=args.cutoffs or [14,31,59,90,120,151,181,212,243,273,304,334]
135     jobs=[(k,d,args.iterations,args.steps,args.tail_days) for k in ↵
    args.kinds for d in cutoffs];worker=train_job
136 else:
137     start=args.start if args.start is not None else ↵
    {'pilot':31,'calibrate':24,'annual':31}.get(args.phase,31)
138     stop=args.stop if args.stop is not None else ↵
    {'pilot':32,'calibrate':31,'annual':365}.get(args.phase,32)
139     days=list(range(start,stop));jobs=[(args.phase,k,c,days,args.count ↵
    ,args.horizon,args.tail_scale,args.grid,args.final) for k in ↵
    args.kinds for c in args.candidates];worker=replay_job
140 begin=time.perf_counter();results=[]
141 with ProcessPoolExecutor(max_workers=args.workers) as pool:
142     futures={pool.submit(worker,j):j for j in jobs}
143     for f in as_completed(futures):
144         result=f.result();results.append(result);print(json.dumps(resu ↵
    lt,ensure_ascii=False),flush=True)
145 log_phase=args.phase+(f'-D{args.tail_days}' if args.phase=='train' and ↵
    args.tail_days!=3 else '')
146 dump(ROOT/f'artifacts/execution-{log_phase}.json',{'command':'python3 ↵
    '+' '.join(sys.argv),'exit_code':0,'runtime_seconds':time.perf_counter ↵
    ()-begin,
147     'environment':{'python':sys.version,'numpy':np.__version__,'platfo ↵
    rm':platform.platform()},'results':results,
148     'seed':20260911 if args.phase=='train' else ↵
    None,'deterministic_reason':None if args.phase=='train' else ↵
    'Deterministic chronological scenario selection, LP and convex ↵
    DP.',
149     'source_hashes':{'p.name:hashlib.sha256(p.read_bytes()).hexdigest() ↵
    for p in (ROOT/'src').glob('*.py')}})
150
151 if __name__=='__main__':main()

```

其余脚本、核验程序与绘图入口保持紧凑排版，全文仍完整列入。

src/analyze_tail.py

```

1 """Summarize the actual trained scalar lower-envelope shape, not the unknown optimum."""
2 from pathlib import Path
3 import json
4 from collections import Counter

```

```

5 from control import prune_cuts
6 ROOT=Path(__file__).resolve().parents[1]
7 def main():
8     models=[]
9     for path in sorted((ROOT/'artifacts/tails').glob('*.json')):
10         if '_D' in path.stem or path.stem.endswith('_14'):continue
11         m=json.loads(path.read_text());assert m['iterations']==500
12         cuts=prune_cuts(m['root_cuts'])
13         models.append({'model':path.stem,'active_root_cut_count':len(cuts),'active_slopes':[float(b) for a,b
            ↪ in cuts]})
14     assert len(models)==44
15     result={'scope':'44 monthly three-day auxiliary lower envelopes on inventory1200..10800,
            ↪ after500iterations. This describes computed lower approximations, not exact value-function shape.',
16         'models':models,'count':len(models),'active_cut_count_distribution':dict(Counter(str(m['active_root_cut']
            ↪ _count')) for m in models)),
17         'single_segment_models':sum(m['active_root_cut_count']==1 for m in models)}
18     result['minimum_active_cuts']=min(m['active_root_cut_count'] for m in models)
19     result['maximum_active_cuts']=max(m['active_root_cut_count'] for m in models)
20     (ROOT/'artifacts/tail-shape-analysis.json').write_text(json.dumps(result,ensure_ascii=False,indent=2)+'\n'
            ↪ ' ')
21     print({k:v for k,v in result.items() if k!='models'})
22 if __name__=='__main__':main()

```

src/check_terminal.py

```

1 """Exact prefix reuse for this two-midnight MPC's global terminal comparison."""
2 from pathlib import Path
3 from concurrent.futures import ProcessPoolExecutor,as_completed
4 import json,sys,time,hashlib
5 import numpy as np
6 from run import ROOT,simulate,dump
7
8 def worker(job):
9     kind,candidate=job;folder='annual_sddp' if candidate.startswith('sddp') else 'annual'
10     path=ROOT/f'artifacts/{folder}/{kind}_{candidate}'
11     a=dict(np.load(str(path)+'.npz'));m=json.loads(Path(str(path)+'.json').read_text())
12     assert np.array_equal(a['days'],np.arange(31,365))
13     assert np.array_equal(a['dates'].astype('datetime64[D]'),np.arange(np.datetime64('2025-02-01'),np.datetime
            ↪ e64('2026-01-01')))
14     assert candidate in ['affine_mpc','markov_mpc'],'Prefix proof is scoped to the linear-tail controllers.'
15     assert m['configuration']['horizon_days']==2 and m['configuration']['final'] is None
16     data=dict(np.load(ROOT/'artifacts/data.npz'));selection=json.loads((ROOT/'artifacts/forecast-selection.js
            ↪ on').read_text())
17     idx=len(a['days'])-2;days=a['days'][idx:].tolist();initial=float(a['state'][idx,0])
18     kwargs={k:m['configuration'][k] for k in ['count','horizon_days','tail_scale','grid']}
19     free,fm=simulate(data,selection,days,kind,candidate,initial=initial,**kwargs)
20     diffs={k:float(np.max(np.abs(free[k]-a[k][idx:]))) for k in ['q','r','c','d','emergency','spill','state']}
21     assert max(diffs.values())<1e-5,diffs
22     fixed,xm=simulate(data,selection,days,kind,candidate,initial=initial,final=6000,**kwargs)
23     combined={k:np.concatenate([v[:idx],fixed[k]],axis=0) for k,v in a.items()}
24     assert np.max(np.abs(combined['state'][1:,0]-combined['state'][:-1,-1]))<1e-6
25     q,r,em,p=[combined[k] for k in ['q','r','emergency','price']]
26     bill=p*(q+1.5*np.maximum(r-q,0)-.5*np.maximum(q-r,0)+5*em)
27     total=float(bill.sum());prefix=float(sum(z['total_cost'] for z in m['daily'][:idx]))
28     assert abs(total-prefix-xm['totals']['total_cost'])<1e-5
29     out=ROOT/'artifacts/global-terminal';out.mkdir(exist_ok=True)
30     np.savez_compressed(out/f'{kind}_{candidate}.npz',**combined)
31     result={'kind':kind,'candidate':candidate,'free_total_cost':m['totals']['total_cost'],'fixed_total_cost':
            ↪ total,
32         'terminal_constraint_increment':total-m['totals']['total_cost'],'prefix_days':idx,'replayed_days':days,
33         'suffix_initial_inventory':initial,'free_suffix_replay_differences':diffs,'free_final_inventory':fm['va
            ↪ lidation']['final_inventory'],
34         'fixed_final_inventory':xm['validation']['final_inventory'],'fixed_suffix_validation':xm['validation'],
35         'fixed_suffix_decisions':xm['decisions'],'fixed_suffix_configuration':xm['configuration'],'fixed_suffix'
            ↪ _daily':xm['daily'],
36         'source_trajectory_sha256':hashlib.sha256(Path(str(path)+'.npz').read_bytes()).hexdigest(),
37         'source_metrics_sha256':hashlib.sha256(Path(str(path)+'.json').read_bytes()).hexdigest(),

```

```

38     'prefix_proof': 'Before Dec30, this fixed two-midnight controller never includes the real year-end in its
    ↪ planning horizon; its physical terminal-reachability interval is also inactive. At the reused
    ↪ midnight all old deliveries are complete and forecast history is identical. This is an exact reuse
    ↪ for this implemented policy, not a proof for an arbitrary long-horizon optimal policy.}'
39 dump(out/f'{kind}_{candidate}.json', result); return result
40
41 def main():
42     start=time.perf_counter(); jobs=[(k,c) for k in ['q2','q3','q4_2','q4_3'] for c in
    ↪ ['affine_mpc','markov_mpc']]
43     results=[]
44     with ProcessPoolExecutor(max_workers=4) as pool:
45         for f in as_completed([pool.submit(worker,j) for j in jobs]):
46             r=f.result(); results.append(r); print(json.dumps({k:r[k] for k in
    ↪ ['kind','candidate','fixed_total_cost','terminal_constraint_increment']}), flush=True)
47     dump(ROOT/'artifacts/global-terminal.json', {'results': results, 'execution': {'command': 'python3 '+'
    ↪ '.join(sys.argv)', 'exit_code': 0, 'runtime_seconds': time.perf_counter()-start}})
48
49 if __name__=='__main__': main()

```

src/continue_pipeline.py

```

1  """Local orchestration of the remaining finite calculations, with no model calls."""
2  from pathlib import Path
3  import subprocess, sys, time, json, shutil
4  ROOT=Path(__file__).resolve().parents[1]
5
6  def complete(paths):
7      for path in paths:
8          try:
9              m=json.loads(path.read_text())
10             if m.get('iterations')!=500: return False
11             except (OSError, ValueError): return False
12     return True
13
14 def run(args):
15     print('START', ' '.join(args), flush=True)
16     subprocess.run([sys.executable,*args], cwd=ROOT, check=True)
17     print('DONE', ' '.join(args), flush=True)
18
19 def main():
20     kinds=['q2','q3','q4_2','q4_3']; cutoffs=[14,31,59,90,120,151,181,212,243,273,304,334]
21     while not complete([ROOT/f'artifacts/tails/{k}_{d}.json' for k in kinds for d in
    ↪ [14,334]]): time.sleep(10)
22     run(['src/run.py', '--phase', 'train', '--cutoffs', '14', '334', '--tail-days', '2', '--iterations', '500', '--work'
    ↪ 'ers', '2'])
23     run(['src/sddp.py', '--cutoff', '31', '--days', '1', '--iterations', '500', '--output', 'tails/q2_31_D1.json'])
24     print('WAIT monthly training models', flush=True)
25     while not complete([ROOT/f'artifacts/tails/{k}_{d}.json' for k in kinds for d in cutoffs]): time.sleep(10)
26     run(['src/run.py', '--phase', 'calibrate_sddp_v2', '--start', '24', '--stop', '31', '--candidates', 'sddp_mpc', 's'
    ↪ 'ddp_markov', '--workers', '3'])
27     select=ROOT/'artifacts/selection.json'; old=json.loads(select.read_text()); shutil.copy2(select, ROOT/'artif'
    ↪ 'acts/selection-v1.json')
28     for k in kinds:
29         vals=old['development_costs'][k]
30         for c in ['sddp_mpc', 'sddp_markov']:
31             vals[c]=json.loads((ROOT/f'artifacts/calibrate_sddp_v2/{k}_{c}.json').read_text())['totals']['tot'
    ↪ 'al_cost']
32             chosen=min(vals, key=vals.get)
33             if vals['markov_mpc']<=min(vals.values())*(1+old['relative_tie_tolerance']): chosen='markov_mpc'
34             old['selected'][k]=chosen
35             assert chosen!='sddp_mpc', 'Unexpected development selection: run its annual candidate before
    ↪ finishing.'
36     old['sddp_training_iterations_development']=500
37     old['v2_note']='Recomputed January SDDP comparisons after finite-end tail truncation and increased
    ↪ training. This selection code reads no annual cost.'
38     select.write_text(json.dumps(old, ensure_ascii=False, indent=2)+'\n'); print('FROZEN
    ↪ SELECTION', old['selected'], flush=True)
39     # The affinely executed SDDP variant lost clearly in January and is retained

```

```

40 # as a development comparison. Annual SDDP comparison uses the selected
41 # Markov feedback, changing only the continuation-value construction.
42 a=subprocess.Popen([sys.executable,'src/run.py','--phase','annual_sddp','--start','31','--stop','365','--'
↳ candidates','sddp_markov','--workers','3'],cwd=ROOT)
43 b=subprocess.Popen([sys.executable,'src/validate.py','--scope','annual-releases','--workers','3'],cwd=ROOT)
↳ T)
44 assert a.wait()==0;assert b.wait()==0
45 print('WAIT main annual pool',flush=True)
46 while True:
47     try:
48         m=json.loads((ROOT/'artifacts/execution-annual.json').read_text())
49         if m['exit_code']==0 and len(m['results'])==20:break
50     except (OSError,ValueError,KeyError):pass
51     time.sleep(10)
52 run(['src/check_terminal.py'])
53 print('ALL NUMERICAL PHASES FINISHED; final assembly and independent audit still required',flush=True)
54
55 if __name__=='__main__':main()

```

src/export_results.py

```

1 """Prepare numerical handoff tables, detailed CSVs and an artifact-tool workbook payload."""
2 from pathlib import Path
3 import csv,json,shutil,sys,time
4 import numpy as np
5
6 ROOT=Path(__file__).resolve().parents[1]
7 OUT=ROOT/'计算结果'; OUT.mkdir(exist_ok=True)
8 def dump(p,x):Path(p).write_text(json.dumps(x,ensure_ascii=False,indent=2)+'\n')
9 def clock(i):return f'{i//6:02d}:{(i%6)*10:02d}'
10 def interval(i,j=None):return clock(i)+'-'+clock(i+1 if j is None else j)
11 def writecsv(name,header,rows):
12     with (OUT/name).open('w',encoding='utf-8-sig',newline='') as f:
13         w=csv.writer(f);w.writerow(header);w.writerows(rows)
14 def events(values,date):
15     out=[];start=None
16     for t in range(145):
17         active=t<144 and values[t]>1e-6
18         if active and start is None:start=t
19         if not active and start is not None:
20             out.append([date,interval(start,t),float(values[start:t].sum())]);start=None
21     return out
22 def markdown_table(header,rows):
23     def fmt(v):return f'{v:,.4f}' if isinstance(v,float) else str(v)
24     return '\n'.join(['| '+' | '.join(header)+' |','| '+'| '.join(['---']*len(header))+'|']+
25         ['| '+' | '.join(fmt(v) for v in r)+' | ' for r in rows])
26
27 def main():
28     started=time.perf_counter()
29     selection=json.loads((ROOT/'artifacts/selection.json').read_text())
30     for kind,candidate in selection['selected'].items():
31         phase='annual_sddp' if candidate.startswith('sddp') else 'annual'
32         for ext in ['json','npz']:shutil.copy2(ROOT/f'artifacts/{phase}/{kind}_{candidate}.{ext}',ROOT/f'arti
↳ facts/{kind}.{ext}')
33     data=dict(np.load(ROOT/'artifacts/data.npz'));q1=json.loads((ROOT/'artifacts/q1.json').read_text());q1.up
↳ date({k:v.tolist() for k,v in
↳ dict(np.load(ROOT/'artifacts/q1.npz')).items()});q1['daily_energy']=float(sum(q1['q']));q1['daily_cos']
↳ t']=float(data['day_price']@np.array(q1['q']))
34     labels=[interval(t) for t in range(144)]
35     notes=['项目','口径'],['官方原题','https://www.mcm.edu.cn/html_cn/node/27b6e148f8113f09b0269f64a02629fb.h
↳ tml'],
36         ['时间轴','原输入00:10至24:00按前10分钟区间均值处理;输出时间段改正为00:00-00:10至23:50-24:00。'],
37         ['单位','购电、充放电、SOC 均为 kWh;费用为元;充放电量在交流母线侧计量。'],
38         ['储能效率','充电和放电各 0.9; SOC= 上一时段 SOC+0.9 充电量-放电量/0.9。'],
39         ['年度边界','只输出2025年2月1日至12月31日。1月为校准,初始SOC6000,1月储能保持6000;主策略SOC跨日连续、
↳ 年末库存自由;一次全局期末6000仅作参考。'],
40         ['计划购电量','0 点承诺量 q;全天购电费列仅为计划成本  $\epsilon p \cdot q$ 。'],
41         ['调整购电量','当天逐次调整后最终执行量 r,非增减差值;全天购电费为计划成本加调整净费用,未含紧急。'],

```

```

42     ['费用汇总','总费用 = 计划成本 +1.5p 增购量-0.5p 退购量 +5p 紧急量; 已购电实物总量 =r+ 紧急量。'],
43     ['退购结算','主口径: 退购部分原电费被50%违约费替代; 相对0点基准逐时最终净额结算。若解释为不退款,
    ↳ 必须改变目标函数并重算, 本目录未将其作为已验证结果。'],
44     ['弃电口径','弃电量为母线总富余弃置, 可能包含已购买电能, 不能直接视为光伏弃电量或用于计算纯光伏利用率。'],
45     ['充放电量','问题 2-4 填写因果实时反馈后的实际充放电, 非 0 点 LP 预计划。'],
46     ['紧急购电量','连续非零 10 分钟段合并; 未发生的日期填无、0。小于 1e-6 kWh 视为数值零。'],
47     ['预测与信息','仅用过往日期数据与已发布的光伏预报; 实际未来电价只用于事后结算。'],
48     ['最优性边界','Q1确定性LP; Q2-4跨日因果仿射策略SAA计划+Markov随机动态规划反馈, SDDP作为末值备选,
    ↳ 为经过验证的可行策略, 不声称完整随机问题全局最优。']]
49 payload={'labels':labels,'notes':notes,'q1':q1,'models':{}}
50 tables={'q1':{ 'purchase':[[labels[t],float(q1['q'] [t])] for t in [60,72,84,96,108,120]],
51     'energy':q1['daily_energy'],'cost':q1['daily_cost'],
52     'battery':[[interval(b,b+24),float(sum(q1['c'] [b:b+24])),float(sum(q1['d'] [b:b+24]))] for b
    ↳ in range(0,144,24)],
53     'soc':[q1['state'] [0],q1['state'] [-1]], 'models':{}}
54 chosen=['2025-03-20','2025-06-21','2025-09-23','2025-12-21']
55 for key in ['q2','q3','q4_2','q4_3']:
56     a=dict(np.load(ROOT/f'artifacts/{key}.npz'));m=json.loads((ROOT/f'artifacts/{key}.json').read_text())
57
58     for i,row in enumerate(m['daily']):
59         row['planned_energy']=float(a['q'] [i].sum());row['adjusted_energy']=float(a['r'] [i].sum())
60     bat=[];emer=[];detail=[];releases=[];selected={}
61     for i,date in enumerate(a['dates']):
62         date=str(date)
63         battery=[[date,interval(b,b+24),float(a['c'] [i,b:b+24].sum()),float(a['d'] [i,b:b+24].sum()),
64             '00:00' if b==0 else '24:00' if b==24 else None,
65             float(a['state'] [i,0]) if b==0 else float(a['state'] [i,-1]) if b==24 else None] for b in
            ↳ range(0,144,24)]
66         ev=events(a['emergency'] [i],date)
67         bat+=battery;emer+=ev or [[date,'无',0.]]
68         for t in range(144):
69             detail.append([date,labels[t]]+[float(a[z] [i,t]) for z in
            ↳ ['q','r','c','d','emergency','spill','price']] +
                [float(a['state'] [i,t]),float(a['state'] [i,t+1])])
70         for k in range(4):
71             for t in range(k*36,144):
72                 if np.isfinite(a['releases'] [i,k,t]):releases.append([date,f'{k*6:02d}:00',labels[t],float(a['releases'] [i,k,t])])
73                 ↳ t(a['releases'] [i,k,t]))
74         if date in chosen:
75             selected[date]={ 'purchase':[[labels[t],float(a['q'] [i,t]),float(a['r'] [i,t])] for t in
            ↳ [60,72,84,96,108,120]],
76                 'daily':m['daily'] [i], 'battery':battery, 'emergency':ev or [[date,'无',0.]]}
77     payload['models'] [key]={ 'dates':a['dates'].tolist(), 'q':a['q'].tolist(), 'r':a['r'].tolist(),
78         'battery':bat, 'emergency':emer, 'daily':m['daily']}
79     tables['models'] [key]={ 'totals':m['totals'], 'selected':selected, 'validation':m['validation']}
80     writecsv(f'{key}_逐时完整策略.csv', ['日期', '时间段', '计划购电_kWh', '最终合同购电_kWh', '充电_kWh', '放电_kWh',
    ↳ 'Wh', '紧急购电_kWh', '弃电_kWh', '电价_元每kWh', '时段初SOC_kWh', '时段末SOC_kWh'], detail)
81     writecsv(f'{key}_调整发布记录.csv', ['日期', '发布时间', '交易时间段', '购电量_kWh', releases])
82     writecsv(f'{key}_费用汇总.csv', list(m['daily'] [0]), [list(row.values()) for row in m['daily']])
83     writecsv(f'{key}_紧急购电.csv', ['日期', '时间段', '购电量_kWh'], emer)
84     writecsv('q1_逐时完整策略.csv', ['时间段', '购电_kWh', '充电_kWh', '放电_kWh', '时段初SOC_kWh', '时段末SOC_kWh'],
85         [[labels[t],q1['q'] [t],q1['c'] [t],q1['d'] [t],q1['state'] [t],q1['state'] [t+1]] for t in
            ↳ range(144)])
86     dump(ROOT/'artifacts/workbook-payload.json',payload);dump(ROOT/'artifacts/handoff-tables.json',tables)
87     text=['# 指定日期结果表','\n全部数值由本地程序直接生成。单位: 电量 kWh、费用元。计划量 q 与最终合同量 r 分列;
    ↳ 实际从电网取电量另加紧急购电。',
88         '\n## 问题 1',markdown_table(['时间段','购电量'],tables['q1'] ['purchase']),
89         f"\n全天购电量 {q1['daily_energy'][:,.4f]} kWh; 全天购电费 {q1['daily_cost'][:,.4f]} 元; 首末SOC均为6000
            ↳ kWh。",
90         markdown_table(['时间段','充电量','放电量'],tables['q1'] ['battery'])]
91     for key,model in tables['models'].items():
92         for date,s in model['selected'].items():
93             text+=['\n## '+key+
            ↳ '+date,markdown_table(['时间段','0点计划购电量q','最终合同购电量r'],s['purchase']),
                markdown_table(['项目','数值'],[[k,v] for k,v in s['daily'].items() if k!='date']),
                markdown_table(['日期','时间段','充电量','放电量','时刻','SOC'],[['' if v is None else v]
            ↳ for v in r] for r in s['battery']]],
                markdown_table(['日期','紧急购电时间段','紧急购电量'],s['emergency'])]
94     (ROOT/'指定日期结果表.md').write_text('\n\n'.join(text)+'\n')
95

```

```

98     dump(ROOT/'artifacts/execution-export.json',{'command':'python3 '+' '.join(sys.argv),'exit_code':0,'runti
    ↪ me_seconds':time.perf_counter()-started,'selected':selection['selected']})
99     print('Exported CSVs, workbook payload and exact selected-day tables.')
100
101 if __name__=='__main__':main()

```

src/finish.py

```

1  """Freeze executed contest results, traceable tables and scientific figures."""
2  from pathlib import Path
3  import json,shutil,hashlib,sys,time
4  import numpy as np
5  import matplotlib
6  matplotlib.use('Agg')
7  import matplotlib.pyplot as plt
8  import matplotlib.dates as mdates
9  from control import prune_cuts
10 from analyze_tail import main as analyze_tails
11
12 ROOT=Path(__file__).resolve().parents[1]
13 KINDS=['q2','q3','q4_2','q4_3']
14 LABELS={'closed_baseline':'每日闭合基线','cross_baseline':'跨日基线','closed_affine':'每日闭合仿射反馈','affine'
    ↪ '_mpc':'跨日仿射反馈','markov_mpc':'跨日Markov反馈','sddp_markov':'SDDP末值+Markov反馈'}
15 def read(f):return json.loads((ROOT/f).read_text())
16 def dump(f,x):(ROOT/f).write_text(json.dumps(x,ensure_ascii=False,indent=2,allow_nan=False)+'\n')
17 def source(kind,candidate):
18     phase='annual_sddp' if candidate.startswith('sddp') else 'annual'
19     return f'artifacts/{phase}/{kind}_{candidate}.json'
20 def table(headers,rows):
21     return '\n'.join(['| '+' | '.join(headers)+' | '+'| '.join(['---']*len(headers))+'| '+'| '+' |
    ↪ '.join(str(v) for v in r)+' | ' for r in rows])
22
23 def main():
24     begin=time.perf_counter();analyze_tails();selection=read('artifacts/selection.json');summary={'selected':
    ↪ selection['selected'],'models':{},'warmup':read('artifacts/warmup.json'),'q1':read('artifacts/q1.json')
    ↪ ',tail_shape_analysis':read('artifacts/tail-shape-analysis.json')}
25     q1=dict(np.load(ROOT/'artifacts/q1.npz'));data=dict(np.load(ROOT/'artifacts/data.npz'))
26     summary['development_sddp_comparison']={k:{'linear_markov':v['markov_mpc'],'sddp_markov':v['sddp_markov']}
    ↪ ',difference_yuan':v['sddp_markov']-v['markov_mpc']} for k,v in
    ↪ selection['development_costs'].items()
27     summary['q1']['total_cost']=float(data['day_price']@q1['q']);summary['q1']['planned_energy']=float(q1['q']
    ↪ ).sum())
28     baseline={};metrics={};rows=[];allcases={}
29     for kind in KINDS:
30         cases={c:read(source(kind,c)) for c in LABELS};allcases[kind]=cases
31         base=cases['closed_baseline']['totals']['total_cost'];selected=selection['selected'][kind]
32         summary['models'][kind]={}
33         for c,m in cases.items():
34             summary['models'][kind][c]={**m['totals'],**m['validation'],'saving_vs_daily_baseline_percent':10
    ↪ 0*(1-m['totals']['total_cost']/base),'source_artifact':source(kind,c)}
35         metric=f'{kind}_total_cost';baseline[metric]=base;metrics[metric]=cases[selected]['totals']['total_co
    ↪ st']
36         for suffix in ['json','npz']:
37             src=ROOT/source(kind,selected).replace('.json','.'+suffix);shutil.copy2(src,ROOT/f'artifacts/{kin
    ↪ d}_{suffix}')
38             rows.append([kind,LABELS[selected],f'{metrics[metric]:.2f}',f'{base:.2f}',f'{100*(1-metrics[metric]
    ↪ /base):.4f}%',f'{cases[selected]['validation']['final_inventory':.4f}"]])
39     tails=[]
40     for path in sorted((ROOT/'artifacts/tails').glob('*.json')):
41         m=json.loads(path.read_text());ev=m['evaluation']
42         tails.append({'model':path.stem,'configuration':m['configuration'],'iterations':m['iterations'],**ev,
43             'estimated_gap_percent':100*(ev['mean_policy_cost']-ev['root_lower_bound'])/ev['mean_policy_cost'],
44             'root_cut_count':len(m['root_cuts'],'active_root_cut_count':len(prune_cuts(m['root_cuts']))})
45     summary['sddp_training']=tails
46     summary['annual_sddp_difference']={k:summary['models'][k]['sddp_markov']['total_cost']-summary['models'][
    ↪ k]['markov_mpc']['total_cost'] for k in KINDS}
47     experiments={p.stem:json.loads(p.read_text()) for p in (ROOT/'artifacts/experiments').glob('*.json')}
48     assert len(experiments)>=15,'Validation experiments not completed'

```

```

49 def compact(m):return
↳ {'total_cost':m['totals']['total_cost'],'emergency_energy':m['totals']['emergency_energy'],
50 'final_inventory':m['validation']['final_inventory'],'configuration':m['configuration'],'design':m['va']
↳ ligation_design']}
51 short_order=['reference','grid161','grid641','scenarios14','horizon3','tail_zero','tail_double','terminal']
↳ 6000','stress_load_up_pv_down','pv_0only','pv_without6','pv_without12','pv_without18','variable_refer']
↳ ence','variable_stress']
52 annual_order=['annual_pv_0only','annual_pv_without6','annual_pv_without12','annual_pv_without18']
53 summary['experiments']={n:compact(experiments[n]) for n in short_order}
54 summary['annual_release_ablations']={n:compact(experiments[n]) for n in annual_order}
55 assert len(summary['experiments'])==15 and len(summary['annual_release_ablations'])==4
56 summary['global_terminal']=sorted(read('artifacts/global-terminal.json')['results'],key=lambda
↳ m:(KINDS.index(m['kind']),['affine_mpc','markov_mpc'].index(m['candidate'])))
57 for m in summary['global_terminal']:
58     if m['candidate']=='affine_mpc':
59         m['saving_vs_closed_affine_same_global_terminal']=summary['models'][m['kind']]['closed_affine']['']
↳ total_cost'-m['fixed_total_cost']
60 for m in summary['annual_release_ablations'].values():
61     m['increment_vs_markov_all_releases']=m['total_cost']-summary['models']['q3']['markov_mpc']['total_co']
↳ st']
62 summary['perfect_information_reference']={k:{z:v[z] for z in
↳ ['objective_cost_yuan','terminal_constraint','initial_inventory_kwh','final_inventory_kwh','checks']}
↳ for k,v in read('review/global-oracle.json')['results'].items()}
63 summary['scope_notes']=['Annual means February-December, with a separate common January initialization.',
64 'Primary endpoint is free; daily-closure comparisons also differ in terminal inventory.
↳ Fixed-global-end sensitivity is reported separately.',
65 'SDDP bounds concern fixed coarse auxiliary models, not the richer original contest process.',
66 'March seven-day experiments are finite subproblems, not annual savings estimates.']
67 dump('artifacts/summary.json',summary);dump('artifacts/baseline-metrics.json',baseline);dump('artifacts/m']
↳ etrics.json',metrics)
68 figure_data={'annual_comparison':{c:{k:summary['models'][k][c]['saving_vs_daily_baseline_percent'] for k
↳ in KINDS} for c in LABELS},'midnight_inventory':{}}
69 plt.rcParams.update({'font.family':'Songti
↳ SC','font.size':11,'axes.unicode_minus':False,'svg.fonttype':'none'})
70 colors=['#777777','#2B718B','#C97840','#597948','#795C91','#AB535A']
71 fig,ax=plt.subplots(figsize=(7.2,3.8),layout='constrained');x=np.arange(4);cs=['cross_baseline','closed_a']
↳ ffine','affine_mpc','markov_mpc','sddp_markov']
72 for j,c in enumerate(cs):ax.plot(x,[summary['models'][k][c]['saving_vs_daily_baseline_percent'] for k in
↳ KINDS],marker=['o','s','^','D','v','P'][j],color=colors[j],label=LABELS[c],lw=1.4)
73 ax.axhline(0,color='#444444',lw=.7);ax.set(xticks=x,xticklabels=['第二问','第三问','第四问·2','第四问·
↳ 3'],ylabel='相对每日闭合基线的费用减少 / %',title='2025年2—12月费用比较 (跨日候选年末自由)
↳ ');ax.grid(axis='y',alpha=.15);ax.legend(fontsize=9,ncol=2,loc='upper
↳ center',bbox_to_anchor=(.5,-.12))
74 savefig(fig,'年度方法比较')
75 fig,axs=plt.subplots(2,2,figsize=(7.2,4.8),layout='constrained',sharex=True,sharey=True)
76 for panel,(ax,k) in enumerate(zip(axs.ravel(),KINDS)):
77     a=dict(np.load(ROOT/f'artifacts/{k}.npz'));dates=a['dates'].astype('datetime64[D]')
78     figure_data['midnight_inventory'][k]={dates:a['dates'].tolist(),'inventory_kwh':a['state'][:,-1].to]
↳ list()}
79     ax.plot(dates,a['state'][:,-1],color='#2B718B',lw=.9);ax.axhline(6000,color='#999999',ls=':',lw=.8)
80     ax.set(title=['第二问','第三问','第四问·2','第四问·3'][panel],ylabel='午夜储能 / kWh if panel%2==0
↳ else '',ylim=(900,11100));ax.grid(alpha=.13)
81     loc=mdates.AutoDateLocator(minticks=3,maxticks=5);ax.xaxis.set_major_locator(loc);ax.xaxis.set_major_]
↳ formatter(mdates.ConciseDateFormatter(loc))
82 fig.suptitle('跨日库存连续传递, 午夜不再固定为 6000 kWh');savefig(fig,'午夜库存轨迹')
83 figure_data['sddp']=plot_sddp()
84 exp=summary['experiments'];keys=['reference','pv_0only','pv_without6','pv_without12','pv_without18']
85 fig,ax=plt.subplots(figsize=(7.2,3.4),layout='constrained');base=exp['reference']['total_cost'];vals=[exp]
↳ [k]['total_cost']-base for k in keys]
86 figure_data['short_release_ablation']=dict(zip(keys,vals));dump('artifacts/figure-data.json',figure_data)
87 ax.bar(np.arange(len(keys)),vals,color=['#2B718B']+['#C97840']*4);ax.axhline(0,color='#555',lw=.7)
88 ax.set(xticks=np.arange(len(keys)),xticklabels=['全部发布','仅0点发布','不使用6点','不使用12点','不使用18点']
↳ [''],ylabel='相对全部发布的费用变化 / 元',title='3月连续七天子问题:
↳ 保持调约权限不变的预报消融');ax.grid(axis='y',alpha=.12);savefig(fig,'预报发布消融')
89 text=['# C 题跨日随机控制建模计算交接',
90 '本目录已运行的主结果为连续SOC的跨日因果仿射计划与Markov随机动态规划反馈。通用SDDP引擎已实现,
↳ 并通过独立小树验证; 其按月训练的粗未来费用被接入相同MPC, 作为备选比较。方法在1月开发期确定, 2—
↳ 12月不再按实际费用重选。论文由用户手工撰写。',

```

91 '详细题意、假设、公式、推导、信息时序和算法限制见[模型与推导](模型与推导.md)。原附件、代码、五份工作簿、完整CSV、
 ↳ 科学图表及审查证据均保存在本目录。',

92 '## 主结果',table(['问题','开发期选定模型','费用/元','每日闭合基线/元','相对费用减少','年末库存/kWh'],rows),

93 '上述主结果允许年末库存自由; 每日闭合基线同时要求每个日末6000, 所以费用差包含中间跨日自由与一次年末库存变化,
 ↳ 不能全部解释成可持续跨日套利。1月暖启动的费用单列, 各方法均从其真实末库存6000衔接2月。',

94 f"第一问确定性LP购电费为{summary['q1']['planned_energy']:.4f} kWh; 保留题给首末6000条件。",## 完整候选对照'
 ↳ 购电量为{summary['q1']['planned_energy']:.4f} kWh; 保留题给首末6000条件。",## 完整候选对照'

95 text.append(table(['方法']+KINDS,[LABELS[c]]+f"{summary['models'][k][c]['total_cost']:.2f}" for k in
 ↳ KINDS] for c in LABELS)))

96 text.append(table(['问题','同Markov反馈下SDDP末值减线性末值费用/元'],[[k,f'{v:.4f}'] for k,v in
 ↳ summary['annual_sddp_difference'].items()))

97 text += ['表中六种候选均实际独立运行整段时间序列, 没有事后逐日挑最低费用。
 ↳ SDDP末值+仿射反馈也已参加1月开发期比较; 因其开发期费用不占优, 全年备选保留SDDP末值+Markov反馈。
 ↳ SDDP未被预设为优胜者; 其数字反映当前辅助随机模型、训练程度和尾值接入方式, 不是对SDDP算法一般优劣的证明。',

98 '## 统一年末库存的跨日对照',

99 table(['问题','跨日反馈','自由年末费用/元','年末6000费用/元','终端约束增量/元'],[m['kind'],LABELS[m['cand']
 ↳ idate]],f"{m['free_total_cost']:.2f}",f"{m['fixed_total_cost']:.2f}",f"{m['terminal_constraint_i']
 ↳ ncrement']:.2f}"] for m in summary['global_terminal'])),

100 '此处仅在12月31日未施加6000, 所有中间午夜仍连续自由。程序复用前332天, 并重置最后两天; 先复现自由末端后缀、
 ↳ 验证每个决策和状态与原轨迹一致, 再增加真正的全年末约束。
 ↳ 复用依据是本实现的两午夜前瞻及此前不活跃的期末可达约束, 不能推广为任意长时域最优策略的结论。',

101 table(['问题','每日闭合仿射费用/元','跨日仿射且年末6000费用/元','同末端费用减少/元'],[m['kind'],f"{summar
 ↳ y['models'][m['kind']]['closed_affine']['total_cost']:.2f}",f"{m['fixed_total_cost']:.2f}",f"{m['
 ↳ saving_vs_closed_affine_same_global_terminal']:.2f}"] for m in summary['global_terminal'] if
 ↳ m['candidate']=='affine_mpc']),

102 '这组对照保持仿射策略、情景数、信息数、前瞻时域和真实年末库存一致, 用于判断移除中间午夜闭合的实际效果;
 ↳ 不能以主策略相对基线的全部收益代替这一单因素比较。',

103 '## 完美信息乐观参考',

104 table(['电价与真实末端','全时域LP费用/元'],[n,f"{m['objective_cost_yuan']:.2f}"] for n,m in
 ↳ summary['perfect_information_reference'].items()),

105 '这四个LP预先知道全部实际负载、光伏和电价, 采用与回放相同的期初及对应期末约束。它们是不可实施的乐观费用参考;
 ↳ 同末端策略费用相对它们的差距同时含信息限制、模型近似和策略类限制, 不能称为求解器或SDDP最优性间隙。独立原始
 ↳ 一对偶核验见review/global-oracle.json。',

106 '## 全年预报发布消融',

107 table(['实验','2—12月费用/元','相对同Markov反馈全部发布增量/元'],[n,f"{m['total_cost']:.2f}",f"{m['inc
 ↳ rement_vs_markov_all_releases']:.2f}"] for n,m in summary['annual_release_ablations'].items()),

108 '此处均保留0、6、12、18点合同调整权限, 只移除指定官方预报; 消融沿全年连续运行。
 ↳ 增量描述当前策略使用该信息后的实测效果, 不等于所有最优策略下的严格信息价值。',

109 '第三问“其他时刻的预报”若指现有四个发布时点之外增加发布频率, 附件并未提供这些额外时点的预报序列。
 ↳ 删除已有发布的消融不能识别新增发布的收益; 需要新增时点的带时间戳预报、误差分布及更新成本,
 ↳ 再按同一信息时序对照。当前结果只支持评估已有发布的利用效果。',

110 '## 参数、预报与压力验证',

111 '以下是预先固定的3月14—20日连续七天子问题, 均独立从6000开始, 除期末6000对照外允许末端自由;
 ↳ 不能把七天差值外推成年度收益。预报消融只改变可用官方发布, 合同调整权限保持一致。',

112 table(['实验','费用/元','紧急电量/kWh','末库存/kWh'],[n,f"{m['total_cost']:.4f}",f"{m['emergency_energ
 ↳ y']:.4f}",f"{m['final_inventory']:.4f}"] for n,m in exp.items()),

113 '压力情景在窗口内使实际负载上升10%、实际光伏下降20%, 既有官方预报保持不变, 后续自建预测可利用已经观察的变化。
 ↳ 这是计算反事实, 不是原附件另有这些观测。',

114 '## SDDP 完成范围',

115 table(['问题','1月开发期线性末值Markov/元','SDDP末值Markov/元','SDDP减线性/元'],[[k,f'{v['linear_markov']
 ↳ :.4f}'],f'{v['sddp_markov']:.4f}'],f'{v['difference_yuan']:.4f}"] for k,v in
 ↳ summary['development_sddp_comparison'].items()),

116 '上述开发期为1月25—31日。费用在0.01%以内的差异按预定规则优先采用较简单的线性末值Markov反馈,
 ↳ 不能把接近浮点精度的差异写成算法改进。全年费用比较见前表。',

117 '通用引擎支持本轮已验证的有限时域、有限阶段独立噪声、连续状态凸线性问题。合同在前置阶段确定,
 ↳ 电池在交付噪声揭示后决策。独立小树对照、全路径策略枚举与割检查见review/sddp-checks.json。',

118 f"正式回放使用的{summary['tail_shape_analysis']['count']}个三日月模型,
 ↳ 在允许库存区间内各有{summary['tail_shape_analysis']['minimum_active_cuts']}—
 ↳ {summary['tail_shape_analysis']['maximum_active_cuts']}条有效根割,
 ↳ 其中{summary['tail_shape_analysis']['single_segment_models']}个为单段。
 ↳ 这里描述的是500次迭代后已得到的下界形状, 不能据此认定真实最优值函数形状。
 ↳ 详见artifacts/tail-shape-analysis.json。",

119 '每月辅助模型只用此前完整日拟合, 两小时聚合、通常三天时域、500次迭代; 真实年末或开发子问题末点前只剩一两天时,
 ↳ 使用相应缩短的辅助模型, 真实末点继续费用为零。其根下界与64条独立模拟策略费用用artifacts/summary.json。
 ↳ 模拟均值及区间不是确定上界; 部分模型可能仍有训练间隙。当前原数据比较使用SDDP末值增强MPC,
 ↳ 不冒称已经直接精确求解全年多阶段随机问题。',

120 '## 写作与结果追溯',

```

121     '指定日期的购电、六段充放电、首末SOC和全部紧急事件见[指定日期结果表](指定日期结果表.md)。
    ↪ 计算结果目录提供五份附件5格式工作簿及逐10分钟CSV，调整量表填写最终生效合同量r，实际取电量为r+紧急电量，
    ↪ 不能将q与r相加。'，
122     '所有可引用结果由本地程序产生并登记至artifacts/result-registry.json。主要限制：经验场景数有限；
    ↪ 前瞻仿射类不模拟未来官方预报创新；Markov下层使用名义未来合同及三箱近似；
    ↪ SDDP辅助模型进行了时间聚合和阶段独立化；没有对电池老化和配网暂态作无数据支撑的扩展。'，
123     '数据仅含2025年一个历史年度，正式比较为2—12月334日的连续回放。它不能证明跨年泛化、
    ↪ 稳态平均成本或未来年度稳定收益。延长运行期与增加独立年份的验证仍需额外数据。'，
124     '独立审查与模拟评分见 review。评分只针对这份建模计算交接，不是尚未手工撰写的论文分数，也不是获奖承诺。']
125     (ROOT/'建模计算交接.md').write_text('\n\n'.join(text)+'\n')
126     dump('artifacts/execution-finish.json',{'command':'python3 '+'
    ↪ '.join(sys.argv),'exit_code':0,'runtime_seconds':time.perf_counter()-begin})
127     print(json.dumps(metrics),flush=True)
128
129 def savefig(fig,name):
130     for ax in fig.axes:ax.spines[['top','right']].set_visible(False)
131     for ext in
    ↪ ['png','svg']:fig.savefig(ROOT/f'figures/{name}.{ext}',dpi=300,bbox_inches='tight',facecolor='white')
132     plt.close(fig)
133
134 def plot_sddp():
135     tail=read('artifacts/tails/q2_31.json');fig,axs=plt.subplots(1,2,figsize=(7.2,3.5),layout='constrained')
136     tr=tail['trace'];axs[0].plot([z['iteration'] for z in tr],[z['root_lower_bound']/1e4 for z in
    ↪ tr],color='#2B718B',label='辅助模型根下界')
137     ev=tail['evaluation'];ci=np.array(ev['normal_approximation_95_interval'])/1e4
138     axs[0].axhspan(*ci,color='#C97840',alpha=.16);axs[0].axhline(ev['mean_policy_cost']/1e4,color='#C97840',l
    ↪ s='--',label='策略均值 (95%近似区间)')
139     axs[0].set(xlabel='SDDP训练迭代',ylabel='三天辅助模型期望费用 /
    ↪ 万元',title='模型下界与独立模拟估计');axs[0].legend(fontsize=9)
140     E=np.linspace(1200,10800,401);cuts=prune_cuts(tail['root_cuts']);v=np.max([a+b*E for a,b in
    ↪ cuts],axis=0);v0=max(a+b*6000 for a,b in cuts)
141     result={'trace':tr,'evaluation':ev,'inventory_kwh':E.tolist(),'relative_value_yuan':(v-v0).tolist(),'tail'
    ↪ '_source':'artifacts/tails/q2_31.json'}
142     axs[1].plot(E,v-v0,color='#2B718B');axs[1].axvline(6000,color='#999999',ls=':');axs[1].set(xlabel='进入未
    ↪ 来模型的库存 / kWh',ylabel='相对6000 kWh的继续费用 / 元',title='清空合同节点的SDDP库存价值')
143     for ax in axs:ax.grid(alpha=.15)
144     savefig(fig,'SDDP 训练与库存价值')
145     return result
146
147 if __name__=='__main__':main()

```

src/freeze_evidence.py

```

1  """Bind existing executed artifacts; never substitute missing evidence with claims."""
2  from pathlib import Path
3  import json,hashlib,platform,sys
4  ROOT=Path(__file__).resolve().parents[1]
5  def read(p):return json.loads((ROOT/p).read_text())
6  def dump(p,x):(ROOT/p).write_text(json.dumps(x,ensure_ascii=False,indent=2,allow_nan=False)+'\n')
7  def sha(p):return hashlib.sha256(p.read_bytes()).hexdigest()
8  def main():
9      m=read('modeling-manifest.json');summary=read('artifacts/summary.json')
10     annual=read('artifacts/execution-annual.json');sddp=read('artifacts/execution-annual_sddp.json')
11     assert annual['exit_code']==0 and len(annual['results'])==20
12     assert sddp['exit_code']==0 and len(sddp['results'])==4
13     assert read('artifacts/execution-train.json')['exit_code']==0
14     assert len(read('artifacts/execution-train.json')['results'])==48
15     for days in [1,2]:
16         record=read(f'artifacts/execution-train-D{days}.json');assert record['exit_code']==0 and
    ↪ len(record['results'])==8
17     for scope,count in [('short',15),('annual-releases',4)]:
18         record=read(f'artifacts/execution-experiments-{scope}.json');assert record['exit_code']==0 and
    ↪ len(record['results'])==count
19     assert all(c['status']=='pass' for c in read('artifacts/workbook-validation.json')['checks'])
20     production=read('review/production-audit.json');assert production['passed'] is True and not
    ↪ production['pending_cases'] and not production['failed_cases']
21     visual=read('artifacts/visual-qa.json');assert visual['status']=='pass'

```

```

22 # Register coherent tables as structured values; this preserves every number
23 # without manufacturing thousands of redundant scalar registry entries.
24 registry=[]
25 def add(rid,path,pointer,value):registry.append({'id':rid,'source_artifact':path,'source_path':pointer,'v'
↪ alue':value,'status':'verified'})
26 for name in ['summary','handoff-tables','selection','figure-data']:
27     path=f'artifacts/{name}.json';obj=read(path)
28     for key,value in obj.items():
29         if key in ['source_hashes','scope_notes','policy']:continue
30         add(name+'.'+key,path,'/'+key.replace('~','-0').replace('/','-1'),value)
31 for name in ['sddp-checks','controller-checks','global-oracle','v2-boundary-review','production-audit']:
32     path=f'review/{name}.json'
33     if (ROOT/path).exists():
34         obj=read(path)
35         for key in ['checks','results','metrics','expected_cases','passed_cases','cases']:
36             if key in obj:add(name+'.'+key,path,'/'+key,obj[key])
37 dump('artifacts/result-registry.json',{'schema_version':1,'scope':'Executed modeling handoff. Structured
↪ table records include every displayed numerical value. No paper authored.','results':registry})
38 m['framing'].update({'objectives':['Minimize Q1 deterministic purchase cost and Q2-4 realized
↪ chronological settlement cost, with physically continuous cross-day inventory.'],
39     'decision_variables':['Original q and effective r per delivery interval','Bus charge/discharge,
↪ inventory, emergency purchase and total spill','Causal affine policy coefficients and approximate
↪ future values'],
40     'constraints':['SOC 1200..10800 kWh, each-direction efficiency0.9, power5000 kW','10-minute balance,
↪ nonnegative purchases, no simultaneous charge/discharge after canonicalization','Allowed contract
↪ revisions only; first6hours unadjustable; original daily q retained for settlement','Q1 daily
↪ terminal6000; Q2-4 main free year-end, exact physical continuity and separate global6000
↪ comparison','Scenario and release nonanticipativity'],
41     'ambiguities':['Interval-end time labels; single-direction90% efficiency; refund interpretation; final
↪ net versus cumulative settlement. See 模型与推导.md'],
42     'assumptions':['Current net load observed before battery action','Free total surplus disposal, no sale;
↪ unspecified degradation/network constants omitted','January common no-battery emergency warmup;
↪ separate cost','Finite empirical scenario approximation and restricted affine/Markov policies']})
43 descriptions={
44     'cross_baseline':('Common-path SAA with greedy physical feedback','Ignores stochastic feedback
↪ effects'),
45     'affine_mpc':('Causal affine-policy SAA-MPC','Restricted affine policy may perform poorly outside
↪ empirical scenarios'),
46     'markov_mpc':('Causal affine planning with Markov stochastic DP battery feedback','Three error bins and
↪ nominal future contracts approximate the full process'),
47     'sddp_markov':('Generic SDDP auxiliary value with the same Markov feedback','Coarse independent-stage
↪ tail can miss fine-resolution forecast dependence'})
48 m['models']['candidates']=[{'name':k,'target_output':v[0],'failure_mode':v[1],'diagnostic':'January
↪ development comparison and complete held-out chronological replay; physical, information and
↪ exact-small-instance audits.','assumptions':['Same accounting and information rules','Details and
↪ limited policy scope in 模型与推导.md'],'data_requirements':['Original load/PV/price history','Released
↪ official forecast where allowed']} for k,v in descriptions.items()]
49 selected=set(summary['selected'].values());assert len(selected)==1,'Update manifest model mapping for a
↪ mixed selection.'
50 m['models']['selected']=next(iter(selected))
51 m['models']['selection_rationale'].update({'objective_fit':'Exact settlement accounting in executed
↪ historical evaluation; January-only choice with0.01% tie preference.','constraint_fit':'Hard
↪ power/SOC/nonanticipation and contract release restrictions, physically carried
↪ inventory.','data_fit':'Chronological mature historical errors and released24h
↪ forecasts.','baseline':'Matched daily-closure and cross-day common-path SAA baselines, with global
↪ terminal comparisons.','risks':['Empirical model misspecification','Approximate tail value','One
↪ historical year, no population optimality guarantee'],'fallback_trigger':'Any physical/information
↪ audit failure blocks delivery; select only from executed January-feasible candidates, never switch
↪ after observing annual costs.'})
52 ids=[k+'_total_cost' for k in ['q2','q3','q4_2','q4_3']]
53 m['models']['baseline'].update({'status':'completed','name':'closed_baseline','command':annual['command']}
↪ , 'exit_code':0,'runtime_seconds':sum(r['runtime_seconds'] for r in annual['results']) if
↪ r['candidate']=='closed_baseline'),'metric_ids':ids})
54 figures=[str(p.relative_to(ROOT)) for p in sorted((ROOT/'figures').glob('*')) if p.suffix in
↪ ['.png','.svg']]

```

```

55 m['execution'].update({'status':'completed','command':annual['command'],'exit_code':0,'runtime_seconds':a
    ↳ nnual['runtime_seconds'],'environment':annual['environment'],'seed':None,'deterministic_reason':'Prim
    ↳ ary replay uses deterministic chronological scenarios and LP/DP. SDDP training seed20260911 and
    ↳ independent evaluation seed9182 are recorded separately.','metric_ids':ids,'figures':figures,'additio
    ↳ nal_execution_records':['artifacts/execution-annual_sddp.json','artifacts/execution-train.json','arti
    ↳ facts/execution-train-D1.json','artifacts/execution-train-D2.json','artifacts/global-terminal.json']})
56 m['validation'].update({'status':'completed','checks':[{'name':n,'status':'pass','evidence':p} for n,p in
    ↳ [(['physical_and_settlement','artifacts/summary.json'),('independent_36_trajectories','review/producti
    ↳ on-audit.json'),('independent_small_tree_sddp','review/sddp-checks.json'),('causality_and_dp','review
    ↳ /controller-checks.json'),('global_oracle','review/global-oracle.json'),('workbook_reconciliation','a
    ↳ rtifacts/workbook-validation.json'),('visual_review','artifacts/visual-qa.json')]]},
57 'sensitivity':['artifacts/summary.json#/experiments: grids161/321/641, scenarios7/14, horizon2/3,
    ↳ tail0/1/2, fixed global endpoint'],
58 'robustness':['artifacts/experiments/stress_load_up_pv_down.json','artifacts/experiments/variable_stres
    ↳ s.json'],
59 'falsification':['Future actual and unreleased forecast perturbations leave prior decisions
    ↳ unchanged','Independent explicit c/d LP versus convex infimal convolution','Full-path SDDP strategy
    ↳ enumeration and lower-cut validity','Free suffix replay before global terminal intervention'],'Annual
    ↳ release ablations keep adjustment rights fixed']})
60 findings_path=ROOT/'review/findings.json'
61 findings=read('review/findings.json') if findings_path.exists() else {}
62 if findings.get('decision')=='pass' and findings.get('independent') is True and not
    ↳ findings.get('unresolved'):
63     reviewed=findings.get('reviewed_artifact_hashes',{})
64     assert reviewed,'Final review must bind the frozen source, numerical results, documents and exported
    ↳ workbooks.'
65     for name,expected in reviewed.items():assert sha(ROOT/name)==expected,('Stale independent
    ↳ review',name)
66     m['review'].update({'status':'completed','independent':True,'reviewer_id':findings['reviewer_id']})
67     m['project'].update({'status':'complete','stage':'modeling_handoff'});m['pending']=[]
68 else:
69     m['project']['status']='in_progress';m['pending']=['Final independent frozen-artifact review']
70 dump('modeling-manifest.json',m)
71 outputs={}
72 for folder in ['artifacts','figures','计算结果','review','src']:
73     for p in (ROOT/folder).rglob('*'):
74         if p.is_file() and '__pycache__' not in p.parts and p.name not in
            ↳ ['reproducibility.json','acceptance.json'] and p.suffix not in ['.pyc']:
75             outputs[str(p.relative_to(ROOT))]=sha(p)
76 for p in ROOT.iterdir():
77     if p.is_file() and p.suffix in ['.md','.sh','.txt']:outputs[p.name]=sha(p)
78 commands={p.name:read(str(p.relative_to(ROOT))) for p in (ROOT/'artifacts').glob('execution-*.json')}
79 dump('artifacts/reproducibility.json',{'command':m['execution']['command'],'seed':m['execution']['seed'],
    ↳ 'input_hashes':m['source']['input_hashes'],'output_hashes':outputs,'execution_records':commands,
80 'library_versions':read('artifacts/library-versions.json'),
81 'source_versions':read('artifacts/source-version-records.json'),
82 'source_hash_semantics':'Early command source_hashes were sampled from disk at completion, not a
    ↳ snapshot of modules already loaded in workers. The hash-verified source-v1 archive binds the
    ↳ original annual non-SDDP, three-day tail and short-validation workers. Later SDPP comparisons use
    ↳ the finite-end v2 correction. Q1 was rerun with zero turnover regularization; settlement code was
    ↳ unchanged. Current sources reproduce the non-SDDP numerical defaults.',
83 'full_reproduction':'复现.sh; exact run records retain historical command spelling. Reviews and visual
    ↳ acceptance must be repeated after regeneration.})
84 print('Frozen evidence; status:',m['project']['status'])
85 if __name__=='__main__':main()

```

src/package_when_ready.py

```

1 """Assemble completed numerical runs and export the five requested result templates."""
2 from pathlib import Path
3 import subprocess,time,json,sys,hashlib,shutil
4 ROOT=Path(__file__).resolve().parents[1]
5 NODE=Path(shutil.which('node') or 'node')
6 PYTHON=Path(sys.executable)
7 def run(args):
8     started=time.perf_counter();print('START',args,flush=True)
9     subprocess.run(args,cwd=ROOT,check=True)

```

```

10     return {'command':args,'exit_code':0,'runtime_seconds':time.perf_counter()-started}
11 def main():
12     assert len(json.loads((ROOT/'artifacts/execution-annual.json').read_text()))['results']==20
13     assert len(json.loads((ROOT/'artifacts/execution-annual_sddp.json').read_text()))['results']==4
14     payload=ROOT/'artifacts/workbook-payload.json';expected_payload=hashlib.sha256(payload.read_bytes()).hexdigest()
15     ↪ igest()
16     logs=[]
17     logs.append(run([str(NODE),'src/build_workbooks.mjs']))
18     logs.append(run([str(PYTHON),'src/verify_workbooks.py']))
19     while True:
20         try:
21             m=json.loads((ROOT/'artifacts/global-terminal.json').read_text())
22             if len(m['results'])==8 and m['execution']['exit_code']==0:break
23         except (OSError,ValueError,KeyError):pass
24         time.sleep(10)
25     logs.append(run([sys.executable,'src/finish.py']));logs.append(run([sys.executable,'src/export_results.py']
26     ↪ '']))
27     assert hashlib.sha256(payload.read_bytes()).hexdigest()==expected_payload,'Workbook payload changed after
28     ↪ early export; rebuild the workbooks and reconcile.'
29     logs.append(run([str(PYTHON),'src/verify_workbooks.py']))
30     (ROOT/'artifacts/execution-package.json').write_text(json.dumps({'results':logs,'exit_code':0,'runtime_se
31     ↪ conds':sum(x['runtime_seconds'] for x in logs)},ensure_ascii=False,indent=2)+'\n')
32     print('EXPORTED AND NUMERICALLY CHECKED; visual and independent final review required.',flush=True)
33 if __name__=='__main__':main()

```

src/prepare_data.py

```

1  """Read-only Excel intake. Run with bundled Python (openpyxl, numpy)."""
2  from pathlib import Path
3  import json, hashlib, shutil, datetime, platform
4  import numpy as np
5  import openpyxl
6
7  ROOT = Path(__file__).resolve().parents[1]
8  SOURCE = ROOT.parents[1] / 'C 题'
9
10 def dump(path, obj):
11     path.write_text(json.dumps(obj, ensure_ascii=False, indent=2)+'\n')
12
13 def rows(path, sheet=0):
14     w = openpyxl.load_workbook(path, read_only=True, data_only=True)
15     s = w.worksheets[sheet] if isinstance(sheet, int) else w[sheet]
16     out = list(s.values)
17     w.close()
18     return out
19
20 def main():
21     inputs = ROOT/'inputs'
22     for p in SOURCE.rglob('*'):
23         if p.is_file() and p.suffix in ['.xlsx','.pdf']:
24             dest=inputs/p.relative_to(SOURCE); dest.parent.mkdir(parents=True,exist_ok=True)
25             shutil.copy2(p,dest)
26     a1=rows(inputs/'附件/附件 1.xlsx')
27     a2l=rows(inputs/'附件/附件 2.xlsx','小区负载')
28     a2v=rows(inputs/'附件/附件 2.xlsx','光伏发电实际功率')
29     a3=rows(inputs/'附件/附件 3.xlsx')
30     a4=rows(inputs/'附件/附件 4.xlsx')
31     dates=[r[0].date().isoformat() for r in a2l[1:]]
32     load=np.array([r[1:] for r in a2l[1:]],float)
33     pv=np.array([r[1:] for r in a2v[1:]],float)
34     price=np.array([r[1:] for r in a4[1:]],float)
35     forecasts=np.array([r[2:] for r in a3[1:]],float).reshape(365,4,24)
36     day=np.array([r[1:] for r in a1[1:]],float)
37     assert dates==[(datetime.date(2025,1,1)+datetime.timedelta(days=i)).isoformat() for i in range(365)]
38     assert [r[0] for r in a2l[1:]]==[r[0] for r in a2v[1:]]==[r[0] for r in a4[1:]]
39     def minute(x):
40         if isinstance(x,datetime.time): return x.hour*60+x.minute
41         if x=='0:00+1': return 1440

```

```

42     h,m=str(x).split(':'); return int(h)*60+int(m)
43     time_axes=[list(a2l[0][1:]),list(a2v[0][1:]),list(a4[0][1:]),[r[0] for r in a1[1:]]]
44     assert all([minute(t) for t in axis]==list(range(10,1441,10)) for axis in time_axes)
45     for i,r in enumerate(a3[1:]):
46         if i%4==0: assert str(r[0])==f'2025-{a2l[1+i//4][0].month}-{a2l[1+i//4][0].day}'
47         assert r[1]==['0:00','6:00','12:00','18:00'][i%4]
48     np.savez_compressed(ROOT/'artifacts/data.npz',load=load,pv=pv,price=price,forecast=forecasts,
49                        day_price=day[:,0],day_load=day[:,1],day_pv=day[:,2],dates=np.array(dates))
50     stats={}
51     for name,x in [(('load',load),('pv',pv),('price',price),('forecast',forecasts),('q1',day))]:
52         stats[name]={ 'shape':list(x.shape), 'count':int(x.size), 'missing':int(np.isnan(x).sum()),
53                       'min':float(x.min()), 'max':float(x.max()), 'negative':int((x<0).sum()),
54                       'quantiles':np.quantile(x,[0,.01,.5,.99,1]).tolist()}
55     assert np.isfinite(x).all() and (x>=0).all()
56     def corr_daily(x):
57         return [float(np.corrcoef(x[:-lag].ravel(),x[lag:].ravel())[0,1]) for lag in [1,7]]
58     stats['lag_correlations']={n:corr_daily(x) for n,x in [(('load',load),('pv',pv),('price',price))]:
59     stats['daily_load_energy_range']=[float(load.sum(1).min()/6),float(load.sum(1).max()/6)]
60     stats['daily_pv_energy_range']=[float(pv.sum(1).min()/6),float(pv.sum(1).max()/6)]
61     audit={'status':'completed', 'stats':stats, 'date_range':[dates[0],dates[-1]], 'evaluation_days':334,
62           'checks':[{'name':n, 'status':'pass'} for n in
63                     ↪ ['finite_nonnegative_values', '365_unique_consecutive_days',
64                       '144_aligned_samples_per_day', 'forecast_4_releases_24_leads', 'source_copies_hashed']],
65           'issues':[{'type':'time_label_mismatch', 'evidence':'Inputs 00:10..24:00; template
66                     ↪ 00:10-00:20..next-day00:10.',
67                     'resolution':'Treat input as right endpoint / interval mean of preceding 10min; correct
68                     ↪ output copy headers.'}],
69           'units':{'load':'kW', 'pv':'kW', 'forecast':'kW', 'price':'CNY/kWh', 'time_step_hours':1/6},
70           'data_policy':'No deletion or imputation. Repeated zero night PV is structural, not missing. Mixed
71                     ↪ Excel time/text types normalized to minutes.',
72           'coverage':'January is calibration; February-December strictly prequential. Raw future data only
73                     ↪ for scoring.'}
74     dump(ROOT/'artifacts/data-audit.json',audit)
75     m=json.loads((ROOT/'modeling-manifest.json').read_text())
76     paths=sorted(p.relative_to(ROOT).as_posix() for p in inputs.rglob('*') if p.is_file())
77     m['source'].update(statement_files=['inputs/C題.pdf'],data_files=[p for p in paths if
78                               ↪ p.endswith('.xlsx')],
79                       input_hashes={p:hashlib.sha256((ROOT/p).read_bytes()).hexdigest() for p in paths},
80                       official_rules={'status':'verified', 'url':'https://www.mcm.edu.cn/html_cn/node/4cd596519c9eb9fbd86639'
81                               ↪ '8f6df0caa3.html',
82                               'version':'2026 revision; modeling handoff only, no competition submission
83                               ↪ claimed', 'reason':None})
84     m['source']['official_statement_url']='https://www.mcm.edu.cn/html_cn/node/27b6e148f8113f09b0269f64a02629'
85     ↪ 'fb.html'
86     m['project']['stage']='modeling'
87     dump(ROOT/'modeling-manifest.json',m)
88     print(json.dumps(stats,ensure_ascii=False))
89
90 if __name__=='__main__': main()

```

src/sddp.py

```

1  """Finite-horizon, risk-neutral SDDP for general convex linear stage models.
2
3  The engine only requires solve(state, noise, next_cuts) -> objective, outgoing
4  state, realized stage cost, input-state dual. BatteryStage is one application.
5  Stagewise-independent noises are observed at stage entry. Here-and-now
6  contracts have their own preceding deterministic stages.
7  """
8  from pathlib import Path
9  from dataclasses import dataclass
10 import sys,json,time,hashlib,argparse
11 import numpy as np
12 from scipy.sparse import coo_matrix,vstack,csr_matrix
13 sys.path.insert(0,str(Path(__file__).resolve().parents[1]/'vendor'))
14 import highsipy
15
16 @dataclass

```

```

17 class StageResult:
18     objective:float
19     state:np.ndarray
20     stage_cost:float
21     gradient:np.ndarray
22
23 class LinearStage:
24     """General LP stage with explicit incoming-state fixing equalities.
25
26     Variables start with x_in, x_out, then stage controls, final theta.
27     The user supplies base matrix/bounds/cost for each finite noise outcome.
28     Costs must exclude theta, which the engine adds with coefficient one.
29     """
30     def __init__(self,dimension,outcomes):
31         self.dimension=dimension;self.outcomes=outcomes
32         self.probabilities=np.array([o['probability'] for o in outcomes])
33         assert np.isclose(self.probabilities.sum(),1)
34     def solve(self,state,noise,next_cuts):
35         o=self.outcomes[noise];D=self.dimension;N=len(o['cost']);theta=N-1
36         fix=coo_matrix((np.ones(D),(np.arange(D),np.arange(D))),shape=(D,N)).tocsr()
37         mats=[fix,o['matrix']];lower=[state,o['row_lower']];upper=[state,o['row_upper']]
38         if next_cuts:
39             coeff=np.array([c[1] for c in next_cuts]);rows,cols=np.nonzero(coeff)
40             mat=coo_matrix((np.r_[coeff[rows,cols],-np.ones(len(next_cuts))],(np.r_[rows,np.arange(len(next_c_
41             ↪ uts)]),np.r_[D+cols,np.full(len(next_cuts),theta)])),shape=(len(next_cuts),N)).tocsr()
42             mats.append(mat);lower.append(np.full(len(next_cuts),-np.inf));upper.append(-np.array([c[0] for c
43             ↪ in next_cuts]))
44             A=vstack(mats,format='csr');lo=np.concatenate(lower);hi=np.concatenate(upper)
45             p=highspy.HighsLp();p.num_col_=N;p.num_row_=len(lo);p.col_cost_=o['cost'].copy();p.col_cost_[theta]=1.
46             p.col_lower_=o['lower'];p.col_upper_=o['upper'];p.row_lower_=lo;p.row_upper_=hi
47             p.a_matrix_.format_=highspy.MatrixFormat.kRowwise;p.a_matrix_.start_=A.indptr;p.a_matrix_.index_=A.in
48             ↪ dices;p.a_matrix_.value_=A.data
49             h=highspy.Highs();h.setOptionValue('output_flag',False);h.setOptionValue('threads',1);h.passModel(p);
50             ↪ h.run()
51             if h.getModelStatus()!=highspy.HighsModelStatus.kOptimal:raise
52             ↪ RuntimeError(h.modelStatusToString(h.getModelStatus()))
53             sol=h.getSolution();x=np.array(sol.col_value)
54             return StageResult(h.getObjectiveValue(),x[D:2*D],float(o['cost']@x),np.array(sol.row_dual[:D]))
55
56 class SDDP:
57     def __init__(self,stages,seed=20260911):
58         self.stages=stages;self.dimension=stages[0].dimension;self.rng=np.random.default_rng(seed);self.seed=
59         ↪ seed
60         self.cuts=[[] for _ in range(len(stages)+1)];self.trace=[]
61     def forward(self,initial):
62         state=np.array(initial,dtype=float);inputs=[];cost=0.
63         for t,stage in enumerate(self.stages):
64             inputs.append(state.copy());k=int(self.rng.choice(len(stage.outcomes),p=stage.probabilities))
65             sol=stage.solve(state,k,self.cuts[t+1]);state=sol.state;cost+=sol.stage_cost
66         return inputs,cost
67     def backward(self,inputs):
68         for t in range(len(self.stages)-1,-1,-1):
69             x=inputs[t];stage=self.stages[t]
70             vals=[stage.solve(x,k,self.cuts[t+1]) for k in range(len(stage.outcomes))]
71             value=sum(p*v.objective for p,v in zip(stage.probabilities,vals))
72             beta=sum(p*v.gradient for p,v in zip(stage.probabilities,vals));alpha=float(value-beta*x)
73             self.cuts[t].append((alpha,beta))
74     def bound(self,initial):
75         stage=self.stages[0]
76         return float(sum(p*stage.solve(np.array(initial),k,self.cuts[1]).objective for k,p in
77         ↪ enumerate(stage.probabilities)))
78     def train(self,initial,iterations=50,exploration=None):
79         for it in range(iterations):
80             x=np.array(initial,dtype=float)
81             if exploration is not None:x=exploration(it,x)
82             path,cost=self.forward(x);self.backward(path)
83             if it==0 or (it+1)%10==0 or it==iterations-1:self.trace.append({'iteration':it+1,'root_lower_boun
84             ↪ d':self.bound(initial),'forward_sample_cost':cost})
85         return self

```

```

78     def evaluate(self, initial, count=128, seed=9182):
79         old=self.rng;self.rng=np.random.default_rng(seed)
80         costs=np.array([self.forward(initial)[1] for _ in range(count)]);self.rng=old
81         se=float(costs.std(ddof=1)/np.sqrt(count))
82         return {'samples':count, 'seed':seed, 'mean_policy_cost':float(costs.mean()), 'standard_error':se,
83                 'normal_approximation_95_interval':[float(costs.mean()-1.96*se),float(costs.mean()+1.96*se)],
84                 'root_lower_bound':self.bound(initial), 'note':'Monte Carlo estimate, not a deterministic upper
            ↪ bound.'}
85
86 def battery_stage(K, kind, hour, net_values=None, prices=None, probabilities=None):
87     D=1+2*K;N=2*D+3;em=2*D;bill=em+1;theta=bill+1
88     # Contract states contain all K deliveries; delivered entries are set to zero.
89     rows=[];cols=[];vs=[];rl=[];ru=[]
90     def row(items, lo, hi):
91         r=len(rl)
92         for c,v in items.items():rows.append(r);cols.append(c);vs.append(v)
93         rl.append(lo);ru.append(hi)
94     def copy(i):row({D+i:1,i:-1},0,0)
95     q=lambda h:1+h
96     r=lambda h:1+K+h
97     if kind=='sign':
98         copy(0)
99         for h in range(K):row({D+r(h):1,D+q(h):-1},0,0)
100    elif kind=='revise':
101        copy(0)
102        for h in range(K):
103            copy(q(h))
104            if h<hour:copy(r(h))
105    else:
106        dt=24/K;max_bus=5000*dt
107        row({D:1,0:-1},-max_bus/.9,.9*max_bus)
108        for h in range(K):
109            if h==hour:
110                row({D+q(h):1},0,0);row({D+r(h):1},0,0)
111            else:copy(q(h));copy(r(h))
112        row({bill:1,r(hour):-1.5,q(hour):.5},0,np.inf)
113        row({bill:1,r(hour):-1.5,q(hour):-1.5},0,np.inf)
114        row({em:1,r(hour):1,D:-.9,0:.9},0,np.inf)
115        row({em:1,r(hour):1,D:-1/.9,0:1/.9},0,np.inf)
116    matrix=coo_matrix((vs,(rows,cols)),shape=(len(rl),N)).tocsr()
117    lower=np.zeros(N);upper=np.full(N,np.inf);lower[0]=lower[D]=1200;upper[0]=upper[D]=10800
118    outcomes=[]
119    if kind!='deliver':net_values=[0];prices=[0];probabilities=[1]
120    for net,p,prob in zip(net_values,prices,probabilities):
121        cost=np.zeros(N);cost[em]=5*p;cost[bill]=p
122        lowers=np.array(rl,dtype=float)
123        if kind=='deliver':lowers[-2:]=net
124        outcomes.append({'probability':float(prob), 'matrix':matrix, 'row_lower':lowers, 'row_upper':np.array(ru)
            ↪ ,dtype=float),
125                        'lower':lower, 'upper':upper, 'cost':cost})
126    return LinearStage(D,outcomes)
127
128 def fit_battery_model(data, cutoff, variable=False, adjust=False, K=12, days=3, window=28):
129     """Coarse auxiliary future model, fitted only to complete past days.
130
131     Per-stage net bins and conditional price means form a declared independent
132     finite innovation model. This model is a terminal-value proxy, not a lower
133     bound for the richer forecast-conditioned contest controller.
134     """
135     assert 144%K==0 and K%4==0 and cutoff>=7
136     hist=np.arange(max(0,cutoff-window),cutoff);width=144//K
137     net=((data['load'][hist]-data['pv'][hist])/6).reshape(len(hist),K,width).sum(2)
138     p=(data['price'][hist] if variable else
        ↪ np.repeat(data['day_price'][None,:],len(hist),axis=0)).reshape(len(hist),K,width).mean(2)
139     stages=[];emissions=[]
140     for k in range(K):
141         edges=np.quantile(net[:,k],[1/3,2/3]);labels=(net[:,k]>edges[0]).astype(int)+(net[:,k]>edges[1]).astype(int)
        ↪ pe(int)
142         vals=[];prices=[];weights=[]

```

```

143         for b in range(3):
144             ids=np.flatnonzero(labels==b)
145             if len(ids):vals.append(float(net[ids,k].mean()));prices.append(float(p[ids,k].mean()));weights.a
↪             ppend(len(ids)/len(hist))
146         emissions.append((vals,prices,weights))
147     for d in range(days):
148         stages.append(battery_stage(K,'sign',0))
149         for k in range(K):
150             if adjust and k and k%(K//4)==0:stages.append(battery_stage(K,'revise',k))
151             stages.append(battery_stage(K,'deliver',k,*emissions[k]))
152     return stages,{'cutoff_day_exclusive':cutoff,'history_days':hist.tolist(),'steps_per_day':K,'horizon_days'
↪     ':days,
153     'variable_price':variable,'adjustments':adjust,'noise_model':'Stagewise-independent net bins;
↪     conditional expected prices; hourly aggregation is an auxiliary-model approximation.'}
154
155 def train_tail(data,cutoff,variable=False,adjust=False,K=12,days=3,iterations=50,seed=20260911):
156     start=time.perf_counter();stages,meta=fit_battery_model(data,cutoff,variable,adjust,K,days)
157     model=SDDP(stages,seed);initial=np.r_[6000.,np.zeros(2*K)]
158     def exploration(i,x):x[0]=np.linspace(1200,10800,9)[i%9];return x
159     model.train(initial,iterations,exploration)
160     ev=model.evaluate(initial,count=64)
161     # Root signing resets contracts: its value depends on incoming inventory only.
162     assert max(np.max(np.abs(b[1:])) for a,b in model.cuts[0])<1e-8
163     rootcuts=[[a,float(b[0])] for a,b in model.cuts[0]]
164     return {'configuration':meta,'seed':seed,'iterations':iterations,'trace':model.trace,'evaluation':ev,
165           'root_cuts':rootcuts,'runtime_seconds':time.perf_counter()-start,
166           'scope':'Valid lower cuts for this fixed coarse auxiliary model; approximate terminal values for the
↪           contest MPC.'}
167
168 def main():
169     p=argparse.ArgumentParser();p.add_argument('--cutoff',type=int,default=31);p.add_argument('--iterations',
↪     type=int,default=50);p.add_argument('--steps',type=int,default=12);p.add_argument('--days',type=int,d
↪     efault=3);p.add_argument('--variable',action='store_true');p.add_argument('--adjust',action='store_tr
↪     ue');p.add_argument('--output',default='sddp-pilot.json');args=p.parse_args()
170     root=Path(__file__).resolve().parents[1];data=dict(np.load(root/'artifacts/data.npz'))
171     out=train_tail(data,args.cutoff,args.variable,args.adjust,args.steps,args.days,args.iterations)
172     out['execution']={'command':'python3 '+' '.join(sys.argv),'exit_code':0,'source_sha256':hashlib.sha256(Pa
↪     th(__file__).read_bytes()).hexdigest()}
173     (root/'artifacts'/args.output).write_text(json.dumps(out,ensure_ascii=False,indent=2)+'\n');print(json.du
↪     mps(out['evaluation']),flush=True)
174
175 if __name__=='__main__':main()

```

src/select_model.py

```

1  """Freeze model choice using January development results only."""
2  from pathlib import Path
3  import json,hashlib,time,sys
4  ROOT=Path(__file__).resolve().parents[1]
5  def main():
6      started=time.perf_counter()
7      result={'selection_period':[24,30],'evaluation_period':[31,364],'relative_tie_tolerance':.0001,
8            'policy':'Choose minimum January development cost; within 0.01% prefer Markov feedback with linear tail.
↪            This script never reads annual costs.',
9            'selected':{},'development_costs':{},'sddp_training_iterations_development':500,'sddp_training_iteratio
↪            ns_evaluation':500}
10     for kind in ['q2','q3','q4_2','q4_3']:
11         vals={}
12         for c in ['cross_baseline','affine_mpc','markov_mpc','sddp_mpc','sddp_markov']:
13             phase='calibrate_sddp_v2' if c.startswith('sddp') else 'calibrate'
14             m=json.loads((ROOT/f'artifacts/{phase}/{kind}_{c}.json').read_text())
15             assert m['configuration']['start_day']==24 and m['configuration']['end_day']==30
16             vals[c]=m['totals']['total_cost']
17             chosen=min(vals,key=vals.get)
18             if vals['markov_mpc']<=min(vals.values())*1.0001:chosen='markov_mpc'
19             result['selected'][kind]=chosen;result['development_costs'][kind]=vals
20     result['source_hashes']={p.name:hashlib.sha256(p.read_bytes()).hexdigest() for p in
↪     (ROOT/'src').glob('*.py')}

```

```

21     result['sddp_training_note']='Corrected development and annual auxiliary models both use500iterations,
    ↪ with finite-end1/2day truncation. Older development results remain archived.'
22     (ROOT/'artifacts/selection.json').write_text(json.dumps(result,ensure_ascii=False,indent=2)+'\n');print(r
    ↪ result['selected'])
23     (ROOT/'artifacts/execution-selection.json').write_text(json.dumps({'command':'python3 '+'
    ↪ '.join(sys.argv),'exit_code':0,'runtime_seconds':time.perf_counter()-started,'selected':result['selec
    ↪ ted'],'reads_annual_costs':False},ensure_ascii=False,indent=2)+'\n')
24 if __name__=='__main__':main()

```

src/validate.py

```

1  """Counterfactual validation and parameter sweeps, distinct from model selection."""
2  from pathlib import Path
3  from concurrent.futures import ProcessPoolExecutor,as_completed
4  import json,time,sys,argparse,hashlib
5  import numpy as np
6  import forecasting
7  from run import ROOT,simulate,dump
8
9  ORIGINAL_FORECAST=forecasting.forecast_as_of
10
11 def forecast_with_releases(allowed):
12     def restricted(data,selection,asof,targets,official,variable):
13         if not official:return ORIGINAL_FORECAST(data,selection,asof,targets,official,variable)
14         pred=ORIGINAL_FORECAST(data,selection,asof,targets,False,variable)
15         issue=max(i for i in range(max(0,(asof//144-1)*144),asof+1,36) if (i%144)//36 in allowed)
16         d,k=divmod(issue//36,4);anchor=data['pv'].ravel()[issue-1] if issue else 0.
17         mask=(targets+1<=issue+144)&(targets+1>=issue)
18         pred['pv'][mask]=np.interp(targets[mask]+1,issue+np.arange(25)*6,np.r_[anchor,data['forecast'][d,k]])
19         pred['net']=(pred['load']-pred['pv'])/6;pred['official_covered_slots']=int(mask.sum())
20     return pred
21     return restricted
22
23 def job(config):
24     data=dict(np.load(ROOT/'artifacts/data.npz'));selection=json.loads((ROOT/'artifacts/forecast-selection.js
    ↪ on').read_text())
25     kind=config.get('kind','q3');start=config.get('start',72);stop=config.get('stop',79)
26     days=list(range(start,stop));settings={'count':7,'horizon_days':2,'tail_scale':1,'grid':321,'final':None}
27     for k in list(settings):
28         if k in config:settings[k]=config[k]
29     if config.get('stress'):
30         data['load'][start:stop]*=1.1;data['pv'][start:stop]*=.8
31     forecasting.forecast_as_of=forecast_with_releases(config['releases']) if 'releases' in config else
    ↪ ORIGINAL_FORECAST
32     a,m=simulate(data,selection,days,kind,'markov_mpc',**settings)
33     m['validation_design']=config
34     out=ROOT/'artifacts/experiments';out.mkdir(exist_ok=True)
35     np.savez_compressed(out/(config['name']+'.npz'),**a);dump(out/(config['name']+'.json'),m)
36     return
    ↪ {'name':config['name'],'total_cost':m['totals']['total_cost'],'runtime_seconds':m['runtime_seconds']}
37
38 def main():
39     parser=argparse.ArgumentParser();parser.add_argument('--workers',type=int,default=4);parser.add_argument(
    ↪ '--scope',choices=['short','annual-releases'],default='short');args=parser.parse_args()
40     # These are matched finite seven-day subproblems, not annual savings estimates.
41     configs=[{'name':'reference','kind':'q3'},{'name':'grid161','grid':161},{'name':'grid641','grid':641},
42             {'name':'scenarios14','count':14},{'name':'horizon3','horizon_days':3},
43             {'name':'tail_zero','tail_scale':0.},{'name':'tail_double','tail_scale':2.},
44             {'name':'terminal6000','final':6000.},{'name':'stress_load_up_pv_down','stress':True},
45             {'name':'pv_0only','releases':[0]},{'name':'pv_without6','releases':[0,2,3]},
46             {'name':'pv_without12','releases':[0,1,3]},{'name':'pv_without18','releases':[0,1,2]},
47             {'name':'variable_reference','kind':'q4_3'},{'name':'variable_stress','kind':'q4_3','stress':True}]
48     if args.scope=='annual-releases':
49         configs=[{'name':'annual_pv_'+name,'kind':'q3','start':31,'stop':365,'releases':releases} for
    ↪ name,releases in [(('0only',[0]),('without6',[0,2,3]),('without12',[0,1,3]),('without18',[0,1,2]))]
50     start=time.perf_counter();results=[]
51     with ProcessPoolExecutor(max_workers=args.workers) as pool:
52         fs=[pool.submit(job,c) for c in configs]

```

```

53     for f in as_completed(fs):r=f.result();results.append(r);print(json.dumps(r),flush=True)
54     dump(ROOT/f'artifacts/execution-experiments-{args.scope}.json',{ 'command':'python3 '+'
    ↪ '.join(sys.argv),'exit_code':0,'runtime_seconds':time.perf_counter()-start,
55         'source_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),'results':results,
56         'scope':'February-December continuous matched release ablations.' if args.scope=='annual-releases'
    ↪ else 'Prespecified seven-day March subproblems; common initial SOC6000 and free endpoint except
    ↪ the terminal comparison. Do not extrapolate their cost differences to annual totals.'})
57
58 if __name__=='__main__':main()

```

src/verify_workbooks.py

```

1  """Read exported workbooks and reconcile them with executed numerical artifacts."""
2  from pathlib import Path
3  import json,hashlib,time
4  import openpyxl
5  import numpy as np
6  ROOT=Path(__file__).resolve().parents[1]
7  begin=time.perf_counter();checks=[]
8  payload=json.loads((ROOT/'artifacts/workbook-payload.json').read_text())
9  for name,key in
    ↪ [ ('result1',None),('result2','q2'),('result3','q3'),('result4-2','q4_2'),('result4-3','q4_3')]:
10     path=ROOT/'计算结果'/(name+'.xlsx');w=openpyxl.load_workbook(path,data_only=True,read_only=True)
11     rows=list(w['计划购电量'].values)
12     if key:
13         a=np.load(ROOT/f'artifacts/{key}.npz');m=payload['models'][key]
14         assert np.max(abs(np.array([r[1:145] for r in rows[1:335]],float)-a['q']))<1e-7
15         assert rows[0][1:145]==tuple(payload['labels'])
16         assert len(rows)==335
17         assert [r[0].strftime('%Y-%m-%d') for r in rows[1:335]]==m['dates']
18         assert all(abs(rows[i+1][145]-a['q'][i].sum())<1e-6 for i in range(334))
19         assert all(abs(rows[i+1][146]-m['daily'][i]['planned_cost'])<1e-6 for i in range(334))
20         if key.endswith('3'):
21             adjusted=list(w['调整购电量'].values)
22             assert np.max(abs(np.array([r[1:145] for r in adjusted[1:335]],float)-a['r']))<1e-7
23             for i,row in enumerate(adjusted[1:335]):
24                 assert abs(row[145]-a['r'][i].sum())<1e-6
25                 d=m['daily'][i];assert
                ↪ abs(row[146]-d['planned_cost']-d['increase_cost']-d['reduction_net_cost'])<1e-6
26         battery=list(w['充放电电量'].values)
27         assert len(battery)==2005
28         for j,(row,expected) in enumerate(zip(battery[1:],m['battery'])):
29             if j%6==0:assert row[0].strftime('%Y-%m-%d')==expected[0]
30             else:assert row[0] is None
31             assert row[1]==expected[1] and np.max(abs(np.array(row[2:4])-expected[2:4]))<1e-7
32             assert row[4]==expected[4]
33             if expected[5] is not None:assert abs(row[5]-expected[5])<1e-7
34         emergency=list(w['紧急购电量'].values)[1:]
35         assert len(emergency)==len(m['emergency'])
36         for row,expected in zip(emergency,m['emergency']):
37             assert row[0].strftime('%Y-%m-%d')==expected[0] and row[1]==expected[1] and
                ↪ abs(row[2]-expected[2])<1e-7
38         fees=list(w['费用汇总'].values);total=sum(r[5] for r in fees[1:335])
39         fields=['planned_cost','increase_cost','reduction_net_cost','emergency_cost','total_cost','planned_en
    ↪ ergy','adjusted_energy','emergency_energy','spill_energy']
40         assert np.max(abs(np.array([r[1:10] for r in fees[1:335]],float)-np.array([d[f] for f in fields] for
    ↪ d in m['daily'])))<1e-6
41         expected=json.loads((ROOT/f'artifacts/{key}.json').read_text())['totals']['total_cost']
42         assert abs(total-expected)<1e-6 and abs(fees[335][5]-expected)<1e-6
43     else:
44         q=payload['q1'];assert abs(rows[145][1]-q['daily_energy'])<1e-6
45         assert abs(rows[146][1]-q['daily_cost'])<1e-6
46         assert np.max(abs(np.array([r[1] for r in rows[1:145]])-q['q']))<1e-7
47         battery=list(w['充放电电量'].values)
48         expected=[sum(q['c'][24*b:24*b+24]),sum(q['d'][24*b:24*b+24])] for b in range(6)]
49         assert np.max(abs(np.array([r[1:3] for r in battery[1:7]],float)-expected))<1e-7
50         assert abs(battery[1][4]-q['state'][0])<1e-7 and abs(battery[2][4]-q['state'][-1])<1e-7
51     for sheet in w:

```

```

52     for row in sheet.values:
53         assert not any(isinstance(v,str) and v.startswith(('#REF!','#DIV/0!','#VALUE!','#NAME?','#N/A','#NUM!','#NULL!','#SPILL!','#CALC!')) for v in row)
54     checks.append({'file':str(path.relative_to(ROOT)),'status':'pass','sheet_count':len(w.sheetnames),'sha256':
55         ↪ ':hashlib.sha256(path.read_bytes()).hexdigest()'};w.close()
56 (ROOT/'artifacts/workbook-validation.json').write_text(json.dumps({'checks':checks,'execution':{'command':'bu
57 ↪ ndled-python
58 ↪ src/verify_workbooks.py','exit_code':0,'runtime_seconds':time.perf_counter()-begin}},ensure_ascii=False,i
59 ↪ ndent=2)+'\n')
60 print(checks)

```

src/build_workbooks.mjs

```

1 import fs from 'node:fs/promises';
2 import {FileBlob,SpreadsheetFile} from '@oai/artifact-tool';
3 const payload=JSON.parse(await fs.readFile('artifacts/workbook-payload.json','utf8'));
4 const out='计算结果';await fs.mkdir(out,{recursive:true});
5 const pairs=[['result1',null],['result2','q2'],['result3','q3'],['result4-2','q4_2'],['result4-3','q4_3']];
6 const dateValue=s=>new Date(`${s}T00:00:00Z`);
7 const style=(s,rows,cols)=>{
8     const r=s.getRangeByIndexes(0,0,rows,cols);r.format.font={name:'Arial',size:11};r.format.rowHeight=20;r.form
9     ↪ at.verticalAlignment='center';
10    s.getRangeByIndexes(0,0,1,cols).format.font={bold:true};
11    s.getRangeByIndexes(0,0,rows,1).format.columnWidth=16;
12    s.getRangeByIndexes(0,1,rows,Math.max(1,cols-1)).format.columnWidth=19;
13 };
14 for(const [file,key] of pairs){
15     if(process.argv[2] && process.argv[2]!==file) continue;
16     const wb=await SpreadsheetFile.importXlsx(await FileBlob.load(`inputs/附件/附件 5/${file}.xlsx`));
17     const qsheets=wb.worksheets.getItem('计划购电量');
18     if(!key){
19         const q=payload.q1;
20         qsheets.getRange('A2:B145').values=payload.labels.map((v,i)=>[v,q.q[i]]);
21         qsheets.getRange('A146:B147').values=[['全天购电量',null],['全天购电费',q.daily_cost]];
22         qsheets.getRange('B146').formulas=[['=SUM(B2:B145)']];qsheets.getRange('B2:B147').setNumberFormat('0.0000');
23         style(qsheets,147,2);qsheets.getRange('A1:A147').format.columnWidth=22;
24         const s=wb.worksheets.getItem('充放电电量');
25         s.getRange('B2:C7').values=Array.from({length:6},(_,b)=>[q.c.slice(b*24,b*24+24).reduce((a,v)=>a+v,0),q.d.s
26         ↪ lice(b*24,b*24+24).reduce((a,v)=>a+v,0)]);
27         s.getRange('E2:E3').values=[[q.state[0]],[q.state[144]]];s.getRange('B2:C7').setNumberFormat('0.0000');
28         style(s,7,5);
29     }else{
30         const m=payload.models[key];
31         for(const name of ['计划购电量',...(key.endsWith('3')?['调整购电量']:[])]){
32             const s=wb.worksheets.getItem(name);const isPlan=name==='计划购电量';
33             s.getRange('B1:E01').values=[payload.labels];
34             s.getRange('B2:E0335').values=isPlan?m.q:m.r;
35             s.getRange('A2:A335').values=m.dates.map(d=>[dateValue(d)]);s.getRange('A2:A335').setNumberFormat('yyyy-mm
36             ↪ -dd');
37             s.getRange('EP2').formulas=[['=SUM(B2:E02)']];s.getRange('EP2:EP335').fillDown();
38             s.getRange('EQ2:EQ335').values=m.daily.map(r=>[r.planned_cost+(isPlan?0:r.increase_cost+r.reduction_net_co
39             ↪ st)]);
40             s.getRange('B2:EQ335').setNumberFormat('0.0000');style(s,335,147);
41             s.freezePanes.freezeRows(1);s.freezeColumns(1);
42         }
43         const bs=wb.worksheets.getItem('充放电电量');bs.getUsedRange().unmerge();bs.getUsedRange().clear({applyTo:'al
44         ↪ l'});
45         bs.getRange('A1:F1').values=[['日期','时间段','充电量','放电量','时刻','储电量']];
46         bs.getRange('A2').write(m.battery.map((r,i)=>[i%6===0?dateValue(r[0]):null,...r.slice(1)]));
47         bs.getRange('A2:A${m.battery.length+1}').setNumberFormat('yyyy-mm-dd');
48         bs.getRange('C2:D${m.battery.length+1}').setNumberFormat('0.0000');
49         bs.getRange('F2:F${m.battery.length+1}').setNumberFormat('0.0000');style(bs,m.battery.length+1,6);
50         const es=wb.worksheets.getItem('紧急购电量');es.getUsedRange().clear({applyTo:'contents'});
51         es.getRange('A1:C1').values=[['日期','购电时间段','购电量']];
52         es.getRange('A2').write(m.emergency.map(r=>[dateValue(r[0]),...r.slice(1)]));
53         es.getRange('A2:A${m.emergency.length+1}').setNumberFormat('yyyy-mm-dd');es.getRange('C2:C${m.emergency.len
54         ↪ gth+1}').setNumberFormat('0.0000');
55         style(es,m.emergency.length+1,3);es.getRange('B1:B${m.emergency.length+1}').format.columnWidth=24;

```

```

50 const cs=wb.worksheets.add('费用汇总');const fields=['planned_cost','increase_cost','reduction_net_cost','e
    ↪ mergency_cost','total_cost','planned_energy','adjusted_energy','emergency_energy','spill_energy'];
51 cs.getRange('A1:J1').values=[[日期','计划费用_元','增购费用_元','退购净费用_元','紧急费用_元','总费用_元','计
    ↪ 划电量_kWh','最终合同电量_kWh','紧急电量_kWh','弃电量_kWh']];
52 cs.getRange('A2').write(m.daily.map(r=>[dateValue(r.date),...fields.map(f=>r[f])]));
53 cs.getRange('A2:A335').setNumberFormat('yyyy-mm-dd');cs.getRange('B2:J336').setNumberFormat('0.0000');
54 cs.getRange('A336').values=[[合计]];
55 for(let j=1;j<10;j++){const
    ↪ col=String.fromCharCode(65+j);cs.getCell(335,j).formulas=[[`=SUM(${col}2:${col}335)`]];
56 style(cs,336,10);cs.getRange('B1:J1').format.wrapText=true;cs.getRange('A1:J1').format.rowHeight=32;
57 }
58 const
    ↪ ns=wb.worksheets.add('口径说明');ns.getRange('A1').write(payload.notes);style(ns,payload.notes.length,2);
59 ns.getRange(`B1:B${payload.notes.length}`).format.columnWidth=115;ns.getRange(`A1:B${payload.notes.length}`)
    ↪ .format.wrapText=true;
60 ns.getRange('A1:B${payload.notes.length}`).format.rowHeight=32;
61 ns.getRange('A1:B1').format.rowHeight=22;
62 const err=await wb.inspect({kind:'match',searchTerm:'#REF!|#DIV/0!|#VALUE!|#NAME\\?|#N/A|#NUM!|#NULL!|#SPILL'
    ↪ !|#CALC!',options:{useRegex:true,maxResults:30,maxChars:1500});
63 console.log(file,'error_scan',err.ndjson);
64 console.log((await wb.inspect({kind:'table',range:'计划购电量!A1:D4',include:'values,formulas',tableMaxRows:4
    ↪ ,tableMaxCols:4,maxChars:1200})).ndjson);
65 await (await SpreadsheetFile.exportXlsx(wb)).save(`${out}/${file}.xlsx`);
66 // Reimport the saved file so previews use its final unmerged layout.
67 const saved=await SpreadsheetFile.importXlsx(await FileBlob.load(`${out}/${file}.xlsx`));
68 for(const s of saved.worksheets.items){
69   const range=s.name==='口径说明'?`A1:B${payload.notes.length}`:s.name==='计划购电量'&&
    ↪ `!key?'A1:B8':s.name==='紧急购电量'?`A1:C8':s.name==='费用汇总'?`A1:J8':`A1:F8`;
70   const blob=await saved.render({sheetName:s.name,range,range:scale:1.3,format:'png'});
71   await fs.writeFile(`artifacts/previews/${file}-${s.name}.png`,new Uint8Array(await blob.arrayBuffer()));
72   if(s.name==='调整购电量'){
73     const revised=await saved.render({sheetName:s.name,range:'A1:AS8',scale:1.3,format:'png'});
74     await fs.writeFile(`artifacts/previews/${file}-${s.name}-6点.png`,new Uint8Array(await
    ↪ revised.arrayBuffer()));
75   }
76 }
77 await fs.rm(`${out}/${file}.xlsx.inspect.ndjson`,{force:true});
78 console.log('EXPORTED',file);
79 }

```

src/preview_saved.mjs

```

1 import fs from 'node:fs/promises';
2 import {FileBlob,SpreadsheetFile} from '@oai/artifact-tool';
3 const file=process.argv[2];
4 if(!/^result(?:1|2|3|4-2|4-3)$/.test(file)) throw Error('Specify one output workbook');
5 const wb=await SpreadsheetFile.importXlsx(await FileBlob.load(`计算结果/${file}.xlsx`));
6 for(const s of wb.worksheets.items){
7   const range=s.name==='口径说明'?`A1:B15`:s.name==='计划购电量'&&file==='result1'?`A1:B8`:s.name==='紧急购电量'
    ↪ `?`A1:C8':s.name==='费用汇总'?`A1:J8':`A1:F8`;
8   const blob=await wb.render({sheetName:s.name,range,range:scale:1.3,format:'png'});
9   await fs.writeFile(`artifacts/previews/${file}-${s.name}.png`,new Uint8Array(await blob.arrayBuffer()));
10 }

```

src/preview_templates.mjs

```

1 import fs from 'node:fs/promises';
2 import {FileBlob,SpreadsheetFile} from '@oai/artifact-tool';
3 await fs.mkdir('artifacts/previews',{recursive:true});
4 for(const name of ['result1','result3']){
5   const wb=await SpreadsheetFile.importXlsx(await FileBlob.load(`inputs/附件/附件 5/${name}.xlsx`));
6   console.log((await wb.inspect({kind:'sheet',include:'id,name',maxChars:1800})).ndjson);
7   for(const s of wb.worksheets.items){
8     const range=s.name==='计划购电量'||s.name==='调整购电量'?`A1:B8':(name==='result1'?`A1:B8':`A1:G6')):`A1:F8`;
9     const blob=await wb.render({sheetName:s.name,range,range:scale:1,format:'png'});
10    await fs.writeFile(`artifacts/previews/template-${name}-${s.name}.png`,new Uint8Array(await
    ↪ blob.arrayBuffer()));

```

```

11 }
12 }

```

review/check-controllers.py

```

1 """Independent, non-annual audit; writes only this review directory.
2
3 Run: python3 review/check-controllers.py
4 Uses separate SciPy LPs with bus charge/discharge variables for Bellman checks.
5 """
6 from pathlib import Path
7 import hashlib, json, sys, time
8 import numpy as np
9 from scipy.optimize import linprog
10
11 ROOT = Path(__file__).resolve().parents[1]
12 sys.path.insert(0, str(ROOT/'src'))
13 import forecasting as fc
14 import control as ct
15 import run as replay
16
17 DATA = dict(np.load(ROOT/'artifacts/data.npz'))
18 SEL = json.loads((ROOT/'artifacts/forecast-selection.json').read_text())
19 REPORT = {'scope': 'controller_implementation_and_development_trajectories_only',
20           'reviewer_id': '/root/independent_review', 'independent': True,
21           'annual_execution_review': 'pending', 'checks': {}}
22 SOURCE = {str(p.relative_to(ROOT)): hashlib.sha256(p.read_bytes()).hexdigest()
23            for p in [ROOT/'src/forecasting.py', ROOT/'src/control.py', ROOT/'src/run.py']}
24
25 def eq(a, b, tol=1e-7):
26     error = float(np.max(np.abs(np.asarray(a)-np.asarray(b))))
27     assert error <= tol, error
28     return error
29
30 def forecast_checks():
31     evidence = []; root_checks = []
32     for day in [31, 59, 60, 150, 333]:
33         for offset in [0, 36, 72, 108, 120]:
34             now = day*144+offset; end = min((day+3)*144, 365*144)
35             targets = np.arange(now, end)
36             changed = {k: v.copy() for k, v in DATA.items()}
37             for k in ['load', 'pv', 'price']:
38                 changed[k].ravel()[now:] = 100000+np.arange(changed[k].size-now)
39             release_index = now//36
40             changed['forecast'].reshape(-1,24)[release_index+1:] += 100000
41             for official, variable in [(False,False),(True,False),(False,True),(True,True)]:
42                 original = fc.forecast_as_of(DATA, SEL, now, targets, official, variable)
43                 mutant = fc.forecast_as_of(changed, SEL, now, targets, official, variable)
44                 for k in ['load','pv','price','net']: eq(original[k], mutant[k], 0)
45                 net, prices, meta = fc.scenarios(DATA, SEL, now, end, official, variable, count=3)
46                 nn, pp, mm = fc.scenarios(changed, SEL, now, end, official, variable, count=3)
47                 eq(net,nn,0); eq(prices,pp,0); assert meta == mm
48                 assert meta['latest_training_target_exclusive'] <= now
49                 mature = [h*144+offset+end-now for h in meta['history_days']]
50                 assert max(mature) <= now
51                 if official:
52                     issue = now//36*36
53                     assert meta['official_covered_slots'] == issue+144-now
54                     d,k = divmod(issue//36,4)
55                     for hour in range(1,25):
56                         target = issue+hour*6-1
57                         if target >= now: eq(original['pv'][target-now], DATA['forecast'][d,k,hour-1], 1e-10)
58                 evidence.append({'asof':now,'official':official,'variable':variable,
59                                 'maturity_slack':now-max(mature),
60                                 'official_covered_slots':meta['official_covered_slots']})
61             if day == 60 and offset in [0,108] and official and variable:
62                 # Same root empirical problem after mutation, independently solved twice.
63                 base = None if offset == 0 else np.full(end-now,150.)

```

```

64         p1=ct.affine_plan(net,prices,now,6000.,base,True,True,.25)
65         p2=ct.affine_plan(nn,pp,now,6000.,base,True,True,.25)
66         root_checks.append({'asof':now,'q_error':eq(p1['q'],p2['q'],1e-5),
67                             'r_error':eq(p1['r'],p2['r'],1e-5)})
68     REPORT['checks']['future_actual_and_release_mutation'] =
    ↪ {'passed':True,'cases':len(evidence),'cases_detail':evidence,'root_lp_checks':root_checks}
69
70 def node_checks():
71     net=np.array([[100,150,800,1200,50,250], [100,150,50,200,1500,300],
72                  [300,400,1000,50,1200,50], [300,400,200,1500,10,800]],float)
73     prices=np.array([.2,.2,.8,1.2,.8,.3])*4)
74     findings=[]
75     for asof in [34,142,144]:
76         base=None if asof==144 else np.array([125.,130.,0,0,0,0])
77         plan=ct.affine_plan(net,prices,asof,6000.,base,True,True,.6)
78         q,r,x=map(plan.get,['scenario_q','scenario_r','scenario_state'])
79         if asof==144:
80             eq(q,np.repeat(q[:1],4,axis=0));eq(r,q)
81         else:
82             eq(q[:,:2],np.repeat(base[None,:2],4,axis=0))
83             if asof==34:
84                 eq(q,np.repeat(base[None,:],4,axis=0))
85                 for a,b in [(0,1),(2,3)]:
86                     eq(q[a,2:],q[b,2:]);eq(r[a,2:],r[b,2:]);eq(x[a,2:],x[b,2:])
87             # Root 06:00 adjustment is shared despite distinct first-slot observations.
88             findings.append({'asof':asof,'lp_violation':plan['lp_violation'],'objective_gap':plan['objective_gap']
    ↪ })
89     p=ct.affine_plan(net,prices,36,6000.,np.full(6,100.),True,True,.6)
90     eq(p['scenario_r'],np.repeat(p['scenario_r'][:1],4,axis=0))
91     REPORT['checks']['pre_release_nonanticipation']={'passed':True,'cases':findings,
92             'test':'Identical pre-release prefixes with different current/future observations have identical node
    ↪ contracts; current battery actions share identical observed prefixes.'}
93
94 def separate_lp(grid, values, E, net_minus_q, price):
95     # Variables x,c,d,e,v; independent physical bus formulation.
96     objective=[0.,0.,0.,5*price,1.]
97     A=[[0.,1.,-1.,-1.,0.]]; b=[-net_minus_q]
98     slopes=np.diff(values)/np.diff(grid)
99     for i,slope in enumerate(slopes):
100         A.append([slope,0,0,0,-1]);b.append(-(values[i]-slope*grid[i]))
101     sol=linprog(objective,A_ub=A,b_ub=b,
102                 A_eq=[[1.,-ct.ETA,1/ct.ETA,0,0]],b_eq=[E],
103                 bounds=[(grid[0],grid[-1]),(0,ct.M),(0,ct.M),(0,None),(None,None)],method='highs')
104     assert sol.success,sol.message
105     return float(sol.fun),float(sol.x[0])
106
107 def dp_checks():
108     rng=np.random.default_rng(742031); error=0.; tests=0
109     for case in range(12):
110         grid=np.linspace(1200,10800,13)
111         slopes=np.sort(rng.uniform(-3,5,len(grid)-1))
112         values=np.r_[0,np.cumsum(slopes*np.diff(grid))]
113         a=float(rng.uniform(-2000,3000));price=float(rng.uniform(.01,2))
114         got=ct.inf_convolution(grid,values,a,price)
115         for j,E in enumerate(grid):
116             exact,_=separate_lp(grid,values,E,a,price)
117             error=max(error,eq(got[j],exact,1e-6));tests+=1
118     net=np.array([[-100,200,1200],[200,900,-100],[700,400,600],[1400,1600,900]],float)
119     prices=np.array([.3,.8,.5],[.2,.7,.6],[.4,.9,.4],[.5,1.1,.7])
120     q=np.array([500.,600.,500.]);dp=ct.MarkovDP(net,prices,q,.35,grid_size=13)
121     edges=np.quantile(net,[1/3,2/3],axis=0).T
122     labels=np.sum(net[:,:,:None]>edges[None,:,:],axis=2)
123     val=np.tile(-.35*dp.grid,(3,1));rec_error=0.;action_error=0.
124     for j in range(2,-1,-1):
125         P=np.empty((3,3))
126         for k in range(3):
127             count=np.array([sum((labels[:,j]==k)&(labels[:,j+1]==kk)) if j<2 else 0 for kk in range(3)])
128             P[k]=(count+1/3)/(count.sum()+1)
129     nxt=P@val;rec_error=max(rec_error,eq(nxt,dp.future[j],1e-6));current=np.empty_like(val)

```

```

130     for k in range(3):
131         ids=np.flatnonzero(labels[:,j]==k)
132         if len(ids)==0:ids=np.arange(len(net))
133         p=float(prices[ids,j].mean())
134         for z,E in enumerate(dp.grid):
135             current[k,z]=np.mean([separate_lp(dp.grid,nxt[k],E,net[s,j]-q[j],p)[0] for s in ids])
136         for s in ids[1:]:
137             if labels[s,j]!=k:continue
138             E=5831.;target=dp.action(j,E,net[s,j],q[j]);delta=target-E
139             cost=5*p*max(0,net[s,j]-q[j]+max(ct.ETA*delta,delta/ct.ETA))+np.interp(target,dp.grid,nxt[k])
140             exact,_=separate_lp(dp.grid,nxt[k],E,net[s,j]-q[j],p)
141             action_error=max(action_error,eq(cost,exact,1e-6))
142         val=current
143     rec_error=max(rec_error,eq(val,dp.initial_values,1e-6))
144     REPORT['checks']['convex_dp_independent_bus_lp']={'passed':True,'inf_convolution_cases':tests,
145         'max_inf_convolution_error':error,'max_bellman_recursion_error':rec_error,
146         'max_action_objective_error':action_error,
147         'scope':'Exact subproblem comparison for the implemented convex PL grid model; not an exact original
        ↪ stochastic control result.'}
148
149 def cut_checks():
150     rng=np.random.default_rng(6201);maxerr=0.;n=0
151     for case in range(30):
152         lines=np.c_[rng.uniform(-1e4,1e5,30),rng.uniform(-10,10,30)]
153         original=np.r_[[0,0],lines];pruned=np.array(ct.prune_cuts(lines.tolist()))
154         xs=[ct.EMIN,ct.EMAX]
155         for i in range(len(original)):
156             for j in range(i):
157                 if original[i,1] == original[j,1]:continue
158                 x=(original[j,0]-original[i,0])/(original[i,1]-original[j,1])
159                 if ct.EMIN<=x<=ct.EMAX:xs.append(x)
160             xs=np.sort(xs);xs=np.r_[xs,(xs[1:]+xs[-1])/2]
161             e=eq(np.max(original[:,0,None]+original[:,1,None]*xs,axis=0),
162                 np.max(pruned[:,0,None]+pruned[:,1,None]*xs,axis=0),1e-6)
163             maxerr=max(maxerr,e);n+=len(xs)
164     REPORT['checks']['tail_cut_envelope_all_breakpoints']={'passed':True,'line_sets':30,'probes':n,'max_error':
        ↪ :maxerr}
165
166 def trajectory_check(path):
167     a=dict(np.load(path));metrics=json.loads(path.with_suffix('.json').read_text());days=a['days']
168     q,r,c,d,em,sp,st=[a[k] for k in ['q','r','c','d','emergency','spill','state']]
169     net=(DATA['load'][days]-DATA['pv'][days])/6
170     assert st.min()>=ct.EMIN-1e-5 and st.max()<=ct.EMAX+1e-5
171     assert max(c.max(),d.max())<=ct.M+1e-5
172     for v in [q,r,c,d,em,sp]:assert v.min()>=-1e-5
173     eq(np.minimum(c,d),0);eq(np.diff(st,axis=1),.9*c-d/.9)
174     eq(r+d+em-c-sp,net);eq(st[1:,0],st[:-1,-1]);eq(st[0,0],6000)
175     eq(a['releases'][:,0],q)
176     is_adjust=metrics['kind'] in ['q3','q4_3'];effective=q.copy()
177     for k in [1,2,3]:
178         assert np.isnan(a['releases'][:,k,:k*36]).all()
179         if is_adjust:
180             assert np.isfinite(a['releases'][:,k,k*36:]).all()
181             effective[:,k*36:]=a['releases'][:,k,k*36:]
182         else:assert np.isnan(a['releases'][:,k,:]).all()
183     eq(effective,r)
184     expected_price=DATA['price'][days] if metrics['kind'].startswith('q4') else
        ↪ np.broadcast_to(DATA['day_price'],q.shape)
185     eq(a['price'],expected_price,0)
186     charges=expected_price*(np.maximum(1.5*r-.5*q,.5*r+.5*q)+5*em)
187     err=eq(charges.sum(),metrics['totals']['total_cost'],1e-5)
188     if metrics['configuration']['closed_daily']:eq(st[:,-1],6000)
189     if metrics['configuration']['final'] is not None:eq(st[-1,-1],metrics['configuration']['final'])
190     for row in metrics['decisions']:
191         assert row['latest_training_target_exclusive']<=row['as_of']
192         if row['tail_model_cutoff'] is not None:
193             date=str(DATA['dates'][row['as_of']//144]);day=row['as_of']//144
194             expected=14 if day<31 else day-int(date[8:10])+1
195             assert row['tail_model_cutoff']==expected and expected*144<=row['as_of']

```

```

196         model_days=row.get('tail_model_days') or 3
197         model=json.loads(replay.tail_file(metrics['kind'],expected,model_days).read_text())
198         # Numeric cutoff in training metadata, not merely the filename.
199         assert model['configuration']['cutoff_day_exclusive']==expected
200         assert model['configuration']['horizon_days']==model_days
201         assert max(model['configuration']['history_days']) < expected
202     return {'file':str(path.relative_to(ROOT)), 'days':len(days), 'cost_error':err,
203           'midnight_final':st[:,-1].tolist(), 'closed':metrics['configuration']['closed_daily'],
204           'sha256':hashlib.sha256(path.read_bytes()).hexdigest()}
205
206 def execution_checks():
207     outcomes=[]
208     # Development runs are evidence of mechanics, not annual performance.
209     for folder in ['calibrate','calibrate_closed','calibrate_sddp']:
210         for path in sorted((ROOT/'artifacts'/folder).glob('*.*npz')):
211             outcomes.append(trajecory_check(path))
212     assert outcomes
213     # Actual execute function with deliberately aggressive targets over two days.
214     result=[]
215     for closed,final in [(False,None),(False,6000.),(True,6000.)]:
216         E=6000.;states=[E]
217         for t in range(288):
218             target=ct.EMAX if t<200 else ct.EMIN
219             E,c,d,em,sp,_=ct.execute(E,target,200.,400.,287-t,closed,t//144,final)
220             eq(E-states[-1],.9*c-d/.9);eq(200+d+em-c-sp,400)
221             assert max(c,d)<=ct.M+1e-6 and min(c,d)==0
222             states.append(E)
223             if final is not None:eq(E,final)
224             if closed:eq(states[144],6000)
225             if not closed and final is None:eq(E,ct.EMIN)
226             result.append({'daily_closed':closed,'global_final':final,'midnight':states[144],'end':E})
227     REPORT['checks']['development_replay_and_contract_reconstruction']={'passed':True,'trajectories':outcomes}
228     REPORT['checks']['physical_cross_day_free_fixed_terminal']={'passed':True,'synthetic_execution':result}
229
230 def real_month_boundary_checks():
231     # The first horizon reaches March, but the decision is still on February 28.
232     path=replay.tail_file('q2',31,1)
233     tail_hash=hashlib.sha256(path.read_bytes()).hexdigest()
234     runs=[]
235     for final in [None,6000.]:
236         arrays,metrics=replay.simulate(DATA,SEL,[58,59,60],'q2','sddp_mpc',count=3,horizon_days=2,final=final)
237         prefix='short-free' if final is None else 'short-fixed'
238         output=ROOT/'review'/f'{prefix}.npz'
239         np.savez_compressed(output,**arrays)
240         output.with_suffix('.json').write_text(json.dumps(metrics,ensure_ascii=False,indent=2,allow_nan=False)
241         ↵ )+'\n')
242         checked=trajecory_check(output)
243         first=metrics['decisions'][0]
244         assert first['end']//144 == 60 and first['tail_model_cutoff']==31 and first['tail_model_days']==1
245         assert first['as_of']//144 == 58
246         checked['first_decision']=first;checked['final_requested']=final
247         runs.append(checked)
248     assert tail_hash==hashlib.sha256(path.read_bytes()).hexdigest()
249     REPORT['checks']['actual_three_day_free_fixed_and_month_version']={
250         'passed':True,'runs':runs,'tail_model':str(path.relative_to(ROOT)),'tail_model_sha256':tail_hash,
251         'scope':'February 28 to March 2, original data, 3-scenario mechanism check; not annual performance
252         ↵ evidence.'}
253
254 def main():
255     start=time.perf_counter()
256     for fn in [forecast_checks,node_checks,dp_checks,cut_checks,execution_checks,real_month_boundary_checks]:
257         print(fn.__name__,flush=True);fn()
258     after={path:hashlib.sha256((ROOT/path).read_bytes()).hexdigest() for path in SOURCE}
259     assert after==SOURCE,'Production sources changed during audit'
260     REPORT['source_hashes']=SOURCE
261     REPORT['execution']={'command':'python3 review/check-controllers.py','exit_code':0,
262         'runtime_seconds':time.perf_counter()-start,'seed':742031}
263     REPORT['passed']=True

```

```

262 (ROOT/'review/controller-checks.json').write_text(json.dumps(REPORT,ensure_ascii=False,indent=2,allow_nan
↪ =False)+'\n')
263 print(json.dumps({'passed':True,'checks':len(REPORT['checks']),'seconds':REPORT['execution']['runtime_sec
↪ onds']}))
264
265 if __name__=='__main__':main()

```

review/check-final-integrity.py

```

1 """Independent consistency checks for the frozen handoff evidence.
2
3 Hash verification is an integrity check, not a substitute for numerical review.
4 Final review/repro/acceptance are deliberately outside their own hash closure.
5 """
6 from pathlib import Path
7 import hashlib, json
8
9 ROOT = Path(__file__).resolve().parents[1]
10 def read(p): return json.loads((ROOT / p).read_text())
11 def sha(p): return hashlib.sha256((ROOT / p).read_bytes()).hexdigest()
12 def check_hashes(items):
13     for name, expected in items.items():
14         assert sha(name) == expected, name
15     return len(items)
16
17 def main():
18     checks = {}
19     registry = read('artifacts/result-registry.json')['results']
20     assert len({r['id'] for r in registry}) == len(registry)
21     for row in registry:
22         obj = read(row['source_artifact'])
23         for part in row['source_path'].split('/')[1:]:
24             part = part.replace('~1', '/').replace('~0', '~')
25             obj = obj[int(part)] if isinstance(obj, list) else obj[part]
26             assert row['status'] == 'verified' and row['value'] == obj, row['id']
27     checks['registry_exact_source_values'] = {'passed': True, 'records': len(registry)}
28     visual = read('artifacts/visual-qa.json')
29     assert visual['status'] == 'pass'
30     checks['visual_hashes'] = {k: check_hashes(visual[k]) for k in
31                               ['preview_hashes', 'figure_hashes', 'workbook_hashes']}
32     wb = read('artifacts/workbook-validation.json')
33     assert wb['execution']['exit_code'] == 0
34     assert len(wb['checks']) == 5 and all(c['status'] == 'pass' for c in wb['checks'])
35     checks['workbook_hashes'] = check_hashes({c['file']: c['sha256'] for c in wb['checks']})
36     versions = read('artifacts/source-version-records.json')['v1']
37     checks['executed_source_archive'] = check_hashes({versions['directory']+'/'+name: value
38                                                         for name, value in versions['files'].items()})
39     repro = read('artifacts/reproducibility.json')
40     checks['input_hashes'] = check_hashes(repro['input_hashes'])
41     # Reviewer reruns and final findings are refreshed by the author's final
42     # freeze; all production and documentation hashes must already be stable.
43     frozen = {n: h for n, h in repro['output_hashes'].items()
44               if not n.startswith('review/') and n not in
45                  ['modeling-manifest.json', 'artifacts/reproducibility.json', 'artifacts/acceptance.json']}
46     checks['frozen_production_outputs'] = check_hashes(frozen)
47     production = read('review/production-audit.json')
48     assert production['passed'] is True and production['passed_cases'] == 36
49     assert not production['pending_cases'] and not production['failed_cases']
50     supplement = read('review/final-supplement.json')
51     assert all(v.get('passed') is True for v in supplement['checks'].values())
52     checks['independent_visual_inspection'] = {
53         'passed': True,
54         'scope': 'Four final scientific PNGs, plus three representative workbook preview ranges; not every
↪ cell visually inspected.',
55         'files': {p: sha(p) for p in [
56             'figures/年度方法比较.png', 'figures/午夜库存轨迹.png',
57             'figures/SDDP 训练与库存价值.png', 'figures/预报发布消融.png',
58             'artifacts/previews/result4-2-充放电量.png',

```

```

59         'artifacts/previews/result3-调整购电量-6 点.png',
60         'artifacts/previews/result1-口径说明.png']],
61         'observations': [
62             'Four figures have readable axes/legends, no visible clipping; close annual curves are accompanied
↳ by exact numeric differences in the handoff.',
63             'Battery dates are visible and grouped per six rows; the six-hour revision view has correct
↳ ten-minute interval headers.',
64             'Workbook SOC in kWh denotes energy inventory, as explicitly distinguished from the normalized SOC
↳ ratio in the model document.',
65             'Workbook notes state the time, settlement, terminal, information and optimality conventions.']}
66     result = {'schema_version': 1, 'reviewer_id': '/root/independent_review', 'independent': True,
67              'passed': True, 'scope': 'frozen_evidence_consistency_and_sampled_visual_review',
68              'checks': checks, 'audit_source_sha256': sha('review/check-final-integrity.py')}
69     (ROOT/'review/final-integrity.json').write_text(json.dumps(result, ensure_ascii=False, indent=2)+'\n')
70     print(json.dumps({'passed': True, 'registry_records': len(registry), 'production_hashes': len(frozen)}))
71
72 if __name__ == '__main__': main()

```

review/check-final-supplement.py

```

1  """Final independent warmup/Q1, development, handoff and workbook checks.
2
3  python3 review/check-final-supplement.py
4  bundled-python review/check-final-supplement.py --workbooks
5  Reuses the reviewer's explicit-spill LP and NumPy ledger modules, not production
6  settlement, forecasting, or controller functions.
7  """
8  from pathlib import Path
9  import importlib.util, hashlib, json, sys, time
10 import numpy as np
11 ROOT=Path(__file__).resolve().parents[1];OUT=ROOT/'review/final-supplement.json'
12 DATA=dict(np.load(ROOT/'artifacts/data.npz'))
13 def load(name):return json.loads((ROOT/name).read_text())
14 def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
15 def module(name,file):
16     spec=importlib.util.spec_from_file_location(name,ROOT/file);m=importlib.util.module_from_spec(spec);spec.
↳ loader.exec_module(m);return m
17 def near(a,b,tol=1e-6):
18     e=float(np.max(abs(np.asarray(a)-np.asarray(b))))
19     assert np.isfinite(e) and e<=tol,e
20     return e
21 def clock(t):return f'{t//6:02d}:{10*(t%6):02d}'
22 def interval(a,b=None):return clock(a)+'-'+clock(a+1 if b is None else b)
23
24 def warmup():
25     a=dict(np.load(ROOT/'artifacts/january-warmup.npz'));m=load('artifacts/warmup.json')
26     assert a['state'].shape==(31,145)
27     for k in ['q','r','c','d']:near(a[k],0,0)
28     near(a['state'],6000,0);n=(DATA['load'][:31]-DATA['pv'][:31])/6
29     near(a['emergency'],np.maximum(n,0),0);near(a['spill'],np.maximum(-n,0),0)
30     near(a['r']+a['emergency']+a['d']-a['c']-a['spill'],n)
31     near(np.diff(a['state'],axis=1),.9*a['c']-a['d']/.9)
32     near(a['state'][1:,0],a['state'][:-1,-1],0)
33     fees={'fixed_price_cost':float(np.sum(5*DATA['day_price']*a['emergency'])),
34           'variable_price_cost':float(np.sum(5*DATA['price'][:31]*a['emergency']))}
35     for k,v in fees.items():near(m[k],v)
36     assert m['days']==31 and m['initial_energy']==6000 and m['february_initial_energy']==6000
37     for k in ['q2','q3','q4_2','q4_3']:
38         a2=np.load(ROOT/f'artifacts/{k}.npz');near(a['state'][-1,-1],a2['state'][0,0],0)
39     return {'passed':True,'days':31,'fees_yuan':fees,'emergency_kwh':float(a['emergency'].sum()),
40           'spill_kwh':float(a['spill'].sum()),'initial_and_february_inventory':6000.,
41           'files':{f:sha(ROOT/f) for f in ['artifacts/january-warmup.npz','artifacts/warmup.json']}}
42
43 def q1():
44     oracle=module('review_oracle','review/check-global-oracle.py')
45     n=(DATA['day_load']-DATA['day_pv'])/6;p=DATA['day_price']
46     exact,_=oracle.one_case(oracle.matrices(144),n,p,6000,'q1_final_independent')
47     a=dict(np.load(ROOT/'artifacts/q1.npz'));m=load('artifacts/q1.json');c,d,E=a['c'],a['d'],a['state']

```

```

48     near(p@a['q'],exact['objective_cost_yuan']);near(m['expected_cost'],exact['objective_cost_yuan']);near(m[
↪ 'lp_objective'],exact['objective_cost_yuan'])
49     near(np.diff(E),.9*c-d/.9);near(np.minimum(c,d),0);near(E[[0,-1]],6000)
50     assert E.min()>=1200-1e-6 and E.max()<=10800+1e-6 and max(c.max(),d.max())<=5000/6+1e-6
51     assert max(0,float(np.max(n+c-d-a['q'])))<1e-6
52     return {'passed':True,'economic_cost_yuan':exact['objective_cost_yuan'],'energy_kwh':float(a['q'].sum()),
53            'duality_gap':exact['checks']['primal_minus_repaired_dual'],
54            'normalization_simultaneous_max':float(np.minimum(c,d).max()),
55            'files':{f:sha(ROOT/f) for f in ['artifacts/q1.json','artifacts/q1.npz']}}
56
57 def development_and_short():
58     aud=module('review_production','review/check-production.py');sel=load('artifacts/selection.json');checked_
↪ =[]
59     for kind in ['q2','q3','q4_2','q4_3']:
60         costs={}
61         for cand in ['cross_baseline','affine_mpc','markov_mpc','sddp_mpc','sddp_markov']:
62             folder='calibrate_sddp_v2' if cand.startswith('sddp') else 'calibrate'
63             f=f'artifacts/{folder}/{kind}_{cand}';a,m,h=aud.read_pair(f)
64             near(a['days'],np.arange(24,31),0)
65             aud.physical(a,kind,6000);aud.releases(a,kind);l=aud.daily_ledger(a)
66             e=aud.compare_reported_ledger(l,m['daily'],m['totals'],a['dates'],a['state'])
67             costs[cand]=float(l['total_cost'].sum());near(costs[cand],sel['development_costs'][kind][cand])
68             for node in m['decisions']:
69                 assert node['latest_training_target_exclusive']<=node['as_of']
70                 if node['tail_cut_count']:
71                     assert node['tail_model_cutoff']==14 and
↪ node['tail_model_days']==min(3,(31*144-node['end']))//144
72                     if node['end']==31*144:assert node['tail_cut_count']==0 and node['tail_price']==0
73                     checked.append({'case':f,'ledger_error':e,'files':h})
74             chosen=min(costs,key=costs.get)
75             if costs['markov_mpc']<=min(costs.values())*(1+sel['relative_tie_tolerance']):chosen='markov_mpc'
76             assert sel['selected'][kind]==chosen
77     for p in sorted((ROOT/'artifacts/experiments').glob('*.json')):
78         if p.stem.startswith('annual_'):continue
79         f=str(p.relative_to(ROOT))[:-5];a,m,h=aud.read_pair(f);cfg=m['configuration'];design=m['validation_de
↪ sign']
80         changed={k:v.copy() for k,v in DATA.items()}
81         if design.get('stress'):
82             changed['load'][72:79]*=1.1;changed['pv'][72:79]*=.8
83         aud.DATA=changed
84         aud.physical(a,m['kind'],6000,terminal=cfg['final']);aud.releases(a,m['kind']);l=aud.daily_ledger(a)
85         e=aud.compare_reported_ledger(l,m['daily'],m['totals'],a['dates'],a['state'])
86         near(a['days'],np.arange(72,79),0)
87         for node in m['decisions']:assert node['latest_training_target_exclusive']<=node['as_of']
88         checked.append({'case':f,'ledger_error':e,'files':h})
89     return {'passed':True,'development_cases':20,'short_experiments':15,'checked':checked,
90            'scope':'Independent numerical ledger/physics for January counterfactual development and March
↪ sensitivity; January choice reconstructed with declared tie rule. No out-of-sample claim for
↪ development scores.'}
91
92 def tables():
93     tab=load('artifacts/handoff-tables.json');selected=load('artifacts/selection.json')['selected'];errs=[]
94     for kind in ['q2','q3','q4_2','q4_3']:
95         a=dict(np.load(ROOT/f'artifacts/{kind}.npz'));source=np.load(ROOT/f'artifacts/annual/{kind}_{selected_
↪ [kind]}.npz')
96         for k in a:
97             if np.issubdtype(a[k].dtype,np.number):assert np.array_equal(a[k],source[k],equal_nan=True)
98             else:assert np.array_equal(a[k],source[k])
99         dates=tab['models'][kind]['selected'];assert
↪ set(dates)=={'2025-03-20','2025-06-21','2025-09-23','2025-12-21'}
100        for date,v in dates.items():
101            i=a['dates'].tolist().index(date)
102            for row,t in zip(v['purchase'],[60,72,84,96,108,120]):
103                assert row[0]==interval(t);errs.append(near(row[1:],a['q'][i,t],a['r'][i,t]))
104            for j,row in enumerate(v['battery']):
105                assert row[0]==date and row[1]==interval(j*24,(j+1)*24)
106                errs.append(near(row[2:4],[a['c'][i,j*24:(j+1)*24].sum(),a['d'][i,j*24:(j+1)*24].sum()))
107                expectedtime='00:00' if j==0 else '24:00' if j==1 else None
108                assert row[4]==expectedtime

```

```

109         if j<2:errs.append(near(row[5],a['state'][i,0 if j==0 else -1]))
110         else:assert row[5] is None
111         mask=a['emergency'][i]>1e-6;starts=np.flatnonzero(mask&~np.r_[False,mask[:i]]);ends=np.flatnonzero
↪ ro(mask&~np.r_[mask[i:],False])+1
112         ev=[[date,interval(int(s),int(e)),float(a['emergency'][i,s:e].sum())] for s,e in zip(starts,ends)]
↪ or [[date,'无',0.]]
113         assert len(ev)==len(v['emergency'])
114         for r,s in zip(ev,v['emergency']):assert r[:2]==s[:2];errs.append(near(r[2],s[2]))
115     return {'passed':True,'selected_dates':16,'max_numeric_error':max(errs),'source':sha(ROOT/'artifacts/handl
↪ off-tables.json'),
116           'meaning':'Four-hour blocks may contain both charging and discharging at different ten-minute
↪ intervals; this does not violate instantaneous mutual exclusion.'}
117
118 def workbooks():
119     import openpyxl
120     cases=[]
121     for name,kind in
↪ [('result1',None),('result2','q2'),('result3','q3'),('result4-2','q4_2'),('result4-3','q4_3')]:
122         p=ROOT/'计算结果'/f'{name}.xlsx';w=openpyxl.load_workbook(p,read_only=True,data_only=True)
123         a=dict(np.load(ROOT/f'artifacts/{kind or "q1"}.npz'));r=list(w['计划购电量'].values)
124         if kind:
125             assert list(r[0][1:145])==[interval(t) for t in range(144)]
126             near(np.array([z[1:145] for z in r[1:335]],float),a['q'])
127             b=list(w['充放电量'].values)[1:]
128             for j,row in enumerate(b):
129                 i=j//6;t=j%6
130                 assert row[1]==interval(t*24,(t+1)*24)
131                 near(row[2:4],[a['c'][i,t*24:(t+1)*24].sum(),a['d'][i,t*24:(t+1)*24].sum())
132                 if t==0:assert row[0].strftime('%Y-%m-%d')==a['dates'][i]
133                 else:assert row[0] is None
134                 if t<2:assert row[4]==('00:00' if t==0 else '24:00');near(row[5],a['state'][i,0 if t==0 else
↪ -1])
135                 else:assert row[4] is None and row[5] is None
136             else:
137                 assert [row[0] for row in r[1:145]]==[interval(t) for t in range(144)]
138                 near([row[1] for row in r[1:145]],a['q'])
139                 b=list(w['充放电量'].values)
140                 near(b[1][4],6000);near(b[2][4],6000)
141                 # The phrase SOC/kWh is a workbook convention for E, not a ratio.
142                 cases.append({'file':str(p.relative_to(ROOT)), 'sha256':sha(p), 'sheet_count':len(w.sheetnames), 'status'
↪ ': 'pass'})
143             w.close()
144     return {'passed':True,'files':cases,'scope':'Independent workbook headers, full q and four-hour battery
↪ sums/inventory directly versus raw NPZ. Other workbook cells are covered by producer readback plus
↪ independently reconciled handoff/production ledgers; no claim to have rerendered all cells.'}
145
146 def main():
147     start=time.perf_counter();only='--workbooks' in sys.argv
148     report=load('review/final-supplement.json') if only else {'scope':'final_independent_supplement','reviewe
↪ r_id':'/root/independent_review','independent':True,'checks':{}}
149     todo=[('workbooks',workbooks)] if only else [('warmup',warmup),('q1',q1),('development_and_short',develop
↪ ment_and_short),('specified_day_tables',tables)]
150     for key,fn in todo:print(key,flush=True);report['checks'][key]=fn()
151     report['passed']=all(v['passed'] for v in report['checks'].values())
152     report.setdefault('executions',[]).append({'command':('bundled-python' if only else 'python3')+
↪ ' review/check-final-supplement.py'+(' --workbooks' if only else
↪ ''), 'exit_code':0, 'runtime_seconds':time.perf_counter()-start})
153     report['review_script_sha256']=sha(__file__)
154     OUT.write_text(json.dumps(report,ensure_ascii=False,indent=2,allow_nan=False)+'\n')
155     print(json.dumps({'passed':report['passed'],'seconds':time.perf_counter()-start}))
156 if __name__=='__main__':main()

```

review/check-global-oracle.py

```

1  """Independent full-horizon perfect-information LP, not an executable policy.
2
3  Run from the project directory: python3 review/check-global-oracle.py
4  Four LPs use all 48,096 actual ten-minute intervals, no daily closure, no

```

```

5 emergency energy, no regularization term, and explicit free spill variables.
6 """
7 from pathlib import Path
8 import hashlib, json, platform, sys, time
9 import numpy as np
10 from scipy.sparse import eye, diags, hstack, vstack, csr_matrix
11
12 ROOT=Path(__file__).resolve().parents[1]
13 sys.path.insert(0,str(ROOT/'vendor'))
14 import highsipy
15
16 ETA=.9; M=5000./6.; EMIN=1200.; EMAX=10800.; INITIAL=6000.
17 OUTPUT=ROOT/'review'
18
19 def sha(path):return hashlib.sha256(Path(path).read_bytes()).hexdigest()
20
21 def matrices(T):
22     I=eye(T,format='csr'); Z=csr_matrix((T,T))
23     D=I-diags(np.ones(T-1),-1,shape=(T,T),format='csr')
24     # Column blocks q, c, d, next E, spill.
25     return vstack([hstack([I,-I,I,Z,-I]),hstack([Z,-ETA*I,I/ETA,D,Z])],format='csr')
26
27 def one_case(A,net,price,terminal,label):
28     begin=time.perf_counter();T=len(net);N=5*T
29     objective=np.r_[price,np.zeros(4*T)]
30     lo=np.r_[np.zeros(3*T),np.full(T,EMIN),np.zeros(T)]
31     hi=np.r_[np.full(T,np.inf),np.full(2*T,M),np.full(T,EMAX),np.full(T,np.inf)]
32     if terminal is not None:lo[4*T-1]=hi[4*T-1]=terminal
33     rhs=np.r_[net,np.r_[INITIAL,np.zeros(T-1)]]
34     problem=highspy.HighsLp();problem.num_col_=N;problem.num_row_=2*T
35     problem.col_cost_=objective;problem.col_lower_=lo;problem.col_upper_=hi
36     problem.row_lower_=rhs;problem.row_upper_=rhs
37     problem.a_matrix_.format_=highspy.MatrixFormat.kRowwise
38     problem.a_matrix_.start_=A.indptr;problem.a_matrix_.index_=A.indices;problem.a_matrix_.value_=A.data
39     solver=highspy.Highs()
40     options={'output_flag':False,'threads':1,'solver':'ipm',
41             'primal_feasibility_tolerance':1e-8,'dual_feasibility_tolerance':1e-8,
42             'ipm_optimality_tolerance':1e-10}
43     for key,value in options.items():
44         assert solver.setOptionValue(key,value)==highspy.HighsStatus.kOk
45     assert solver.passModel(problem)==highspy.HighsStatus.kOk
46     status=solver.run();model_status=solver.getModelStatus()
47     assert status==highspy.HighsStatus.kOk,status
48     assert model_status==highspy.HighsModelStatus.kOptimal,solver.modelStatusToString(model_status)
49     sol=solver.getSolution();info=solver.getInfo()
50     assert sol.value_valid and sol.dual_valid
51     x=np.asarray(sol.col_value);y=np.asarray(sol.row_dual);z=np.asarray(sol.col_dual)
52     q,c,d,E,spill=np.split(x,5)
53     cost=float(price@q);raw_objective=float(solver.getObjectiveValue())
54     finite=np.isfinite(hi)
55     # Solver reduced-cost dual objective, with infeasible infinite-bound signs
56     # reported separately. An independently repaired certificate follows.
57     raw_dual=float(rhs@y+np.sum(lo*np.maximum(z,0))+
58                   np.sum(hi[finite]*np.minimum(z[finite],0)))
59     stationarity=float(abs(objective-A.T@y-z).max())
60     infinite_sign_violation=float(max(0,-z[~finite].min()))
61
62     # For unbounded-above q and spill, dual feasibility is exactly  $0 \leq y_{\text{bus}} \leq p$ .
63     # All other columns have finite bounds and need no reduced-cost sign limit.
64     # Clipping these rows constructs a feasible dual directly, independently
65     # recomputing every reduced cost rather than relying on the solver report.
66     certificate_y=y.copy();certificate_y[:T]=np.clip(certificate_y[:T],0,price)
67     certificate_z=objective-A.T@certificate_y
68     assert certificate_z[~finite].min()>=-1e-12
69     dual_bound=float(rhs@certificate_y+np.sum(lo*np.maximum(certificate_z,0))+
70                   np.sum(hi[finite]*np.minimum(certificate_z[finite],0)))
71     gap=cost-dual_bound
72     relative_gap=gap/max(1,abs(cost))
73     eq_residual=float(abs(A@x-rhs).max())

```

```

74     bounds_residual=float(max(0,np.max(lo-x),np.max(x[finite]-hi[finite])))
75
76     # Any simultaneous charge/discharge is removed while keeping E and q.
77     # Extra bus supply is sent to the explicit free spill variable.
78     simultaneous_before=float(np.minimum(c,d).max())
79     remove=np.minimum(c,d/(ETA*ETA))
80     canonical_c=c-remove;canonical_d=d-ETA*ETA*remove
81     canonical_spill=spill+(1-ETA*ETA)*remove
82     state=np.r_[INITIAL,E]
83     balance_error=float(abs(q+canonical_d-canonical_c-canonical_spill-net).max())
84     soc_error=float(abs(np.diff(state)-ETA*canonical_c+canonical_d/ETA).max())
85     power=max(canonical_c.max(),canonical_d.max())*6
86     simultaneous_after=float(np.minimum(canonical_c,canonical_d).max())
87     midnight=state[np.arange(144,T+1,144)]
88     end_error=0. if terminal is None else abs(float(state[-1])-terminal)
89     checks={'matrix_equality_max_abs':eq_residual,'variable_bound_violation':bounds_residual,
90            'economic_objective_recompute_error':abs(cost-raw_objective),
91            'dual_stationarity_max_abs':stationarity,'raw_infinite_bound_sign_violation':infinite_sign_violat
↵ ion,
92            'dual_certificate_repair_max_abs':float(abs(certificate_y-y).max()),
93            'dual_certificate_unbounded_column_violation':float(max(0,-certificate_z[-finite].min())),
94            'primal_minus_repaired_dual':gap,'relative_primal_dual_gap':relative_gap,
95            'canonical_bus_balance_max_abs':balance_error,'canonical_soc_recurrence_max_abs':soc_error,
96            'canonical_soc_min':float(state.min()),'canonical_soc_max':float(state.max()),
97            'canonical_power_max_kw':float(power),'simultaneous_before':simultaneous_before,
98            'simultaneous_after':simultaneous_after,'terminal_error':end_error}
99     assert max(eq_residual,bounds_residual,balance_error,soc_error,end_error)<1e-5,checks
100     assert state.min()>=EMIN-1e-5 and state.max()<=EMAX+1e-5 and power<=5000+1e-5
101     assert min(q.min(),canonical_c.min(),canonical_d.min(),canonical_spill.min())>=-1e-5
102     assert abs(gap)<max(.01,abs(cost)*1e-8) and stationarity<1e-6,checks
103     result={'label':label,'status':solver.modelStatusToString(model_status),'terminal_constraint':terminal,
104            'objective_cost_yuan':cost,'solver_objective_yuan':raw_objective,
105            'raw_dual_objective_yuan':raw_dual,'repaired_dual_lower_bound_yuan':dual_bound,
106            'initial_inventory_kwh':INITIAL,'final_inventory_kwh':float(state[-1]),
107            'ordinary_purchase_kwh':float(q.sum()),'spill_kwh':float(canonical_spill.sum()),
108            'charge_bus_kwh':float(canonical_c.sum()),'discharge_bus_kwh':float(canonical_d.sum()),
109            'midnight_inventories_kwh':midnight.tolist(),
110            'checks':checks,'iterations':{'ipm':int(info.ipm_iteration_count),
111                                         'crossover':int(info.crossover_iteration_count),
112                                         'simplex':int(info.simplex_iteration_count)},
113            'runtime_seconds':time.perf_counter()-begin,'solver_options':options,
114            'solver_report':{'max_primal_infeasibility':float(info.max_primal_infeasibility),
115                           'max_dual_infeasibility':float(info.max_dual_infeasibility)}}
116     arrays={'q':q,'c':canonical_c,'d':canonical_d,'state':state,'spill':canonical_spill}
117     return result,arrays
118
119 def main():
120     start=time.perf_counter();source=sha(__file__);data_path=R00T/'artifacts/data.npz';input_hash=sha(data_pa
↵ th)
121     data=dict(np.load(data_path));net=((data['load'][31:]-data['pv'][31:])/6).ravel()
122     T=len(net);assert T==48096
123     prices={'fixed':np.tile(data['day_price'],334),'variable':data['price'][31:].ravel()}
124     assert all(p.min()>0 and len(p)==T for p in prices.values())
125     A=matrices(T);results={};trajectories={}
126     for kind,price in prices.items():
127         for name,terminal in [('free',None),('fixed6000',6000.)]:
128             label=f'{kind}_{name}';print('Solving '+label,flush=True)
129             result,arrays=one_case(A,net,price,terminal,label)
130             results[label]=result
131             trajectories.update({'{label}_{k}':v for k,v in arrays.items()})
132             print(json.dumps({'label':label,'cost':result['objective_cost_yuan'],
133                              'gap':result['checks']['primal_minus_repaired_dual'],
134                              'seconds':result['runtime_seconds']}),flush=True)
135     for kind in prices:
136         assert results[f'{kind}_free']['objective_cost_yuan']<=results[f'{kind}_fixed6000']['objective_cost_y
↵ uan']+1e-5
137     archive=OUTPUT/'global-oracle-trajectories.npz'
138     np.savez_compressed(archive,**trajectories)
139     assert source==sha(__file__) and input_hash==sha(data_path)

```

```

140     report={'schema_version':1,'reviewer_id':'/root/independent_review','independent':True,
141            'scope':'Full-horizon perfect-information optimistic reference, not a causal policy or SDDP gap.',
142            'passed':True,'evaluation':{'start_date':str(data['dates'][31]),'end_date':str(data['dates'][-1]),
143                                       'days':334,'ten_minute_intervals':T,'initial_inventory_kwh':INITIAL,
144                                       'daily_closure':False,'emergency_purchase_allowed':False,
145                                       'free_spill':True,'turnover_regularizer':0.,
146                                       'eta_each_direction':ETA,'power_kw':5000,'soc_bounds_kwh':[EMIN,EMAX]},
147            'lower_bound_argument':'For any feasible causal strategy with identical initial and true terminal
    ↪ conditions, use perfect-information ordinary purchase q_oracle=r+emergency with its battery
    ↪ and spill.  $\phi(q,r) \geq r$  and  $5*emergency \geq emergency$ , so positive-price cost cannot increase.
    ↪ Removing nonanticipativity therefore gives a lower bound. The free-terminal bound must not be
    ↪ presented as a same-terminal gap for a fixed-terminal policy.',
148            'dual_certificate':'Clip the bus-balance row dual to [0,price], recompute  $z=c-A^T y$ , and evaluate
    ↪  $b^T y + \sum(z_{\text{positive}}*lower) + \sum(z_{\text{negative}}*finite\_upper)$ . All infinite-upper columns are q
    ↪ and spill and have nonnegative reduced costs after repair. Numerical tolerances are explicitly
    ↪ reported; this is not exact rational arithmetic.',
149            'results':results,'source_sha256':source,'data_npz_sha256':input_hash,
150            'trajectory_file':str(archive.relative_to(ROOT)),'trajectory_sha256':sha(archive),
151            'execution':{'command':'python3 review/check-global-oracle.py','exit_code':0,
152                          'runtime_seconds':time.perf_counter()-start,'python':sys.version,
153                          'numpy':np.__version__, 'highs':highspy.Highs().version(),'platform':platform.platform()
    ↪ m(),
154                          'deterministic_reason':'Four deterministic sparse LPs; no sampling or random seed.')}}
155     (OUTPUT/'global-oracle.json').write_text(json.dumps(report,ensure_ascii=False,indent=2,allow_nan=False)+'\n')
    ↪ '\n')
156     print(json.dumps({'passed':True,'total_seconds':report['execution']['runtime_seconds']}),flush=True)
157
158 if __name__=='__main__':main()

```

review/check-production.py

```

1  """Independent audit of 36 full production trajectories; no solver/model calls.
2
3  python3 review/check-production.py                # partial live inventory
4  python3 review/check-production.py --require-complete # final numerical gate
5
6  Energy, costs, physics, release reconstruction, and calendar rules below are
7  implemented independently with NumPy. No production module is imported.
8  Prior code/mutation/LP reviews are identified as reused evidence, not rerun here.
9  """
10 from pathlib import Path
11 import hashlib, json, sys, time
12 import numpy as np
13
14 ROOT=Path(__file__).resolve().parents[1]
15 OUT=ROOT/'review/production-audit.json'
16 ETA=.9; M=5000/6; EMIN=1200.; EMAX=10800.; INITIAL=6000.
17 DAYS=np.arange(31,365); DATES=np.arange(np.datetime64('2025-02-01'),np.datetime64('2026-01-01'))
18 TEND=365*144; TOL=1e-5
19 KINDS=['q2','q3','q4_2','q4_3']
20 METHODS=['closed_baseline','cross_baseline','affine_mpc','markov_mpc','closed_affine']
21 ABLATIONS={'only': [0], 'without6': [0,2,3], 'without12': [0,1,3], 'without18': [0,1,2]}
22 DATA=dict(np.load(ROOT/'artifacts/data.npz'))
23 MODEL_CACHE={}
24
25 class Pending(Exception):pass
26 def sha(p):return hashlib.sha256(Path(p).read_bytes()).hexdigest()
27 def require(test,message):
28     if not bool(test):raise AssertionError(message)
29 def near(a,b,label,tol=TOL):
30     aa=np.asarray(a);bb=np.asarray(b)
31     err=float(np.max(np.abs(aa-bb))) if aa.size else 0.
32     require(np.isfinite(err) and err<=tol,f'{label}: {err}')
33     return err
34 def read_pair(stem):
35     j=ROOT/(stem+'.json');n=ROOT/(stem+'.npz')
36     if not j.exists() or not n.exists():raise Pending('Missing '+', '.join(str(p.relative_to(ROOT)) for p in
    ↪ [j,n] if not p.exists()))

```

```

37     before={str(p.relative_to(ROOT)):sha(p) for p in [j,n]}
38     try:
39         meta=json.loads(j.read_text());arrays=dict(np.load(n))
40     except (ValueError,EofError,OSError) as e:raise Pending('Incomplete/unreadable pair: '+str(e))
41     after={str(p.relative_to(ROOT)):sha(p) for p in [j,n]}
42     if before!=after:raise Pending('Files changed during read')
43     return arrays,meta,after
44
45 def daily_ledger(a):
46     q,r,p,e=a['q'],a['r'],a['price'],a['emergency']
47     ledger={'planned_cost':np.sum(p*q,axis=1),
48             'increase_cost':np.sum(1.5*p*np.maximum(r-q,0),axis=1),
49             'reduction_net_cost':np.sum(-.5*p*np.maximum(q-r,0),axis=1),
50             'emergency_cost':np.sum(5*p*e,axis=1),
51             'emergency_energy':e.sum(1),'spill_energy':a['spill'].sum(1),
52             'projection_count':np.sum(a['projection']>1e-6,axis=1)}
53     ledger['total_cost']=sum(ledger[k] for k in
    ↪ ['planned_cost','increase_cost','reduction_net_cost','emergency_cost'])
54     near(ledger['total_cost'],np.sum(p*(np.maximum(1.5*r-.5*q,.5*r+.5*q)+5*e),axis=1),'two independent fee
    ↪ forms')
55     return ledger
56
57 def physical(a,kind,initial,terminal=None,closed=False):
58     nd=len(a['days']);days=np.asarray(a['days'])
59     for key in ['q','r','c','d','emergency','spill','price','projection']:
60         require(a[key].shape==(nd,144),key+' shape')
61         require(np.isfinite(a[key]).all(),key+' finite')
62     require(a['state'].shape==(nd,145),'state shape')
63     require(np.isfinite(a['state']).all(), 'state finite')
64     near(a['dates'].astype('datetime64[D]').astype(int),DATA['dates'][days].astype('datetime64[D]').astype(int)
    ↪ t),'data date mapping',0)
65     q,r,c,d,e,w,E=[a[k] for k in ['q','r','c','d','emergency','spill','state']]
66     net=(DATA['load'][days]-DATA['pv'][days])/6
67     prices=DATA['price'][days] if kind.startswith('q4') else np.broadcast_to(DATA['day_price'],(nd,144))
68     near(a['price'],prices,'actual settlement price',0)
69     nonnegative=float(max(0,-min(a[k].min() for k in ['q','r','c','d','emergency','spill','projection'])))
70     require(nonnegative<=TOL,'negative physical energy')
71     result={'balance_max_abs':near(r+e+d-c-w,net,'bus balance'),
72            'soc_recurrence_max_abs':near(np.diff(E,axis=1),ETA*c-d/ETA,'SOC recurrence'),
73            'midnight_jump_max_abs':near(E[1:,0],E[:-1,-1],'midnight continuity'),
74            'soc_min':float(E.min()),'soc_max':float(E.max()),
75            'power_max_kw':float(max(c.max(),d.max())*6),
76            'simultaneous_max':near(np.minimum(c,d),0,'simultaneous charge/discharge'),
77            'initial_inventory':float(E[0,0]),'final_inventory':float(E[-1,-1]),
78            'negative_energy_violation':nonnegative}
79     near(E[0,0],initial,'inherited initial inventory')
80     require(result['soc_min']>=EMIN-TOL and result['soc_max']<=EMAX+TOL,'SOC limits')
81     require(result['power_max_kw']<=5000+TOL,'power limit')
82     if closed:near(E[:,-1],6000,'daily closure')
83     if terminal is not None:near(E[-1,-1],terminal,'true terminal inventory')
84     return result
85
86 def releases(a,kind):
87     rel=a['releases'];nd=len(a['days']);require(rel.shape==(nd,4,144),'release shape')
88     require(np.isfinite(rel[:,0]).all(), 'midnight contracts missing')
89     near(rel[:,0],a['q'],'original q locked to midnight snapshot')
90     effective=rel[:,0].copy();adjust=kind in ['q3','q4_3']
91     for k in range(1,4):
92         require(np.isnan(rel[:,k:k*36]).all(), 'revision affects already delivered slots')
93         if adjust:
94             require(np.isfinite(rel[:,k:k*36]).all(), 'missing permitted revision')
95             require(rel[:,k:k*36:].min()>=-TOL,'negative revised quantity')
96             effective[:,k:k*36]=rel[:,k:k*36:]
97         else:require(np.isnan(rel[:,k]).all(), 'Q2-type unauthorized revision')
98     err=near(effective,a['r'],'latest legal release reconstructs final r')
99     near(a['r'][:,0:36],a['q'][:,0:36],'first six hours locked')
100    return {'passed':True,'final_r_reconstruction_error':err,'official_revision_nodes_per_day':4 if adjust
    ↪ else 1,

```

```

101         'meaning': 'r is delivery-wise final effective quantity; every original q equals its daily 00:00
↪ snapshot.']}
102
103 def compare_reported_ledger(ledger, reported_daily, reported_totals, dates, E):
104     require(len(reported_daily)==len(dates), 'daily record length')
105     maxerr=0.
106     for i, row in enumerate(reported_daily):
107         require(str(row['date'])==str(dates[i]), 'daily report date')
108         for key, value in ledger.items(): maxerr=max(maxerr, near(row[key], value[i], f'daily {key}'))
109         maxerr=max(maxerr, near(row['ending_inventory'], E[i,-1], 'daily ending inventory'))
110     if reported_totals is not None:
111         for key, value in ledger.items(): maxerr=max(maxerr, near(reported_totals[key], np.sum(value), f'annual
↪ {key}'))
112     return maxerr
113
114 def model_metadata(kind, cutoff, days):
115     suffix=' ' if days==3 else f'_D{days}'
116     relative=f'artifacts/tails/{kind}_{cutoff}{suffix}.json'
117     if relative not in MODEL_CACHE:
118         path=ROOT/relative
119         if not path.exists(): raise Pending('Missing used tail '+relative)
120         m=json.loads(path.read_text()); cfg=m['configuration']
121         require(cfg['cutoff_day_exclusive']==cutoff, 'tail cutoff metadata')
122         require(cfg['horizon_days']==days and cfg['steps_per_day']==12, 'tail horizon/granularity')
123         require(cfg['history_days']==list(range(max(0, cutoff-28), cutoff)), 'tail mature complete-day window')
124         require(cfg['variable_price']==kind.startswith('q4'), 'tail price setting')
125         require(cfg['adjustments']==(kind in ['q3', 'q4_3']), 'tail adjustment setting')
126         require(m['iterations']==500, 'tail iterations differ from frozen development/evaluation budget')
127         cuts=np.asarray(m['root_cuts']); require(cuts.ndim==2 and cuts.shape[1]==2 and
↪ np.isfinite(cuts).all(), 'one-dimensional tail cuts')
128         MODEL_CACHE[relative]={ 'sha256': sha(path), 'configuration': cfg, 'cut_count': len(cuts), 'iterations': m['i
↪ terations']}
129     return relative, MODEL_CACHE[relative]
130
131 def information(a, meta, kind, candidate, config, allowed=None):
132     rows=meta['decisions']; require(len(rows)==len(a['days'])*4, 'four planning nodes per day')
133     closed=bool(config['closed_daily']); official=kind in ['q3', 'q4_3']; sddp=candidate.startswith('sddp')
134     horizon=int(config['horizon_days']); count=int(config['count']); scale=float(config['tail_scale'])
135     used=set(); slack=[]; tail_hist={}; maxinv=0.; cover={}
136     for i, day in enumerate(a['days']):
137         for k in range(4):
138             row=rows[i*4+k]; now=int(day)*144+k*36; end=min((int(day)+horizon)*144, TEND); length=end-now
139             require(row['as_of']==now and row['release']==k and row['end']==end, 'planning absolute clock')
140             require(row['date']==str(a['dates'][i]), 'planning date')
141             maxinv=max(maxinv, near(row['inventory'], a['state'][i, k*36], 'planning input SOC'))
142             # Derive all matured historical issues first, then use the most
143             # recent 28; this is independent of the producer's search shortcut.
144             historical=[h for h in range(7, int(day)) if h*144+k*36+length<=now] [-28:]
145             if count and len(historical)>count:
146                 positions=np.rint(np.linspace(0, len(historical)-1, count)).astype(int)
147                 historical=[historical[j] for j in positions]
148             require(row['history_days']==historical, 'mature residual block selection')
149             last=max(historical)*144+k*36+length
150             require(row['latest_training_target_exclusive']==last and last<=now, 'last actual training target')
151             slack.append(now-last)
152             if official:
153                 nodes=range(4) if allowed is None else allowed
154                 issue=max(d*144+j*36 for d in [int(day)-1, int(day)] for j in nodes if d*144+j*36<=now)
155                 covered=max(0, min(end, issue+144)-now)
156             else: covered=0
157             require(row['official_covered_slots']==covered, 'already released full-24h coverage')
158             cover[str(covered)]=cover.get(str(covered), 0)+1
159             use_tail=sddp and not closed and end<TEND
160             if use_tail:
161                 month=a['dates'][i].astype('datetime64[M]').astype('datetime64[D]')
162                 cutoff=int((month-np.datetime64('2025-01-01')).astype(int))
163                 D=min(3, (TEND-end)//144)
164                 require(D>=1 and row['tail_model_cutoff']==cutoff and row.get('tail_model_days')==D, 'as-of
↪ month or remaining-day tail mismatch')

```

```

165         path,model=model_metadata(kind,cutoff,D);used.add(path)
166         require(1<=row['tail_cut_count']<=model['cut_count']+1,'active cut count')
167         near(row['tail_price'],0,'SDDP replaces linear tail',0)
168         tail_hist[str(D)]=tail_hist.get(str(D),0)+1
169     else:
170         require(row['tail_model_cutoff'] is None and row['tail_cut_count']==0 and
171         ↪ row.get('tail_model_days') is None,'unexpected tail model')
172         if closed or end==TEND:near(row['tail_price'],0,'true endpoint or constant closed tail',0)
173         elif not kind.startswith('q4'):
174             near(row['tail_price'],scale*float(np.quantile(DATA['day_price'],.25))/ETA,'fixed-price
175             ↪ linear tail',1e-10)
176         else:require(np.isfinite(row['tail_price']) and row['tail_price']>=0,'variable-price linear
177         ↪ tail finite')
178     return {'passed':True,'decisions':len(rows),'minimum_training_maturity_slack_slots':int(min(slack)),
179         'input_SOC_max_error':maxinv,'official_coverage_histogram':cover,
180         'tail_days_histogram':tail_hist,'used_model_files':sorted(used),
181         'scope_limit':'Checks recorded time/index permissions and file metadata. Nonanticipativity of the
182         ↪ producing algorithm additionally relies on separately executed source/mutation reviews; it
183         ↪ cannot be proved from one realized trajectory alone.'}
184
185 def ordinary_case(stem,kind,candidate,allowed=None):
186     a,m,hs=read_pair(stem)
187     near(a['days'],DAYS,'full annual day indices',0)
188     near(a['dates'].astype('datetime64[D]').astype(int),DATES.astype(int),'full annual calendar',0)
189     require(m['kind']==kind and m['candidate']==candidate,'method identity')
190     cfg=m['configuration']
191     require(cfg['start_day']==31 and cfg['end_day']==364 and cfg['final'] is None,'main evaluation endpoints')
192     require(cfg['count']==7 and cfg['horizon_days']==2 and cfg['tail_scale']==1 and cfg['grid']==321,'matched
193     ↪ production configuration')
194     require(cfg['closed_daily']==candidate.startswith('closed'),'daily closure configuration')
195     require(cfg.get('initial',6000)==6000,'annual initial inventory configuration')
196     if allowed is not None:
197         design=m['validation_design']
198         require(design['kind']=='q3' and design['start']==31 and design['stop']==365 and
199         ↪ design['releases']==allowed,'release-ablation design')
200         require(not design.get('stress',False),'annual ablation must not change actual data')
201     phys=physical(a,kind,6000,closed=cfg['closed_daily']);rel=releases(a,kind);ledger=daily_ledger(a)
202     fee_error=compare_reported_ledger(ledger,m['daily'],m['totals'],a['dates'],a['state'])
203     for key,value in phys.items():near(m['validation'][key],value,'reported physical '+key)
204     info=information(a,m,kind,candidate,cfg,allowed)
205     return {'status':'pass','kind':kind,'candidate':candidate,'files':hs,'days':len(a['days']),
206         'independent_totals':{k:float(v.sum()) for k,v in ledger.items()},
207         'daily_and_annual_cost_reconciliation_max_abs':fee_error,'physical':phys,'releases':rel,'informat
208         ↪ ion':info}
209
210 def terminal_case(stem,kind,candidate):
211     a,m,hs=read_pair(stem);source=f'artifacts/annual/{kind}_{candidate}'
212     old,om,oh=read_pair(source)
213     require(m['kind']==kind and m['candidate']==candidate,'terminal case identity')
214     require(m['source_trajectory_sha256']==oh[source+'.npz'],'terminal source trajectory hash')
215     for key in
216     ↪ ['source_metrics_sha256','fixed_suffix_configuration','fixed_suffix_daily','fixed_suffix_decisions']:
217         if key not in m:raise Pending('Missing terminal suffix audit evidence '+key)
218     require(m['source_metrics_sha256']==oh[source+'.json'],'terminal source metrics hash')
219     require(m['prefix_days']==332 and m['replayed_days']==[363,364],'terminal splice boundary')
220     near(a['days'],DAYS,'terminal annual days',0)
221     near(a['dates'].astype('datetime64[D]').astype(int),DATES.astype(int),'terminal annual calendar',0)
222     require(set(a)==set(old),'spliced array columns')
223     prefix_error=0.
224     for key in a:
225         if np.issubdtype(a[key].dtype,np.number):
226             aa=a[key][:332];bb=old[key][:332]
227             require(np.array_equal(np.isnan(aa),np.isnan(bb)),'prefix NaN masks '+key)
228             prefix_error=max(prefix_error,near(np.nan_to_num(aa),np.nan_to_num(bb),'unchanged prefix '+key,0))
229         else:require(np.array_equal(a[key][:332],old[key][:332]),'unchanged prefix '+key)
230     phys=physical(a,kind,6000,terminal=6000);rel=releases(a,kind);ledger=daily_ledger(a)
231     source_ledger=daily_ledger(old)
232     near(m['free_total_cost'],source_ledger['total_cost'].sum(),'independent original free cost')
233     near(m['fixed_total_cost'],ledger['total_cost'].sum(),'independent fixed cost')

```

```

225 near(m['terminal_constraint_increment'],ledger['total_cost'].sum()-source_ledger['total_cost'].sum(),'ter
    ↪ minal cost difference')
226 require(max(m['free_suffix_replay_differences'].values())<=TOL,'producer reported free suffix replay
    ↪ failed')
227 suffix={k:v[332:] for k,v in a.items()};cfg=m['fixed_suffix_configuration'];original=om['configuration']
228 for key in ['count','horizon_days','tail_scale','grid']:require(cfg[key]==original[key],'fixed suffix
    ↪ configuration '+key)
229 require(cfg['start_day']==363 and cfg['end_day']==364 and cfg['final']==6000 and not
    ↪ cfg['closed_daily'],'suffix endpoints')
230 near(cfg['initial'],old['state'][332,0],'suffix initial configuration')
231 near(m['suffix_initial_inventory'],old['state'][332,0],'reported inherited SOC')
232 near(suffix['state'][0,0],old['state'][331,-1],'physical splice SOC')
233 sp=physical(suffix,kind,float(old['state'][332,0]),terminal=6000)
234 for key,value in sp.items():near(m['fixed_suffix_validation'][key],value,'suffix reported physical '+key)
235 sl=daily_ledger(suffix)
236 fee_error=compare_reported_ledger(sl,m['fixed_suffix_daily'],None,suffix['dates'],suffix['state'])
237 info=information(suffix,{k:v[332:] for k,v in old.items()},kind,candidate,cfg)
238 prefix_info=information({k:v[332:] for k,v in old.items()},
239                        {'decisions':om['decisions'][:332*4]},kind,candidate,original)
240 near(ledger['total_cost'].sum(),source_ledger['total_cost'][:332].sum()+sl['total_cost'].sum(),'prefix
    ↪ plus fixed suffix fee')
241 return {'status':'pass','kind':kind,'candidate':candidate,'files':hs,'source_files':oh,'days':334,
242         'independent_totals':{k:float(v.sum()) for k,v in ledger.items()}, 'physical':phys,'releases':rel,
243         'prefix_exact_max_error':prefix_error,'suffix_daily_cost_reconciliation_max_abs':fee_error,
244         'information':{'prefix':prefix_info,'fixed_suffix':info},
245         'replay_scope':'Independently verifies stored source hashes, every reused prefix element, splice
    ↪ SOC, suffix logs/fees and physical constraints. Does not rerun all eight controllers; prior v2
    ↪ review independently replayed both Q2 linear-tail cases.'}

246
247 def main():
248     begin=time.perf_counter();expected=[]
249     for kind in KINDS:
250         expected += [(f'artifacts/annual/{kind}_{c}','kind,c,None,False) for c in METHODS]
251         expected += [(f'artifacts/annual_sddp/{kind}_sddp_markov','kind','sddp_markov',None,False)]
252         expected += [(f'artifacts/experiments/annual_pv_{n}','q3','markov_mpc',a,False) for n,a in
    ↪ ABLATIONS.items()]
253         expected += [(f'artifacts/global-terminal/{k}_{c}','k,c,None,True) for k in KINDS for c in
    ↪ ['affine_mpc','markov_mpc']]
254         require(len(expected)==36,'expected inventory count')
255         cases={};pending=[];failed=[]
256         for stem,kind,candidate,allowed,terminal in expected:
257             try:
258                 result=terminal_case(stem,kind,candidate) if terminal else
    ↪ ordinary_case(stem,kind,candidate,allowed)
259                 cases[stem]=result;print('PASS '+stem,flush=True)
260             except Pending as exc:
261                 cases[stem]={ 'status':'pending','reason':str(exc)};pending.append(stem)
262             except Exception as exc:
263                 cases[stem]={ 'status':'fail','error_type':type(exc).__name__,'reason':str(exc)};failed.append(stem)
    ↪ m)
264                 print('FAIL '+stem+' '+str(exc),flush=True)
265         reused={}
266         for name in ['controller-checks.json','sddp-checks.json','v2-boundary-review.json','global-oracle.json']:
267             p=ROOT/'review'/name
268             if p.exists():reused[str(p.relative_to(ROOT))]=sha(p)
269         status='fail' if failed else 'pending' if pending else 'all_36_pass_within_numerical_scope'
270         code=1 if failed else 2 if pending and '--require-complete' in sys.argv else 0
271         report={'schema_version':1,'reviewer_id':'/root/independent_review','independent':True,
272               'scope':'Production numerical trajectories and recorded information permissions only; no ranking,
    ↪ score, or final project acceptance.',
273               'status':status,'passed':not failed and not pending,'all_available_cases_passed':not failed,
274               'expected_cases':36,'passed_cases':sum(v['status']=='pass' for v in cases.values()),
275               'pending_cases':pending,'failed_cases':failed,'cases':cases,'used_tail_models':MODEL_CACHE,
276               'formula_independence':'No production module imported. Bus balance, battery recurrence, fee
    ↪ piecewise identity, daily/annual aggregation, revision replay, calendar/maturity, tail-file
    ↪ calendar routing, and prefix splice checks are independent NumPy calculations in this file.',
    ↪ 'reused_evidence':reused,
277

```

```

278     'reuse_limitations': 'Source nonanticipativity, future-data mutation, LP/SDDP dual validity, and
    ↪ deterministic fixed-suffix policy reproduction are covered by named earlier reviews. This
    ↪ script does not pretend that recorded timestamps alone prove those algorithmic properties and
    ↪ does not re-solve optimization models.',
279     'audit_source_sha256': sha(__file__), 'data_npz_sha256': sha(ROOT/'artifacts/data.npz'),
280     'observed_production_source_hashes': {str(p.relative_to(ROOT)): sha(p) for p in
    ↪ (ROOT/'src').glob('*.*py')},
281     'execution': {'command': 'python3 review/check-production.py'+(' --require-complete' if
    ↪ '--require-complete' in sys.argv else '')},
282     'exit_code': code, 'runtime_seconds': time.perf_counter()-begin, 'python': sys.version, 'n|
    ↪ umpy': np.__version__,
283     'deterministic_reason': 'Exact calendar/index comparisons and deterministic array
    ↪ formulas; no sampling.'}}
284     OUT.write_text(json.dumps(report, ensure_ascii=False, indent=2, allow_nan=False)+'\n')
285     note=f'''# 生产轨迹独立审计
286
287 审查人 `~/root/independent_review`。本次快照：应有36组，已通过{report['passed_cases']}组，待补{len(pending)}组，
    ↪ 失败{len(failed)}组；状态 `{status}`。详细逐组结论、原文件哈希与缺项见production-audit.json。这里不作候选排名、
    ↪ 模拟评分或最终项目验收。
288
289 运行 `python3 review/check-production.py` 更新现有产物快照。最终运行 `python3 review/check-production.py
    ↪ --require-complete`：缺项退出2，实质检查失败退出1，仅全部36组通过才退出0。
290
291 目标为20个普通年度、4个SDDP年度、4个全年预报消融、8个全局期末6000对照。每条年度轨迹覆盖334日、48,096个10分钟段、
    ↪ 1336个重规划节点。
292
293 本脚本不导入任何生产模型模块。费用用两种等价分段表达式独立重算，并逐日及全年核对；能量平衡、SOC递推/边界、功率、互斥、
    ↪ 午夜连续与真实末端直接由数组检验。原q必须等于0点发布快照，最终r由每段最新合法发布重建；已交付段不能调约，
    ↪ Q2不得有日内调约，头6小时r=q。
294
295 记录中的as-of、发布时点、当前库存、完整24小时预报覆盖、成熟连续历史块均重新按绝对时刻核算。SDDP检查当前月份、
    ↪ 剩余D1/D2/D3、H=T零末值及实际被引用模型的训练截止/配置和哈希。全局期末对照另检查源NPZ与源JSON双哈希、
    ↪ 全部332日前缀数组逐值一致、继承库存、固定后缀日志/费用和拼接。
296
297 复用的既有证据在JSON中逐项列出并哈希：算法非预见性与未来扰动、独立DP/LP比较、SDDP小树及对偶割、v2后缀实际复演。
    ↪ 它们没有在本脚本中重跑；仅凭一条实测轨迹及其时间戳本身，无法证明生产程序没有读取不可见信息。
    ↪ 本脚本也不从保存的NPZ声称重新验证了所有上层训练LP最优性。
298 '''
299     (ROOT/'review/production-audit.md').write_text(note)
300     print(json.dumps({'status': status, 'passed': report['passed_cases'], 'pending': len(pending), 'failed': len(fai|
    ↪ led),
301     'seconds': report['execution']['runtime_seconds']}, flush=True)
302     raise SystemExit(code)
303
304 if __name__ == '__main__': main()

```

review/check-sddp.py

```

1  """Independent extensive-form LP and exhaustive policy audit of finite SDDP."""
2  from pathlib import Path
3  import sys, json, hashlib
4  import numpy as np
5  from scipy.optimize import linprog
6  ROOT=Path(__file__).resolve().parents[1]
7  sys.path.insert(0, str(ROOT/'src'))
8  import sddp
9
10 K=4; ETA=.9; M=5000*24/K; D=1+2*K
11 NET=[[0., 2000.], [1000., 3000.], [-500., 1000.], [800., 2500.]]
12 PRICES=[.2, 1.3, .4, 1.8]
13 INITIAL=np.r_[6000., np.zeros(2*K)]
14
15 def specification(adjust):
16     spec=[('sign', 0)]
17     for k in range(K):
18         if adjust and k: spec.append(('revise', k))
19         spec.append(('deliver', k))

```

```

20     return spec
21
22 def extensive(spec,state,start=0,clairvoyant=False):
23     """Separate bus c/d variables and direct tree recursion, no producer matrices.
24
25     A variable is a dict {index:1}; scalar constants represent fixed state.
26     Signing/revision controls are created BEFORE the next outcome branches.
27     """
28     bounds=[];cost=[];equal=[];rhs=[];upper=[];ub=[]
29     def var(bound=(0,None),weight=0.):
30         j=len(bounds);bounds.append(bound);cost.append(weight);return {j:1.}
31     def constraint(terms,right,equality=False):
32         row={};constant=0.
33         for weight,expr in terms:
34             if isinstance(expr,dict):
35                 for j,v in expr.items():row[j]=row.get(j,0.)+weight*v
36             else:constant+=weight*float(expr)
37         (equal if equality else upper).append(row);(rhs if equality else ub).append(right-constant)
38     def visit(t,E,q,r,prob):
39         if t==len(spec):return
40         kind,hour=spec[t]
41         if kind=='sign':
42             newq=[var() for _ in range(K)]
43             visit(t+1,E,newq,newq.copy(),prob)
44         elif kind=='revise':
45             newr=[r[k] if k<hour else var() for k in range(K)]
46             visit(t+1,E,q,newr,prob)
47         else:
48             for demand in NET[hour]:
49                 node_prob=prob/2;price=PRICES[hour]
50                 c=var((0,M));d=var((0,M));em=var(weight=node_prob*5*price)
51                 bill=var(weight=node_prob*price);end=var((1200,10800))
52                 constraint([(1,end),(-1,E),(-ETA,c),(1/ETA,d)],0,True)
53                 constraint([(-1,r[hour]),(-1,d),(-1,em),(1,c)],-demand)
54                 constraint([(1.5,r[hour]),(-.5,q[hour]),(-1,bill)],0)
55                 constraint([(1.5,r[hour]),(.5,q[hour]),(-1,bill)],0)
56                 qq=q.copy();rr=r.copy();qq[hour]=0.;rr[hour]=0.
57                 visit(t+1,end,qq,rr,node_prob)
58     visit(start,float(state[0]),list(map(float,state[1:1+K])),list(map(float,state[1+K:])),1.)
59     def matrix(rows):
60         a=np.zeros((len(rows),len(bounds)))
61         for i,row in enumerate(rows):
62             for j,v in row.items():a[i,j]=v
63         return a
64     res=linprog(cost,A_ub=matrix(upper),b_ub=ub,A_eq=matrix(equal),b_eq=rhs,bounds=bounds,method='highs')
65     assert res.success,res.message
66     return float(res.fun)
67
68 def policy_value(model,state,t=0):
69     if t==len(model.stages):return 0.
70     stage=model.stages[t];total=0.
71     for k,prob in enumerate(stage.probabilities):
72         sol=stage.solve(state,k,model.cuts[t+1])
73         total+=prob*(sol.stage_cost+policy_value(model,sol.state,t+1))
74     return float(total)
75
76 def one_case(adjust):
77     spec=specification(adjust)
78     stages=[sddp.battery_stage(K,kind,h,NET[h],[PRICES[h]]*2,[.5,.5]) for kind,h in spec]
79     exact=extensive(spec,INITIAL);model=sddp.SDDP(stages,seed=2121+int(adjust))
80     def explore(i,x):x[0]=[1200.,3600.,6000.,8400.,10800.][i%5];return x
81     batches=[]
82     for batch in range(5):
83         model.train(INITIAL,iterations=100,exploration=explore)
84         lower=model.bound(INITIAL);policy=policy_value(model,INITIAL)
85         assert lower<=exact+1e-5 and policy>=exact-1e-5
86         batches.append({'iterations':(batch+1)*100,'lower_bound':lower,'exact_tree_cost':exact,'exact_policy_
87             ↪ cost':policy})
88     if policy-lower<1e-5:break

```

```

88     assert policy_lower<1e-5,('SDDP not converged on toy',adjust,batches)
89     # Check every cut at independently valued state probes at every stage.
90     probes=[] for _ in stages:
91     for E in [1200.,3600.,6000.,8400.,10800.]:
92         x=INITIAL.copy();x[0]=E;inputs,_=model.forward(x)
93         for t,state in enumerate(inputs):probes[t].append(state.copy())
94     # Include nonzero q/r states so both refund and increase branches are tested.
95     for t,(kind,h) in enumerate(spec):
96         if kind=='sign':continue
97         for qval,rval in [(600.,300.), (600.,900.)]:
98             x=INITIAL.copy();x[1+h:1+K]=qval;x[1+K+h:]=rval;probes[t].append(x)
99     cut_max=-np.inf;probe_count=0;upper_max=-np.inf
100    for t,states in enumerate(probes):
101        for x in states:
102            truth=extensive(spec,x,t);cuts=model.cuts[t]
103            cut_max=max(cut_max,max(a+b*x-truth for a,b in cuts));probe_count+=1
104            assert max(a+b*x for a,b in cuts)<=truth+1e-5
105    # Root signing discards fully delivered contracts, so root gradients vanish.
106    root_qr=max(np.abs(b[1:]).max() for _,b in model.cuts[0]);assert root_qr<1e-7
107    # Deterministic one-delivery accounting: q100,r80 ->90; q100,r120 ->130.
108    fee_checks=[]
109    delivery=sddp.battery_stage(K,'deliver',0,[0.],[1.],[1.])
110    for qval,rval,want in [(100.,80.,90.), (100.,120.,130.)]:
111        x=INITIAL.copy();x[1]=qval;x[1+K]=rval
112        sol=delivery.solve(x,0,[]);assert abs(sol.stage_cost-want)<1e-7
113        assert abs(sol.state[1])+abs(sol.state[1+K])<1e-8
114        fee_checks.append({'q':qval,'r':rval,'expected':want,'actual':sol.stage_cost})
115    return {'adjustments':adjust,'stages':len(stages),'full_path_count':16,'status':'pass',
116            'independent_extensive_cost':exact,'sddp_lower_bound':lower,'enumerated_policy_cost':policy,
117            'bound_policy_gap':policy_lower,'training_batches':batches,'cut_state_probes':probe_count,
118            'all_cut_max_lower_bound_violation':cut_max,'root_unused_contract_gradient_max_abs':root_qr,
119            'one_delivery_fee_checks':fee_checks}
120
121    def main():
122        before=hashlib.sha256((ROOT/'src/sddp.py').read_bytes()).hexdigest()
123        cases=[one_case(False),one_case(True)]
124        after=hashlib.sha256((ROOT/'src/sddp.py').read_bytes()).hexdigest();assert before==after,'Source changed
125        ↳ during review run'
126        result={'status':'pass_for_finite_battery_sddp','scope':'Independent K4 one-day finite-tree tests,
127        ↳ with/without pre-noise revisions; exact extensive LP and all-path policy values. Not validation of
128        ↳ original-data MPC terminal quality or universal negative-cost models.',
129                'cases':cases,'reviewed_source_sha256':before,
130                'input':{'K':K,'days':1,'net_support_kwh':NET,'prices_yuan_per_kwh':PRICES,'probabilities':[.5,.5]
131                ↳ },'initial_state':INITIAL.tolist()},
132                'initial_bound_condition':'Every supplied theta lower bound must be a valid remaining-value lower
133                ↳ bound. BatteryStage uses zero because all remaining costs are nonnegative.'}
134        (ROOT/'review/sddp-checks.json').write_text(json.dumps(result,ensure_ascii=False,indent=2)+'\n')
135        print(json.dumps(result,ensure_ascii=False))
136    if __name__=='__main__':main()

```

review/check-v2-boundaries.py

```

1  """Independent limited audit of v2 terminal handling and economic-only Q1.
2
3  python3 review/check-v2-boundaries.py
4  python3 review/check-v2-boundaries.py --month-only
5  The latter completes the actual cross-month test when its D1 model is ready.
6  """
7  from pathlib import Path
8  import hashlib, importlib.util, json, sys, time
9  import numpy as np
10
11  ROOT=Path(__file__).resolve().parents[1]
12  sys.path.insert(0,str(ROOT/'src'))
13  import run as rr
14  import control as cc
15  import dispatch
16

```

```

17 DATA=dict(np.load(ROOT/'artifacts/data.npz'))
18 SEL=json.loads((ROOT/'artifacts/forecast-selection.json').read_text())
19 OUT=ROOT/'review/v2-boundary-review.json'
20 FILES=['src/run.py','src/control.py','src/dispatch.py','src/check_terminal.py','src/sddp.py']
21
22 def sha(path):return hashlib.sha256(Path(path).read_bytes()).hexdigest()
23 def hashes():return {p:sha(ROOT/p) for p in FILES}
24 def error(a,b):return float(np.max(np.abs(np.asarray(a)-np.asarray(b))))
25 def check(a,b,tol=1e-6):
26     v=error(a,b);assert v<=tol,v;return v
27
28 def q1_check():
29     spec=importlib.util.spec_from_file_location('independent_oracle',ROOT/'review/check-global-oracle.py')
30     oracle=importlib.util.module_from_spec(spec);spec.loader.exec_module(oracle)
31     net=(DATA['day_load']-DATA['day_pv'])/6;p=DATA['day_price']
32     solved,arrays=oracle.one_case(oracle.matrices(144),net,p,6000.,'q1_independent')
33     producer=dispatch.solve(net[None,:],p[None,:],hard=True)
34     saved=json.loads((ROOT/'artifacts/q1.json').read_text());a=dict(np.load(ROOT/'artifacts/q1.npz'))
35     values=[producer['expected_cost'],producer['lp_objective'],saved['expected_cost'],saved['lp_objective'],f]
    ↪ load(p@a['q'])]
36     for value in values:check(value,solved['objective_cost_yuan'],1e-6)
37     c,d,q,E=[a[k] for k in ['c','d','q','state']]
38     checks={'max_simultaneous':check(np.minimum(c,d),0),
39             'soc_recurrence':check(np.diff(E),.9*c-d/.9),
40             'initial':check(E[0],6000),'terminal':check(E[-1],6000),
41             'max_shortage_without_emergency':float(max(0,(net+c-d-q).max())),
42             'max_economic_objective_difference':max(abs(v-solved['objective_cost_yuan']) for v in values)}
43     assert checks['max_shortage_without_emergency']<1e-6
44     assert E.min()>=1200-1e-6 and E.max()<=10800+1e-6
45     assert min(q.min(),c.min(),d.min())>=-1e-6 and max(c.max(),d.max())<=5000/6+1e-6
46     # Algebraic normalization on deliberately simultaneous random flows.
47     rng=np.random.default_rng(250611);c=rng.uniform(0,5000/6,1000);d=rng.uniform(0,5000/6,1000)
48     rem=np.minimum(c,d/.9**2);c1=c-rem;d1=d-.9**2*rem
49     checks['normalization_delta_error']=check(.9*c-d/.9,.9*c1-d1/.9)
50     checks['normalization_net_supply_error']=check(d1-c1-(d-c),(1-.9**2)*rem)
51     checks['normalization_simultaneous']=check(np.minimum(c1,d1),0)
52     assert (c1>=-1e-10).all() and (d1>=-1e-10).all()
53     return {'passed':True,'cost_yuan':solved['objective_cost_yuan'],'independent_duality_gap':solved['checks']
    ↪ ['primal_minus_repaired_dual'],
54             'checks':checks,'q1_artifact_sha256':{'json':sha(ROOT/'artifacts/q1.json'),'npz':sha(ROOT/'artifa
    ↪ cts/q1.npz')},
55     'scope':'Explicit-spill independent LP agrees with economic-only producer and regenerated
    ↪ artifact; normalization preserves SOC and permits extra free spill.'}
56
57 def route_check():
58     class Intercept(Exception):pass
59     original=rr.tail_file;rows=[]
60     def record(kind,cutoff,days=3):
61         rows.append({'kind':kind,'cutoff':cutoff,'days':days});raise Intercept()
62     rr.tail_file=record
63     try:
64         for length,expected in [(3,1),(4,2),(5,3),(8,3)]:
65             try:rr.simulate(DATA,SEL,list(range(58,58+length)),'q2','sddp_mpc',count=3)
66             except Intercept:pass
67             else:raise AssertionError('Expected a tail request')
68             assert rows[-1]=={'kind':'q2','cutoff':31,'days':expected}
69             # H=T: there must be no tail request; nonstandard initial proves it is
70             # inherited even when this standalone replay starts inside the year.
71             a,m=rr.simulate(DATA,SEL,[363,364],'q2','sddp_mpc',count=3,initial=7123.)
72             check(a['state'][0,0],7123)
73             assert all(z['tail_model_days'] is None and z['tail_cut_count']==0 and z['tail_price']==0 for z in
    ↪ m['decisions'])
74     finally:rr.tail_file=original
75     # Numeric reachable-set proof for every prefix action, independent of state.
76     prefix_last_day=362;T=365*144
77     remaining=T-(np.arange(31*144,(prefix_last_day+1)*144))-1
78     lower=6000-remaining*.9*(5000/6);upper=6000+remaining*(5000/6)/.9
79     assert lower.max()<=1200 and upper.min()>=10800
80     return {'passed':True,'intercepted_requests':rows,'actual_zero_tail_decisions':len(m['decisions']),

```

```

81     'optional_initial_kwh':7123., 'prefix_last_date':str(DATA['dates'][prefix_last_day]),
82     'minimum_remaining_steps_in_prefix':int(remaining.min()),
83     'steps_sufficient_for_entire_SOC_range':7,
84     'minimum_prefix_reachability_slack_kwh':float(min(1200-lower.max(),upper.min()-10800)),
85     'scope':'Intercepted file requests test routing only. The zero-tail case executes actual original
    ↪ data.')}
86
87 def suffix_check():
88     rows=[]
89     for candidate in ['affine_mpc','markov_mpc']:
90         path=ROOT/f'artifacts/annual/q2_{candidate}.npz'
91         a=dict(np.load(path));m=json.loads(path.with_suffix('.json').read_text())
92         check(a['days'],np.arange(31,365),0)
93         config=m['configuration'];assert config['horizon_days']==2 and config['final'] is None
94         kw={k:config[k] for k in ['count','horizon_days','tail_scale','grid']}
95         ix=len(a['days'])-4;init=float(a['state'][ix,0]);days=a['days'][ix:].tolist()
96         free,fm=rr.simulate(DATA,SEL,days,'q2',candidate,initial=init,**kw)
97         fixed,xm=rr.simulate(DATA,SEL,days,'q2',candidate,initial=init,final=6000.,**kw)
98         two,tm=rr.simulate(DATA,SEL,days[-2:], 'q2',candidate,initial=float(a['state'][-2,0]),final=6000.,**kw)
99         full_error={};prefix_error={};suffix_error={}
100        for key in ['q','r','c','d','emergency','spill','state']:
101            full_error[key]=check(free[key],a[key][ix:],1e-5)
102            prefix_error[key]=check(fixed[key][:2],free[key][:2],1e-5)
103            suffix_error[key]=check(fixed[key][2:],two[key],1e-5)
104        check(fixed['state'][-1,-1],6000)
105        check(two['state'][0,0],a['state'][-2,0])
106        check(fixed['state'][1:,0],fixed['state'][:-1,-1])
107        rows.append({'candidate':candidate,'source_sha256':sha(path),'initial_four_day_inventory':init,
108                    'suffix_initial_inventory':float(a['state'][-2,0]),
109                    'free_replay_max_differences':full_error,'fixed_prefix_max_differences':prefix_error,
110                    'two_day_vs_four_day_fixed_suffix_differences':suffix_error,
111                    'free_final_inventory':fm['validation']['final_inventory'],'fixed_final_inventory':xm['v']
112                    ↪ alidation']['final_inventory']})
113    return {'passed':True,'cases':rows,
114            'scope':'Two Q2 linear-tail policies only; four-day replay, same two-day prefix under free/fixed
    ↪ global end, and exact match to inherited two-day suffix. This does not establish reuse across
    ↪ changed SDDP tail model versions.'}
114
115 def month_check():
116     path=rr.tail_file('q2',31,1)
117     if not path.exists():return
    ↪ {'passed':False,'status':'pending_input','required_file':str(path.relative_to(ROOT))}
118     model=json.loads(path.read_text());meta=model['configuration']
119     assert meta['cutoff_day_exclusive']==31 and meta['horizon_days']==1 and max(meta['history_days'])<31
120     a,m=rr.simulate(DATA,SEL,[58,59,60],'q2','sddp_mpc',count=3,initial=6432.)
121     check(a['state'][0,0],6432)
122     check(a['state'][1:,0],a['state'][:-1,-1])
123     for z in m['decisions']:
124         if z['end']<61*144:
125             assert z['tail_model_cutoff']==31 and z['tail_model_days']==1 and z['tail_cut_count']>0
126         else:assert z['tail_model_days'] is None and z['tail_cut_count']==0
127     return {'passed':True,'status':'executed','model_sha256':sha(path),'decisions':m['decisions'],
128            'midnight_inventories':a['state'][:, -1].tolist(),'validation':m['validation'],
129            'scope':'Actual February 28 root horizon reaches March; uses known February cutoff with only one
    ↪ remaining auxiliary day, then no tail at H=T.'}
130
131 def main():
132     start=time.perf_counter();before=hashes();month_only='--month-only' in sys.argv
133     if month_only:
134         report=json.loads(OUT.read_text())
135         # Only check_terminal may receive added guards without affecting earlier
136         # runtime tests; production numerical modules must still match.
137         for key in ['src/run.py','src/control.py','src/dispatch.py','src/sddp.py']:
138             assert report['source_hashes'][key]==before[key],key
139         report['checks']['actual_cross_month_D1']=month_check()
140     else:
141         report={'schema_version':1,'reviewer_id': '/root/independent_review','independent':True,
142                'scope':'v2_boundary_and_q1_only','annual_final_review':'pending','checks':{}}
```

```

143         for name,fn in
            ↪ [(('q1_economic_and_normalization',q1_check),('tail_routing_and_reachable_prefix',route_check),
144                ('inherited_suffix_replay',suffix_check),('actual_cross_month_D1',month_check))]:
145             print(name,flush=True);report['checks'][name]=fn()
146         after=hashes()
147         for key in ['src/run.py','src/control.py','src/dispatch.py','src/sddp.py']:assert
            ↪ before[key]==after[key],key
148         report['source_hashes']=after
149         report['passed']=all(v['passed'] for v in report['checks'].values())
150         report['status']='pass_within_scope' if report['passed'] else 'pending_actual_D1_model_test'
151         report.setdefault('executions',[]).append({'command':'python3 review/check-v2-boundaries.py'+(
            ↪ --month-only' if month_only else ''),
152                'exit_code':0,'runtime_seconds':time.perf_counter()-start})
153         OUT.write_text(json.dumps(report,ensure_ascii=False,indent=2,allow_nan=False)+'\n')
154         print(json.dumps({'status':report['status'],'seconds':time.perf_counter()-start}),flush=True)
155
156 if __name__=='__main__':main()

```

revision/q1_and_billing.py

```

1  """Independent periodic-inventory LP and two billing interpretations.
2  Run: python3 revision/q1_and_billing.py ; immutable source results read only.
3  """
4  from pathlib import Path
5  import json, hashlib, csv, platform, sys
6  import numpy as np
7  from scipy.optimize import linprog
8  from scipy.sparse import coo_matrix
9  ROOT=Path(__file__).resolve().parents[1]; OUT=ROOT/'revision/numeric-audit';OUT.mkdir(exist_ok=True)
10 BASE=ROOT.parent/'C 题 _ 跨日随机控制'
11 sha=lambda p:hashlib.sha256(Path(p).read_bytes()).hexdigest()
12 data=dict(np.load(ROOT/'artifacts/data.npz'));T=144; eta=.9; power=5000/6
13 # Independent variable layout q[144], c[144], d[144], E[145].
14 net=(data['day_load']-data['day_pv'])/6;p=data['day_price'];n=4*T+1
15 obj=np.r_[p,np.zeros(3*T+1)]
16 ri=[];ci=[];vv=[]
17 for t in range(T):
18     for c,v in [(T+t,-eta),(2*T+t,1/eta),(3*T+t,-1),(3*T+t+1,1)]:ri.append(t);ci.append(c);vv.append(v)
19 ri.extend([T,T]);ci.extend([3*T,4*T]);vv.extend([-1,1])
20 Aeq=coo_matrix((vv,(ri,ci)),shape=(T+1,n)).tocsr()
21 Aub=coo_matrix((np.tile([-1,1,-1],T),(np.repeat(np.arange(T),3),np.array([t,T+t,2*T+t] for t in
            ↪ range(T))).ravel()))).shape=(T,n)).tocsr()
22 resall=[]
23 for fixed in [6000,None]:
24     bounds=[(0,None)]*T+[(0,power)]*(2*T)+[(1200,10800)]*(T+1)
25     if fixed is not None:bounds[3*T]=(fixed,fixed)
26     z=linprog(obj,A_ub=Aub,b_ub=-net,A_eq=Aeq,b_eq=np.zeros(T+1),bounds=bounds,method='highs',options={'dual_fea'
            ↪ sibility_tolerance':1e-8,'primal_feasibility_tolerance':1e-8})
27     assert z.success,z.message
28     q=z.x[:T];c=z.x[T:2*T].copy();d=z.x[2*T:3*T].copy();E=z.x[3*T:]
29     remove=np.minimum(c,d/eta**2);c-=remove;d-=eta**2*remove
30     spill=q+d-c-net
31     dual=-net@z.ineqlin.marginals+sum(lo*m for (lo,hi),m in zip(bounds,z.lower.marginals) if lo is not
            ↪ None)+sum(hi*m for (lo,hi),m in zip(bounds,z.upper.marginals) if hi is not None)
32     check={'balance_max_abs':float(abs(q+d-c-spill-net).max()),'soc_recurrence_max_abs':float(abs(np.diff(E)-eta
            ↪ *c+d/eta).max()),'cyclic_gap':float(abs(E[-1]-E[0])), 'power_max_kw':float(max(c.max(),d.max())*6), 'soc_m
            ↪ in':float(E.min()),'soc_max':float(E.max()),'simultaneous_charge_discharge_max':float(np.minimum(c,d).ma
            ↪ x()),'spill_min':float(spill.min()),'duality_gap':float(abs(z.fun-dual))}
33     assert max(check[k] for k in ['balance_max_abs','soc_recurrence_max_abs','cyclic_gap','duality_gap'])<1e-6
34     assert check['soc_min']>=1200-1e-6 and check['soc_max']<=10800+1e-6 and check['power_max_kw']<=5000+1e-6 and
            ↪ check['spill_min']>=-1e-6
35     tag='fixed6000' if fixed else 'free_cyclic'
36     np.savez_compressed(OUT/f'q1_{tag}.npz',q=q,c=c,d=d,state=E,spill=spill)
37     resall.append({'model':tag,'total_cost':float(p@q),'initial_inventory':float(E[0]),'final_inventory':float(E
            ↪ [-1]),'purchase_energy':float(q.sum()),'validation':check})
38     resall[1]['saving']=resall[0]['total_cost']-resall[1]['total_cost'];resall[1]['saving_percent']=100*resall[1]
            ↪ ['saving']/resall[0]['total_cost']
39     assert abs(resall[0]['total_cost']-json.loads((ROOT/'artifacts/q1.json').read_text())['expected_cost'])<1e-6

```

```

40 rows=[];daily=[];inputs={}
41 for endpoint in ['free','fixed6000']:
42     for kind in ['q2','q3','q4_2','q4_3']:
43         f=BASE/'artifacts'/'('annual' if endpoint=='free' else
44             ↪ 'global-terminal')/f'{kind}_markov_mpc.npz';a=dict(np.load(f));inputs[str(f)]=sha(f)
45         q,r,em,price=[a[k] for k in ['q','r','emergency','price']];red=np.maximum(q-r,0);inc=np.maximum(r-q,0)
46         old=price*(q+1.5*inc-.5*red+5*em);new=price*(q+1.5*inc+.5*red+5*em)
47         # Causal shadow-policy construction: retain q, battery and emergency; refuse returns and spill surplus.
48         rn=np.maximum(q,r);sn=a['spill']+red
49         dominated_free=price*(q+1.5*np.maximum(rn-q,0)+5*em)
50         balance=rn+em+a['d']-a['c']-sn-(data['load'][a['days']]-data['pv'][a['days']])/6
51         assert abs(balance).max()<1e-6 and np.min(price)>=0
52         assert abs((new-old-price*red).max())<1e-6
53         row={'endpoint':endpoint,'kind':kind,'original_bill':float(old.sum()),'alternative_bill_same_trajectory':fl
54             ↪ oat(new.sum()),'bill_increase':float((new-old).sum()),'bill_increase_percent':float(100*(new-old).sum()
55             ↪ /old.sum()),'retreated_kwh':float(red.sum()),'no_return_shadow_policy_bill':float(dominated_free.sum())
56             ↪ ,'shadow_bill_saving_vs_repriced':float((new-dominated_free).sum()),'shadow_balance_max_abs':float(abs(
57             ↪ balance).max()),'initial_inventory':float(a['state'][0,0]),'final_inventory':float(a['state'][-1,-1]),'
58             ↪ min_price':float(price.min())}
59         rows.append(row)
60     for i,date in enumerate(a['dates']):daily.append({'endpoint':endpoint,'kind':kind,'date':str(date),'origina
61         ↪ l_bill':float(old[i].sum()),'alternative_bill_same_trajectory':float(new[i].sum()),'bill_increase':floo
62         ↪ t((new[i]-old[i]).sum()),'no_return_shadow_policy_bill':float(dominated_free[i].sum())})
63 for name,rs in [('billing-summary',rows),('billing-daily',daily)]:
64     with (OUT/f'{name}.csv').open('w',encoding='utf-8-sig',newline='') as
65         ↪ f:w=csv.DictWriter(f,fieldnames=list(rs[0]));w.writeheader();w.writerows(rs)
66 report={'status':'PASS','q1':resall,'billing':rows,'interpretation':Alternative billing reprices identical
67     ↪ trajectories; shadow no-return policy is feasible but neither is claimed optimal under alternative
68     ↪ billing.','input_sha256':{str(ROOT/'artifacts/data.npz'):sha(ROOT/'artifacts/data.npz'),**inputs},'script
69     ↪ _sha256':sha(__file__),'python':sys.version,'platform':platform.platform()}
70 (OUT/'q1-billing-audit.json').write_text(json.dumps(report,ensure_ascii=False,indent=2)+'\n')
71 print(json.dumps({'q1':resall,'billing':rows},ensure_ascii=False,indent=2))

```

revision/billing_january_replan.py

```

1 """Causal Jan25-31 paired sensitivity. Original src remains immutable.
2 Only replace convex delivered-contract epigraph from max(1.5R-.5Q,.5R+.5Q)
3 to max(1.5R-.5Q,-.5R+1.5Q); same physics, forecasts, tail and feedback.
4 """
5 from pathlib import Path
6 import sys,json,inspect,hashlib,time
7 from concurrent.futures import ProcessPoolExecutor,as_completed
8 import numpy as np
9 ROOT=Path(__file__).resolve().parents[1]
10 sys.path.insert(0,str(ROOT.parent/'C 题 _ 跨日随机控制/vendor'))
11 sys.path.insert(0,str(ROOT/'src'))
12 import run,control,dispatch
13 OUT=ROOT/'revision/numeric-audit';OUT.mkdir(exist_ok=True)
14 def worker(job):
15     kind,alternative=job
16     source=inspect.getsource(control.affine_plan)
17     replacements={'1.5R+.5Q-F': '-.5R+1.5Q-F', '-.5Q0-.5R0': '-1.5Q0+.5R0', '.5*rs+.5*qs': '-.5*rs+1.5*qs'}
18     if alternative:
19         for old,new in replacements.items():
20             assert source.count(old)==1,(old,source.count(old))
21             source=source.replace(old,new)
22     namespace=dict(control.__dict__);exec(compile(source,'billing_epigraph_variant','exec'),namespace)
23     run.affine_plan=namespace['affine_plan']
24     run.settlement=lambda q,r,em,p:dispatch.settlement(q,r,em,p,refund=False)
25     else:run.affine_plan=control.affine_plan;run.settlement=dispatch.settlement
26     data=dict(np.load(ROOT/'artifacts/data.npz'));selection=json.loads((ROOT/'artifacts/forecast-selection.json'
27     ↪ ).read_text())
28     a,m=run.simulate(data,selection,list(range(24,31)),kind,'markov_mpc',final=6000.)
29     q,r,em,p=[a[k] for k in ['q','r','emergency','price']]
30     bill=p*(q+1.5*np.maximum(r-q,0)+.5 if alternative else -.5)*np.maximum(q-r,0)+5*em
31     assert abs(bill.sum()-m['totals']['total_cost'])<1e-6
32     m['billing_variant']='additive_penalty' if alternative else
33     ↪ 'replacement_refund';m['transformed_planner_sha256']=hashlib.sha256(source.encode()).hexdigest()

```

```

32 m['retreated_kwh']=float(np.maximum(q-r,0).sum());m['billing_recompute_gap']=float(abs(bill.sum()-m['totals']
↪  ]['total_cost']))
33 stem=OUT/f'jan_{kind}_{m["billing_variant"]}'
34 np.savez_compressed(str(stem)+'.npz',**a);Path(str(stem)+'.json').write_text(json.dumps(m,ensure_ascii=False)
↪  ,indent=2)+'\n')
35 return {'kind':kind,'variant':m['billing_variant'],'total_cost':m['totals']['total_cost'],'retreated_kwh':m[
↪  'retreated_kwh'],'validation':m['validation'],'runtime_seconds':m['runtime_seconds']}
36 if __name__=='__main__':
37     results=[]
38     with ProcessPoolExecutor(max_workers=4) as pool:
39         for fut in as_completed([pool.submit(worker,j) for j in [(k,a) for k in ['q3','q4_3'] for a in
↪         [False,True]])]):
40             r=fut.result();results.append(r);print(json.dumps(r,ensure_ascii=False),flush=True)
41 (OUT/'january-billing-replan.json').write_text(json.dumps({'status':'PASS','results':results,'scope':'January
↪  25-31 only; unchanged pre-existing parameters; no formal-period tuning. Not full-year optimal
↪  values.','script_sha256':hashlib.sha256(Path(__file__).read_bytes()).hexdigest(),ensure_ascii=False,ind
↪  ent=2)+'\n')

```

revision/verify_billing_january.py

```

1 """Independent validation of stored January variant outputs, no planner imports."""
2 from pathlib import Path
3 import numpy as np,json,hashlib,csv
4 ROOT=Path(__file__).resolve().parents[1];O=ROOT/'revision/numeric-audit';data=np.load(ROOT/'artifacts/data.npz'
↪  z'); rows=[];files={}
5 for k in ['q3','q4_3']:
6     for variant in ['replacement_refund','additive_penalty']:
7         p=O/f'jan_{k}_{variant}.npz';a=dict(np.load(p));m=json.loads(p.with_suffix('.json').read_text());files[str(
↪         p)]=hashlib.sha256(p.read_bytes()).hexdigest()
8         q,r,em,px,c,d,s,E=[a[t] for t in ['q','r','emergency','price','c','d','spill','state']]
9         bill=px*(q+1.5*np.maximum(r-q,0))+(-.5 if variant=='replacement_refund' else .5)*np.maximum(q-r,0)+5*em
10        phys=max(float(abs(r+em+d-c-s-(data['load'][a['days']]-data['pv'][a['days']])/6).max()),float(abs(np.diff(E)
↪        ,axis=1)-.9*c+d/.9).max()),float(abs(E[1:,0]-E[:-1,-1]).max()))
11        assert phys<1e-6 and abs(E[0,0]-6000)<1e-6 and abs(E[-1,-1]-6000)<1e-6
12        assert E.min(>=1200-1e-6 and E.max(<=10800+1e-6 and np.minimum(c,d).max(<1e-6 and
↪        max(c.max(),d.max())*6<=5000+1e-6
13        assert abs(bill.sum()-m['totals']['total_cost'])<1e-6
14        assert np.array_equal(a['days'],np.arange(24,31))
15        assert np.min([q.min(),r.min(),em.min(),s.min(),c.min(),d.min()])>=-1e-6
16        # Before a release, all effective deliveries must equal the already published latest valid contract.
17        for i in range(7):
18            assert abs(a['releases'][i,0]-q[i]).max(<1e-6
19            for j in range(4):
20                assert abs(a['releases'][i,j,j*36:(j+1)*36]-r[i,j*36:(j+1)*36]).max(<1e-6
21            repriced=px*(q+1.5*np.maximum(r-q,0)+.5*np.maximum(q-r,0)+5*em)
22            rows.append({'kind':k,'variant':variant,'original_meter_bill':float(bill.sum()),'new_meter_bill':float(repr
↪            iced.sum()),'retreated_kwh':float(np.maximum(q-r,0).sum()),'max_physics_error':phys})
23 for k in ['q3','q4_3']:
24     old,new=[next(r for r in rows if r['kind']==k and r['variant']==v) for v in
↪     ['replacement_refund','additive_penalty']]
25     new['improvement_vs_old_policy_new_meter']=old['new_meter_bill']-new['new_meter_bill']
26     new['new_bill_increase_vs_old_bill']=new['new_meter_bill']-old['original_meter_bill']
27     new['new_bill_increase_percent']=100*new['new_bill_increase_vs_old_bill']/old['original_meter_bill']
28     r={'status':'PASS','rows':rows,'input_sha256':files,'source_sha256':{str(p):hashlib.sha256(p.read_bytes()).he
↪     xdigest() for p in
↪     [ROOT/'revision/billing_january_replan.py',ROOT/'artifacts/data.npz',ROOT/'artifacts/forecast-selection.j
↪     son',*sorted((ROOT/'src').glob('*.py'))]},'scope':'Independent recomputation of outputs; January
↪     development diagnostic, not an out-of-sample performance claim.'}
29 (O/'january-billing-independent.json').write_text(json.dumps(r,ensure_ascii=False,indent=2)+'\n');print(json.
↪     dumps(rows,ensure_ascii=False,indent=2))

```

revision/strong_baseline.py

```

1 from pathlib import Path
2 import json,sys,hashlib,concurrent.futures
3 import numpy as np
4 ROOT=Path(__file__).resolve().parents[1];BASE=ROOT.parent/'C 题 _ 跨日随机控制'

```

```

5 sys.path.insert(0,str(BASE/'src'))
6 from run import simulate
7 OUT=ROOT/'revision/results';OUT.mkdir(exist_ok=True)
8 def one(kind):
9     p=BASE/f'artifacts/annual/{kind}_cross_baseline'
10    a=dict(np.load(p.with_suffix('.npz')));m=json.loads(p.with_suffix('.json').read_text());data=dict(np.load(BA
    ↪ SE/'artifacts/data.npz'));selection=json.loads((BASE/'artifacts/forecast-selection.json').read_text())
11    days=a['days'][-2:].tolist();initial=float(a['state'][-2,0]);kw={k:m['configuration'][k] for k in
    ↪ ['count','horizon_days','tail_scale','grid']}
12    free,fm=simulate(data,selection,days,kind,'cross_baseline',initial=initial,**kw)
13    differences={k:float(np.max(np.abs(free[k]-a[k][-2:]))) for k in
    ↪ ['q','r','c','d','emergency','spill','state']};assert max(differences.values())<1e-5
14    fixed,fx=simulate(data,selection,days,kind,'cross_baseline',initial=initial,final=6000,**kw)
15    combined={k:np.concatenate([v[:-2],fixed[k]]) for k,v in a.items()}
16    assert np.max(np.abs(combined['state'][1:,0]-combined['state'][:-1,-1]))<1e-6
17    bill=(combined['price']*(combined['q']+1.5*np.maximum(combined['r']-combined['q'],0)-.5*np.maximum(combined[
    ↪ 'q']-combined['r'],0)+5*combined['emergency'])).sum(1)
18    main=dict(np.load(BASE/f'artifacts/global-terminal/{kind}_markov_mpc.npz'));mb=(main['price']*(main['q']+1.5
    ↪ *np.maximum(main['r']-main['q'],0)-.5*np.maximum(main['q']-main['r'],0)+5*main['emergency'])).sum(1)
19    diff=bill-mb;rng=np.random.default_rng(20260911);starts=rng.integers(0,328,size=(10000,48));indices=(starts[
    ↪ :,None]+np.arange(7)).reshape(10000,-1)[:,:334];ci=np.quantile(diff[indices].sum(1),[.025,.975])
20    np.savez_compressed(OUT/f'{kind}_cross_baseline_fixed.npz',**combined)
21    result={'kind':kind,'baseline_total':float(bill.sum()),'main_total':float(mb.sum()),'saving':float(diff.sum(
    ↪ )), 'saving_pct':float(diff.sum()/bill.sum()*100), 'saving_95ci':ci.tolist(), 'positive_days':int((diff>1e-
    ↪ 6).sum()), 'negative_days':int((diff <
    ↪ -1e-6).sum()), 'prefix_replay_differences':differences, 'suffix_validation':fx['validation'], 'baseline_dai
    ↪ ly':bill.tolist(), 'main_daily':mb.tolist(), 'source_sha256':hashlib.sha256(p.with_suffix('.npz').read_byt
    ↪ es()).hexdigest(), 'prefix_scope': 'Exact suffix reuse for unchanged two-midnight greedy model; free suffix
    ↪ reproduced before fixed suffix.'}
22    (OUT/f'{kind}_strong_baseline.json').write_text(json.dumps(result,indent=2)+'\n');return result
23 def export_pairs(rows):
24     import csv
25     with (OUT/'daily-paired-costs.csv').open('w',newline='',encoding='utf-8-sig') as f:
26         w=csv.writer(f);w.writerow(['kind','date','baseline_yuan','main_yuan','saving_yuan'])
27         for row in rows:
28             dates=np.load(OUT/f'{row['kind']}_cross_baseline_fixed.npz')['dates']
29             for date,b,m in
    ↪ zip(dates,row['baseline_daily'],row['main_daily']):w.writerow([row['kind'],str(date),b,m,b-m])
30 if __name__=='__main__':
31     if '--export-only' in sys.argv:r=json.loads((OUT/'strong_baseline.json').read_text())
32     else:
33         with concurrent.futures.ProcessPoolExecutor(max_workers=2) as
    ↪ pool:r=list(pool.map(one,['q2','q3','q4_2','q4_3']))
34     (OUT/'strong_baseline.json').write_text(json.dumps(r,indent=2)+'\n')
35     export_pairs(r)

```

revision/build_required_tables.py

```

1 from pathlib import Path
2 import numpy as np,json,hashlib
3 ROOT=Path(__file__).resolve().parents[1]; BASE=ROOT.parent/'C 题 _ 跨日随机控制';T=ROOT/'tables'
4 DATES=['2025-03-20','2025-06-21','2025-09-23','2025-12-21']
5 def h(t):return f'{t//6:02d}:{t%6*10:02d}'
6 def span(a,b):return h(a)+'--'+h(b)
7 def f(x):return f'{x:,.2f}'
8 def table(caption,cols,body):return '\\begin{table}[H]\\centering\\small\\setlength{\\tabcolsep}{4pt}\\caption{
    ↪ n{'+caption+'}\\n\\begin{tabular}{'+cols+'}\\toprule\\n'+body+'\\n\\bottomrule\\end{tabular}\\end{table}\\n'
9 checks=[]
10 emergency=[]
11 for kind,label in [('q2','问题二'),('q3','问题三'),('q4_2','问题四日前'),('q4_3','问题四日内')]:
12     p=BASE/f'artifacts/global-terminal/{kind}_markov_mpc.npz';a=dict(np.load(p));blocks=[];events=[]
13     for day in DATES:
14         i=list(a['dates'].astype(str)).index(day);q,r,c,d,e,price=[a[k][i] for k in
    ↪ ['q','r','c','d','emergency','price']];ordinary=float((price*(q+1.5*np.maximum(r-q,0)-.5*np.maximum(q-r
    ↪ ,0))).sum());ec=float((5*price*e).sum())
15     # All values from fixed-terminal full trajectories. Actual physical purchase is r+e.
16     lines=['时间段 & 购电量 & 时间段 & 购电量 & 时间段 & 购电量\\\\\\\\\\\\\\\\midrule']

```

```

17 for hrs in [[10,12,14],[16,18,20]]:
18     cells=[]
19     for hh in hrs:
20         j=hh*6;v=f(r[j]) if kind in ('q2','q4_2') else f(r[j])+(''+f(q[j]))+'';cells += [span(j,j+1),v]
21     lines.append(' & '.join(cells)+'\\\\')
22 lines += ['\\midrule','全天合同购电量 & \\multicolumn{2}{r}{'+f(r.sum())+'} & 全天合同购电费 &
↪ \\multicolumn{2}{r}{'+f(ordinary)+'}\\\\','全天实际购电量 & \\multicolumn{2}{r}{'+f((r+e).sum())+'} &
↪ 全天总购电费 & \\multicolumn{2}{r}{'+f(ordinary+ec)+'}\\\\']
23 caption=label+' '+day+'购电结果 (电量 kWh, 费用元) '
24 s='\\subsubsection*{'+label+'': '+day+'\\n'+table(caption,'lr lr lr','\\n'.join(lines))
25 if kind in ('q3','q4_3'):s+='\\noindent\\small 购电量列为最终合同量, 括号内为 0 时原合同。\\normalsize\\n'
26 lines=['时间段 & 充电量 & 放电量 & 时间段 & 充电量 & 放电量\\\\\\midrule']
27 for j in [0,48,96]:lines.append(' & '.join([span(j,j+24),f(c[j:j+24].sum()),f(d[j:j+24].sum()),span(j+24,j+
↪ 48),f(c[j+24:j+48].sum()),f(d[j+24:j+48].sum())]+''\\\\'))
28 lines+=['\\midrule','0:00库存 & \\multicolumn{2}{r}{'+f(a['state'][i,0])+'} & 24:00库存 &
↪ \\multicolumn{2}{r}{'+f(a['state'][i,-1])+'}\\\\']
29 s+=table(label+' '+day+'储能结果 (kWh) ', 'lrr lrr','\\n'.join(lines));blocks.append('\\par\\noindent\\begin{m
↪ inipage}{\\textwidth}\\n'+s+'\\end{minipage}\\par\\n')
30 ev=[];start=None
31 for j in range(145):
32     active=j<144 and e[j]>1e-6
33     if active and start is None:start=j
34     if not active and start is not None:ev.append((span(start,j),f(e[start:j].sum())));start=None
35 events.append(ev or ['无','0.00'])
36 checks.append({'kind':kind,'date':day,'sum_q':float(q.sum()),'sum_r':float(r.sum()),'physical_purchase':flo
↪ at((r+e).sum()),'ordinary_cost':ordinary,'emergency_cost':ec,'total_cost':ordinary+ec,'charge':float(c.
↪ sum()),'discharge':float(d.sum()),'initial':float(a['state'][i,0]),'final':float(a['state'][i,-1]),'eve
↪ nts':events[-1],'source_sha256':hashlib.sha256(p.read_bytes()).hexdigest()})
37 (T/f'specified-{kind}.tex').write_text('\\n\\par\\medskip\\n'.join(blocks))
38 emergency.append((label,events))
39 emer_parts=[]
40 for label,events in emergency:
41     lines=[' & '.join('\\multicolumn{2}{c}{'+day+'}' for day in DATES)+'\\\\','时间段 & 购电量 & 时间段 & 购电量 &
↪ 时间段 & 购电量 & 时间段 & 购电量\\\\\\midrule']
42 for row in range(max(map(len,events))):lines.append(' & '.join(v for ev in events for v in (ev[row] if
↪ row<len(ev) else ('','')))+'\\\\')
43 emer_parts.append('\\textbf{'+label+'}\\\\\\n\\begin{tabular}{lr lr lr
↪ lr}\\toprule\\n'+\\n'.join(lines)+'\\n\\bottomrule\\end{tabular}')
44 (T/'specified-emergency.tex').write_text('\\begin{table}[H]\\centering\\footnotesize\\setlength{\\tabcolsep}{
↪ 3.5pt}\\n\\caption{指定日期紧急购电 (kWh; 连续非零区间, 数值零阈值为$10^{-6}$ kWh)
↪ }\\n'+\\n\\par\\medskip\\n'.join(emer_parts)+'\\n\\end{table}\\n')
45 (ROOT/'revision/required-tables.json').write_text(json.dumps(checks,ensure_ascii=False,indent=2)+'\\n');print(
↪ '16 dates;32 daily tables;4 emergency tables')
46
47 # Q1 official-format tables, from the same saved deterministic optimum.
48 a=dict(np.load(BASE/'artifacts/q1.npz'));data=dict(np.load(BASE/'artifacts/data.npz'))
49 lines=['时间段 & 购电量 & 时间段 & 购电量 & 时间段 & 购电量\\\\\\midrule']
50 for hrs in [[10,12,14],[16,18,20]]:
51     cells=[]
52     for hh in hrs:cells += [span(hh*6,hh*6+1),f(a['q'][hh*6])]
53     lines.append(' & '.join(cells)+'\\\\')
54 lines += ['\\midrule','全天购电量 & \\multicolumn{2}{r}{'+f(a['q'].sum())+'} & 全天购电费 &
↪ \\multicolumn{2}{r}{'+f(a['q']@data['day_price'])+'}\\\\']
55 (T/'q1buy.tex').write_text(table('问题一购电结果 (电量kWh, 费用元) ', 'lr lr
↪ lr','\\n'.join(lines)).replace('\\begin{tabular}',r'\\label{tab:q1buy}\\begin{tabular}',1))
56 lines=['时间段 & 充电量 & 放电量 & 时间段 & 充电量 & 放电量\\\\\\midrule']
57 for j in [0,48,96]:
58     cells=[]
59     for t in [j,j+24]:cells += [span(t,t+24),f(a['c'][t:t+24].sum()),f(a['d'][t:t+24].sum())]
60     lines.append(' & '.join(cells)+'\\\\')
61 lines += ['\\midrule','0:00库存 & \\multicolumn{2}{r}{'+f(a['state'][0])+'} & 24:00库存 &
↪ \\multicolumn{2}{r}{'+f(a['state'][-1])+'}\\\\']
62 (T/'q1bat.tex').write_text(table('问题一充放电与首末库存 (kWh) ', 'lrr lrr','\\n'.join(lines)))

```

revision/revision_figures.py

```
1 from pathlib import Path
```

```

2 import numpy as np
3 import matplotlib;matplotlib.use('Agg')
4 import matplotlib.pyplot as plt
5 from matplotlib import font_manager
6 R=Path(__file__).resolve().parents[1];B=R.parent/'C 题 _ 跨日随机控制'
7 for f in ['/System/Library/Fonts/STHeiti Medium.ttc']:
8     if Path(f).exists():font_manager.fontManager.addfont(f)
9 plt.rcParams.update({'font.family':['Heiti TC','Arial Unicode MS','DejaVu
    ↳ Sans'],'font.size':10,'axes.unicode_minus':False,'axes.spines.top':False,'axes.spines.right':False})
10 a=np.load(B/'artifacts/global-terminal/q3_markov_mpc.npz');day=list(a['dates']).astype(str).index('2025-03-20')
    ↳ ');x=np.arange(144)/6
11 fig,axes=plt.subplots(3,1,figsize=(8,5.6),sharex=True,layout='constrained')
12 axes[0].step(x,a['q'][day],where='post',label='0时合同',lw=1.1,color='#477a9e');axes[0].step(x,a['r'][day],wh
    ↳ ere='post',label='最终合同',lw=1.1,color='#d27f42');axes[0].set_ylabel('合同电量 /
    ↳ kWh');axes[0].legend(ncol=2,loc='upper right')
13 axes[1].bar(x,a['emergency'][day],width=1/6,align='edge',color='#bc4f4c');axes[1].set_ylabel('紧急电量 /
    ↳ kWh');axes[1].text(.98,.83,'紧急补购发生于09:20-10:00',transform=axes[1].transAxes,ha='right')
14 axes[2].plot(np.arange(145)/6,a['state'][day]/1000,color='#357c6f');axes[2].set_ylabel('内部库存 /
    ↳ MWh',xlabel='时刻 / h',xlim=(0,24),xticks=np.arange(0,25,3))
15 for ax in axes:
16     ax.grid(axis='y',alpha=.2)
17     for t in [6,12,18]:ax.axvline(t,ls='-',lw=.7,color='#999999')
18 fig.savefig(R/'figures/contracts-revision.png',dpi=200);fig.savefig(R/'figures/contracts-revision.pdf');plt.c
    ↳ lose(fig)

```

revision/base_figures.py

```

1 from pathlib import Path
2 import json,numpy as np
3 import matplotlib;matplotlib.use('Agg')
4 import matplotlib.pyplot as plt
5 from matplotlib import font_manager
6 O=Path(__file__).resolve().parents[1];R=O.parent/'C 题 _ 跨日随机控制'
7 font_manager.fontManager.addfont('/System/Library/Fonts/STHeiti Medium.ttc')
8 plt.rcParams.update({'font.family':'Heiti TC','axes.unicode_minus':False,'font.size':10,'axes.spines.top':Fal
    ↳ se,'axes.spines.right':False,'figure.dpi':160,'savefig.dpi':220})
9 C=json.loads((O/'revision/base-artifacts/result-claims.json').read_text());H=json.loads((O/'revision/base-art
    ↳ ifacts/handoff-tables.json').read_text());K=list(C['models']);labels=['问题二','问题三','问题四日前','问题
    ↳ 四日内'];colors=['#2b8c91','#cb655d','#7683b4','#d79a57']
10 def save(name):plt.savefig(O/'figures/{name}.png',bbox_inches='tight');plt.close()
11 def table(name,caption,head,rows,spec=None):
12     spec=spec or 'l'+r'*(len(head)-1)
13     lines=['\\begin{table}[htbp]\\centering\\small',r'\\caption{'+caption+'}',r'\\begin{tabular}{'+spec+'}',r'\\top
    ↳ rule','&'.join(head)+r'\\',r'\\midrule']
14     lines+=['&'.join(str(x) for x in row)+r'\\' for row in rows];lines
    ↳ +=[r'\\bottomrule\\end{tabular}\\end{table}']
15 # Table reproduction is handled by revision/build_required_tables.py.
16 pass
17 table('main','同真实初末库存下的334日账单与配对节省 (C; 费用单位: 万元)
    ↳ ',['机制','主策略','每日闭合基线','节省','降幅'],[[l,f"{C['models'][k]['main']['total_cost']/1e4:.2f}",f"{C
    ↳ ['models'][k]['baseline_daily_greedy']['total_cost']/1e4:.2f}",f"{C['models'][k]['paired_vs_daily_greedy']
    ↳ }['cumulative_yuan']/1e4:.2f}",f"{100*C['models'][k]['paired_vs_daily_greedy']['cumulative_yuan']/C['mode
    ↳ ls'][k]['baseline_daily_greedy']['total_cost']:.2f}\\%"} for k,l in zip(K,labels)])
18 table('mechanisms','分别隔离跨日状态与库存价值反馈的闭环对照 (C; 万元)
    ↳ ',['机制','每日闭合仿射','跨日仿射','跨日节省','反馈再节省'],[[l,f"{C['models'][k]['closed_affine']['total_
    ↳ cost']/1e4:.2f}",f"{C['models'][k]['cross_affine']['total_cost']/1e4:.2f}",f"{C['models'][k]['cross_savin
    ↳ g_yuan']/1e4:.2f}",f"{C['models'][k]['feedback_saving_yuan']/1e4:.2f}"} for k,l in zip(K,labels)])
19 table('ablation','固定主策略框架下删减已提供预报的配对回放 (C; 万元)
    ↳ ',['保留预报','总费用','相对完整预报增加'],[['全部',f"{C['models'][k]['q3']['main']['total_cost']/1e4:.2f}",'0
    ↳ .00']]+[['0only':'只保留0时','without6':'删除6时','without12':'删除12时','without18':'删除18时']n],f"{v['
    ↳ full_fixed_totals']['total_cost']/1e4:.2f}",f"{v['extra_cost_yuan']/1e4:.2f}"} for n,v in
    ↳ C['ablation'].items())
20 table('bootstrap','日均配对节省的移动块95\\%百分位区间 (C; 元/日)',['机制','日均节省','3日块','7日块','14日块'],
    ↳ [[l,f"{C['models'][k]['paired_vs_daily_greedy']['mean_yuan']:.1f}"+['+', '.join(f'{x:.1f}' for x in
    ↳ C['models'][k]['paired_vs_daily_greedy']['bootstrap'][str(b)]['mean_95_percentile_yuan'])+']' for b in
    ↳ [3,7,14]] for k,l in zip(K,labels)])

```

```

21 table('stability','同年度时间稳定性诊断 (C) ',['机制','正日数','负日数','正月数/11','一阶相关'],[[l,v['positive_
    ↪ days'],v['negative_days'],f"{v['positive_months']}/11",f"{v['lag1_correlation']:.3f}"] for k,l in
    ↪ zip(K,labels) for v in C['models'][k]['paired_vs_daily_greedy']]])
22 table('monthly','逐月配对累计节省 (C; 万元)
    ↪ ',['月份']+labels,[[m]+f"{C['models'][k]['paired_vs_daily_greedy']['monthly_yuan'][m]/1e4:.3f}" for k in
    ↪ K] for m in C['models'][k]['q2']['paired_vs_daily_greedy']['monthly_yuan']]])
23 table('dev','固定终点开发期复核: 2025年1月25日至31日 (C; 元)
    ↪ ',['候选']+labels,[[c.replace('_',r'\_')+f"{C['development'][k]['costs_yuan'][c]:.2f}" for k in K] for c
    ↪ in C['development'][k]['q2']['costs_yuan']]])
24 table('sensitivity','既有七日敏感性实验: 3月14日至20日, 自由末端 (C) ',['配置','费用/元','末库存/kWh','相对参考/元']
    ↪ ',[[n.replace('_',r'\_')+f"{v['totals']['total_cost']:.2f}",f"{v['validation']['final_inventory']:.2f}"]],
    ↪ f"{v['totals']['total_cost']-C['sensitivity']['reference']['totals']['total_cost']:.2f}" for n,v in
    ↪ C['sensitivity'].items()])
25 table('q1buy','问题一指定时段购电量 (C; kWh) ',['时间段','购电量'],[[t,f"{v:.2f}"] for t,v in
    ↪ H['q1']['purchase']]])
26 table('q1bat','问题一分块电池动作 (C; 母线侧kWh) ',['时间段','充电量','放电量'],[[t,f"{c:.2f}",f"{d:.2f}"] for
    ↪ t,c,d in H['q1']['battery']]])
27 for k,l in zip(K,labels):
28 rows=[]
29 for date,s in H['models'][k]['selected'].items():
30 rows+=[[date,t,f"{q:.2f}",f"{r:.2f}"] for t,q,r in s['purchase']]
31 table('specified-'+k,l+'指定日期、指定时段合同 (C; kWh) ',['日期','时段','原合同','最终合同'],rows,'llrr')
32 table('oracle','同路径、同真实终点的完美信息参照 (B/C) ',['机制','主策略/万元','乐观参照/万元','参照距离'],[[l,f"{
    ↪ C['models'][k]['main']['total_cost']/1e4:.2f}",f"{C['models'][k]['perfect_information_cost_yuan']/1e4:.2f}
    ↪ }",f"{C['models'][k]['optimistic_distance_percent']:.2f}\%"] for k,l in zip(K,labels)]]
33 E=np.linspace(1.2,10.8,300);plt.figure(figsize=(5.7,3.5));plt.fill_between(E,np.maximum(1.2,E-(5/6)/.9),np.mi
    ↪ nimum(10.8,E+.9*5/6),color='#bfe6e4');plt.plot(E,E,'--',color='#555555',label='不动作');plt.plot(E,np.min
    ↪ imum(10.8,E+.9*5/6),color=colors[0],label='最大充电');plt.plot(E,np.maximum(1.2,E-(5/6)/.9),color=colors[
    ↪ 1],label='最大放电');plt.xlabel('当前库存 E / MWh');plt.ylabel('下一库存 x /
    ↪ MWh');plt.legend(fontsize=9);save('reachable')
34 a=dict(np.load(R/'artifacts/q1.npz'));data=dict(np.load(R/'artifacts/data.npz'));fig,ax=plt.subplots(figsize=
    ↪ (7,2.8));ax.plot(np.arange(145)/6,a['state']/1000,color=colors[0],label='库存');ax.set_xlabel('时刻 /
    ↪ h',ylabel='库存 / MWh',xlim=(0,24));p=ax.twinx();p.step(np.arange(144)/6,data['day_price'],where='post',c
    ↪ olor=colors[1],alpha=.55,label='电价');p.set_ylabel('元/kWh');ax.legend(loc='upper
    ↪ left');p.legend(loc='upper right');save('q1')
35 fig,ax=plt.subplots(figsize=(7,2.8));
36 for i,(k,l) in enumerate(zip(K,labels)):
37 v=C['models'][k]['paired_vs_daily_greedy'];ci=v['bootstrap']['7']['mean_95_percentile_yuan'];ax.errorbar(v['
    ↪ mean_yuan'],i,xerr=[v['mean_yuan']-ci[0]],ci[1]-v['mean_yuan']],fmt='o',capsize=4,color=colors[i]);ax
    ↪ .text(ci[1]+20,i,f"{v['mean_yuan']:.0f} [{ci[0]:.0f}, {ci[1]:.0f}]",va='center',fontsize=9)
38 ax.set_yticks(range(4),labels);ax.set_xlim(0,2050);ax.axvline(0,color='#777777',ls='--');ax.set_xlabel('日均
    ↪ 节省及7日块95%区间 / 元');save('bootstrap')
39 fig,axes=plt.subplots(1,2,figsize=(8,2.9));
40 for ax,k,title in zip(axes,['q3','q4_3'],['固定价日内机制','变动价日内机制']):
41 v=C['models'][k];parts=['planned_cost','increase_cost','reduction_net_cost','emergency_cost'];delta=np.array
    ↪ ([v['main'][p]-v['baseline_daily_greedy'][p] for p in parts])/1e4;ax.bar(range(4),delta,color=[colors[1]
    ↪ if x>0 else colors[0] for x in delta]);ax.axhline(0,color='#777777',lw=.6);ax.set_xticks(range(4),['原计
    ↪ 划','增购','退购净额','紧急']);ax.set_title(title);ax.set_ylabel('相对基线的费用变化 / 万元');
42 for j,x in enumerate(delta):ax.text(j,x+(1 if x>=0 else -1),f"{x:.2f}",ha='center',va='bottom' if x>=0 else
    ↪ 'top',fontsize=9)
43 fig.tight_layout();save('decomposition')
44 fig,ax=plt.subplots(figsize=(7,2.8));a=dict(np.load(O/'artifacts/q3.npz'));i=list(a['dates']).index('2025-09-
    ↪ 23');ax.step(np.arange(144)/6,a['q'][i]*6/1000,label='原合同',color=colors[0]);ax.step(np.arange(144)/6,a
    ↪ ['r'][i]*6/1000,label='最终合同',color=colors[2]);ax.step(np.arange(144)/6,a['emergency'][i]*6/1000,label
    ↪ ='紧急购电',color=colors[1]);ax.set_xlabel='9月23日时刻 / h',ylabel='功率 /
    ↪ MW',xlim=(0,24));ax.legend(ncol=3);save('contracts')
45 if all((O/f'artifacts/gain-{k}.json').exists() for k in K):
46 table('gains','全年主策略规划LP增益触边诊断 (C; 容差$10^{-6}$)
    ↪ ',['机制','规划次数','增益数','触边数','触边比例'],[[l,len(v['records']),v['gain_entries'],v['near_bound_e
    ↪ ntries'],f"{v['near_bound_fraction']*100:.2f}\%"] for k,l in zip(K,labels) for v in
    ↪ [json.loads((O/f'artifacts/gain-{k}.json').read_text())]])
47 print('Generated registered tables and six argument-focused figures.')

```

revision/prediction-control/audit.py

```

1 """Read-only frozen-model audits; all generated evidence stays in revision/."""
2 from pathlib import Path

```

```

3 import sys,json,hashlib,time,argparse, csv
4 import numpy as np
5 HERE=Path(__file__).resolve().parent
6 BASE=HERE.parents[1].parent/'C 题 _ 跨日随机控制'
7 sys.path[0]=[str(BASE/'src'),str(BASE/'vendor')]
8 from forecasting import base_forecast,forecast_as_of,FAMILIES
9 from dispatch import solve,settlement
10 from control import execute
11
12 def sha(p):return hashlib.sha256(p.read_bytes()).hexdigest()
13 def dump(name,obj):(HERE/name).write_text(json.dumps(obj,ensure_ascii=False,indent=2,allow_nan=False)+'\n')
14 def setup():return dict(np.load(BASE/'artifacts/data.npz'),json.loads((BASE/'artifacts/forecast-selection.js'
    ↪ on').read_text()))
15 def score(y,p):
16     e=np.asarray(p)-np.asarray(y)
17     return
    ↪ {'n':int(e.size),'MAE':float(abs(e).mean()),'RMSE':float(np.sqrt(np.mean(e*e))),'bias':float(e.mean())}
18 def predict():
19     data,selection=setup();out={'selection':selection,'units':{'load':'kW','pv':'kW','price':'yuan/kWh','net':'k'
    ↪ W},'development_note':'Retrospective January15-31 selection-window scores, not a prospectively frozen
    ↪ January deployment. February-December predictor is frozen before
    ↪ February1.','base_forecasts':[],'operational_forecasts':[]}
20     for scope,days in [('development',range(14,31)),('formal',range(31,365))]:
21         for key in ['load','pv','price']:
22             actual=data[key][list(days)]
23             for fam in FAMILIES:
24                 pred=np.array([base_forecast(data,key,d*144,np.arange(d*144,(d+1)*144),fam) for d in days])
25                 out['base_forecasts'].append({'scope':scope,'target':key,'family':fam,'selected':fam==selection[key]['sel'
    ↪ ected'],**score(actual,pred)})
26     # Every target is counted exactly once: release forecast for its next6 hours.
27     for official in [False,True]:
28         errors={k:[[],[]] for k in ['load','pv','price','net']}
29         for d in days:
30             for t in [0,36,72,108]:
31                 now=d*144+t;targets=np.arange(now,now+36)
32                 p=forecast_as_of(data,selection,now,targets,official,True)
33                 for key in errors:
34                     y=(data['load'].ravel()[targets]-data['pv'].ravel()[targets]) if key=='net' else
    ↪ data[key].ravel()[targets]
35                     pred=p[key]*6 if key=='net' else p[key]
36                     errors[key][0].extend(y);errors[key][1].extend(pred)
37                 for k,(y,p) in errors.items():out['operational_forecasts'].append({'scope':scope,'official_pv':official,'t'
    ↪ arget':k,'horizon':'nonoverlapping next6h
    ↪ at0/6/12/18h','includes_observed_error_correction':True,**score(y,p)})
38     # Deliberate perturbation of future actuals and unissued forecasts at4 release nodes.
39     leaks=[]
40     for t in [0,36,72,108]:
41         now=170*144+t;targets=np.arange(now,now+144)
42         a=forecast_as_of(data,selection,now,targets,True,True)
43         changed={k:v.copy() for k,v in data.items()}
44         for k in ['load','pv','price']:changed[k].ravel()[now:]+=12345
45         changed['forecast'].reshape(-1,24)[now//36+1:]+=12345
46         b=forecast_as_of(changed,selection,now,targets,True,True)
47         gap=max(float(abs(a[k]-b[k]).max()) for k in ['net','load','pv','price']);assert gap==0
48         leaks.append({'asof':now,'max_forecast_change':gap})
49     out['future_information_perturbation']=leaks
50     out['input_sha256']={str(BASE/f):sha(BASE/f) for f in
    ↪ ['artifacts/data.npz','artifacts/forecast-selection.json','src/forecasting.py']}
51     dump('forecast-errors.json',out)
52     import matplotlib;matplotlib.use('Agg');import matplotlib.pyplot as plt
53     from matplotlib import font_manager
54     font_manager.fontManager.addfont('/System/Library/Fonts/STHeiti Medium.ttc')
55     plt.rcParams.update({'font.family':'Heiti TC','axes.unicode_minus':False,'font.size':10,'axes.spines.top':Fa'
    ↪ lse,'axes.spines.right':False,'pdf.fonttype':42})
56     fig,axes=plt.subplots(3,2,figsize=(11,8),sharex=True)
57     records=[]
58     names={'load':'负荷 / kW','pv':'光伏 / kW','net':'净需求 / kW'}
59     for col,date in enumerate(['2025-06-21','2025-09-23']):

```

```

60 d=int(np.flatnonzero(data['dates']==date)[0]);history={k:[] for k in ['load','pv','net']};official={k:[] for
    ↪ k in history}
61 for t in [0,36,72,108]:
62     targets=np.arange(d*144+t,d*144+t+36)
63     for flag,target in [(False,history),(True,official)]:
64         pred=forecast_as_of(data,selection,d*144+t,targets,flag,True)
65         for key in target:target[key].extend(pred[key]*6 if key=='net' else pred[key])
66 for row,key in enumerate(history):
67     y=data['load'][d]-data['pv'][d] if key=='net' else data[key][d]
68     ax=axes[row,col];h=(np.arange(144)+.5)/6
69     ax.plot(h,y,label='实测',color='black',lw=1.4)
70     ax.plot(h,history[key],label='历史预测 + 在线误差修正',color='#4771b2',lw=1)
71     if key!='load':ax.plot(h,official[key],label='官方光伏 + 历史负荷',color='#d66c32',lw=1)
72     for edge in [6,12,18]:ax.axvline(edge,color='0.8',lw=.5,ls='--')
73     ax.set_ylabel(names[key]);ax.grid(alpha=.18);ax.set_title(date if row==0 else '');ax.set_xlim(0,24)
74     if row==2:ax.set_xlabel('时刻 / h (十分钟均值)')
75     for t in range(144):records.append({'date':date,'slot':t,**{k+'_actual_kw':float(data['load'][d,t]-data['pv
    ↪ '[d,t]) if k=='net' else float(data[k][d,t]) for k in history},**{k+'_history_kw':history[k][t] for k
    ↪ ' in history},**{k+'_official_kw':official[k][t] for k in history}})
76 axes[1,0].legend(loc='upper left',fontsize=8);fig.tight_layout();fig.savefig(HERE/'forecast-vs-actual.pdf');
    ↪ fig.savefig(HERE/'forecast-vs-actual.png',dpi=180);plt.close(fig)
77 with (HERE/'forecast-vs-actual.csv').open('w') as
    ↪ f:w=csv.DictWriter(f,records[0]);w.writeheader();w.writerows(records)
78 # Pairs seen in paper: verify both full-precision and 2decimal outputs.
79 dup=[]
80 for kind in ['q3','q4_3']:
81     p=BASE/f'artifacts/global-terminal/{kind}_markov_mpc.npz';a=dict(np.load(p))
82     for date,slots in [(('2025-06-21',[108,120]),('2025-09-23',[72,84]))]:
83         i=int(np.flatnonzero(a['dates']==date)[0]);values=[float(a['q'][i,t]) for t in slots]
84         dup.append({'kind':kind,'date':date,'slots':slots,'original_contract_kwh':values,'exactly_equal':values[0]
    ↪ ==values[1],'absolute_difference_kwh':abs(values[0]-values[1]),'two_decimal_equal':f'{values[0]:.2f}'=
    ↪ f'{values[1]:.2f}','source_sha256':sha(p)})
85 dump('duplicate-contract-audit.json',{'status':'VALUES_VERIFIED','records':dup,'interpretation':The
    ↪ requested values are executed original contracts q, read directly from full-precision NPZ. Equality alone
    ↪ cannot prove zero intercept or affine degeneracy; current-day q is an independent intercept for each
    ↪ slot, because release<=asof prevents a current-day gain term.})
86 print('prediction and duplicate audits complete',flush=True)
87
88 def warmup():
89     data,_=setup();a={k:np.zeros((31,144)) for k in
    ↪ ['q','r','c','d','emergency','spill'];a['state']=np.zeros((31,145));E=6000.;daily=[]
90     for day in range(31):
91         net=(data['load'][day]-data['pv'][day])/6;a['state'][day,0]=E
92         # Jan1 no completed history: inherited no-action emergency policy.
93         if day:
94             p=(data['load'][day-1]-data['pv'][day-1])/6
95             plan=solve(p[None,:],data['day_price'][None,:],initial=6000.,terminal=6000.)
96             a['q'][day]=plan['q'];a['r'][day]=plan['q']
97         for t in range(144):
98             if day==0:target=E
99             else:
100                 bal=a['r'][day,t]-net[t];target=E+.9*max(bal,0)-max(-bal,0)/.9
101                 E,c,d,em,sp,pr=execute(E,target,a['r'][day,t],net[t],143-t,True,t,6000.)
102                 for k,v in [(('c',c),('d',d),('emergency',em),('spill',sp)):a[k][day,t]=v
103                 a['state'][day,t+1]=E
104         daily.append({'date':str(data['dates'][day]),'fixed':settlement(a['q'][day],a['r'][day],a['emergency'][day]
    ↪ ,data['day_price']),'variable':settlement(a['q'][day],a['r'][day],a['emergency'][day],data['price'][day]
    ↪ )})
105     net=(data['load'][:31]-data['pv'][:31])/6
106     check={'bus':float(abs(a['r']+a['emergency']+a['d']-a['c']-a['spill']-net).max()),'soc':float(abs(np.diff(a[
    ↪ 'state'],axis=1)-.9*a['c']+a['d']/.9).max()),'midnight':float(abs(a['state'][1:,0]-a['state'][:-1,-1]).m
    ↪ ax()),'closure':float(abs(a['state'][:,0,-1]-6000).max()),'minimum':float(a['state'].min()),'maximum':
    ↪ float(a['state'].max()),'power_kw':float(max(a['c'].max(),a['d'].max())*6),'simultaneous':float(np.minim
    ↪ um(a['c'],a['d']).max())}
107     assert max(check[k] for k in ['bus','soc','midnight','closure','simultaneous'])<1e-6 and
    ↪ check['power_kw']<=5000+1e-6 and check['minimum']>=1200-1e-6 and check['maximum']<=10800+1e-6
108     np.savez_compressed(HERE/'january-causal-greedy.npz',**a)

```

```

109 dump('january-causal-greedy.json',{'status':'PASS','policy':'January1 no battery use/no contract because no
    ↳ prior day exists; January2-31 one previous complete day forecast, deterministic daily-closed LP contract
    ↳ and physically projected greedy battery. Same contract schedule in all tasks; daily given tariff used as
    ↳ fixed planning price. No January15-31 selector or future data
    ↳ used.','daily':daily,'cost_fixed':sum(x['fixed']['total_cost'] for x in
    ↳ daily),'cost_variable':sum(x['variable']['total_cost'] for x in daily),'validation':check,'formal_evalua
    ↳ tion_unchanged':True,'february1_inventory':E,'input_sha256':sha(BASE/'artifacts/data.npz')}})
110 print('causal January warmup complete',flush=True)
111 if __name__=='__main__':
112     p=argparse.ArgumentParser();p.add_argument('mode',choices=['predict','warmup','all']);args=p.parse_args()
113     if args.mode in ['predict','all']:predict()
114     if args.mode in ['warmup','all']:warmup()

```

revision/prediction-control/duplicate_lp_probe.py

```

1 """Diagnostic only: test whether reported equal current-day contracts are unique."""
2 from pathlib import Path
3 import sys,json
4 import numpy as np
5 from scipy.sparse import vstack,csr_matrix
6 HERE=Path(__file__).resolve().parent
7 BASE=HERE.parents[1].parent/'C 题 _ 跨日随机控制'
8 sys.path[0]=[str(BASE/'src'),str(BASE/'vendor')]
9 import control
10 from forecasting import scenarios
11 D=dict(np.load(BASE/'artifacts/data.npz'));S=json.loads((BASE/'artifacts/forecast-selection.json').read_text(
    ↳ ));raw=control.lp_solve;records=[]
12 for kind in ['q3','q4_3']:
13     a=dict(np.load(BASE/'artifacts/global-terminal/{kind}_markov_mpc.npz'))
14     for date,slots in [('2025-06-21',[108,120]),('2025-09-23',[72,84])]:
15         ix=int(np.flatnonzero(a['dates']==date)[0]);day=int(a['days'][ix]);now=day*144;end=(day+2)*144;net,p,meta=s
            ↳ cenarios(D,S,now,end,True,kind=='q4_3',count=7);captured={}
16         def capture(obj,A,b,bounds,solver='ipm'):
17             sol=raw(obj,A,b,bounds,solver);captured.update(obj=obj,A=A,b=b,bounds=bounds,sol=sol);return sol
18         control.lp_solve=capture
19         plan=control.affine_plan(net,p,now,float(a['state'][ix,0]),None,True,True,float(np.quantile(p[:,-144:],.25)
            ↳ )/.9)
20         gap=float(abs(a['q'][ix]-plan['q'][:144]).max());assert gap<1e-5
21         obj=captured['obj'];primary=captured['sol'].fun;tol=1e-7;A=vstack([captured['A'],csr_matrix(obj[None,:])]);
            ↳ b=np.r_[captured['b'],primary+tol];secondary=np.zeros_like(obj);secondary[slots[0]]=1;secondary[slots[1]
            ↳ ]]=-1
22         lo=raw(secondary,A,b,captured['bounds']);hi=raw(-secondary,A,b,captured['bounds']);records.append({'kind':k
            ↳ ind,'date':date,'slots':slots,'contract_replay_max_abs':gap,'original_pair_difference':float(plan['q'][
            ↳ slots[0]]-plan['q'][slots[1]]),'optimal_difference_min':float(secondary@lo.x),'optimal_difference_max':
            ↳ float(secondary@hi.x),'objective_allowance_yuan':tol,'primary_objective_yuan':primary,'min_solution_pri
            ↳ mary_increase':float(obj@lo.x-primary),'max_solution_primary_increase':float(obj@hi.x-primary),'primary
            ↳ _gain_columns_for_current_q':'identically zero by release/asof restriction; current q has independent
            ↳ per-slot intercept','interpretation':'secondary LP over near-optimal face, not physical re-execution or
            ↳ adopted strategy'})
23 (HERE/'duplicate-lp-probe.json').write_text(json.dumps({'records':records},indent=2));print(json.dumps(record
    ↳ s))

```

revision/prediction-control/gain_sensitivity.py

```

1 """January-only post-review sensitivity, NOT retrospective parameter selection.
2 Original modules stay unchanged. The LP wrapper changes only gain bounds and
3 records columns identically absent from BOTH constraint matrix and objective.
4 """
5 from pathlib import Path
6 import sys,json,time,hashlib
7 from concurrent.futures import ProcessPoolExecutor,as_completed
8 import numpy as np
9 HERE=Path(__file__).resolve().parent
10 BASE=HERE.parents[1].parent/'C 题 _ 跨日随机控制'
11 sys.path[0]=[str(BASE/'src'),str(BASE/'vendor')]
12 import run,control
13

```

```

14 def job(kind,bound):
15     start=time.perf_counter();records=[];raw=control.lp_solve
16     def measured(obj,A,b,bounds,solver='ipm'):
17         cols=[j for j,v in enumerate(bounds) if v==(-10,10)];new=list(bounds)
18         for j in cols:new[j]=(-bound,bound)
19         sol=raw(obj,A,b,new,solver);Ac=A.tocsc(copy=True);Ac.eliminate_zeros()
20         null=(np.diff(Ac.indptr)[cols]==0)&(np.asarray(obj)[cols]==0)
21         vals=sol.x[cols];touch=abs(vals)>=bound-1e-6
22         records.append({'gain_count':len(cols),'touch_count':int(touch.sum()),'zero_columns':int(null.sum()),'zero_
        ↳ columns_touch':int((null&touch).sum()),'active_columns':int((~null).sum()),'active_touch':int((~null&to
        ↳ uch).sum()),'zero_column_indices':[int(j) for j,n in zip(cols,null) if n], 'active_touch_indices':[int(j)
        ↳ for j,n in zip(cols,-null&touch) if n], 'gain_values':vals.tolist()})
23     return sol
24 control.lp_solve=measured
25 data=dict(np.load(BASE/'artifacts/data.npz'));selection=json.loads((BASE/'artifacts/forecast-selection.json'
        ↳ ).read_text())
26 a,m=run.simulate(data,selection,list(range(24,31)),kind,'markov_mpc',count=7,horizon_days=2,tail_scale=1.,gr
        ↳ id=321,final=6000.)
27 summary={k:sum(r[k] for r in records) for k in
        ↳ ['gain_count','touch_count','zero_columns','zero_columns_touch','active_columns','active_touch']}
28 summary['active_touch_fraction']=summary['active_touch']/summary['active_columns'];summary['raw_touch_fracti
        ↳ on']=summary['touch_count']/summary['gain_count']
29 out={'kind':kind,'gain_bound':bound,'dates':'2025-01-25 through
        ↳ 2025-01-31','initial':6000,'final':6000,'status':'POST_REVIEW_DIAGNOSTIC_NOT_MODEL_SELECTION','total_cos
        ↳ t':m['totals'] ['total_cost'],'totals':m['totals'],'validation':m['validation'],'column_statistics':summa
        ↳ ry,'records':records,'daily':m['daily'],'runtime_seconds':time.perf_counter()-start,'source_sha256':{'f:h
        ↳ ashlib.sha256((BASE/'src'/f).read_bytes()).hexdigest() for f in
        ↳ ['control.py','run.py','forecasting.py']}}
30 np.savez_compressed(HERE/f'gain-{kind}-{bound}.npz',**a)
31 (HERE/f'gain-{kind}-{bound}.json').write_text(json.dumps(out,ensure_ascii=False,indent=2));return {k:v for
        ↳ k,v in out.items() if k not in ['records','daily','source_sha256']}
32 if __name__=='__main__':
33     results=[]
34     with ProcessPoolExecutor(max_workers=4) as pool:
35         fs=[pool.submit(job,k,b) for k in ['q2','q3','q4_2','q4_3'] for b in [10,20]]
36         for f in as_completed(fs):r=f.result();results.append(r);print(json.dumps(r),flush=True)
37     pairs=[]
38     for k in ['q2','q3','q4_2','q4_3']:
39         a=next(r for r in results if r['kind']==k and r['gain_bound']==10);b=next(r for r in results if r['kind']==k
        ↳ and r['gain_bound']==20)
40         pairs.append({'kind':k,'cost_gain10':a['total_cost'],'cost_gain20':b['total_cost'],'saving_gain20':a['total
        ↳ _cost']-b['total_cost'],'saving_percent':100*(a['total_cost']-b['total_cost'])/a['total_cost'],'gain10_
        ↳ active_touch':a['column_statistics'],'gain20_active_touch':b['column_statistics']})
41 (HERE/'gain-summary.json').write_text(json.dumps({'status':'PASS_JANUARY_DIAGNOSTIC','notes':'Frozen
        ↳ historical predictor selected on January15-31. This is a retrospective development-window diagnostic; no
        ↳ claim that January selection was known at January25. No formal-result-based tuning or adoption. Zero
        ↳ column means exactly zero coefficient in objective and all constraints, verified in assembled LP. Annual
        ↳ raw bound ratios cannot be corrected from old scalar counts without replay; do not extrapolate these
        ↳ January ratios to whole year.','comparisons':pairs,'results':results},indent=2))

```

revision/prediction-control/verify_outputs.py

```

1 """Direct array physics and bill recomputation, without importing model code."""
2 from pathlib import Path
3 import numpy as np,json,hashlib,platform,sys
4 H=Path(__file__).resolve().parent
5 B=H.parents[1].parent/'C 题 _ 跨日随机控制';D=np.load(B/'artifacts/data.npz')
6 records=[]
7 for stem in ['january-causal-greedy']+['f'gain-{k}-{g}' for k in ['q2','q3','q4_2','q4_3'] for g in [10,20]]:
8     a=np.load(H/(stem+'.npz'));m=json.loads((H/(stem+'.json')).read_text());warm=stem.startswith('january');days
        ↳ =np.arange(31) if warm else a['days']
9     y=(D['load'][days]-D['pv'][days])/6
10    residuals={'bus':float(abs(a['r']+a['emergency']+a['d']-a['c']-a['spill']-y).max()),'soc':float(abs(np.diff(
        ↳ a['state'],axis=1)-.9*a['c']+a['d']/.9).max()),'midnight':float(abs(a['state'][1:,0]-a['state'][:-1,-1])
        ↳ .max()),'terminal':float(abs(a['state'][-1,-1]-6000)), 'initial':float(abs(a['state'][0,0]-6000)), 'simult
        ↳ aneous':float(np.minimum(a['c'],a['d'])).max(), 'lower':float(max(0,1200-a['state'].min())), 'upper':float
        ↳ (max(0,a['state'].max()-10800)), 'power':float(max(0,max(a['c']).max(),a['d'].max()-5000/6))}

```

```

11 prices=[('fixed',np.broadcast_to(D['day_price'],a['q'].shape)),('variable',D['price'][days])] if warm else
    ↳ [('total',D['price'][days] if 'q4' in stem else np.broadcast_to(D['day_price'],a['q'].shape))]
12 for name,p in prices:
13     cost=np.sum(p*(a['q']+1.5*np.maximum(a['r']-a['q'],0)-.5*np.maximum(a['q']-a['r'],0)+5*a['emergency']))
14     target=m['cost_'+name] if warm else m['total_cost'];residuals['bill_'+name]=float(abs(cost-target))
15     assert max(residuals.values())<1e-5,(stem,residuals)
16     records.append({'output':stem,'checks':residuals,'status':'PASS'})
17 files=[p for p in H.iterdir() if p.suffix in ['.py','.json','.npz','.csv','.pdf','.png','.tex'] and p.name not
    ↳ in ['verification.json','evidence-manifest.json']]
18 (H/'verification.json').write_text(json.dumps({'status':'PASS_ARRAY_RECOMPUTATION','independence':'Separate
    ↳ algebraic validator; author-generated, not a substitute for external independent
    ↳ review.','cases':records},indent=2))
19 (H/'evidence-manifest.json').write_text(json.dumps({'python':sys.version,'numpy':np.__version__,'platform':pl
    ↳ atform.platform(),'deterministic':True,'files':{p.name:hashlib.sha256(p.read_bytes()).hexdigest() for p in
    ↳ files}},indent=2))
20 print('9 output trajectories and bills PASS')

```

reference/scripts/register_results.py

```

1 """Independently recompute fixed-terminal bills, feasibility and paired statistics."""
2 from pathlib import Path
3 import json,sys,shutil,hashlib
4 import numpy as np
5 O=Path(__file__).resolve().parents[1];R=O.parent.parent/'C 题 _ 跨日随机控制'
6 KINDS=['q2','q3','q4_2','q4_3'];data=dict(np.load(R/'artifacts/data.npz'))
7 def dump(p,x):p.write_text(json.dumps(x,ensure_ascii=False,indent=2,allow_nan=False)+'\n')
8 def read(p):return dict(np.load(p))
9 def metrics(a):
10     q,r,e,p=(a[k] for k in ['q','r','emergency','price'])
11     parts={'planned_cost':(p*q).sum(1),'increase_cost':(1.5*p*np.maximum(r-q,0)).sum(1),'reduction_net_cost':(-.
    ↳ 5*p*np.maximum(q-r,0)).sum(1),'emergency_cost':(5*p*e).sum(1)}
12     parts['total_cost']=sum(parts.values());parts['emergency_energy']=e.sum(1);parts['spill_energy']=a['spill'].
    ↳ sum(1);parts['projection_count']=(a['projection']>1e-6).sum(1)
13     bal=r+e+a['d']-a['c']-a['spill']-(data['load'][a['days']]-data['pv'][a['days']])/6
14     val={'balance_max_abs':float(abs(bal).max()),'soc_recurrence_max_abs':float(abs(np.diff(a['state'],axis=1)-.
    ↳ 9*a['c']+a['d']/.9).max()),'midnight_jump_max_abs':float(abs(a['state'][1:,0]-a['state'][:-1,-1]).max())
    ↳ , 'soc_min':float(a['state'].min()),'soc_max':float(a['state'].max()),'power_max_kw':float(6*max(a['c'].m
    ↳ ax(),a['d'].max())),'simultaneous_max':float(np.minimum(a['c'],a['d']).max()),'initial_inventory':float(
    ↳ a['state'][0,0]),'final_inventory':float(a['state'][-1,-1])}
15     assert max(val[k] for k in
    ↳ ['balance_max_abs','soc_recurrence_max_abs','midnight_jump_max_abs','simultaneous_max'])<1e-5
16     assert val['soc_min']>=1200-1e-5 and val['soc_max']<=10800+1e-5 and val['power_max_kw']<=5000+1e-5
17     assert abs(val['initial_inventory']-6000)<1e-5 and abs(val['final_inventory']-6000)<1e-5
18     return {'daily':[{ 'date':str(d),**{k:float(v[i]) for k,v in
    ↳ parts.items()},'ending_inventory':float(a['state'][i,-1])} for i,d in
    ↳ enumerate(a['dates'])], 'totals':{k:float(v.sum()) for k,v in parts.items()}, 'validation':val}
19 def stats(delta,dates,seed):
20     n=len(delta);res={'cumulative_yuan':float(delta.sum()),'mean_yuan':float(delta.mean()),'positive_days':int((
    ↳ delta>1e-8).sum()),'negative_days':int((delta<
    ↳ -1e-8).sum()),'zero_days':int((abs(delta)<=1e-8).sum()),'lag1_correlation':float(np.corrcoef(delta[:-1],
    ↳ delta[1:])[0,1]),'bootstrap':{}}
21     for b in [3,7,14]:
22         rng=np.random.default_rng(np.random.SeedSequence([seed,b]));starts=rng.integers(0,n-b+1,size=(10000,int(np.
    ↳ ceil(n/b))))
23         ids=(starts[:,None]+np.arange(b)).reshape(10000,-1)[:,:n];means=delta[ids].mean(1);ci=np.quantile(means,[
    ↳ .025,.975])
24         res['bootstrap'][str(b)]={'mean_95_percentile_yuan':ci.tolist(),'cumulative_95_percentile_yuan':(ci*n).toli
    ↳ st(), 'replicates':10000, 'seed_sequence':[seed,b], 'method':'overlapping noncircular moving blocks,
    ↳ concatenate then truncate to 334'}
25     months=np.array([str(d)[7:] for d in dates]);res['monthly_yuan']={m:float(delta[months==m].sum()) for m in
    ↳ np.unique(months)};res['positive_months']=sum(v>1e-8 for v in res['monthly_yuan'].values());return res
26 def main():
27     claims={'source_commit':'59af4e054e079f18484889b6bc899bd83cc1527f','evaluation':['2025-02-01','2025-12-31'],
    ↳ 'days':334,'initial_final_kwh':[6000,6000],'claim_classes':{'A':'mathematical result under stated
    ↳ assumptions','B':'model-specific optimum or bound','C':'finite-sample numerical finding','D':'scope or
    ↳ extrapolation boundary'},'models':{,'ablation':{,'development':{,'sensitivity':{}}
28     oracle=json.loads((R/'review/global-oracle.json').read_text()['results'])

```

```

29 for ki,k in enumerate(KINDS):
30     a=read(R/f'artifacts/global-terminal/{k}_markov_mpc.npz');m=metrics(a);np.savez_compressed(0/f'artifacts/{k}
    ↳ }.npz',**a);dump(0/f'artifacts/{k}.json',m)
31     b=metrics(read(R/f'artifacts/annual/{k}_closed_baseline.npz'));af=metrics(read(R/f'artifacts/global-termina
    ↳ l/{k}_affine_mpc.npz'));closed=metrics(read(R/f'artifacts/annual/{k}_closed_affine.npz'))
32     dc=np.array([x['total_cost'] for x in b['daily']])~np.array([x['total_cost'] for x in
    ↳ m['daily']]);bound=oracle['variable_fixed6000' if k.startswith('q4') else
    ↳ 'fixed_fixed6000']['objective_cost_yuan']
33     claims['models'][k]={ 'class': 'C', 'main': m['totals'], 'validation': m['validation'], 'baseline_daily_greedy': b[
    ↳ 'totals'], 'cross_affine': af['totals'], 'closed_affine': closed['totals'], 'cross_saving_yuan': closed['tota
    ↳ ls']['total_cost']-af['totals']['total_cost'], 'feedback_saving_yuan': af['totals']['total_cost']-m['tota
    ↳ ls']['total_cost'], 'paired_vs_daily_greedy': stats(dc,a['dates'],202609110+ki), 'perfect_information_cost
    ↳ _yuan': bound, 'optimistic_distance_percent': 100*(m['totals']['total_cost']-bound)/bound}
34     vals={c:json.loads((0/f'artifacts/development_fixed/{k}_{c}.json').read_text())['totals']['total_cost'] for
    ↳ c in
    ↳ ['cross_baseline', 'affine_mpc', 'markov_mpc', 'sddp_mpc', 'sddp_markov']};chosen=min(vals,key=vals.get)
35     if vals['markov_mpc']<=min(vals.values())*1.0001:chosen='markov_mpc'
36     claims['development'][k]={ 'costs_yuan': vals, 'selected': chosen, 'retrospective_terminal_consistency_check': Tr
    ↳ ue}
37     for name in ['0only', 'without6', 'without12', 'without18']:
38         m=metrics(read(0/f'artifacts/ablation_fixed/q3_{name}.npz'));claims['ablation'][name]={ 'class': 'C', 'full_fi
    ↳ xed_totals': m['totals'], 'extra_cost_yuan': m['totals']['total_cost']-claims['models']['q3']['main']['tot
    ↳ al_cost'], 'validation': m['validation']}
39     for name in ['reference', 'grid161', 'grid641', 'scenarios14', 'horizon3', 'tail_zero', 'tail_double', 'stress_load
    ↳ _up_pv_down']:
40         p=R/f'artifacts/experiments/{name}.json'
41         if p.exists():
42             m=json.loads(p.read_text());claims['sensitivity'][name]={ 'source': str(p.relative_to(R)), 'configuration': m[
    ↳ 'configuration'], 'totals': m['totals'], 'validation': m['validation']}
43         claims['q1']=json.loads((R/'artifacts/q1.json').read_text())
44         claims['warmup']=json.loads((R/'artifacts/summary.json').read_text())['warmup']
45         claims['gain_diagnostic']={k:{kk:vv for kk,vv in
    ↳ json.loads((0/f'artifacts/gain-{k}.json').read_text()).items() if kk!='records'} for k in KINDS if
    ↳ (0/f'artifacts/gain-{k}.json').exists()}
46         claims['gain_scope_note']='All bounded gain variables, including structurally unused or degenerate columns.
    ↳ Boundary frequency is not effective feedback saturation.'
47         claims['provenance']={str(p.relative_to(R.parent)): hashlib.sha256(p.read_bytes()).hexdigest() for p in
    ↳ list((R/'artifacts/global-terminal').glob('*.*npz'))+list((0/'artifacts/ablation_fixed').glob('*.*npz'))}
48         dump(0/'artifacts/result-claims.json',claims)
49         for name in
    ↳ ['data.npz', 'q1.npz', 'q1.json', 'selection.json']:shutil.copy2(R/'artifacts'/name,0/'artifacts'/name)
50         source=(R/'src/export_results.py').read_text();start=source.index('    for kind,candidate in
    ↳ selection');end=source.index('        data=dict',start)
51         source=source[start]+source[end:];source=source.replace('主策略SOC跨日连续、年末库存自由;
    ↳ 一次全局期末6000仅作参考。','主策略SOC跨日连续，真实年末固定6000；全部CSV和工作簿采用同一固定终点轨迹。')
52         (0/'scripts/export_fixed.py').write_text(source)
53         print(json.dumps({k:{ 'cost': v['main']['total_cost'], 'cross': v['cross_saving_yuan'], 'feedback': v['feedback_sa
    ↳ ving_yuan'], 'bootstrap': v['paired_vs_daily_greedy']} for k,v in
    ↳ claims['models'].items()}),ensure_ascii=False))
54 if __name__=='__main__':main()

```

reference/scripts/gain_replay.py

```

1 """Re-solve frozen main planning LPs at saved actual states; no policy change."""
2 from pathlib import Path
3 import sys,json,time
4 import numpy as np
5 from concurrent.futures import ProcessPoolExecutor
6 O=Path(__file__).resolve().parents[1];R=O.parent.parent/'C 题 _ 跨日随机控制'
7 sys.path.insert(0,str(R/'src'))
8 import control
9 from forecasting import scenarios
10 def job(kind):
11     data=np.load(R/'artifacts/data.npz');selection=json.loads((R/'artifacts/forecast-selection.json').read
    ↳ _text())
12     a=dict(np.load(R/f'artifacts/global-terminal/{kind}_markov_mpc.npz'))
13     official=kind in ['q3','q4_3'];variable=kind.startswith('q4');records=[];raw=control.lp_solve;probe=[]

```

```

14 def measured(*args,**kw):
15     sol=raw(*args,**kw);bounds=kw.get('bounds',args[3] if len(args)>3 else None)
16     ix=[j for j,b in enumerate(bounds) if b[0]==-10 and b[1]==10]
17     g=sol.x[ix];probe.append((len(g),int(np.sum(abs(g))>=10-1e-6)),float(max(abs(g),default=0)));return sol
18     control.lp_solve=measured
19     for i,day in enumerate(a['days']):
20         for t in [0,36,72,108]:
21             now=int(day)*144+t;end=min((int(day)+2)*144,365*144)
22             net,prices,meta=scenarios(data,selection,now,end,official,variable,count=7)
23             base=None if t==0 else np.r_[a['q'][i,t:],np.zeros(end-(int(day)+1)*144)]
24             terminal=6000. if end==365*144 else None
25             lam=0. if terminal is not None else float(np.quantile(prices[:,-min(144,len(prices[0])):],.25))/0.9
26             plan=control.affine_plan(net,prices,now,float(a['state'][i,t]),base,official,True,lam,terminal,None,daily_)
27             ↪ closed=False)
28             expected=a['q'][i] if t==0 else a['releases'][i,t//36,t:]
29             found=plan['q'][:144] if t==0 else plan['r'][:144-t]
30             gap=float(np.max(abs(expected-found))) if t==0 or official else 0.
31             assert gap<1e-5,(kind,day,t,gap)
32             n,b,m=probe[-1];records.append({'date':str(a['dates'][i]),'slot':t,'gain_entries':n,'near_bound_entries':b}
33             ↪ , 'max_abs':m, 'contract_replay_max_abs':gap})
34 result={'kind':kind,'scope':'all 1336 planning LPs on saved fixed-terminal main
35 ↪ trajectory','records':records,'gain_entries':sum(r['gain_entries'] for r in
36 ↪ records),'near_bound_entries':sum(r['near_bound_entries'] for r in
37 ↪ records),'max_contract_replay_error':max(r['contract_replay_max_abs'] for r in records)}
38 result['near_bound_fraction']=result['near_bound_entries']/result['gain_entries']
39 (0/f'artifacts/gain-{kind}.json').write_text(json.dumps(result,ensure_ascii=False,indent=2));return {k:v for
40 ↪ k,v in result.items() if k!='records'}
41 if __name__=='__main__':
42     with ProcessPoolExecutor(max_workers=4) as ex:
43         for r in ex.map(job,['q2','q3','q4_2','q4_3']):print(json.dumps(r),flush=True)

```

reference/scripts/run_supplement.py

```

1 """Supplemental terminal-matched audit; imports frozen main code unchanged."""
2 from pathlib import Path
3 from concurrent.futures import ProcessPoolExecutor,as_completed
4 import sys,json,time,hashlib
5 import numpy as np
6 O=Path(__file__).resolve().parents[1];R=O.parent.parent/'C 题 _ 跨日随机控制'
7 sys.path.insert(0,str(R/'src'))
8 import run,control,forecasting,validate
9
10 def dump(p,x):
11     p.parent.mkdir(parents=True,exist_ok=True);p.write_text(json.dumps(x,ensure_ascii=False,indent=2,allow_nan=F)
12     ↪ else)+'\n')
13
14 def sha(p):return hashlib.sha256(p.read_bytes()).hexdigest()
15
16 def worker(job):
17     mode,kind,candidate=job;start=time.perf_counter();data=dict(np.load(R/'artifacts/data.npz'));selection=json.
18     ↪ loads((R/'artifacts/forecast-selection.json').read_text())
19     gain=[];original=control.lp_solve
20     def measured(obj,A,b,bounds,solver='ipm'):
21         sol=original(obj,A,b,bounds,solver);ids=[i for i,v in enumerate(bounds) if v==(-10,10)]
22         x=sol.x[ids];gain.append({'count':len(ids),'near_bound':int((abs(x)>=10-1e-6).sum()),'max_abs':float(abs(x)
23         ↪ .max()) if len(x) else 0})
24     return sol
25     control.lp_solve=measured
26     if mode=='development':
27         arrays,m=run.simulate(data,selection,list(range(24,31)),kind,candidate,final=6000.)
28         target=0/'artifacts/development_fixed'/f'{kind}_{candidate}'
29         m['audit_design']={'original_selection':'January free-terminal; retained as historical selection
30         ↪ record','supplement':'January fixed-terminal retrospective consistency check; February-December not used
31         ↪ to choose','initial':6000,'final':6000}
32     elif mode=='ablation':
33         name=candidate;allowed={'0only':[0],'without6':[0,2,3],'without12':[0,1,3],'without18':[0,1,2]}[name]
34         forecasting.forecast_as_of=validate.forecast_with_releases(allowed)
35         source=R/f'artifacts/experiments/annual_pv_{name}';a=dict(np.load(str(source)+'.npz'));initial=float(a['sta
36         ↪ te'][-2,0]);days=a['days'][-2:].tolist()

```

```

30 free, fm=run.simulate(data, selection, days, 'q3', 'markov_mpc', initial=initial)
31 diffs={k:float(abs(free[k]-a[k][-2:]).max()) for k in ['q', 'r', 'c', 'd', 'emergency', 'spill', 'state']}
32 assert max(diffs.values())<1e-5, diffs
33 fixed, m=run.simulate(data, selection, days, 'q3', 'markov_mpc', initial=initial, final=6000.)
34 arrays={k:np.concatenate([v[:-2], fixed[k]]) for k, v in a.items()}
35 m['prefix_reuse']={'days':332, 'original_trajectory':str(source.relative_to(R))+'.npz', 'sha256':sha(Path(str(
    ↪ (source)+'.npz'))), 'free_suffix_max_differences':diffs, 'proof_scope':'unchanged two-midnight policy, no
    ↪ terminal-visible plan before Dec30 and inactive earlier terminal reachability'}
36 target=0/'artifacts/ablation_fixed'/f'q3_{name}'
37 else:raise ValueError(mode)
38 m['gain_probe']={'scope':mode, 'lp_calls':len(gain), 'gain_entries':sum(x['count'] for x in
    ↪ gain), 'near_bound_entries':sum(x['near_bound'] for x in gain), 'max_abs':max((x['max_abs'] for x in
    ↪ gain), default=0), 'method':'read returned primal values for bounds exactly [-10,10], no
    ↪ objective/constraints/actions altered'}
39 m['execution']={'job':job, 'exit_code':0, 'runtime_seconds':time.perf_counter()-start, 'script_sha256':sha(Path(
    ↪ (__file__)), 'source_hashes':{p.name:sha(p) for p in (R/'src').glob('*.py')}}
40 target.parent.mkdir(exist_ok=True, parents=True); np.savez_compressed(str(target)+'.npz', **arrays); dump(Path(s
    ↪ tr(target)+'.json'), m)
41 return {'job':job, 'runtime_seconds':m['execution']['runtime_seconds'], 'cost':m['totals']['total_cost'], 'gain'
    ↪ _probe':m['gain_probe']}
42
43 def main():
44 jobs=[('development', k, c) for k in run.KINDS for c in
    ↪ ['cross_baseline', 'affine_mpc', 'markov_mpc', 'sddp_mpc', 'sddp_markov']] + [('ablation', 'q3', x) for x in
    ↪ ['0only', 'without6', 'without12', 'without18']]
45 started=time.perf_counter(); done=[]
46 with ProcessPoolExecutor(max_workers=4) as pool:
47 for f in as_completed([pool.submit(worker, j) for j in jobs]):
48 r=f.result(); done.append(r); print(json.dumps(r, ensure_ascii=False), flush=True); dump(0/'artifacts/supplemen
    ↪ t-progress.json', {'completed':len(done), 'total':len(jobs), 'results':done})
49 dump(0/'artifacts/supplement-execution.json', {'command':'python3 theory_revision/scripts/run_supplement.py',
    ↪ 'exit_code':0, 'runtime_seconds':time.perf_counter()-started, 'results':done})
50 if __name__=='__main__':main()

```

reference/scripts/export_fixed.py

```

1 """Prepare numerical handoff tables, detailed CSVs and an artifact-tool workbook payload."""
2 from pathlib import Path
3 import csv, json, shutil, sys, time
4 import numpy as np
5
6 ROOT=Path(__file__).resolve().parents[1]
7 OUT=ROOT/'计算结果'; OUT.mkdir(exist_ok=True)
8 def dump(p, x): Path(p).write_text(json.dumps(x, ensure_ascii=False, indent=2)+'\n')
9 def clock(i): return f'{i//6:02d}:{(i%6)*10:02d}'
10 def interval(i, j=None): return clock(i)+'-'+clock(i+1 if j is None else j)
11 def writcsv(name, header, rows):
12     with (OUT/name).open('w', encoding='utf-8-sig', newline='') as f:
13         w=csv.writer(f); w.writerow(header); w.writerows(rows)
14 def events(values, date):
15     out=[]; start=None
16     for t in range(145):
17         active=t<144 and values[t]>1e-6
18         if active and start is None: start=t
19         if not active and start is not None:
20             out.append([date, interval(start, t), float(values[start:t].sum())]); start=None
21     return out
22 def markdown_table(header, rows):
23     def fmt(v): return f'{v:,.4f}' if isinstance(v, float) else str(v)
24     return '\n'.join(['| '+' | '.join(header)+' |', '| '+'| '.join(['---']*len(header))+'|'] +
25         ['| '+' | '.join(fmt(v) for v in r)+' |' for r in rows])
26
27 def main():
28     started=time.perf_counter()
29     selection=json.loads((ROOT/'artifacts/selection.json').read_text())

```

```

30 data=dict(np.load(ROOT/'artifacts/data.npz'));q1=json.loads((ROOT/'artifacts/q1.json').read_text());q1.up_
    ↳ date({k:v.tolist() for k,v in
    ↳ dict(np.load(ROOT/'artifacts/q1.npz')).items()});q1['daily_energy']=float(sum(q1['q']));q1['daily_cos_
    ↳ t']=float(data['day_price']@np.array(q1['q']))
31 labels=[interval(t) for t in range(144)]
32 notes=[['项目','口径'],['官方原题','https://www.mcm.edu.cn/html_cn/node/27b6e148f8113f09b0269f64a02629fb.h
    ↳ tml'],
33     ['时间轴','原输入00:10至24:00按前10分钟区间均值处理;输出时间段改正为00:00-00:10至23:50-24:00。'],
34     ['单位','购电、充放电、SOC 均为 kWh;费用为元;充放电电量在交流母线侧计量。'],
35     ['储能效率','充电和放电各 0.9; SOC= 上一时段 SOC+0.9 充电量-放电量/0.9。'],
36     ['年度边界','只输出2025年2月1日至12月31日。1月为校准,初始SOC6000,1月储能保持6000;主策略SOC跨日连续,
    ↳ 真实年末未固定6000;全部CSV和工作簿采用同一固定终点轨迹。'],
37     ['计划购电量','0 点承诺量 q; 全天购电费列仅为计划成本  $\Sigma p \cdot q$ 。'],
38     ['调整购电量','当天逐次调整后最终执行量 r, 非增减差值; 全天购电费为计划成本加调整净费用, 未含紧急。'],
39     ['费用汇总','总费用 = 计划成本 +1.5p 增购量-0.5p 退购量 +5p 紧急量; 已购电实物总量 =r+ 紧急量。'],
40     ['退购结算','主口径: 退购部分原电费被50%违约费替代; 相对0点基准逐时最终净额结算。若解释为不退款,
    ↳ 必须改变目标函数并重算, 本目录未将其作为已验证结果。'],
41     ['弃电口径','弃电量为母线总富余弃置, 可能包含已购买电能, 不能直接视为光伏弃电量或用于计算纯光伏利用率。'],
42     ['充放电量','问题 2-4 填写因果实时反馈后的实际充放电, 非 0 点 LP 预计划。'],
43     ['紧急购电量','连续非零 10 分钟段合并; 未发生的日期填无、0。小于  $1e-6$  kWh 视为数值零。'],
44     ['预测与信息','仅用过往日期数据与已发布的光伏预报; 实际未来电价只用于事后结算。'],
45     ['最优性边界','Q1确定性LP; Q2-4跨日因果仿射策略SAA计划+Markov随机动态规划反馈, SDDP作为末值备选,
    ↳ 为经过验证的可行策略, 不声称完整随机问题全局最优。']]
46 payload={'labels':labels,'notes':notes,'q1':q1,'models':{}}
47 tables={'q1':{ 'purchase':[[labels[t],float(q1['q'][t])] for t in [60,72,84,96,108,120]],
48     'energy':q1['daily_energy'],'cost':q1['daily_cost'],
49     'battery':[[interval(b,b+24),float(sum(q1['c'][b:b+24])),float(sum(q1['d'][b:b+24]))] for b
    ↳ in range(0,144,24)],
50     'soc':[q1['state'][0],q1['state'][-1]],'models':{}}
51 chosen=['2025-03-20','2025-06-21','2025-09-23','2025-12-21']
52 for key in ['q2','q3','q4_2','q4_3']:
53     a=dict(np.load(ROOT/f'artifacts/{key}.npz'));m=json.loads((ROOT/f'artifacts/{key}.json').read_text())
54
55     for i,row in enumerate(m['daily']):
56         row['planned_energy']=float(a['q'][i].sum());row['adjusted_energy']=float(a['r'][i].sum())
57     bat=[];emer=[];detail=[];releases=[];selected=[]
58     for i,date in enumerate(a['dates']):
59         date=str(date)
60         battery=[[date,interval(b,b+24),float(a['c'][i,b:b+24].sum()),float(a['d'][i,b:b+24].sum()),
61             '00:00' if b==0 else '24:00' if b==24 else None,
62             float(a['state'][i,0]) if b==0 else float(a['state'][i,-1]) if b==24 else None] for b in
            ↳ range(0,144,24)]
63         ev=events(a['emergency'][i],date)
64         bat+=battery;emer+=ev or [[date,'无',0.]]
65         for t in range(144):
66             detail.append([date,labels[t]]+[float(a[z][i,t]) for z in
                ↳ ['q','r','c','d','emergency','spill','price']] +
                    [float(a['state'][i,t]),float(a['state'][i,t+1])])
67
68         for k in range(4):
69             for t in range(k*36,144):
70                 if np.isfinite(a['releases'][i,k,t]):releases.append([date,f'{k*6:02d}:00',labels[t],floe
                    ↳ a['releases'][i,k,t]))
71         if date in chosen:
72             selected[date]={ 'purchase':[[labels[t],float(a['q'][i,t]),float(a['r'][i,t])] for t in
                ↳ [60,72,84,96,108,120]],
73                 'daily':m['daily'][i],'battery':battery,'emergency':ev or [[date,'无',0.]]}
74         payload['models'][key]={ 'dates':a['dates'].tolist(),'q':a['q'].tolist(),'r':a['r'].tolist(),
75             'battery':bat,'emergency':emer,'daily':m['daily']}
76     tables['models'][key]={ 'totals':m['totals'],'selected':selected,'validation':m['validation']}
77     writecsv(f'{key}_逐时完整策略.csv',[ '日期','时间段','计划购电_kWh','最终合同购电_kWh','充电_kWh','放电_k
    ↳ Wh','紧急购电_kWh','弃电_kWh','电价_元每kWh','时段初SOC_kWh','时段末SOC_kWh'],detail)
78     writecsv(f'{key}_调整发布记录.csv',[ '日期','发布时间','交易时间段','购电量_kWh'],releases)
79     writecsv(f'{key}_费用汇总.csv',list(m['daily'][0]),[list(row.values()) for row in m['daily']])
80     writecsv(f'{key}_紧急购电.csv',[ '日期','时间段','购电量_kWh'],emer)
81     writecsv('q1_逐时完整策略.csv',[ '时间段','购电_kWh','充电_kWh','放电_kWh','时段初SOC_kWh','时段末SOC_kWh'],
82         [[labels[t],q1['q'][t],q1['c'][t],q1['d'][t],q1['state'][t],q1['state'][t+1]] for t in
            ↳ range(144)])
83     dump(ROOT/'artifacts/workbook-payload.json',payload);dump(ROOT/'artifacts/handoff-tables.json',tables)

```

```

84     text=['# 指定日期结果表', '\n全部数值由本地程序直接生成。单位：电量 kWh、费用元。计划量 q 与最终合同量 r 分列；
    ↪ 实际从电网取电量另加紧急购电。',
85         '\n## 问题 1', markdown_table(['时间段', '购电量'], tables['q1']['purchase']),
86         f"\n全天购电量 {q1['daily_energy']:, .4f} kWh；全天购电费 {q1['daily_cost']:, .4f} 元；首末SOC均为6000
    ↪ kWh。",
87         markdown_table(['时间段', '充电量', '放电量'], tables['q1']['battery'])])
88     for key, model in tables['models'].items():
89         for date, s in model['selected'].items():
90             text+=['\n## '+key+
    ↪ '+date', markdown_table(['时间段', '0点计划购电量q', '最终合同购电量r'], s['purchase']),
91                     markdown_table(['项目', '数值'], [[k, v] for k, v in s['daily'].items() if k != 'date']),
92                     markdown_table(['日期', '时间段', '充电量', '放电量', '时刻', 'SOC'], [(v if v is None else v)
    ↪ for v in r] for r in s['battery'])),
93                     markdown_table(['日期', '紧急购电时间段', '紧急购电量'], s['emergency'])])
94     (ROOT/'指定日期结果表.md').write_text('\n\n'.join(text)+'\n')
95     dump(ROOT/'artifacts/execution-export.json', {'command': 'python3 '+' '.join(sys.argv), 'exit_code': 0, 'run_time':
    ↪ me_seconds: time.perf_counter()-started, 'selected': selection['selected']})
96     print('Exported CSVs, workbook payload and exact selected-day tables.')
97
98 if __name__ == '__main__': main()

```

reference/scripts/paper_assets.py

```

1 from pathlib import Path
2 import json, numpy as np
3 import matplotlib; matplotlib.use('Agg')
4 import matplotlib.pyplot as plt
5 from matplotlib import font_manager
6 O=Path(__file__).resolve().parents[1]; R=O.parent.parent/'C 题 _ 跨日随机控制'
7 font_manager.fontManager.addfont('/System/Library/Fonts/STHeiti Medium.ttc')
8 plt.rcParams.update({'font.family': 'Heiti TC', 'axes.unicode_minus': False, 'font.size': 10, 'axes.spines.top': False,
    ↪ se, 'axes.spines.right': False, 'figure.dpi': 160, 'savefig.dpi': 220})
9 C=json.loads((O/'artifacts/result-claims.json').read_text()); H=json.loads((O/'artifacts/handoff-tables.json').
    ↪ .read_text()); K=list(C['models']); labels=['问题二', '问题三', '问题四日前', '问题四日内']; colors=['#2b8c91', '#
    ↪ cb655d', '#7683b4', '#d79a57']
10 def save(name): plt.savefig(O/f'figures/{name}.png', bbox_inches='tight'); plt.close()
11 def table(name, caption, head, rows, spec=None):
12     spec=spec or 'l'+r'*(len(head)-1)
13     lines=['\\begin{table}[htbp]\\centering\\small', r'\\caption{'+caption+'}', r'\\begin{tabular}{'+spec+'}', r'\\top'
    ↪ rule', ' & '.join(head)+r' \\', r'\\midrule']
14     lines+=[' & '.join(str(x) for x in row)+r' \\' for row in rows]; lines
    ↪ +=['r'\\bottomrule\\end{tabular}\\end{table}']
15     (O/f'tables/{name}.tex').write_text('\n'.join(lines)+'\n')
16 table('main', '同真实初末库存下的334日账单与配对节省 (C; 费用单位: 万元)
    ↪ ', ['机制', '主策略', '每日闭合基线', '节省', '降幅'], [[l, f"{C['models'][k]['main']['total_cost']/1e4:.2f}", f"{C[
    ↪ 'models'][k]['baseline_daily_greedy']['total_cost']/1e4:.2f}", f"{C['models'][k]['paired_vs_daily_greedy']
    ↪ }['cumulative_yuan']/1e4:.2f}", f"{100*C['models'][k]['paired_vs_daily_greedy']['cumulative_yuan']/C['mode
    ↪ ls'][k]['baseline_daily_greedy']['total_cost']:.2f}\\%"} for k, l in zip(K, labels)])
17 table('mechanisms', '分别隔离跨日状态与库存价值反馈的闭环对照 (C; 万元)
    ↪ ', ['机制', '每日闭合仿射', '跨日仿射', '跨日节省', '反馈再节省'], [[l, f"{C['models'][k]['closed_affine']['total_
    ↪ cost']/1e4:.2f}", f"{C['models'][k]['cross_affine']['total_cost']/1e4:.2f}", f"{C['models'][k]['cross_savin
    ↪ g_yuan']/1e4:.2f}", f"{C['models'][k]['feedback_saving_yuan']/1e4:.2f}"] for k, l in zip(K, labels)])
18 table('ablation', '固定主策略框架下删减已提供预报的配对回放 (C; 万元)
    ↪ ', ['保留预报', '总费用', '相对完整预报增加'], [['全部', f"{C['models'][k]['q3']['main']['total_cost']/1e4:.2f}", '0'
    ↪ '.00']] + [[f'0only': '只保留0时', 'without6': '删除6时', 'without12': '删除12时', 'without18': '删除18时'] for n, f in
    ↪ full_fixed_totals]['total_cost']/1e4:.2f}", f"{v['extra_cost_yuan']/1e4:.2f}"] for n, v in
    ↪ C['ablation'].items())
19 table('bootstrap', '日均配对节省的移动块95\\%百分位区间 (C; 元/日)', ['机制', '日均节省', '3日块', '7日块', '14日块'],
    ↪ [[l, f"{C['models'][k]['paired_vs_daily_greedy']['mean_yuan']:.1f}" + f'+' + '.join(f'{x:.1f}' for x in
    ↪ C['models'][k]['paired_vs_daily_greedy']['bootstrap'][str(b)]['mean_95_percentile_yuan']) + f'+' for b in
    ↪ [3, 7, 14]] for k, l in zip(K, labels)])
20 table('stability', '同年度时间稳定性诊断 (C)', ['机制', '正日数', '负日数', '正月数/11', '一阶相关'], [[l, v['positive_
    ↪ days'], v['negative_days'], f"{v['positive_months']/11}", f"{v['lag1_correlation']:.3f}"] for k, l in
    ↪ zip(K, labels) for v in [C['models'][k]['paired_vs_daily_greedy']]])
21 table('monthly', '逐月配对累计节省 (C; 万元)
    ↪ ', ['月份']+labels, [[m]+f"{C['models'][k]['paired_vs_daily_greedy']['monthly_yuan'][m]/1e4:.3f}" for k in
    ↪ K] for m in C['models'][k]['q2']['paired_vs_daily_greedy']['monthly_yuan']]

```

```

22 table('dev','固定终点开发期复核: 2025年1月25日至31日 (C; 元)
    ↪ ',[f'候选']+labels,[c.replace('_',r'\_')+f'{C['development'][k]['costs_yuan'][c]:.2f}" for k in K] for c
    ↪ in C['development'][q2]['costs_yuan'])
23 table('sensitivity','既有七日敏感性实验: 3月14日至20日, 自由末端 (C) ',[f'配置','费用/元','末库存/kWh','相对参考/元'
    ↪ ],[n.replace('_',r'\_'),f'{v['totals']['total_cost']:.2f}',f'{v['validation']['final_inventory']:.2f}'],
    ↪ f'{v['totals']['total_cost']-C['sensitivity']['reference']['totals']['total_cost']:.2f}" for n,v in
    ↪ C['sensitivity'].items())
24 table('q1buy','问题一指定时段购电量 (C; kWh) ',[f'时间段','购电量'],[t,f'{v:.2f}' for t,v in
    ↪ H['q1']['purchase']])
25 table('q1bat','问题一分块电池动作 (C; 母线侧kWh) ',[f'时间段','充电量','放电量'],[t,f'{c:.2f}',f'{d:.2f}' for
    ↪ t,c,d in H['q1']['battery']])
26 for k,l in zip(K,labels):
27     rows=[]
28     for date,s in H['models'][k]['selected'].items():
29         rows+=[[date,t,f'{q:.2f}',f'{r:.2f}' for t,q,r in s['purchase']]
30     table('specified-'+k,l+'指定日期、指定时段合同 (C; kWh) ',[f'日期','时段','原合同','最终合同'],rows,'llrr')
31 table('oracle','同路径、同真实终点的完美信息参照 (B/C) ',[f'机制','主策略/万元','乐观参照/万元','参照距离'],[l,f'{
    ↪ C['models'][k]['main']['total_cost']/1e4:.2f}',f'{C['models'][k]['perfect_information_cost_yuan']/1e4:.2f}
    ↪ }',f'{C['models'][k]['optimistic_distance_percent']:.2f}%' for k,l in zip(K,labels)])
32 E=np.linspace(1.2,10.8,300);plt.figure(figsize=(5.7,3.5));plt.fill_between(E,np.maximum(1.2,E-(5/6)/.9),np.mi
    ↪ nimum(10.8,E+.9*5/6),color='#bfe6e4');plt.plot(E,E,'--',color='#555555',label='不动作');plt.plot(E,np.min
    ↪ imum(10.8,E+.9*5/6),color=colors[0],label='最大充电');plt.plot(E,np.maximum(1.2,E-(5/6)/.9),color=colors[
    ↪ 1],label='最大放电');plt.xlabel('当前库存 E / MWh');plt.ylabel('下一库存 x /
    ↪ MWh');plt.legend(fontsize=9);save('reachable')
33 a=dict(np.load(R/'artifacts/q1.npz'));data=dict(np.load(R/'artifacts/data.npz'));fig,ax=plt.subplots(figsize=
    ↪ (7,2.8));ax.plot(np.arange(145)/6,a['state']/1000,color=colors[0],label='库存');ax.set_xlabel('时刻 /
    ↪ h',ylabel='库存 / MWh',xlim=(0,24));p=ax.twinx();p.step(np.arange(144)/6,data['day_price'],where='post',c
    ↪ olor=colors[1],alpha=.55,label='电价');p.set_ylabel('元/kWh');ax.legend(loc='upper
    ↪ left');p.legend(loc='upper right');save('q1')
34 fig,ax=plt.subplots(figsize=(7,2.8));
35 for i,(k,l) in enumerate(zip(K,labels)):
36     v=C['models'][k]['paired_vs_daily_greedy'];ci=v['bootstrap'][7]['mean_95_percentile_yuan'];ax.errorbar(v['
    ↪ mean_yuan'],i,xerr=[v['mean_yuan']-ci[0],[ci[1]-v['mean_yuan']]],fmt='o',capsize=4,color=colors[i]);ax
    ↪ .text(ci[1]+20,i,f'{v["mean_yuan"]:.0f} [{ci[0]:.0f}, {ci[1]:.0f}]',va='center',fontsize=9)
37 ax.set_yticks(range(4),labels);ax.set_xlim(0,2050);ax.axvline(0,color='#777777',ls='--');ax.set_xlabel('日均
    ↪ 节省及7日块95%区间 / 元');save('bootstrap')
38 fig,axes=plt.subplots(1,2,figsize=(8,2.9));
39 for ax,k,title in zip(axes,['q3','q4_3'],[f'固定价日内机制','变动价日内机制']):
40     v=C['models'][k];parts=[f'planned_cost','increase_cost','reduction_net_cost','emergency_cost'];delta=np.array
    ↪ ([v['main'][p]-v['baseline_daily_greedy'][p] for p in parts])/1e4;ax.bar(range(4),delta,color=[colors[1]
    ↪ if x>0 else colors[0] for x in delta]);ax.axhline(0,color='#777777',lw=.6);ax.set_xticks(range(4),[f'原计
    ↪ 划','增购','退购净额','紧急']);ax.set_title(title);ax.set_ylabel('相对基线的费用变化 / 万元');
41 for j,x in enumerate(delta):ax.text(j,x+(1 if x>=0 else -1),f'{x:.2f}',ha='center',va='bottom' if x>=0 else
    ↪ 'top',fontsize=9)
42 fig.tight_layout();save('decomposition')
43 fig,ax=plt.subplots(figsize=(7,2.8));a=dict(np.load(R/'artifacts/q3.npz'));i=list(a['dates']).index('2025-09-
    ↪ 23');ax.step(np.arange(144)/6,a['q'][i]*6/1000,label='原合同',color=colors[0]);ax.step(np.arange(144)/6,a
    ↪ ['r'][i]*6/1000,label='最终合同',color=colors[2]);ax.step(np.arange(144)/6,a['emergency'][i]*6/1000,label
    ↪ ='紧急购电',color=colors[1]);ax.set_xlabel('9月23日时刻 / h',ylabel='功率 /
    ↪ MW',xlim=(0,24));ax.legend(ncol=3);save('contracts')
44 if all((O/f'artifacts/gain-{k}.json').exists() for k in K):
45     table('gains','全年主策略规划LP增益触边诊断 (C; 容差$10^{-6}$)
        ↪ ',[f'机制','规划次数','增益数','触边数','触边比例'],[l,len(v['records']),v['gain_entries'],v['near_bound_e
        ↪ ntries'],f'{v["near_bound_fraction"]*100:.2f}%' for k,l in zip(K,labels) for v in
        ↪ [json.loads((O/f'artifacts/gain-{k}.json').read_text())]])
46 print('Generated registered tables and six argument-focused figures.')

```

reproduce.sh

```

1 #!/bin/sh
2 set -eu
3 cd "$(dirname "$0")"
4 PY=${MICROGRID_PYTHON:-python3}
5 mode=${1:-check}
6 test -f ../C 题 _ 跨日随机控制/artifacts/global-terminal/q2_markov_mpc.npz
7 case "$mode" in
8     supplement)

```

```

9     "$PY" revision/q1_and_billing.py
10    "$PY" revision/strong_baseline.py
11    "$PY" revision/billing_january_replan.py
12    "$PY" revision/prediction-control/audit.py all
13    "$PY" revision/prediction-control/gain_sensitivity.py
14    "$PY" revision/prediction-control/duplicate_lp_probe.py
15    ;;
16    check) ;;
17    *) echo 'Usage: sh reproduce.sh [check|supplement]' >&2; exit 2 ;;
18    esac
19    "$PY" reference/scripts/register_results.py > reference/artifacts/reproduce-statistics.log
20    "$PY" revision/verify_billing_january.py
21    "$PY" revision/prediction-control/verify_outputs.py
22    "$PY" revision/strong_baseline.py --export-only
23    "$PY" revision/build_required_tables.py
24    "$PY" revision/restore/build_primary_evidence.py > revision/restore/reproduce-primary.log
25    "$PY" revision/restore/control_diagnostics.py > revision/restore/reproduce-diagnostics.log
26    sh compile.sh
27    "$PY" revision/final_math_review.py
28    mkdir -p output/pdf
29    cp main.pdf output/pdf/基于跨日随机控制与价值反馈的微网购电和储能协同调度 _ 证据完整修订.pdf

```

revision/final_math_review.py

```

1  """Final mathematical/data review, scoped separately from source appendix/PDF QA."""
2  from pathlib import Path
3  import json,hashlib,subprocess,numpy as np
4  R=Path(__file__).resolve().parents[1];B=R.parent/'C 题 _ 跨日随机控制'
5  main=(R/'main.tex').read_text();app=(R/'technical-appendix.tex').read_text();checks=[]
6  rows=json.loads((R/'revision/results/strong-baseline.json').read_text());data=np.load(R/'artifacts/data.npz')
7  for row in rows:
8      k=row['kind'];A=dict(np.load(R/f'revision/results/{k}_cross_baseline_fixed.npz'));M=dict(np.load(B/f'artifac
↳ ts/global-terminal/{k}_markov_mpc.npz'))
9      bills=[]
10     for name,a in [('cross_greedy',A),('main',M)]:
11         q,r,e,p,c,d,w,E=[a[x] for x in ['q','r','emergency','price','c','d','spill','state']]
12         assert np.array_equal(a['days'],np.arange(31,365)); assert E[0,0]==6000 and abs(E[-1,-1]-6000)<1e-6
13         assert abs(E[1:,0]-E[:-1,-1]).max(<1e-6
14         assert abs(np.diff(E,axis=1)-.9*c+d/.9).max(<1e-6
15         assert abs(r+e+d-c-w-(data['load'][a['days']]-data['pv'][a['days']])/6).max(<1e-6
16         assert np.minimum(c,d).max(<1e-6 and E.min(>=1200-1e-6 and E.max(<=10800+1e-6 and
↳ max(c.max(),d.max())*6<=5000+1e-6
17         bills.append((p*(q+1.5*np.maximum(r-q,0)-.5*np.maximum(q-r,0)+5*e)).sum(1))
18         diff=bills[0]-bills[1]
19         assert abs(diff.sum()-row['saving'])<1e-6
20         rng=np.random.default_rng(20260911);starts=rng.integers(0,328,size=(10000,48));inds=(starts[:,None]+np.ara
↳ rge(7)).reshape(10000,-1)[:,:334]
21         ci=np.quantile(diff[inds].sum(1),[.025,.975]);assert np.max(abs(ci-row['saving_95ci']))<1e-6
22         assert f'{row["saving"]/10000:.2f}' in main
23         checks.append({'kind':k,'status':'PASS','bill':float(bills[1].sum()),'saving':float(diff.sum()),'bootstrap_9
↳ 5ci':ci.tolist()})
24     findings=[]
25     def add(id,where,msg,required=True):findings.append({'id':id,'location':where,'message':msg,'required':requir
↳ ed})
26     if '图示9月23日' in main and '2025年3月20日' in main:add('F1','main.tex:官方预报图前段','图前正文仍称9月23日,
↳ 图题已为3月20日,须改为同一日期。')
27     if '本稿数值均对应式' in main:add('F2','main.tex:合同结算凸性末段','已补另一计费解释与敏感性表,
↳ 不应再称本稿全部数值均按原phi;应限定为主结果。')
28     if any(x in main for x in ['(B/D)', '(B/C/D)']):add('F3','main.tex:上层定位/参数表题','仅A/B定义后仍有B/D、
↳ B/C/D残留,用户意见15尚未处理完整。')
29     if '手里这度电' in main:add('F4','main.tex: 库存反馈节开头','仍有用户点名的口语表达,意见 16 未完整处理。')
30     if '独立声明的SeedSequence' in app and not all(z in app for z in ['原每日闭合基线','default\\_rng(20260911)',
↳ '共享相同']):add('F5','technical-appendix.tex:移动块实现','新增强基线四问均default_rng(20260911),
↳ 与每机制独立SeedSequence笼统表述不符;区分原对照与新增对照,注明共同索引不影响各边际区间。')
31     add('L1','tables/gain-revision.tex','有效列删除和界限20测试只覆盖一月七日,不能视为已给出全年有效列触边比例。
↳ 当前正文文明确限制范围,属于可接受但应保留的局限。',False)
32     check17={str(i):'ADDRESSED_IN_CONTENT' for i in range(1,18)}

```

```

33 check17.update({'1':'CONTENT_CHECKED_RENDER_PENDING','2':'SOURCE_APPENDIX_SEPARATE_REVIEW_PENDING','7':'DEVEL'
↳ OPMENT_SCOPE_ONLY_DISCLOSED','10':'NEEDS_TEXT_DATE_FIX' if any(f['id']=='F1' for f in findings) else
↳ 'ADDRESSED_IN_CONTENT','15':'NEEDS_LABEL_CLEANUP' if any(f['id']=='F3' for f in findings) else
↳ 'ADDRESSED_IN_CONTENT','16':'NEEDS_LANGUAGE_CLEANUP' if any(f['id']=='F4' for f in findings) else
↳ 'ADDRESSED_IN_CONTENT','17':'PDF_RENDER_REVIEW_PENDING'})
34 pdf_checks={}
35 if (R/'main.pdf').exists():
36 abstract=subprocess.check_output(['/opt/homebrew/bin/pdftotext','-f','1','-l','1','-layout',str(R/'main.pdf'
↳ ),'-'],text=True)
37 body=subprocess.check_output(['/opt/homebrew/bin/pdftotext','-f','1','-l','23','-layout',str(R/'main.pdf'),'
↳ -'],text=True)
38 expected=[1,384.42,1,346.18,1,465.46,1,425.21,20.24,23.13,15.29,22.89,35,126.95,17.93]
39 for v in expected:assert v in abstract,('abstract missing',v)
40 for v in [35118.60,14073209.01,14907515.48,345349.60,384732.76,8550,8.35,33.78,36.62,35.6
↳ 0,42.96]:assert v in body,('body missing',v)
41 pdf_checks={'status':'PASS_NUMERICAL_TEXT_ONLY','abstract_values':expected,'body_values':[35118.60,140732
↳ 09.01,14907515.48,345349.60,384732.76],'pdf_sha256':hashlib.sha256((R/'main.pdf').read_bytes()).h
↳ exdigest(),'visual_layout_checked':False}
42 files=[R/'main.tex',R/'technical-appendix.tex',*sorted((R/'tables').glob('*revision.tex')),R/'tables/strong-b
↳ aseline.tex',R/'revision/results/strong-baseline.json',R/'revision/numeric-audit/q1-billing-audit.json',R
↳ /'revision/numeric-audit/january-billing-independent.json']
43 report={'status':'NEEDS_EDITORIAL_CORRECTION' if any(f['required'] for f in findings) else
↳ 'PASS_MATH_AND_DATA_SCOPE_ONLY','review_scope':'Independent mathematics, causal/terminal/billing
↳ interpretation, recomputed strong-baseline trajectories/bills/bootstrap and numeric insertions. Does not
↳ approve unfinished source appendix or unrendered final PDF.','model_recompute_blockers':[],'checks':check
↳ s,'pdf_numerical_text':pdf_checks,'findings':findings,'seventeen_items':check17,'pending':['Full
↳ source-code appendix completeness and executable packaging','Final PDF per-page rendering/layout
↳ verification'],'hashes':{str(p.relative_to(R)):hashlib.sha256(p.read_bytes()).hexdigest() for p in files}}
44 (R/'revision/final-math-review.json').write_text(json.dumps(report,ensure_ascii=False,indent=2)+'\n')
45 s='# 独立数学与控制终稿审查\n\n 状态: '+report['status']+'. 本报告不替代尚未完成的源码附录和 PDF 渲染验收.\n\n'
46 s+='已重新读取四种同初末6000主轨迹和新增跨日贪心, 独立核对334日、母线平衡、库存递推、午夜连续、功率与充放电互斥;
↳ 重新计算全部账单和10000次七日块区间。主摘要四问费用与新增节省相符。Q1周期初值、
↳ 另一计费公式及其一月重规划与独立数值证据一致。当前没有需要回到模型阶段重算的已发现错误.\n\n'
47 s+='PDF第1页摘要核对四问总费与跨日贪心主对照; 每日闭合33.78至42.96万元作为跨日复位附加效应保留在摘要末段与正文主表。
↳ 8550 kWh/8.35元的周期初值对照保留在正文。此检查只核对数值文本, 不替代页面视觉验收.\n\n'
48 s+='数学结构检查: 合同上图、同时充放电消元、可达域、有限紧域凸PWL卷积、实际反馈与硬终端投影的区分均有充分边界;
↳ 未发现把辅助SDDP下界冒充原问题因果下界, 或把完美信息差额冒充可实现算法损失的论断。
↳ Jan开发选型回顾性与正式区间冻结性已区分.\n\n'
49 for f in findings:s+=f- '{f["id"]}' ('+(需修改' if f['required'] else '已披露局限')+f): {f["location"]}.
↳ {f["message"]}\n'
50 s+='\n17项逐项状态见JSON。指定日期结果已包含全天合同/实际购电费用与电量、六个四小时充放电块、两端库存及紧急区间;
↳ 源代码和最终PDF仍需另行完成检查, 不能给整篇无条件PASS.\n'
51 (R/'revision/final-math-review.md').write_text(s)
52 print(json.dumps({'status':report['status'],'findings':findings,'checks':checks},ensure_ascii=False,indent=2))

```

revision/restore/build_primary_evidence.py

```

1 from pathlib import Path
2 import json, hashlib, numpy as np
3 import matplotlib
4 matplotlib.use('Agg')
5 import matplotlib.pyplot as plt
6 from matplotlib.font_manager import FontProperties
7 R=Path(__file__).resolve().parents[2]
8 BASE=R.parent/'C 题 _ 跨日随机控制'
9 font=FontProperties(fname='/System/Library/Fonts/STHeiti Medium.ttc')
10 plt.rcParams.update({'font.family':font.get_name(),'axes.unicode_minus':False,'font.size':10,'axes.spines.top'
↳ :False,'axes.spines.right':False,'savefig.dpi':220})
11 KS=['q2','q3','q4_2','q4_3'];labels=['问题二','问题三','问题四日前','问题四日内']
12 result={};ledger={}
13 def parts(a):
14 q,r,p,e=[a[k] for k in ['q','r','price','emergency']]
15 return
↳ np.array([(p*q).sum(1),(1.5*p*np.maximum(r-q,0)).sum(1),(-.5*p*np.maximum(q-r,0)).sum(1),(5*p*e).sum(1)])
16 for k in KS:
17 mpath=BASE/f'artifacts/global-terminal/{k}_markov_mpc.npz';bpath=R/f'revision/results/{k}_cross_baseline_fix
↳ ed.npz'

```

```

18 a=np.load(mpath);b=np.load(bpath);pm=parts(a);pb=parts(b);diff=(pb-pm).sum(0);dates=a['dates'];months=np.array
    ↪ ay([str(d)[:7] for d in dates]);n=len(diff)
19 st={'baseline_cost':float(pb.sum()),'main_cost':float(pm.sum()),'saving':float(diff.sum()),'mean':float(diff)
    ↪ .mean()),'positive_days':int((diff>1e-6).sum()),'negative_days':int((diff<
    ↪ -1e-6).sum()),'zero_days':int((abs(diff)<=1e-6).sum()),'monthly':{x:float(diff[months==x].sum()) for x in
    ↪ np.unique(months)},'lag1':float(np.corrcoef(diff[:-1],diff[1:])[0,1]),'bootstrap':{},'component_delta_yu
    ↪ an':(pm-pb).sum(1).tolist(),'main_path':str(mpath.resolve()),'baseline_path':str(bpath.resolve()),'main_
    ↪ sha256':hashlib.sha256(mpath.read_bytes()).hexdigest(),'baseline_sha256':hashlib.sha256(bpath.read_bytes()
    ↪ ).hexdigest()}
20 for block in [3,7,14]:
21 rng=np.random.default_rng(20260911);starts=rng.integers(0,n-block+1,size=(10000,int(np.ceil(n/block))))ix=
    ↪ (starts[:,None]+np.arange(block)).reshape(10000,-1)[:,:n]
22 st['bootstrap'][str(block)]=np.quantile(diff[ix].mean(1),[.025,.975]).tolist()
23 st['positive_months']=sum(v>1e-6 for v in st['monthly'].values());result[k]=st
24 result['_method']={'replicates':10000,'seed':20260911,'blocks':[3,7,14],'method':'overlapping noncircular
    ↪ moving blocks; independent default_rng reset for each mechanism and block; 334
    ↪ truncation','scope':'2025-02-01 through 2025-12-31; same initial/final 6000 kWh; single-year time
    ↪ stability'}
25 (R/'revision/restore/primary-evidence.json').write_text(json.dumps(result,ensure_ascii=False,indent=2)+'\n')
26 def table(file,caption,cols,head,rows):
27 s='\\begin{table}[H]\\centering\\small\\caption{'+caption+'}\\begin{tabular}{'+cols+'}\\toprule\\n'+head+
    ↪ r' \\midrule'+'\n'+'\n'.join(r+r' \\' for r in
    ↪ rows)+'\\n\\bottomrule\\end{tabular}\\end{table}\\n';(R/'tables'/file).write_text(s)
28 table('bootstrap-primary.tex','相对跨日贪心的日均节省与移动块95%区间 (元/日)','lrrrr','机制 & 日均节省 & 3日块
    ↪ & 7日块 & 14日块',[labels[i]+f' & {result[k]["mean"]:.1f}'+'.join(' & ['+', '.join(f'{v:.1f}' for v in
    ↪ result[k]['bootstrap'][str(b)])+']' for b in [3,7,14]) for i,k in enumerate(KS)])
29 table('stability-primary.tex','相对跨日贪心的日、月稳定性','lrrrr','机制 & 正日数 & 负日数 & 正月数/11 &
    ↪ 一阶相关',[f'{labels[i]} & {result[k]["positive_days"]}& {result[k]["negative_days"]}&
    ↪ {result[k]["positive_months"]}/11 & {result[k]["lag1"]:.3f}' for i,k in enumerate(KS)])
30 table('monthly-primary.tex','主策略相对跨日贪心的逐月累计节省 (万元)','lrrrr','月份 & 问题二 & 问题三 &
    ↪ 问题四日前 & 问题四日内',[m+''.join(f' & {result[k]["monthly"][m]/10000:.2f}' for k in KS) for m in
    ↪ result['q2']['monthly']]
31 fig,ax=plt.subplots(figsize=(7,3.0));colors=['#36a9ad','#ed8581','#36a9ad','#e6a072']
32 for i,k in enumerate(KS):
33 st=result[k];lo,hi=st['bootstrap']['7'];ax.errorbar(st['mean'],3-i,xerr=[[st['mean']-lo],[hi-st['mean']]],fm
    ↪ t='o',color=colors[i],capsize=4,linewidth=2)
34 ax.text(870,3-i,f'{st["mean"]:.0f} [{lo:.0f}, {hi:.0f}]',va='center',fontSize=10)
35 ax.axvline(0,color='aaa',ls='--',lw=1);ax.set_yticks(range(4),labels[::-1]);ax.set_xlim(-30,1180);ax.set_yli
    ↪ m(-.6,3.7);ax.set_xlabel('日均节省 / 元 (七日移动块95%区间)
    ↪ ');ax.text(870,3.55,'均值及区间',fontSize=10);ax.grid(axis='x',alpha=.15);fig.tight_layout();fig.savefig(R
    ↪ '/figures/bootstrap-primary.png');plt.close(fig)
36 fig,ax=plt.subplots(1,2,figsize=(9,3.6))
37 for ax,k in zip(axes,['q3','q4_3']):
38 st=result[k];vals=np.array(st['component_delta_yuan']/10000;start=st['baseline_cost']/10000;end=st['main_co
    ↪ st']/10000
39 ax.bar(0,start,color='#92cdd0',width=.65);running=start
40 for j,v in enumerate(vals):
41 ax.bar(j+1,abs(v),bottom=min(running,running+v),width=.65,color='#e88885' if v>0 else
    ↪ '#42afb2');ax.plot([j+.34,j+.66],[running,running],c='#999',lw=.8)
42 ax.text(j+1,running+v+(1 if v>=0 else -1),f'{v:+.2f}',ha='center',va='bottom' if v>=0 else
    ↪ 'top',fontSize=9);running+=v
43 ax.bar(5,end,color='#369d9f',width=.65);ax.plot([4.34,4.66],[end,end],c='#999',lw=.8)
44 ax.text(0,start+1,f'{start:.2f}',ha='center',fontSize=9);ax.text(5,end+1,f'{end:.2f}',ha='center',fontSize=9)
45 ax.set_ylim(min(start,end,np.min(start+np.cumsum(vals)))-12,max(start,end,np.max(start+np.cumsum(vals))+13)
    ↪ );ax.set_xticks(range(6),['跨日\\n贪心','原合同','增购','退购\\n净额','紧急电','主策略']);ax.set_ylabel('费用 /
    ↪ 万元');ax.set_title(labels[KS.index(k)]+f'净节省 {st["saving"]}/10000:.2f'
    ↪ 万元');ax.grid(axis='y',alpha=.15)
46 fig.tight_layout();fig.savefig(R/'figures/decomposition-primary.png');plt.close(fig)
47 print(json.dumps({k:{x:result[k][x] for x in
    ↪ ['saving','mean','positive_days','negative_days','positive_months','bootstrap','component_delta_yuan']}
    ↪ for k in KS},ensure_ascii=False,indent=2))
48 # The horizon increment has its own baseline and seed; never use primary CIs here.
49 nxt=json.loads((R/'review/restoration-control-diagnostics.json').read_text())
50 dlt=np.array(nxt['annual']['q2']['runs']['midnight']['daily_cost_yuan'])-np.array(nxt['annual']['q2']['runs']
    ↪ ['fixed72']['daily_cost_yuan'])
51 diag={'saving':float(dlt.sum()),'mean':float(dlt.mean()),'positive_days':int((dlt>1e-6).sum()),'negative_days'
    ↪ ':int((dlt<
    ↪ -1e-6).sum()),'zero_days':int((abs(dlt)<=1e-6).sum()),'bootstrap':{},'seed':20260912,'replicates':10000}

```

```

52 for block in [3,7,14]:
53     rng=np.random.default_rng(20260912);starts=rng.integers(0,335-block,size=(10000,int(np.ceil(334/block))));id_j
    ↪ s=(starts[:,None]+np.arange(block)).reshape(10000,-1)[:,:334]
54     diag['bootstrap'][str(block)]=np.quantile(dlt[ids].mean(1),[.025,.975]).tolist()
55 (R/'revision/restore/q2-horizon-increment.json').write_text(json.dumps(diag,indent=2)+'\n')

```

review/restoration_control_diagnostics.py

```

1  """Read-only annual horizon contrasts and retrospective forecast moment audit."""
2  from pathlib import Path
3  import sys, json, hashlib
4  import numpy as np
5
6  HERE = Path(__file__).resolve().parent
7  ROOT = HERE.parents[1] / 'C 题 _ 下一代随机控制'
8  sys.path.insert(0, str(ROOT / 'src'))
9  from nextgen_scenarios import ScenarioFactory
10
11 def digest(p): return hashlib.sha256(p.read_bytes()).hexdigest()
12 data = dict(np.load(ROOT/'artifacts/data.npz'))
13 selection = json.loads((ROOT/'artifacts/forecast-selection.json').read_text())
14 ids = {'midnight':'cc3ebe6953b57007', 'fixed48':'5c5eb1d7988dccc0', 'fixed72':'7a7d9f1f5f762810'}
15 out = {'source':'local complete formal JSON+NPZ, recomputed independently', 'annual':{}, 'moments':{},
    ↪ 'files':{}}
16 source_versions=set()
17 for kind in ('q2','q3','q4_2','q4_3'):
18     rows = {}
19     for mode, uid in ids.items():
20         p = ROOT/f'artifacts/formal/{kind}_{uid}.npz'; j = p.with_suffix('.json')
21         if not p.exists() or not j.exists():
22             rows[mode] = {'available':False}; continue
23         a = dict(np.load(p)); m = json.loads(j.read_text()); c = m['configuration']
24         source_versions.add(json.dumps(m['source_hashes'],sort_keys=True))
25         assert np.array_equal(a['days'],np.arange(31,365))
26         assert a['state'].shape == (334,145)
27         assert abs(a['state'][0,0]-6000)<1e-5 and abs(a['state'][-1,-1]-6000)<1e-5
28         assert c['fusion_weight']==1 and c['scenario_method']=='legacy' and c['upper']=='legacy' and
    ↪ c['lower']=='M0'
29         q,r,e,price = [a[k] for k in ('q','r','emergency','price')]
30         bill = price*(r+.5*np.abs(r-q)+5*e)
31         daily = bill.sum(1); recompute_gap = abs(daily.sum()-m['totals']['total_cost'])
32         assert recompute_gap<1e-5
33         balance = r+e+a['d']-a['c']-a['spill']-(data['load'][31:]-data['pv'][31:])/6
34         recurrence = np.diff(a['state'],axis=1)-.9*a['c']+a['d']/.9
35         assert np.max(abs(balance))<1e-5 and np.max(abs(recurrence))<1e-5
36         dec = m['decisions']
37         rows[mode] = {'available':True,'configuration':c,'cost_yuan':float(daily.sum()),
38             'daily_cost_yuan':daily.tolist(), 'recompute_gap_yuan':float(recompute_gap),
39             'balance_max_abs':float(np.max(abs(balance))),
    ↪ 'recurrence_max_abs':float(np.max(abs(recurrence))),
40             'nonmidnight_core_end_count':sum(v['end']%144!=0 for v in dec),
41             'effective_windows':sorted(set(v['effective_window'] for v in dec)),
42             'core_lengths':sorted(set(v['end']-v['as_of'] for v in dec))}
43         out['files'][str(p.relative_to(ROOT))]=digest(p)
44         out['files'][str(j.relative_to(ROOT))]=digest(j)
45     comparisons = {}
46     for old,new in (('midnight','fixed48'),('fixed48','fixed72'),('midnight','fixed72')):
47         if not rows[old]['available'] or not rows[new]['available']: continue
48         differences={k:[rows[old]['configuration'][k],v] for k,v in rows[new]['configuration'].items() if
    ↪ rows[old]['configuration'].get(k)!=v}
49         delta=np.array(rows[old]['daily_cost_yuan'])-np.array(rows[new]['daily_cost_yuan'])
50         comparisons[old+'_to_'+new]={'configuration_differences':differences,'saving_yuan':float(delta.sum()),
51             'saving_percent':float(100*delta.sum()/rows[old]['cost_yuan']),
    ↪ 'positive_days':int(np.sum(delta>1e-6)),
52             'negative_days':int(np.sum(delta < -1e-6)), 'zero_days':int(np.sum(abs(delta)<=1e-6))}
53     out['annual'][kind]={ 'runs':rows,'comparisons':comparisons}
54 assert len(source_versions)==1, 'Horizon comparisons must use identical source versions'
55 out['annual_shared_source_hashes']=json.loads(next(iter(source_versions)))

```

```

56
57 # Replicate only the as-of prediction interface. No optimization or annual selection.
58 masked={k:v.copy() for k,v in data.items()}
59 for name in ('load','pv','price'): masked[name][31:]=np.nan
60 masked['forecast'][31:]=np.nan
61 official=ScenarioFactory(masked,selection,True,False,fusion_weight=1.,official_correction=0.)
62 historical=ScenarioFactory(masked,selection,False,False)
63 for name,start,stop in (('forecast_development_Jan15_24',14,24),('controller_development_Jan25_31',24,31)):
64     eo=[]; eh=[]
65     for day in range(start,stop):
66         for phase in (0,36,72,108):
67             a=day*144+phase; target=masked['pv'].ravel()[a:a+36]
68             eo.extend(official.forecast(a,36)['pv']-target)
69             eh.extend(historical.forecast(a,36)['pv']-target)
70     eo=np.array(eo); eh=np.array(eh)
71     A=float(eo@eo/len(eo)); B=float(eh@eh/len(eh)); C=float(eo@eh/len(eo)); D=float(np.mean((eo-eh)**2))
72     beta=float(np.clip((B-C)/D,0,1)) if D>0 else None
73     grid=[0,.25,.5,.75,1]+([[] if beta is None else [beta]])
74     out['moments'][name]={ 'dates': [str(data['dates'][start]),str(data['dates'][stop-1])],
75         'origins': 4*(stop-start), 'prediction_horizon_slots': 36, 'paired_slots': len(eo),
76         'A_kw2': A, 'B_kw2': B, 'C_kw2': C, 'D_kw2': D, 'beta_star_sample_mse': beta,
77         'official_bias_kw': float(eo.mean()), 'historical_bias_kw': float(eh.mean()),
78         'error_correlation': float(np.corrcoef(eo,eh)[0,1]),
79         'weights': [{ 'beta': b, 'mse_kw2': float(np.mean((b*eo+(1-b)*eh)**2)),
80             'mae_kw': float(np.mean(abs(b*eo+(1-b)*eh))) } for b in grid] }
81 out['files']['artifacts/data.npz']=digest(ROOT/'artifacts/data.npz')
82 out['files']['artifacts/forecast-selection.json']=digest(ROOT/'artifacts/forecast-selection.json')
83 out['files']['script']=digest(Path(__file__))
84 registered=json.loads((ROOT/'artifacts/development-forecast-selection.json').read_text())
85 fresh=out['moments']['forecast_development_Jan15_24']['weights']
86 mae_diffs=[abs(v['mae_kw']-next(z['january_pv_mae_kw'] for z in registered['rows'] if
    ↪ z['fusion_weight']==v['beta'] and z['official_correction']==0)) for v in fresh if v['beta'] in
    ↪ (0,.25,.5,.75,1)]
87 assert max(mae_diffs)<1e-9
88 out['forecast_mae_registration_max_abs_diff_kw']=max(mae_diffs)
89 out['boundary_effect']={'identified':False,'reason': 'No matched counterfactual with identical fixed72
    ↪ horizon/scenario pool/tail rule but full outstanding-contract continuation. Midnight-vs-fixed48 also
    ↪ changes actual horizon and mature history; fixed48-vs-fixed72 changes future deliveries/history/tail price
    ↪ interval. Completion metadata omits outside-core delivery cost and is not a measured error.'}
90 out['selection_scope']='Retrospective January moment diagnosis. Fixed historical forecast families were
    ↪ selected using Jan15-31, overlapping both reported windows. No Feb-Dec observations used in these moment
    ↪ calculations; this is not unbiased held-out validation, and beta_star is an in-sample MSE optimum, not a
    ↪ bill optimum or replacement for beta=.75.'
91 (HERE/'restoration-control-diagnostics.json').write_text(json.dumps(out,ensure_ascii=False,indent=2)+'\n')
92 lines=['# 控制结构恢复性诊断','', '本报告由同目录 restoration_control_diagnostics.py 实际计算; 不修改主论文、
    ↪ 模型或年度轨迹。','',
93     '## 年度时域单配置因素对照','', '| 机制 | 旧两午夜/元 | 固定48h/元 | 固定72h/元 | 48h+72h节省/元 |
    ↪ 两午夜+72h节省/元 |','|---|---|---|---|---|---|']
94 for k,v in out['annual'].items():
95     r=v['runs']; c=v['comparisons']
96     lines.append(f"| {k} | {r['midnight']['cost_yuan']:.2f} | {r['fixed48']['cost_yuan']:.2f} |
    ↪ {r['fixed72']['cost_yuan']:.2f} | {c['fixed48_to_fixed72']['saving_yuan']:.2f} |
    ↪ {c['midnight_to_fixed72']['saving_yuan']:.2f} |")
97 lines+=['','四机制各三组完整334日JSON/NPZ均已找到并重算。比较统一首末6000、beta=1、legacy场景和上层、
    ↪ MO反馈及其余参数; 固定48与72只改变配置horizon_hours, 旧午夜与固定48只改变horizon_mode。它们是配置因素对照,
    ↪ 时域变化还同时改变合法成熟历史池和末值取价区间, 不是固定所有诱导量的纯物理前瞻效应。
    ↪ 该表不使用beta=.75的远端统一候选, 避免把融合收益混入时域对照。正负日数、完整日账单、配置差及哈希见JSON。',' ',
    ↪ '## 开发期配对预测误差矩','',
98     '所有误差为PV预测减实际功率, 单位kW; A=mean(e0^2)、B=mean(eH^2)、C=mean(e0 eH)、D=mean((e0-eH)^2),
    ↪ 不以方差替代有偏误差的二阶矩。每个0/6/12/18时原点预测接下来36段, 窗口内每个目标出现一次。','',
99     '| 窗口 | 配对段数 | A/kW^2 | B/kW^2 | C/kW^2 | D/kW^2 | 样本MSE最优beta
    ↪ |','|---|---|---|---|---|---|']
100 for k,v in out['moments'].items():
101     lines.append(f"| {k} | {v['paired_slots']} | {v['A_kw2']:.6f} | {v['B_kw2']:.6f} | {v['C_kw2']:.6f} |
    ↪ {v['D_kw2']:.6f} | {v['beta_star_sample_mse']:.6f} |")
102 lines+=['','beta*=clip((B-C)/D,0,1), D>0。MSE与MAE不同, 更与受不对称紧急费和库存制约的闭环账单不同;
    ↪ 不能把此样本beta替代现有.75配置。完整候选权重的MSE、MAE、偏差与相关系数保存在JSON。',' ',

```

```

103 '1月15—24日是现有融合MAE选择窗口；1月25—31日是控制器内部开发窗口。固定历史预测族已使用1月15—31日选型，
    ↪ 因此两组结果均为事后内部开发诊断，不能称历史逐原点独立预测验证或无偏留出估计。
    ↪ 脚本将2月至12月观测及官方发布置NaN，原点预测接口进一步遮蔽该原点真实量及未发布预报；
    ↪ 这些限制排除额外未来输入，却不消除预测族选型的1月重叠。','，
104 '## 非午夜边界合同近似能否量化','，
105 '当前数据不能识别其独立费用影响。所需反事实是相同固定72h、相同历史池、相同预测和执行条件下，
    ↪ 将库存未值替换为含未交付合同的继续状态模型，再配对全年运行；现有缓存没有这一匹配对照。
    ↪ 旧午夜与固定48h并非该反事实，因为两者实际时域不同；48与72也改变了历史池和未值时段。','，
106 '可以核对非午夜边界出现次数、核心长度、后缀登记和逐段账单，但不能把域外后缀的名义购电费、
    ↪ 完美信息差或时域差冒充截断误差。正式执行每日日初签完整当日合同，年度结算覆盖全部实际交付；
    ↪ 缺少的是规划继续价值精确度的匹配反事实，不是年度账单遗漏。','，
107 '复现：在任意工作目录执行`python3 review/restoration_control_diagnostics.py`。
    ↪ 该脚本只读源缓存并重写本诊断MD/JSON。']
108 (HERE/'restoration-control-diagnostics.md').write_text('\n'.join(lines)+'\n')
109 print(json.dumps({'annual':{k:v['comparisons'] for k,v in
    ↪ out['annual'].items(),'moments':out['moments']},ensure_ascii=False))

```

下一代诊断对应的模型源码

以下列出固定时域与融合诊断的独立模型版本。dispatch.py、forecasting.py 与 sddp.py 和前述主链逐字节相同，直接复用已列全文；其余四份源码如下。

nextgen-source/control.py

```

1  """Causal affine-policy SAA MPC and one-dimensional convex Markov DP."""
2  from pathlib import Path
3  from types import SimpleNamespace
4  import sys
5  import numpy as np
6  from scipy.sparse import coo_matrix,csr_matrix,vstack,eye
7  sys.path.insert(0,str(Path(__file__).resolve().parents[1]/'vendor'))
8  import highspy
9
10 ETA=.9; M=5000/6; EMIN=1200.; EMAX=10800.; INITIAL=6000.
11
12 def lp_solve(obj,A,b,bounds,solver='ipm'):
13     A=A.tocsr();p=highspy.HighsLp();p.num_col_=len(obj);p.num_row_=len(b)
14     p.col_cost_=np.asarray(obj);p.col_lower_=np.array([a if a is not None else -np.inf for a,z in
    ↪ bounds]);p.col_upper_=np.array([z if z is not None else np.inf for a,z in bounds])
15     p.row_lower_=np.full(len(b),-np.inf);p.row_upper_=np.asarray(b)
16     p.a_matrix_.format_=highspy.MatrixFormat.kRowwise;p.a_matrix_.start_=A.indptr;p.a_matrix_.index_=A.indices
    ↪ s;p.a_matrix_.value_=A.data
17     h=highspy.Highs();h.setOptionValue('output_flag',False);h.setOptionValue('threads',1);h.setOptionValue('s
    ↪ olver',solver);h.passModel(p);h.run()
18     if h.getModelStatus()!=highspy.HighsModelStatus.kOptimal:raise
    ↪ RuntimeError(h.modelStatusToString(h.getModelStatus()))
19     x=np.array(h.getSolution().col_value)
20     return SimpleNamespace(x=x,fun=h.getObjectiveValue(),violation=float(max(0,np.max(A@x-b))))
21
22 def features(net):
23     err=net.mean(0);ema=np.zeros_like(err);avg=np.zeros_like(err);z=np.zeros(len(net))
24     for t in range(net.shape[1]):
25         z=.8*z+.2*err[:,t];ema[:,t]=z;avg[:,t]=err[:,max(0,t-35):t+1].mean(1)
26     return err,ema,avg
27
28 def affine_plan(net,prices,asof,E,base=None,adjust=False,affine=True,tail_price=0.,terminal=None,tail_cuts=No
    ↪ ne,daily_closed=False,solver='ipm'):
29     """Node-time affine recourse. Future contracts use ONLY pre-release errors.
30
31     Scenario states are feasible in-sample; unseen paths are projected physically.
32     Future official forecast innovations are omitted from this restricted policy
33     class, while actual current releases are used by the scenario producer.
34     """
35     S,T=net.shape;K=S*T;clock=np.arange(asof,asof+T); day0=asof//144
36     blocks=clock//36-clock[0]//36;B=int(blocks.max()+1);days=clock//144-day0;D=int(days.max()+1)
37     err,ema,avg=features(net)

```

```

38 # a_q, a_r, a_E, g_q, g_r, g_E, emergency, delivered contract charge.
39 aq=0; ar=T; ae=2*T; gq=3*T; gr=gq+2*D; ge=gr+2*B; em=ge+2*B; bill=em+K; theta=bill+K; N=theta+(S if tail_cuts
↪ else 0)
40 def expr(which):
41     rows=[]; cols=[]; vals=[]; constant=np.zeros(K)
42     for s in range(S):
43         for j,ts in enumerate(clock):
44             row=s*T+j; di=days[j]; bi=blocks[j]
45             if which=='Q' and di==0 and base is not None:
46                 constant[row]=base[j]; continue
47             if which=='Q': a=aq+j; g=gq+2*di; reveal=(ts//144)*144-asof
48             elif which=='R': a=ar+j; g=gr+2*bi; reveal=(ts//36)*36-asof
49             else: a=ae+j; g=ge+2*bi; reveal=j+1
50             rows.append(row); cols.append(a); vals.append(1.)
51             if affine and reveal>0:
52                 values=(err[s,j],ema[s,j]) if which=='E' else (avg[s,reveal-1],ema[s,reveal-1])
53                 for k,val in enumerate(values): rows.append(row); cols.append(g+k); vals.append(val)
54     return coo_matrix((vals,(rows,cols)),shape=(K,N)).tocsr(),constant
55 Q,q0=expr('Q'); R,r0=expr('R') if adjust else (Q,q0); X,x0=expr('E')
56 if adjust:
57     firstblock=np.tile(clock%144<36,S).astype(float)
58     from scipy.sparse import diags
59     keep=diags(firstblock); R=keep@Q+(eye(K)-keep)@R; r0=firstblock*q0+(1-firstblock)*r0
60 shift=coo_matrix((np.ones(S*(T-1)),(np.concatenate([np.arange(s*T+1,(s+1)*T) for s in
↪ range(S)]),np.concatenate([np.arange(s*T,(s+1)*T-1) for s in range(S)]))),shape=(K,K)).tocsr()
61 Delta=X-shift@X; delta0=np.zeros(K); delta0[np.arange(S)*T]=-E
62 U=coo_matrix((np.ones(K),(np.arange(K),em+np.arange(K))),shape=(K,N)).tocsr()
63 F=coo_matrix((np.ones(K),(np.arange(K),bill+np.arange(K))),shape=(K,N)).tocsr()
64 mats=[-Q,-R,X,-X,Delta,-Delta,ETA*Delta-R-U,Delta/ETA-R-U,1.5*R-.5*Q-F,.5*R+.5*Q-F]
65 rhs=[q0,r0,np.full(K,EMAX),np.full(K,-EMIN),np.full(K,ETA*M)-delta0,np.full(K,M/ETA)+delta0,
66 r0-net.ravel()-ETA*delta0,r0-net.ravel()-delta0/ETA,.5*q0-1.5*r0,-.5*q0-.5*r0]
67 last=np.arange(S)*T+T-1
68 if terminal is not None:
69     mats +=[X[last],-X[last]]; rhs +=[np.full(S,terminal),np.full(S,-terminal)]
70 if daily_closed:
71     mids=np.flatnonzero(np.tile((clock+1)%144==0,S))
72     mats +=[X[mids],-X[mids]]; rhs +=[np.full(len(mids),6000.),np.full(len(mids),-6000.)]
73 if tail_cuts:
74     Theta=coo_matrix((np.ones(S),(np.arange(S),theta+np.arange(S))),shape=(S,N)).tocsr()
75     for alpha,beta in tail_cuts:mats.append(beta*X[last]-Theta); rhs.append(np.full(S,-alpha))
76 obj=np.zeros(N); obj[em:bill]=(5*prices/S).ravel(); obj[bill:theta]=(prices/S).ravel()
77 if tail_cuts:obj[theta:]=1/S
78 obj=-tail_price*np.asarray(X[last].mean(0)).ravel()
79 bounds=[(None,None)]*(3*T)+[(-10,10) if affine else (0,0)]*(2*D+4*B)+[(0,None)]*(2*K+(S if tail_cuts else
↪ 0))
80 sol=lp_solve(obj,vstack(mats),np.concatenate(rhs),bounds,solver); x=sol.x
81 qs=(Q@x+q0).reshape(S,T); rs=(R@x+r0).reshape(S,T); xs=(X@x).reshape(S,T)
82 prev=np.c_[np.full(S,E),xs[:,-1]]; delta=xs-prev
83 emer=np.maximum(net-rs+np.maximum(ETA*delta,delta/ETA),0)
84 true=float(np.mean(np.sum(prices*(np.maximum(1.5*rs-.5*qs,.5*rs+.5*qs)+5*emer,axis=1))-tail_price*xs[:,-1]
↪ .mean()))
85 if tail_cuts:true+=float(np.maximum(0,np.max([a+b*xs[:,-1] for a,b in tail_cuts],axis=0)).mean())
86 return {'q':qs.mean(0),'r':rs.mean(0),'a':x[ae:ae+T],'g':x[ge:ge+2*B].reshape(B,2),
87         'center':net.mean(0),'asof':asof,'lp_violation':sol.violation,'objective_gap':abs(true-sol.fun),
88         'scenario_q':qs,'scenario_r':rs,'scenario_state':xs,'forecast_contract_gain':x[gq:gr],
89         'all_gain':x[gq:em],
90         'gain_active_columns':np.asarray(abs(vstack(mats)[: ,gq:em]).sum(axis=0)).ravel(>1e-12)}
91
92 def prune_cuts(cuts):
93     """Remove globally inactive lines on [EMIN, EMAX], preserving the envelope."""
94     by_slope={}
95     for a,b in [[0.,0.]]+list(cuts):by_slope[b]=max(a,by_slope.get(b,-np.inf))
96     hull=[]; starts=[]
97     for b,a in sorted(by_slope.items()):
98         start=-np.inf
99         while hull:
100             pa,pb=hull[-1]; start=(pa-a)/(b-pb)
101             if start>starts[-1]:break
102             hull.pop(); starts.pop()

```

```

103         if not hull:start=-np.inf
104         hull.append([a,b]);starts.append(start)
105     keep=[line for i,line in enumerate(hull) if starts[i]<=EMAX and (i==len(hull)-1 or starts[i+1]>=EMIN)]
106     probes=np.linspace(EMIN,EMAX,301)
107     assert np.max(np.abs(np.max([a+b*probes for a,b in [[0.,0.]]+list(cuts)],axis=0)-np.max([a+b*probes for
        ↪ a,b in keep],axis=0)))<1e-6
108     return keep
109
110 def execute(E,target,q,net,remaining,closed=False,t=0,final=6000.):
111     lo=max(EMIN,E-M/ETA);hi=min(EMAX,E+ETA*M)
112     if closed: remaining=143-t
113     if final is not None:
114         lo=max(lo,final-remaining*ETA*M);hi=min(hi,final+remaining*M/ETA)
115     if lo>hi+1e-6:raise RuntimeError('Unreachable terminal inventory')
116     nex=float(np.clip(target,lo,hi));delta=nex-E;c=max(delta,0)/ETA;d=max(-delta,0)*ETA
117     balance=q+d-c-net
118     return nex,c,d,max(-balance,0),max(balance,0),abs(nex-target)
119
120 def inf_convolution(grid,V,net_minus_q,price):
121     """Exact convex PL infimal convolution, then interpolate on the state grid.
122
123     min_x V(x)+5p max(0,n-q+g(x-E)). Slopes of two convex PL
124     functions are merged. Interpolation is an upper chord approximation.
125     """
126     a=net_minus_q;ys=np.array(sorted(set([-ETA*M,M/ETA,0.,float(np.clip(a/ETA,-ETA*M,M/ETA)),float(np.clip(a*
        ↪ ETA,-ETA*M,M/ETA))])))
127     cost=5*price*np.maximum(0,a+np.maximum(-ETA*ys,-ys/ETA))
128     lengths=np.r_[np.diff(grid),np.diff(ys)];slopes=np.r_[np.diff(V)/np.diff(grid),np.diff(cost)/np.diff(ys)]
129     order=np.argsort(slopes,kind='stable');knots=np.r_[grid[0]+ys[0],grid[0]+ys[0]+np.cumsum(lengths[order])]
130     values=np.r_[V[0]+cost[0],V[0]+cost[0]+np.cumsum(lengths[order]*slopes[order])]
131     return np.interp(grid,knots,values)
132
133 class MarkovDP:
134     """Observable net-load bins, conditional price means, continuous actions.
135
136     Future provisional contracts use the planner's common nominal quantities.
137     Grid value interpolation approximates convex Bellman functions; it is not
138     advertised as SDDP or a global lower bound.
139     """
140     def __init__(self,net,prices,q,tail_price,grid_size=321,terminal=None,tail_cuts=None):
141         S,T=net.shape;self.grid=np.linspace(EMIN,EMAX,grid_size);self.T=T
142         cuts=np.quantile(net,[1/3,2/3],axis=0).T;self.cuts=cuts
143         labels=(net>cuts[:,0]).astype(int)+(net>cuts[:,1]).astype(int)
144         self.future=np.zeros((T,3,grid_size));self.price=np.zeros((T,3))
145         # A finite convex terminal penalty approximates the target in this DP;
146         # hard true-end reachability is enforced separately in execution.
147         v=np.repeat((-tail_price*self.grid)[None,:],3,axis=0)
148         if tail_cuts:v=np.repeat(np.maximum(0,np.max([a+b*self.grid for a,b in
        ↪ tail_cuts],axis=0))[None,:],3,axis=0)
149         if terminal is not None:v=np.repeat((1000*np.abs(self.grid-terminal))[None,:],3,axis=0)
150         for j in range(T-1,-1,-1):
151             counts=np.zeros((3,3))
152             if j<T-1:np.add.at(counts,(labels[:,j],labels[:,j+1]),1.)
153             P=(counts+1/3)/(counts.sum(1,keepdims=True)+1)
154             nxt=P@v;self.future[j]=nxt;cur=np.empty_like(v)
155             for k in range(3):
156                 ids=np.flatnonzero(labels[:,j]==k)
157                 if not len(ids):ids=np.arange(S)
158                 price=float(prices[ids,j].mean());self.price[j,k]=price
159                 cur[k]=np.mean([inf_convolution(self.grid,nxt[k],net[s,j]-q[j],price) for s in ids],axis=0)
160             v=cur
161         self.initial_values=v
162
163     def action(self,j,E,net,q):
164         k=int(net>self.cuts[j,0])+int(net>self.cuts[j,1]);a=net-q
165         lo=max(EMIN,E-M/ETA);hi=min(EMAX,E+ETA*M)
166         # A convex PL minimum occurs at a value knot or an immediate-cost kink.
167         candidates=np.unique(np.r_[self.grid[(self.grid>=lo)&(self.grid<=hi)],lo,hi,np.clip([E,E-a/ETA,E-a*ET
        ↪ A],lo,hi)])

```

```

168         delta=candidates-E;cost=5*self.price[j,k]*np.maximum(0,a+np.maximum(ETA*delta,delta/ETA))+np.interp(c_j
↪ candidates,self.grid,self.future[j,k])
169     return float(candidates[np.argmin(cost)])

```

nextgen-source/nextgen_run.py

```

1  """Incremental chronological replay with the frozen original meter/executor."""
2  from pathlib import Path
3  from dataclasses import asdict
4  import argparse
5  import hashlib
6  import json
7  import time
8  import numpy as np
9  from control import ETA, execute, prune_cuts
10 from dispatch import settlement
11 from nextgen_scenarios import Config, ScenarioFactory, weighted_quantile
12 from nextgen_control import affine_plan, WeightedMarkovDP, EnhancedMarkovDP
13
14 ROOT = Path(__file__).resolve().parents[1]
15 KINDS = {'q2': (False, False), 'q3': (True, False), 'q4_2': (False, True), 'q4_3': (True, True)}
16
17
18 def dump(path, obj):
19     path = Path(path); path.parent.mkdir(parents=True, exist_ok=True)
20     path.write_text(json.dumps(obj, ensure_ascii=False, indent=2, allow_nan=False)+'\n')
21
22
23 def simulate(data, selection, days, kind, config, initial=6000., progress=None):
24     source_hashes = {p.name: hashlib.sha256(p.read_bytes()).hexdigest() for p in
↪ sorted((ROOT/'src').glob('*.py'))}
25     official, variable = KINDS[kind]; days = list(days); count = len(days)
26     arrays = {k: np.zeros((count, 144)) for k in ['q', 'r', 'c', 'd', 'emergency', 'spill', 'price',
↪ 'projection']}
27     arrays['state'] = np.zeros((count, 145)); arrays['releases'] = np.full((count, 4, 144), np.nan)
28     factory = ScenarioFactory(data, selection, official, variable, config.alpha,
29                             config.fusion_weight, config.official_correction)
30     started = time.perf_counter(); E = float(initial); evaluation_end = (days[-1]+1)*144
31     decisions = []; daily = []; lp_violation = objective_gap = 0.; tails = {}
32     planning_seconds = feedback_seconds = scenario_seconds = 0.
33     for i, day in enumerate(days):
34         arrays['state'][i, 0] = E
35         actual = (data['load'][day]-data['pv'][day])/6
36         scoring_price = data['price'][day] if variable else data['day_price']
37         arrays['price'][i] = scoring_price
38         for t in range(144):
39             now = day*144+t
40             if t % 36 == 0:
41                 end = min(now+6*config.horizon_hours if config.horizon_mode == 'fixed'
42                         else (day+config.horizon_hours//24)*144, evaluation_end)
43                 stage_started = time.perf_counter(); bundle = factory.build(now, end, config)
44                 scenario_seconds += time.perf_counter()-stage_started
45                 net, prices, weights = (bundle[k] for k in ('net', 'prices', 'weights'))
46                 base = None if t == 0 else np.r_[arrays['q'][i, t:], np.zeros(end-(day+1)*144)]
47                 terminal = config.final if end == evaluation_end else None
48                 lam = 0.; cuts = None; cutoff = tail_days = None
49                 if end < evaluation_end:
50                     if config.tail == 'linear':
51                         last_prices = prices[:, -min(144, end-now):]
52                         lam = config.tail_scale*float(weighted_quantile(last_prices.ravel(), [.25],
53                             np.repeat(weights, last_prices.shape[1]))[0])/ETA
54                     elif config.tail == 'sddp':
55                         if end % 144: raise ValueError('Existing SDDP tail is valid only at midnight')
56                         cutoff = 14 if day < 31 else day-int(str(data['dates'][day])[8:10])+1
57                         tail_days = min(3, (evaluation_end-end)//144)
58                         suffix = '' if tail_days == 3 else '_D'+str(tail_days)
59                         path = ROOT.parent/'C题_下一代基线复现'/f'artifacts/tails/{kind}_{cutoff}{suffix}.json'
60                         if path not in tails: tails[path] = json.loads(path.read_text())

```

```

61         assert tails[path]['configuration']['cutoff_day_exclusive']*144 <= now
62         cuts = prune_cuts(tails[path]['root_cuts'])
63         elif config.tail != 'zero': raise ValueError(config.tail)
64         plan = affine_plan(bundle, now, E, config, base, official, lam, terminal, cuts)
65         planning_seconds += plan['runtime_seconds']
66         lp_violation = max(lp_violation, plan['lp_violation']); objective_gap = max(objective_gap,
        ↪ plan['objective_gap'])
67         if t == 0:
68             arrays['q'][i] = np.maximum(plan['q'][:144], 0); arrays['r'][i] = arrays['q'][i]
69             arrays['releases'][i, 0] = arrays['q'][i]
70         elif official:
71             arrays['r'][i, t:] = np.maximum(plan['r'][:144-t], 0)
72             arrays['releases'][i, t//36, t:] = arrays['r'][i, t:]
73             effective = plan['r'].copy(); locked = 36 if official else 144-t
74             effective[:locked] = arrays['r'][i, t:t+locked]
75             stage_started = time.perf_counter()
76             if config.lower == 'M0':
77                 dp = WeightedMarkovDP(net, prices, effective, weights, lam, config.grid, config.bins,
        ↪ terminal, cuts)
78             elif config.lower == 'M1':
79                 dp = EnhancedMarkovDP(bundle, effective, now, config, lam, terminal, cuts)
80             else: raise ValueError(config.lower)
81             feedback_seconds += time.perf_counter()-stage_started
82             active = t; zn, ep, zp = bundle['seed'][1:]
83             record = {'**bundle': bundle['meta'], 'date': str(data['dates'][day]), 'release': t//36,
84                     'inventory': E, 'tail_price': lam, 'tail_model_cutoff': cutoff,
85                     'tail_model_days': tail_days, 'tail_cut_count': len(cuts) if cuts else 0,
86                     'contract_information_cutoff': now, 'current_delivery_observation_in_contract':
        ↪ False,
87                     'gain_active_count': plan['gain_active_count'], 'gain_bound_hit_count':
        ↪ plan['gain_bound_hit_count'],
88                     'gain_max_abs': plan['gain_max_abs'], 'lp_violation': plan['lp_violation'],
89                     'objective_recompute_gap': plan['objective_gap'], 'solver_seconds':
        ↪ plan['runtime_seconds'],
90                     'partial_contract_completion': plan['future_partial_contract_completion'],
91                     'contract_information_cutoffs': plan['contract_information_cutoffs']}
92             if config.lower == 'M1': record['enhanced_markov'] = dp.diagnostics()
93             decisions.append(record)
94             j = t-active
95             error = actual[t]-bundle['forecast']['net'][j]; zn = (1-config.alpha)*zn+config.alpha*error
96             state = np.array([error, zn, ep, zp]) if variable else np.array([error, zn])
97             target = dp.action(j, E, actual[t], arrays['r'][i, t], state)
98             E, c, discharge, emergency, spill, projection = execute(E, target, arrays['r'][i, t], actual[t],
        ↪ evaluation_end-now-1, final=config.final)
99             for name, value in [(('c', c), ('d', discharge), ('emergency', emergency), ('spill', spill)),
        ↪ (('projection', projection))]:
100                 arrays[name][i, t] = value
101                 arrays['state'][i, t+1] = E
102                 # Price is disclosed only after the battery action has been committed.
103                 ep = data['price'][day, t]-bundle['forecast']['price'][j]
104                 zp = (1-config.alpha)*zp+config.alpha*ep
105                 cost = settlement(arrays['q'][i], arrays['r'][i], arrays['emergency'][i], scoring_price)
106                 daily.append({'date': str(data['dates'][day]), '**cost',
107                             'emergency_energy': float(arrays['emergency'][i].sum()),
108                             'spill_energy': float(arrays['spill'][i].sum()), 'ending_inventory': E,
109                             'projection_count': int(np.sum(arrays['projection'][i] > 1e-6))})
110             if progress and ((i+1) % 7 == 0 or i+1 == count):
111                 dump(progress, {'kind': kind, 'configuration_id': config.identity(), 'completed_days': i+1,
112                             'requested_days': count, 'last_date': str(data['dates'][day]),
113                             'elapsed_seconds': time.perf_counter()-started, 'status': 'running'})
114             balance = arrays['r']+arrays['emergency']+arrays['d']-arrays['c']-arrays['spill']-(data['load'][days]-dat)
        ↪ a['pv'][days])/6
115             validation = {'balance_max_abs': float(np.abs(balance).max()),
116                         'soc_recurrence_max_abs': float(np.abs(np.diff(arrays['state'],
        ↪ axis=1)-ETA*arrays['c']+arrays['d']/ETA).max()),
117                         'midnight_jump_max_abs': float(np.abs(arrays['state'][1:, 0]-arrays['state'][:-1,
        ↪ -1]).max(initial=0)),
118                         'soc_min': float(arrays['state'].min()), 'soc_max': float(arrays['state'].max()),
119                         'power_max_kw': float(6*max(arrays['c'].max(), arrays['d'].max()))),

```

```

120     'simultaneous_max': float(np.minimum(arrays['c'], arrays['d']).max()),
121     'initial_inventory': initial, 'final_inventory': E, 'lp_violation_max': lp_violation,
122     'lp_objective_recompute_gap': objective_gap}
123     assert validation['balance_max_abs'] < 1e-5 and validation['soc_recurrence_max_abs'] < 1e-5
124     assert validation['midnight_jump_max_abs'] < 1e-5 and validation['soc_min'] >= 1200-1e-5 and
    ↪ validation['soc_max'] <= 10800+1e-5
125     assert config.final is None or abs(E-config.final) < 1e-5
126     arrays['days'] = np.array(days); arrays['dates'] = data['dates'][days]
127     keys = ['planned_cost', 'increase_cost', 'reduction_net_cost', 'emergency_cost', 'total_cost',
    ↪ 'emergency_energy', 'spill_energy', 'projection_count']
128     metrics = {'kind': kind, 'candidate': config.identity(),
129               'configuration': {**asdict(config), 'initial': initial, 'closed_daily': False, 'start_day':
    ↪ days[0], 'end_day': days[-1]},
130               'totals': {k: float(sum(row[k] for row in daily)) for k in keys},
131               'validation': validation, 'daily': daily, 'decisions': decisions,
132               'runtime_seconds': time.perf_counter()-started,
133               'timing': {'planning_seconds': planning_seconds, 'feedback_build_seconds': feedback_seconds,
    ↪ 'scenario_seconds': scenario_seconds},
134               'source_hashes': source_hashes}
135     assert source_hashes == {p.name: hashlib.sha256(p.read_bytes()).hexdigest() for p in
    ↪ sorted((ROOT/'src').glob('*.*py'))}, 'Source changed during replay; do not certify this run'
136     return arrays, metrics
137
138
139 def run_case(kind, configuration, start, stop, output):
140     config = Config(**configuration); data = dict(np.load(ROOT/'artifacts/data.npz'))
141     selection = json.loads((ROOT/'artifacts/forecast-selection.json').read_text())
142     output = Path(output); output.parent.mkdir(parents=True, exist_ok=True)
143     arrays, metrics = simulate(data, selection, range(start, stop), kind, config,
    ↪ progress=output.with_suffix('.progress.json'))
144     np.savez_compressed(output.with_suffix('.npz'), **arrays); dump(output.with_suffix('.json'), metrics)
145     dump(output.with_suffix('.progress.json'), {'status': 'complete', 'completed_days': stop-start})
146     return {'kind': kind, 'configuration': configuration, 'output': str(output), 'cost':
    ↪ metrics['totals']['total_cost'], 'runtime_seconds': metrics['runtime_seconds']}
147
148
149 if __name__ == '__main__':
150     p = argparse.ArgumentParser(); p.add_argument('--kind', default='q2'); p.add_argument('--config',
    ↪ default='{}')
151     p.add_argument('--start', type=int, default=24); p.add_argument('--stop', type=int, default=31)
152     p.add_argument('--output', default=str(ROOT/'artifacts/development/single'))
153     args = p.parse_args(); print(json.dumps(run_case(args.kind, json.loads(args.config), args.start,
    ↪ args.stop, args.output), ensure_ascii=False))

```

nextgen-source/nextgen_control.py

```

1  """Probability-consistent causal affine planning and convex value feedback."""
2  import time
3  import numpy as np
4  from scipy.sparse import coo_matrix, vstack, eye, diags
5  from scipy.spatial.distance import cdist
6  from control import ETA, M, EMIN, EMAX, lp_solve, inf_convolution, MarkovDP, affine_plan as legacy_affine_plan
7  from nextgen_scenarios import weighted_quantile
8
9
10 def affine_plan(bundle, asof, inventory, config, base=None, adjust=False,
11               tail_price=0., terminal=None, tail_cuts=None):
12     started = time.perf_counter()
13     net, prices, weights = (bundle[k] for k in ('net', 'prices', 'weights'))
14     S, T = net.shape; K = S*T; clock = np.arange(asof, asof+T)
15     if config.upper == 'legacy' and config.alpha == .2 and config.gain_bound == 10. and np.all(weights ==
    ↪ 1/S):
16         if tail_cuts and (asof+T) % 144:
17             raise ValueError('Existing SDDP inventory cuts require midnight')
18         answer = legacy_affine_plan(net, prices, asof, inventory, base, adjust, True, tail_price, terminal,
    ↪ tail_cuts)
19         active = answer.pop('gain_active_columns'); gains = answer.pop('all_gain')
20         answer.update({'runtime_seconds': time.perf_counter()-started,

```

```

21         'gain_active_count': int(active.sum()),
22         'gain_bound_hit_count': int(np.sum((np.abs(gains)>=10*(1-1e-6)) & active)),
23         'gain_max_abs': float(np.max(np.abs(gains[active])), initial=0)),
24         'contract_information_cutoffs': {'original': (clock//144*144).tolist(), 'effective':
        ↪ (clock//36*36 if adjust else clock//144*144).tolist()}},
25         'future_partial_contract_completion': 'outside-core suffix fixed to nonnegative origin
        ↪ forecast, no tail delivery cost'})
26     return answer
27     blocks = clock//36-clock[0]//36; B = int(blocks.max()+1)
28     days = clock//144-asof//144; D = int(days.max()+1)
29     if config.upper == 'legacy':
30         error = net-weights@net; ema = np.empty_like(error); avg = np.empty_like(error)
31         z = np.zeros(S)
32         for j in range(T):
33             z = (1-config.alpha)*z+config.alpha*error[:, j]; ema[:, j] = z
34             avg[:, j] = error[:, max(0, j-35):j+1].mean(1)
35             energy_features = np.stack((error, ema), axis=2)
36             previous_features = np.stack((avg, ema), axis=2)
37     elif config.upper == 'state':
38         energy_features = bundle['states'].copy()
39         previous_features = energy_features.copy()
40         if energy_features.shape[-1] == 4:
41             # Battery state has p(t-1); next release may use the newly disclosed pt.
42             ep = prices-bundle['forecast']['price']
43             zp = np.full(S, bundle['seed'][3])
44             for j in range(T):
45                 zp = (1-config.alpha)*zp+config.alpha*ep[:, j]
46                 previous_features[:, j, 2:] = np.c_[ep[:, j], zp]
47                 scale = max(float(np.std(bundle['history_net_error'])),
        ↪ 1e-8)/max(float(np.std(bundle['history_price_error'])), 1e-8)
48                 energy_features[:, :, 2:] *= scale; previous_features[:, :, 2:] *= scale
49                 energy_features -= np.einsum('s,std->td', weights, energy_features)
50                 previous_features -= np.einsum('s,std->td', weights, previous_features)
51     else:
52         raise ValueError(config.upper)
53     Fdim = energy_features.shape[2]
54     aq = 0; ar = T; ae = 2*T; gq = 3*T; gr = gq+Fdim*D
55     ge = gr+Fdim*B; em = ge+Fdim*B; bill = em+K; theta = bill+K
56     N = theta+(S if tail_cuts else 0)
57
58     def expression(which):
59         rows, cols, values = [], [], []; constant = np.zeros(K)
60         for s in range(S):
61             for j, ts in enumerate(clock):
62                 row = s*T+j; di = days[j]; bi = blocks[j]
63                 if which == 'Q' and di == 0 and base is not None:
64                     constant[row] = base[j]; continue
65                 if which == 'Q':
66                     intercept, gain, reveal = aq+j, gq+Fdim*di, ts//144*144-asof
67                 elif which == 'R':
68                     intercept, gain, reveal = ar+j, gr+Fdim*bi, ts//36*36-asof
69                 else:
70                     intercept, gain, reveal = ae+j, ge+Fdim*bi, j+1
71                 rows.append(row); cols.append(intercept); values.append(1.)
72                 if reveal > 0:
73                     feature = energy_features[s, j] if which == 'E' else previous_features[s, reveal-1]
74                     for k, value in enumerate(feature):
75                         rows.append(row); cols.append(gain+k); values.append(value)
76             return coo_matrix((values, (rows, cols)), shape=(K, N)).tocsr(), constant
77
78     Q, q0 = expression('Q'); R, r0 = expression('R') if adjust else (Q, q0)
79     X, _ = expression('E')
80     if adjust:
81         first = np.tile(clock % 144 < 36, S).astype(float); keep = diags(first)
82         R = keep@Q+(eye(K)-keep)@R; r0 = first*q0+(1-first)*r0
83     shift = coo_matrix((np.ones(S*(T-1)),
84         (np.concatenate([np.arange(s*T+1, (s+1)*T) for s in range(S)]),
85         np.concatenate([np.arange(s*T, (s+1)*T-1) for s in range(S)]))), shape=(K, K)).tocsr()
86     Delta = X-shift@X; delta0 = np.zeros(K); delta0[np.arange(S)*T] = -inventory

```

```

87     U = coo_matrix((np.ones(K), (np.arange(K), em+np.arange(K))), shape=(K, N)).tocsr()
88     C = coo_matrix((np.ones(K), (np.arange(K), bill+np.arange(K))), shape=(K, N)).tocsr()
89     mats = [-Q, -R, X, -X, Delta, -Delta, ETA*Delta-R-U, Delta/ETA-R-U,
90             1.5*R-.5*Q-C, .5*R+.5*Q-C]
91     rhs = [q0, r0, np.full(K, EMAX), np.full(K, -EMIN),
92            np.full(K, ETA*M)-delta0, np.full(K, M/ETA)+delta0,
93            r0-net.ravel()-ETA*delta0, r0-net.ravel()-delta0/ETA,
94            .5*q0-1.5*r0, -.5*q0-.5*r0]
95     last = np.arange(S)*T+T-1
96     if terminal is not None:
97         mats += [X[last], -X[last]]; rhs += [np.full(S, terminal), np.full(S, -terminal)]
98     if tail_cuts:
99         if (asof+T) % 144:
100             raise ValueError('Existing inventory-only SDDP cuts require a midnight contract-reset interface')
101             Theta = coo_matrix((np.ones(S), (np.arange(S), theta+np.arange(S))), shape=(S, N)).tocsr()
102             for intercept, slope in tail_cuts:
103                 mats.append(slope*X[last]-Theta); rhs.append(np.full(S, -intercept))
104     obj = np.zeros(N); obj[em:bill] = (5*prices*weights[:, None]).ravel()
105     obj[bill:theta] = (prices*weights[:, None]).ravel()
106     if tail_cuts:
107         obj[theta:] = weights
108     obj -= tail_price*np.asarray(weights@X[last]).ravel()
109     bounds = [(None, None)]*(3*T)+[(-config.gain_bound, config.gain_bound)]*(Fdim*(D+2*B))+[(0, None)]*(2*K+(S
↪ if tail_cuts else 0))
110     sol = lp_solve(obj, vstack(mats), np.concatenate(rhs), bounds)
111     qs = (Q@sol.x+q0).reshape(S, T); rs = (R@sol.x+r0).reshape(S, T)
112     xs = (X@sol.x).reshape(S, T); delta = xs-np.c_[np.full(S, inventory), xs[:, :-1]]
113     emergency = np.maximum(net-rs+np.maximum(ETA*delta, delta/ETA), 0)
114     bills = np.sum(prices*(np.maximum(1.5*rs-.5*qs, .5*rs+.5*qs)+5*emergency), axis=1)
115     recomputed = float(weights@(bills-tail_price*xs[:, -1]))
116     if tail_cuts:
117         recomputed += float(weights@np.maximum(0, np.max([a+b*xs[:, -1] for a, b in tail_cuts], axis=0)))
118     gains = sol.x[gq:em]; bound_hit = np.abs(gains) >= config.gain_bound*(1-1e-6)
119     # Inactive root-node gains do not count toward the binding-frequency denominator.
120     active_columns = np.asarray(abs(vstack(mats)[:, gq:em]).sum(axis=0)).ravel() > 1e-12
121     return {'q': weights@qs, 'r': weights@rs, 'scenario_q': qs, 'scenario_r': rs,
122            'scenario_state': xs, 'lp_violation': sol.violation,
123            'objective_gap': abs(recomputed-sol.fun), 'objective': sol.fun,
124            'runtime_seconds': time.perf_counter()-started,
125            'gain_active_count': int(active_columns.sum()),
126            'gain_bound_hit_count': int(np.sum(bound_hit & active_columns)),
127            'gain_max_abs': float(np.max(np.abs(gains[active_columns])), initial=0),
128            'contract_information_cutoffs': {'original': (clock//144*144).tolist(),
129                                           'effective': (clock//36*36 if adjust else
↪ clock//144*144).tolist()},
130            'future_partial_contract_completion': 'outside-core suffix fixed to nonnegative forecast from this
↪ origin; suffix delivery costs omitted by inventory-tail truncation'}
131
132
133 class WeightedMarkovDP:
134     def __init__(self, net, prices, q, weights, tail_price, grid_size=321, bins=3, terminal=None,
↪ tail_cuts=None):
135         self.legacy = None
136         if bins == 3 and np.allclose(weights, 1/len(weights), rtol=0, atol=1e-15):
137             self.legacy = MarkovDP(net, prices, q, tail_price, grid_size, terminal, tail_cuts)
138             return
139         S, T = net.shape; self.grid = np.linspace(EMIN, EMAX, grid_size)
140         self.cuts = np.array([weighted_quantile(net[:, j], np.arange(1, bins)/bins, weights) for j in
↪ range(T)])
141         labels = np.sum(net[:, :, None] > self.cuts[None], axis=2)
142         self.future = np.empty((T, bins, grid_size)); self.price = np.empty((T, bins))
143         v = np.tile(terminal_values(self.grid, tail_price, terminal, tail_cuts), (bins, 1))
144         for j in range(T-1, -1, -1):
145             counts = np.zeros((bins, bins))
146             if j < T-1:
147                 np.add.at(counts, (labels[:, j], labels[:, j+1]), S*weights)
148                 P = (counts+1/bins)/(counts.sum(1, keepdims=True)+1)
149                 nxt = P@v; self.future[j] = nxt; cur = np.empty_like(v)
150                 for k in range(bins):

```

```

151         ids = np.flatnonzero(labels[:, j] == k)
152         if not len(ids): ids = np.arange(S)
153         w = weights[ids]/weights[ids].sum(); price = float(w@prices[ids, j]); self.price[j, k] = price
154         cur[k] = w@np.array([inf_convolution(self.grid, nxt[k], net[s, j]-q[j], price) for s in ids])
155         v = cur
156         self.initial_values = v
157
158     def action(self, j, E, net, q, state=None):
159         if self.legacy is not None:
160             return self.legacy.action(j, E, net, q)
161         k = int(np.sum(net > self.cuts[j]))
162         return minimize_action(self.grid, self.future[j, k], E, net-q, self.price[j, k])
163
164
165 def terminal_values(grid, price, terminal, cuts):
166     if terminal is not None:
167         return 1000*np.abs(grid-terminal)
168     if cuts:
169         return np.maximum(0, np.max([a+b*grid for a, b in cuts], axis=0))
170     return -price*grid
171
172
173 def minimize_action(grid, future, E, net_minus_q, price):
174     lo = max(EMIN, E-M/ETA); hi = min(EMAX, E+ETA*M); a = net_minus_q
175     candidates = np.unique(np.r_[grid[(grid >= lo) & (grid <= hi)], lo, hi, np.clip([E, E-a/ETA, E-a*ETA], lo,
176     ↪ hi)])
177     delta = candidates-E
178     values = 5*price*np.maximum(0, a+np.maximum(ETA*delta, delta/ETA))+np.interp(candidates, grid, future)
179     return float(candidates[np.argmin(values)])
180
181 def nonnegative_kernel(points, query, scales, prior):
182     distances = cdist(np.atleast_2d(query)/scales, points/scales)**2
183     # Unit bandwidth in legal historical standard deviations, fixed before development.
184     logits = -.5*distances+np.log(np.maximum(prior, 1e-300))[None]
185     w = np.exp(logits-logits.max(1, keepdims=True)); w /= w.sum(1, keepdims=True)
186     return w
187
188
189 class EnhancedMarkovDP:
190     """Origin-conditioned VAR + empirical innovations, convex inventory chords.
191
192     External states use scenario support with nonnegative kernel interpolation.
193     This is an approximate finite-support kernel, not a true-process lower bound.
194     Current price is integrated before choosing the physical action.
195     """
196     def __init__(self, bundle, q, asof, config, tail_price, terminal=None, tail_cuts=None):
197         net, prices, states, weights = (bundle[k] for k in ('net', 'prices', 'states', 'weights'))
198         S, T = net.shape; D = states.shape[2]; self.grid = np.linspace(EMIN, EMAX, config.grid)
199         self.support = states; self.weights = weights; self.variable = D == 4
200         self.future = np.empty((T, S, config.grid)); self.price = np.empty((T, S))
201         self.models = []; self.alpha = config.alpha; self.forecast_price = bundle['forecast']['price']
202         hx = bundle['history_states']; rn, rp = bundle['history_net_error'], bundle['history_price_error']
203         X = hx[:, :-1].reshape(-1, D)
204         Y = np.stack((rn[:, 1:], rp[:, :-1]), axis=2).reshape(-1, 2) if self.variable else rn[:,
205     ↪ 1:].reshape(-1, 1)
206         phases = ((bundle['origins'][:, None]+np.arange(T-1)) % 144//36).ravel()
207         scales = np.std(X, axis=0)
208         # Each error and corresponding EMA share a scale to preserve the EMA algebra.
209         scales[2:] = max(float(np.std(rn)), 1e-8)
210         if self.variable: scales[2:] = max(float(np.std(rp)), 1e-8)
211         self.scales = scales
212         for phase in range(4):
213             ids = np.flatnonzero(phases == phase)
214             if not len(ids): ids = np.arange(len(X))
215             design = np.c_[np.ones(len(ids)), X[ids]/scales]
216             regularizer = np.diag(np.r_[0., np.full(D, config.ridge*len(ids))])
217             coefficients = np.linalg.solve(design.T@design+regularizer, design.T@Y[ids])
218             residuals = Y[ids]-design@coefficients

```

```

218         # Deterministic farthest-first representatives, masses assigned by nearest
219         # empirical innovation. Actual historical residuals, never Gaussian draws.
220         residual_scale = np.maximum(residuals.std(0), 1e-8)
221         standardized = residuals/residual_scale
222         chosen = [int(np.argmax(np.sum(standardized**2, axis=1)))]; distance =
        ↪ np.sum((standardized-standardized[chosen[0]])**2, axis=1)
223     for _ in range(min(config.innovation_count, len(ids))-1):
224         ix = int(np.argmax(distance))
225         if distance[ix] < 1e-14: break
226         chosen.append(ix); distance = np.minimum(distance, np.sum((standardized-standardized[ix])**2,
        ↪ axis=1))
227     labels = np.argmax(cdist(standardized, standardized[chosen]), axis=1)
228     mass = np.bincount(labels, minlength=len(chosen))/len(labels)
229     selected = residuals[chosen]; mean_error = mass@selected
230     # Keep unshifted empirical innovations; report reduced-measure mean error.
231     a = coefficients[1:].T/scales[None]; b = coefficients[0]
232     A = np.empty((D, D)); intercept = np.empty(D)
233     A[0] = a[0]; A[1] = config.alpha*a[0]; A[1, 1] += 1-config.alpha
234     intercept[:2] = b[0], config.alpha*b[0]
235     if self.variable:
236         A[2] = a[1]; A[3] = config.alpha*a[1]; A[3, 3] += 1-config.alpha
237         intercept[2:] = b[1], config.alpha*b[1]
238     self.models.append({'A': A, 'b': intercept, 'innovations': selected, 'mass': mass,
239                       'rows': len(ids), 'reduced_innovation_mean': mean_error})
240     v = np.tile(terminal_values(self.grid, tail_price, terminal, tail_cuts), (S, 1))
241     for j in range(T-1, -1, -1):
242         phase = (asof+j) % 144//36
243         nxt, mean_price = self.transition(states[:, j], phase, self.forecast_price[j])
244         self.price[j] = mean_price
245         if j == T-1:
246             future = np.tile(v[0], (S, 1))
247         else:
248             model = self.models[phase]; kernels = nonnegative_kernel(states[:, j+1], nxt.reshape(-1, D),
        ↪ scales, weights)
249             P = np.einsum('sik,i->sk', kernels.reshape(S, -1, S), model['mass'])
250             future = P@v
251         self.future[j] = future
252         # Collocate the SAME continuous kernel used by action(). A Gaussian
253         # kernel is not cardinal: even at a support point it mixes neighbors.
254         # Applying it only online would define a different greedy policy.
255         support_kernel = nonnegative_kernel(states[:, j], states[:, j], scales, weights)
256         collocated_future = support_kernel@future
257         collocated_price = support_kernel@mean_price
258         v = np.array([inf_convolution(self.grid, collocated_future[s], net[s, j]-q[j],
        ↪ collocated_price[s]) for s in range(S)])
259     self.initial_values = v
260     self.asof = asof
261
262     def transition(self, x, phase, baseline_price):
263         x = np.atleast_2d(x); model = self.models[phase]; innovation = model['innovations']
264         predicted = x@model['A'].T+model['b']
265         next_state = np.repeat(predicted[:, None], len(innovation), axis=1)
266         next_state[:, :, 0] += innovation[:, 0]; next_state[:, :, 1] += self.alpha*innovation[:, 0]
267         if self.variable:
268             error = predicted[:, None, 2]+innovation[None, :, 1]
269             price = np.maximum(.001, baseline_price+error)
270             error = price-baseline_price
271             next_state[:, :, 2] = error
272             next_state[:, :, 3] = (1-self.alpha)*x[:, None, 3]+self.alpha*error
273             mean_price = price@model['mass']
274         else:
275             mean_price = np.full(len(x), baseline_price)
276         return next_state, mean_price
277
278     def action(self, j, E, net, q, state):
279         w = nonnegative_kernel(self.support[:, j], state, self.scales, self.weights)[0]
280         # The same nonnegative interpolation defines the declared approximate kernel
281         # and its continuation value; coefficients never depend on inventory.
282         future = w@self.future[j]

```

```

283         price = float(w@self.price[j])
284         return minimize_action(self.grid, future, E, net-q, price)
285
286     def diagnostics(self):
287         return {'model': 'origin-conditioned empirical VAR with convex inventory interpolation',
288                 'scale': self.scales.tolist(), 'phases': [
289                     {k: v.tolist() if isinstance(v, np.ndarray) else v for k, v in model.items()}
290                     for model in self.models]}

```

nextgen-source/nextgen_scenarios.py

```

1  """As-of conditional trajectory measures; no future realization in planning."""
2  from dataclasses import dataclass, asdict
3  import hashlib
4  import json
5  import numpy as np
6  from scipy.spatial.distance import cdist
7  from forecasting import forecast_as_of
8
9
10 @dataclass(frozen=True)
11 class Config:
12     horizon_hours: int = 48
13     window: int = 28
14     scenarios: int = 7
15     alpha: float = .2
16     gain_bound: float = 10.
17     grid: int = 321
18     bins: int = 3
19     scenario_method: str = 'legacy'
20     upper: str = 'legacy'
21     lower: str = 'M0'
22     tail_scale: float = 1.
23     tail: str = 'linear'
24     horizon_mode: str = 'fixed'
25     final: float | None = 6000.
26     ridge: float = 1e-4
27     innovation_count: int = 10
28     fusion_weight: float = 1.
29     official_correction: float = 0.
30
31     def identity(self):
32         return hashlib.sha256(json.dumps(asdict(self), sort_keys=True).encode()).hexdigest()[:16]
33
34
35 def weighted_quantile(values, quantiles, weights):
36     values = np.asarray(values); weights = np.asarray(weights)
37     if np.allclose(weights, weights[0], rtol=0, atol=1e-15):
38         return np.quantile(values, quantiles)
39     order = np.argsort(values, kind='stable'); w = weights[order] / weights.sum()
40     return np.interp(quantiles, np.cumsum(w) - .5*w, values[order])
41
42
43 def reduce_measure(distance, mass, count, max_swaps=20):
44     """Deterministic weighted forward selection plus PAM swaps, not global PAM."""
45     n = len(mass); count = min(count, n)
46     selected = [int(np.argmin(distance @ mass))]
47     closest = distance[:, selected[0]].copy()
48     while len(selected) < count:
49         improvement = mass @ (closest[:, None] - np.minimum(closest[:, None], distance))
50         improvement[selected] = -np.inf
51         chosen = int(np.argmax(improvement))
52         if improvement[chosen] <= 1e-12:
53             break # Duplicate trajectories do not create independent representatives.
54         selected.append(chosen); closest = np.minimum(closest, distance[:, chosen])
55     swaps = 0
56     for _ in range(max_swaps):
57         old = float(mass @ np.min(distance[:, selected], axis=1)); best = old; choice = None

```

```

58     for position in range(len(selected)):
59         for replacement in range(n):
60             if replacement in selected:
61                 continue
62                 candidate = selected.copy(); candidate[position] = replacement
63                 cost = float(mass @ np.min(distance[:, candidate], axis=1))
64                 if cost < best - 1e-10:
65                     best, choice = cost, candidate
66             if choice is None:
67                 break
68             selected = choice; swaps += 1
69     selected = np.array(sorted(selected), dtype=int)
70     assignments = np.argmin(distance[:, selected], axis=1)
71     reduced = np.bincount(assignments, weights=mass, minlength=len(selected))
72     positive = reduced > 0
73     selected, reduced = selected[positive], reduced[positive]
74     reduced /= reduced.sum()
75     transport = float(mass @ np.min(distance[:, selected], axis=1))
76     return selected, reduced, {'transport_distance': transport, 'pam_swaps': swaps}
77
78
79 def state_paths(net_error, price_error, seeds, alpha, variable):
80     """Battery-node state: current n error, its EMA, lagged p error/EMA."""
81     S, T = net_error.shape
82     seeds = np.broadcast_to(seeds, (S, 4))
83     x = np.empty((S, T, 4 if variable else 2))
84     zn = seeds[:, 1].copy(); zp = seeds[:, 3].copy(); ep = seeds[:, 2].copy()
85     for j in range(T):
86         zn = (1-alpha)*zn + alpha*net_error[:, j]
87         x[:, j, :2] = np.c_[net_error[:, j], zn]
88         if variable:
89             x[:, j, 2:] = np.c_[ep, zp]
90             ep = price_error[:, j]
91             zp = (1-alpha)*zp + alpha*ep
92     return x
93
94
95 class ScenarioFactory:
96     def __init__(self, data, selection, official, variable, alpha=.2,
97                 fusion_weight=1., official_correction=0.):
98         self.data, self.selection = data, selection
99         self.official, self.variable, self.alpha = official, variable, alpha
100        self.fusion_weight, self.official_correction = fusion_weight, official_correction
101        self.forecasts = {}; self.blocks = {}
102        n = (data['load']-data['pv']).ravel()/6; p = data['price'].ravel()
103        self.ledger = np.full((len(n), 4), np.nan)
104        en = n[144:]-n[:-144]; ep = p[144:]-p[:-144]
105        zn, zp = en[0], ep[0]
106        for j in range(len(en)):
107            zn = (1-alpha)*zn+alpha*en[j]; zp = (1-alpha)*zp+alpha*ep[j]
108            self.ledger[j+144] = en[j], zn, ep[j], zp
109
110    def seed(self, asof):
111        assert asof > 144
112        seed = self.ledger[asof-1].copy()
113        assert np.isfinite(seed).all()
114        return seed
115
116    def forecast(self, origin, length):
117        key = origin, length
118        if key not in self.forecasts:
119            visible = dict(self.data)
120            for name in ('load', 'pv', 'price'):
121                visible[name] = self.data[name].copy()
122                visible[name].ravel()[origin:] = np.nan
123            visible['forecast'] = self.data['forecast'].copy()
124            visible['forecast'].reshape(-1, 24)[origin//36+1:] = np.nan
125            pred = forecast_as_of(visible, self.selection, origin,
126                                np.arange(origin, origin+length), self.official, self.variable)

```

```

127         if self.official and (self.fusion_weight != 1. or self.official_correction != 0.):
128             targets = np.arange(origin, origin+length)
129             historical = forecast_as_of(visible, self.selection, origin, targets, False, self.variable)
130             covered = targets+1 <= origin//36*36+144
131             adjustment = 0.
132             if self.official_correction:
133                 errors = []
134                 for past_issue in range(max(144, origin-36)//36*36, origin, 36):
135                     observed = np.arange(max(origin-36, past_issue), min(origin, past_issue+36))
136                     if not len(observed): continue
137                     old = forecast_as_of(visible, self.selection, past_issue, observed, True,
138                                         ↪ self.variable)
139                     errors.extend((visible['pv'].ravel()[observed]-old['pv']).tolist())
140                     adjustment = self.official_correction*float(np.mean(errors)) if errors else 0.
141                     official_pv = np.maximum(0, pred['pv']+adjustment*np.exp(-(targets-origin)/36.))
142                     pred['pv'][covered] =
143                     ↪ self.fusion_weight*official_pv[covered]+(1-self.fusion_weight)*historical['pv'][covered]
144                     pred['net'] = (pred['load']-pred['pv'])/6
145                     assert np.isfinite(pred['net']).all() and np.isfinite(pred['price']).all()
146                     self.forecasts[key] = pred
147             return self.forecasts[key]
148
149 def block(self, origin, length, asof):
150     assert origin+length <= asof, 'Unripe historical residual block'
151     key = origin, length
152     if key not in self.blocks:
153         past = self.forecast(origin, length); targets = slice(origin, origin+length)
154         en = (self.data['load'].ravel()[targets]-self.data['pv'].ravel()[targets])/6-past['net']
155         ep = self.data['price'].ravel()[targets]-past['price']
156         self.blocks[key] = en, ep
157     return self.blocks[key]
158
159 def build(self, asof, end, config):
160     assert config.alpha == self.alpha, 'Factory and policy EMA parameters differ'
161     assert config.fusion_weight == self.fusion_weight and config.official_correction ==
162     ↪ self.official_correction
163     T = end-asof; day, phase = divmod(asof, 144)
164     origins = np.array([h*144+phase for h in range(7, day)
165                         if h*144+phase+T <= asof])[-config.window:], dtype=int)
166
167     if not len(origins):
168         raise ValueError('No mature historical blocks')
169     pairs = [self.block(int(h), T, asof) for h in origins]
170     rn, rp = np.array([v[0] for v in pairs]), np.array([v[1] for v in pairs])
171     seeds = np.array([self.seed(int(h)) for h in origins]); seed = self.seed(asof)
172     dimension = 4 if self.variable else 2
173     scales = np.maximum(np.std(seeds[:, :dimension], axis=0, ddof=1 if len(seeds)>1 else 0), 1e-8)
174     standardized = seeds[:, :dimension]/scales
175     origin_distance = np.linalg.norm((seeds[:, :dimension]-seed[:dimension])/scales, axis=1)
176     pair_distance = cdist(standardized, standardized)
177     nonzero = pair_distance[pair_distance > 1e-10]
178     bandwidth = float(np.median(nonzero)) if len(nonzero) else 1.
179
180     if config.scenario_method == 'conditional':
181         logweights = -.5*(origin_distance/bandwidth)**2
182         mass = np.exp(logweights-logweights.max()); mass /= mass.sum()
183     else:
184         mass = np.full(len(origins), 1/len(origins))
185     ns = max(float(np.std(rn)), 1e-8); ps = max(float(np.std(rp)), 1e-8)
186     joint = np.c_[rn/ns, rp/ps] if self.variable else rn/ns
187     distances = cdist(joint, joint)/np.sqrt(joint.shape[1])
188     if config.scenario_method == 'legacy':
189         selected = np.arange(len(origins))
190         if len(selected) > config.scenarios:
191             selected = np.linspace(0, len(selected)-1, config.scenarios).round().astype(int)
192             weights = np.full(len(selected), 1/len(selected))
193             reduction = {'transport_distance': float(mass @ distances[:, selected].min(1)), 'pam_swaps': 0}
194         elif config.scenario_method in ('medoids', 'conditional'):
195             selected, weights, reduction = reduce_measure(distances, mass, config.scenarios)
196     else:
197         raise ValueError(config.scenario_method)

```

```

193     current = self.forecast(asof, T)
194     completion_length = (-end) % 144
195     completion = np.maximum(0, self.forecast(asof, T+completion_length)['net'][T:]) if completion_length
↪     else np.array([])
196     net = current['net'][None, :]+rn[selected]
197     prices = np.maximum(.001, current['price'][None, :]+rp[selected]) if self.variable else
↪     np.tile(current['price'], (len(selected), 1))
198     if config.scenario_method == 'legacy':
199         # Preserve even the floating-point evaluation order of the accepted
200         # generator for the single-factor horizon compatibility experiments.
201         net = np.array([current['net']+(self.data['load'].ravel()[h:h+T]-self.data['pv'].ravel()[h:h+T])/]
↪         6-self.forecast(int(h), T)['net'] for h in origins[selected]])
202     if self.variable:
203         prices = np.array([np.maximum(.001,
↪         current['price']+self.data['price'].ravel()[h:h+T]-self.forecast(int(h), T)['price']) for
↪         h in origins[selected]])
204     xn = net-current['net']; xp = prices-current['price']
205     states = state_paths(xn, xp, seed, config.alpha, self.variable)
206     history_states = state_paths(rn, rp, seeds, config.alpha, self.variable)
207     tails = rn.sum(1); threshold = weighted_quantile(tails, [.95], mass)[0]
208     metadata = {
209         'as_of': asof, 'end': end, 'horizon_slots': T,
210         'history_origins': origins.tolist(), 'history_days': (origins//144).tolist(),
211         'latest_training_target_exclusive': int(origins.max()+T),
212         'maturity_end': int(origins.max()+T), 'requested_window': config.window,
213         'effective_window': len(origins), 'requested_scenarios': config.scenarios,
214         'effective_scenarios': len(selected), 'representative_origins': origins[selected].tolist(),
215         'candidate_weights': mass.tolist(), 'scenario_weights': weights.tolist(),
216         'weight_ess': float(1/(mass@mass)), 'bandwidth': bandwidth,
217         'state_seed': seed.tolist(), 'state_seed_observation_cutoff': asof-1,
218         'seed_reference': 'previous-day same-slot forecast, EMA initialized at first valid error',
219         'state_reference': 'fixed current forecast origin; deterministic restart at next planning origin',
220         'scenario_method': config.scenario_method, 'tail95_representative_retained':
↪         bool(np.any(tails[selected] >= threshold)),
221         'max_net_residual_retained': bool(np.argmax(rn.max(1)) in selected),
222         'max_price_residual_retained': bool(np.argmax(rp.max(1)) in selected) if self.variable else None,
223         'official_covered_slots': current['official_covered_slots'], **reduction}
224     metadata['outside_core_original_contract_completion'] = {
225         'start': end, 'end': end+completion_length, 'forecast_information_cutoff': asof,
226         'scheduled_release': end//144*144 if completion_length else None,
227         'q_kwh': completion.tolist(), 'feedback_gain': 0.,
228         'rule': 'common nonnegative current-origin forecast; suffix is a causal extension, not optimized
↪         delivery',
229         'delivery_cost_in_core_objective': False}
230     return {'net': net, 'prices': prices, 'weights': weights, 'states': states,
231            'history_states': history_states, 'history_net_error': rn,
232            'history_price_error': rp, 'history_seeds': seeds, 'mass': mass,
233            'forecast': current, 'seed': seed, 'origins': origins, 'meta': metadata}

```